
Sail Rust Book

The Rust, Arrow, and DataFusion Guide



Alexy Khrabrov ∈ LakeSail Team

First Pair Press

A codebase-first edition with PDF, EPUB, HTML, and Obsidian Vault formats

Sail Rust Book

The Rust, Arrow, and DataFusion Guide

Alexy Khrabrov ∈ LakeSail Team

September 2026

Contents

Reader Guide: How This Book Builds	9
0.1 Source boundaries for the 0.7.2 revision	9
0.2 Chapter Links	10
0.3 Concept Progression	10
0.4 Code Reading Strategy	11
0.5 What To Look For In Code Excerpts	12
1 Architecture Overview	12
1.1 The Big Pieces	13
1.2 Spark Connect Is the Front Door	15
1.3 pysail Starts the Rust Server	16
1.4 Sail Uses DataFusion as the Query Kernel	16
1.5 Local Execution Is Direct DataFusion Execution	17
1.6 Cluster Execution Adds a Driver, Workers, Stages, and Shuffles	17
1.7 Shuffle Is Arrow Data Movement	20
1.8 Functions Are Both Planning-Time and Execution-Time Concerns	20
1.9 Where the Implemented Extensions Fit	20
1.10 A First Reading Path Through the Code	21
1.11 Chapter Takeaways	22
2 Rust Foundations in Sail	22
2.1 The Rust Shape of Sail	23
2.2 Shared Ownership With Arc	23
2.3 Trait Objects: dyn Trait	24
2.4 Local and Cluster Runners Share One Interface	25
2.5 Send, Sync, and 'static	26
2.6 Async Traits	26
2.7 Actors: The Control Plane in Rust	27
2.8 Typed Session Extensions	28
2.9 Builders and Mutators	29
2.10 Downcasting Extension Nodes	30
2.11 Error Types	31
2.12 A Mini Example: What Happens to a Query Plan Type	31
2.13 How Rust Shapes the Implemented Extension Boundary	32
2.14 Reading Exercises	32
2.15 Chapter Takeaways	33
2.16 Allocation and accounting in the current source	33
3 Spark Connect	33
3.1 Definitive Spark Connect References	34
3.2 Choosing a session factory as an embedder	34
3.3 The Main Files	35
3.4 The gRPC Service Surface	36
3.5 Sessions: Spark State Inside DataFusion State	37

3.6	Relations Become Sail Specs	38
3.7	SQL Is Also Routed Through the Front Door	40
3.8	Commands	40
3.9	The Core Execution Path	42
3.10	Response Streams and Arrow Batches	42
3.11	Reattachable Operations	43
3.12	Interrupts	45
3.13	Analyze Plan	45
3.14	Config	46
3.15	Data Types and Spark Compatibility	47
3.16	Errors Become Spark Exceptions	47
3.17	Artifacts and Python Distribution	48
3.18	Registering Functions and Data Sources	48
3.19	What Spark Connect Means for Extensions	49
3.20	Reading Exercises	50
3.21	Chapter Takeaways	51
4	PySpark and pysail	51
4.1	The Main Files	52
4.2	Package Shape	52
4.3	Starting Sail From Python	54
4.4	The Global Runtime	56
4.5	Releasing the GIL	57
4.6	Connecting With PySpark	57
4.7	pysail Is Not a PySpark Fork	58
4.8	Python UDFs Enter Through Spark Connect	58
4.9	UDF Kinds	59
4.10	Loading a PySpark UDF Payload	60
4.11	Executing Python UDFs in Process	60
4.12	Python Conversion Code	61
4.13	Config for Python UDF Behavior	62
4.14	UDTFs and Python-Side Analysis	62
4.15	Python Data Sources	63
4.16	Testing PySpark Compatibility	63
4.17	Why PySpark Versions Matter	63
4.18	What pysail Means for Extensions	64
4.19	A Small End-to-End Example	65
4.20	Reading Exercises	66
4.21	Chapter Takeaways	66
5	Apache Arrow	67
5.1	Definitive References	67
5.2	Where Arrow Lives In Sail	67
5.3	The Arrow Mental Model	69
5.4	Why Arc Shows Up Everywhere	70

5.5	RecordBatch As The Unit Of Execution	73
5.6	Spark Connect Output: Arrow IPC In Protobuf Clothing	73
5.7	Schema Conversion: Arrow Back To Spark Types	75
5.8	Extension Types: GeoArrow And Variant	76
5.9	PyArrow: The Python Boundary	77
5.10	UDF Kinds And Arrow-Native Execution	78
5.11	Distributed Shuffle: Arrow As The Exchange Unit	79
5.12	Row Round-Robin Repartition	80
5.13	Flight SQL: Arrow On The Network	80
5.14	Arrow IPC Versus Flight	82
5.15	A Small End-To-End Example	83
5.16	What Arrow Does Not Solve By Itself	84
5.17	Extension Design Pattern: Arrow-First Semantics	85
5.18	Common Mistakes When Learning Arrow In Sail	85
5.19	Reading Exercise: Follow One Batch	85
5.20	Reading Exercise: Follow One Python UDF	86
5.21	How This Prepares Us For DataFusion	86
5.22	Native Extension Ownership Across FFI	87
5.23	Takeaways	87
6	Apache DataFusion	87
6.1	Definitive References	88
6.2	Where DataFusion Lives In Sail	88
6.3	The Critical Function: <code>resolve_and_execute_plan</code>	91
6.4	<code>SessionContext</code> , <code>SessionState</code> , And <code>SessionConfig</code>	92
6.5	Typed Session Extensions	93
6.6	Sail's Query Planner	94
6.7	Logical Extension Nodes	96
6.8	Physical Extension Planning	97
6.9	Physical Plans And <code>ExecutionPlan</code>	98
6.10	Plan Properties: Partitioning, Boundedness, Emission	98
6.11	Logical Optimizers	99
6.12	Physical Optimizers	99
6.13	Function Mapping: Spark Names To DataFusion Expressions	100
6.14	Vectorized UDFs	101
6.15	Table Functions And Providers	102
6.16	DataFusion As Local Engine, Sail As Distributed Engine	103
6.17	Spark Semantics On A DataFusion Kernel	104
6.18	Extension Implications	105
6.19	Reading Exercise: Trace <code>range</code>	105
6.20	Reading Exercise: Trace <code>repartition</code>	106
6.21	Reading Exercise: Trace A Spark Function	106
6.22	Takeaways	107
7	From Physical Plan To Job Graph	107

7.1	The Core Files	107
7.2	Local Runner Versus Cluster Runner	108
7.3	What A Job Graph Represents	111
7.4	StageInputExec: A Placeholder, Not An Operator	112
7.5	Stage Inputs And Input Modes	113
7.6	Output Distribution	114
7.7	How JobGraph::try_new Starts	114
7.8	Rewrite 1: Global Limit Needs One Input Partition	117
7.9	Rewrite 2: Collect-Left Hash Joins And Unmatched Rows	117
7.10	Recursive Stage Splitting	117
7.11	Partition Usage: Single Versus Shared	120
7.12	create_shuffle	120
7.13	Merge And Rescale Boundaries	121
7.14	Driver Stages	122
7.15	Topological Order	123
7.16	Replicas And Repeated Consumption	123
7.17	From Job Graph To Job Topology	124
7.18	Region Dependencies	124
7.19	Task Definitions	125
7.20	Job Output	126
7.21	A Worked Example: Hash Aggregate	126
7.22	A Worked Example: Global Limit	127
7.23	A Worked Example: Broadcast-Like Shared Input	128
7.24	Why This Design Works	128
7.25	Extension Implications	129
7.26	Reading Exercise: Trace A Repartition Boundary	129
7.27	Reading Exercise: Trace Job Acceptance	129
7.28	Worker-Native Regions in the Extension Branch	130
7.29	Takeaways	130
8	Drivers, Workers, Tasks, And Streams	131
8.1	The Core Files	132
8.2	The Actor Runtime	133
8.3	Driver Actor: The Coordinator	134
8.4	Worker Actor: Lifecycle and Services	134
8.5	Worker Launch And Registration	134
8.6	Worker Health	136
8.7	From Job To Task Regions	137
8.8	Task Attempts	138
8.9	Task Regions And Cascading Cancellation	138
8.10	Task Assignment	139
8.11	Scaling Workers	140
8.12	Building A Task Definition	140
8.13	Dispatching Tasks	141

8.14	Running a Task on a Worker	142
8.15	Why The Task Monitor Drains The Stream	142
8.16	Stream Accessor: Actors As Readers And Writers	143
8.17	Stream Identity	143
8.18	Local Stream Ownership	144
8.19	Memory Streams And Replicas	144
8.20	Arrow Flight For Task Streams	144
8.21	Peer Tracking	145
8.22	Cleanup and Stream Tracking	146
8.23	Job Output	146
8.24	One Query Lifecycle	146
8.25	Failure And Retry Story	147
8.26	Why This Design Fits Rust	148
8.27	Extension Implications	148
8.28	Reading Exercise: Follow A Task To A Worker	148
8.29	Reading Exercise: Follow A Stream Fetch	149
8.30	Reading Exercise: Follow Worker Loss	149
8.31	Takeaways	150
9	Shuffle And Data Movement	150
9.1	Current transport and ownership map	150
9.2	Code Map	151
9.3	The Vocabulary Of Movement	152
9.4	From Job Graph To Task Definition	154
9.5	The Runtime Rewrite	155
9.6	Writing Shuffle Output	157
9.7	Reading Shuffle Input	158
9.8	Stream Readers, Writers and Sinks	159
9.9	Local Memory Streams	160
9.10	Arrow Flight As The Remote Shape	160
9.11	A Hash Shuffle Walkthrough	161
9.12	Other Movement Patterns	163
9.13	Failure, Attempts, And Early Termination	164
9.14	Extension Use and Qualification Boundaries	164
9.15	Reading Exercise	165
9.16	Takeaways	165
10	The Sail Spec And Plan Resolver	165
10.1	Code Map	166
10.2	Why Sail Has A Spec Layer	168
10.3	The Full Planning Pipeline	168
10.4	Spark Connect To Sail Spec	171
10.5	SQL To Sail Spec	172
10.6	The Resolver Entry Point	173
10.7	Resolver State	173

10.8	Query Resolution	175
10.9	Reading Tables And Data Sources	176
10.10	Project Resolution And Rewriters	177
10.11	Attribute Resolution	178
10.12	Function Resolution	179
10.13	Commands Become Logical Plans Too	180
10.14	Logical Extension Nodes	181
10.15	Repartition As A Bridge To Distributed Execution	182
10.16	WithRelations And Plan IDs	183
10.17	Commands In Spark Connect Execution	184
10.18	A Worked Example: DataFrame Filter And Project	185
10.19	A Worked Example: Repartition By Customer	185
10.20	A Worked Example: Registered Python UDF	186
10.21	Extension Implications	187
10.22	Resolver Contract and Package Boundary	187
10.23	Design Rules For Future Extensions	187
10.24	Reading Exercise	188
10.25	Takeaways	188
11	Functions, UDFs, And Codecs	189
11.1	Code Map	189
11.2	The Function Lifecycle	191
11.3	Spec Representation	192
11.4	Built-In Function Registry	193
11.5	Function Resolution Order	194
11.6	Registered PySpark UDFs	197
11.7	Inline PySpark UDFs	197
11.8	Scalar Python Execution	198
11.9	Grouped Aggregate Python UDFs	199
11.10	UDTFs And Stream UDFs	200
11.11	The Python Stream Bridge	202
11.12	The Remote Execution Codec	202
11.13	Encoding Scalar UDFs	203
11.14	Decoding Scalar UDFs	204
11.15	Native Package Codecs on the Experimental Branch	205
11.16	Encoding Aggregate And Window UDFs	205
11.17	Encoding Stream UDFs	206
11.18	Worker Sessions And Built-Ins	206
11.19	Distributed Example: Built-In Function	207
11.20	Distributed Example: Python Scalar UDF	207
11.21	Distributed Example: Python UDTF	208
11.22	Codec Design Lessons	211
11.23	Extension Implications	211
11.24	Extension Codecs on the Implementation Branch	212

11.25	Registration and Review Boundaries	213
11.26	Reading Exercise	213
11.27	Takeaways	214
12	Catalogs, Lakehouse Tables, and Data Sources	214
12.1	Code Map	215
12.2	Catalogs: Names Before Data	215
12.3	Catalog Providers	216
12.4	DataSource Is a Logical Planning Interface	217
12.5	Registration and Capabilities	217
12.6	Options, Locations and Time Travel	218
12.7	Listing Writes	218
12.8	LakeSource: Metadata and Row-Level Semantics	218
12.9	Delta and Iceberg Have Their Own Write Paths	219
12.10	Python Data Sources	219
12.11	Storage in the Graph Extensions	220
12.12	Contributor Checks	220
13	Extension Architecture in the Implemented Branch	220
13.1	Four Graph Paths and a Spatial Consumer	221
13.2	Code Map	221
13.3	The Protocol Actually Implemented	222
13.4	Package Discovery and Compatibility	223
13.5	Planning Without Eager Execution	224
13.6	Sedona: Fields Are Part of Correctness	224
13.7	Pecan and Grenada: Use the Existing Relational Engine	225
13.8	Banda: Explicit Native State and Memory Ownership	225
13.9	Argentea: Worker-Local Native Partitions	225
13.10	Failure Semantics and Observability	226
13.11	Build, Run and Review	227
13.12	Review the Host Contract by Responsibility	227
14	Arrow Flight SQL	228
14.1	Code Map	228
14.2	Why Flight SQL Exists Beside Spark Connect	228
14.3	The Service Shape	229
14.4	Phase One: GetFlightInfo	229
14.5	Phase Two: DoGet	230
14.6	The Shared Session Model	230
14.7	Commands	231
14.8	Metrics	231
14.9	Spark Connect Versus Flight SQL	231
14.10	Extension Boundaries Across Front Doors	231
14.11	Takeaways	232
15	Custom Nodes And Optimizers	232
15.1	Code Map	232

15.2	The Pattern	233
15.3	Logical Node Inventory	233
15.4	Example: Range	234
15.5	Example: Explicit Repartition	234
15.6	The Extension Physical Planner	234
15.7	Logical Optimizers	235
15.8	Physical Optimizers	235
15.9	Contributor Checklist	235
15.10	Extension Implications	236
15.11	Opaque Native Regions and Visible Relational Work	236
15.12	Takeaways	236
16	Local And Streaming Execution	237
16.1	Code Map	237
16.2	One Trait, Two Execution Modes	237
16.3	Local Execution	238
16.4	Cluster Execution	238
16.5	Streaming Starts As A Logical Rewrite	239
16.6	Stream Sources	239
16.7	Flow Event Schema	239
16.8	Streaming Physical Nodes	240
16.9	Streaming Query Lifecycle	240
16.10	Current Boundaries	240
16.11	Execution Modes of the Graph Extensions	240
16.12	Takeaways	241
17	Testing Spark Compatibility	241
17.1	Code Map	242
17.2	The Test Pyramid	242
17.3	Gold Tests	242
17.4	Parser Round Trips	243
17.5	PySpark Integration Tests	243
17.6	Flight Tests	244
17.7	Plan Inspection	244
17.8	Local Versus Cluster Testing	244
17.9	Testing New Functions	245
17.10	Testing New Table Formats Or Catalogs	245
17.11	Testing the Implemented Extension Boundaries	245
17.12	Reproduce a Worker Qualification	246
17.13	Failure Evidence Must Establish the Cause	247
17.14	Measurements and Qualification Are Separate	247
17.15	Takeaways	248
18	Feature Playbooks	248
18.1	Adding A Spark Function	248
18.2	Adding A Python UDF Path	249

18.3	Adding A Catalog Backend	249
18.4	Adding a Data Source	250
18.5	Adding A Logical Plan Node	250
18.6	Adding A Physical Optimizer Rule	250
18.7	Adding A Streaming Source	251
18.8	Adding Distributed Codec Support	251
18.9	Adding a Separately Packaged Native Extension	251
18.10	Adding a Worker-Native Algorithm	252
18.11	Debugging A Compatibility Bug	252
18.12	Takeaways	252
19	Roadmap And Codebase Navigation	253
19.1	Start By Task	254
19.2	Route Extension Work by Ownership	254
19.3	Start By Symptom	255
19.4	What Is Mature	255
19.5	What Is Evolving	256
19.6	Capability Snapshot	256
19.7	Reading The Crate Graph	257
19.8	The Files Worth Bookmarking	258
19.9	The Definitive Mental Model	258
19.10	Closing	258
20	The 0.7.2 Preparation Snapshot	259
20.1	Changes That Matter for Extension Authors	259
20.2	The Current Crate Map	259
20.3	Reading Source and Evidence Together	260
20.4	Build the Ordinary Book Formats	261
20.5	The Code-Navigation Vault	261
20.6	Continue from a Concrete Boundary	261

Reader Guide: How This Book Builds

This book is mostly meant to be read in order, but it is also a code companion. Chapters 1-13 walk the core system from architecture through extension design. Chapters 14-20 are definitive-guide companion chapters: they fill protocol, optimizer, streaming, testing, contribution, and navigation coverage that readers will want after the main pass.

0.1 Source boundaries for the 0.7.2 revision

This revision prepares for Sail 0.7.2 using upstream main, whose workspace still declares 0.7.1. It does not announce a released 0.7.2. The upstream source snapshot is 85d06ce08825dfba4e066e9a5f54123e40829d43; the separate extension snapshot is dcd44f287b8422a62abd06d2410aaf96e225e136 on querygraph/sail, branch work/extensions-traversal-bench.

Upstream includes the session-factory embedding entry point from merged #2630. The extension branch adds the experimental protocol, native package integration, and distributed graph implementation discussed later. A successful test on that branch does not establish availability in upstream Sail or a released wheel. Functional process-cluster, physical two-host, and benchmark evidence are separate: each claim must identify the executable and native package it tested.

0.2 Chapter Links

The book is organized into seven reading paths:

- System shape: Chapters 1 and 2 establish the full query pipeline and the Rust patterns that make it possible.
- Front doors: Chapters 3, 4, and 14 show how user intent enters Sail through PySpark, Spark Connect, SQL, Flight SQL, gRPC, protobufs, and Python packaging.
- Columnar runtime: Chapters 5 and 6 introduce the Arrow data model and the DataFusion query engine Sail builds on.
- Distribution: Chapters 7, 8, 9, and 16 show how one DataFusion plan becomes local execution, streaming execution, distributed tasks, and Arrow stream movement.
- Spark semantics: Chapters 10, 11, 12, and 15 explain how Spark-compatible names, expressions, functions, commands, tables, writes, custom logical nodes, and optimizer rules become executable DataFusion objects.
- Extension design: Chapter 13 follows the implemented experimental extension architecture and distinguishes it from upstream embedding capabilities.
- Practice and navigation: Chapters 17, 18, 19, and 20 cover verification, feature work, codebase navigation, and the pinned upstream and extension surfaces.

0.3 Concept Progression

The chapters deliberately introduce concepts before relying on them. This section uses a list instead of a table so the PDF can break cleanly across pages.

- Spark Connect unresolved plans appear first in Chapter 3, then return in Chapters 4 and 10. Chapter 13 uses them to explain why extensions must preserve Spark-facing behavior.
- Spark Connect extension messages appear first in Chapter 3 and are elaborated in Chapter 10. Chapter 13 treats `Relation.extension`, `Command.extension`, and `Expression.extension` as the plan-time extension ABI.
- Flight SQL as a second front door appears first in Chapter 14 and connects back to Chapters 6 and 10. Chapter 13 uses it to motivate protocol-independent registration.
- Rust trait objects and Arc appear first in Chapter 2 and return in Chapters 6, 8, 11, 12, and 16. Chapter 13 uses them for execution-time extension capabilities.
- Arrow `RecordBatch` streams appear first in Chapter 5 and return in Chapters 8, 9, 14, and 16. Chapters 11 and 13 use them for UDFs and custom operators that execute on Arrow batches.
- DataFusion logical and physical plans appear first in Chapter 6 and return in Chapters 7, 10, and 15. Chapter 13 uses them for optimizer rules and physical planners.
- Job graphs and stages appear first in Chapter 7 and return in Chapters 8 and 9. Chapters 13 and 18 use them to show how extension plans survive distributed execution.

- Streaming flow events appear first in Chapter 16 and connect back to Chapters 5 and 15. Chapter 18 uses them for streaming sources that must emit the right event schema.
- Typed session extensions appear first in Chapter 2 and return in Chapters 6, 11, 12, 14, and 16. Chapter 13 explains implemented extension contracts as session services.
- Function resolution and codecs appear in Chapter 11 and return in Chapters 17 and 18. They are the core reason the execution-time extension boundary needs worker-side registration and serialization.
- The table format registry appears in Chapter 12 and returns in Chapter 18. It is the strongest existing model for extension registration.
- The two extension boundaries appear in Chapter 1 and return in Chapters 3, 10, 11, 13, and 18. Chapter 13 separates plan-time and execution-time contributions.
- The Obsidian code vault appears in Chapter 20 and in the vault edition. It is the navigable companion where book chapters, code files, fragments, crates, and subsystems are linked directly.

0.4 Code Reading Strategy

Each chapter has a code map, but these are the highest-leverage excerpts to read first:

- How does a Spark Connect request enter Sail? Read `crates/sail-spark-connect/src/service/plan_executor.rs` and `crates/sail-spark-connect/src/server.rs`.
- How does Sail create a session? Read `crates/sail-session/src/session_factory/server.rs`.
- How does Sail customize DataFusion planning? Read `crates/sail-session/src/planner.rs`.
- How does a physical plan become distributed work? Read `crates/sail-execution/src/job_graph/planner.rs` and `crates/sail-execution/src/job_runner.rs`.
- How do tasks run on workers? Read `crates/sail-execution/src/task_runner/actor/core.rs`.
- How does shuffle move Arrow batches? Read `crates/sail-execution/src/plan/shuffle_write.rs`, `crates/sail-execution/src/plan/shuffle_read.rs`, and `crates/sail-execution/src/stream/`.
- How do Spark functions become DataFusion functions? Read `crates/sail-plan/src/resolver/expression/function.rs` and `crates/sail-plan/src/function/`.
- How do Python UDFs execute? Read `crates/sail-python-udf/src/udf/` and `crates/sail-python-udf/src/stream.rs`.
- How do custom functions and plans reach workers? Read `crates/sail-execution/src/proto/codec.rs`.
- How do catalogs and file formats plug in? Read `crates/sail-catalog/src/manager/mod.rs`, `crates/sail-common-datafusion/src/datasource.rs`, and `crates/sail-session/src/formats.rs`.
- How do lakehouse row-level operations work? Read `crates/sail-delta-lake/`, `crates/sail-iceberg/`, `crates/sail-logical-plan/`, and `crates/sail-session/src/formats.rs`.
- How does Flight SQL enter the same engine? Read `crates/sail-flight/src/service.rs`.

- Where are custom logical nodes planned physically? Read `crates/sail-session/src/planner.rs`.
- Where are optimizer rules registered? Read `crates/sail-session/src/session_factory/server.rs` and `crates/sail-physical-optimizer/src/lib.rs`.
- How does structured streaming change a plan? Read `crates/sail-plan/src/streaming/rewriter.rs` and `crates/sail-common-datafusion/src/streaming/event/schema.rs`.
- How should I add a feature safely? Read Chapter 18, Feature Playbooks.
- How do I jump from prose to exact code fragments? Read Chapter 20, Current Codebase Edition, and use the generated Obsidian vault.

0.5 What To Look For In Code Excerpts

The best excerpts in this book are not chosen because they are short. They are chosen because they reveal a boundary:

- a protobuf boundary,
- a Python/Rust boundary,
- a Spark/DataFusion semantic boundary,
- a local/distributed execution boundary,
- a driver/worker serialization boundary,
- a catalog/table-format boundary,
- or an extension registration boundary.

When reading a snippet, ask what it converts from and what it converts to. Sail’s architecture is mostly a sequence of careful conversions.

Navigation: Start Chapter 1: Architecture Overview

1 Architecture Overview

Sail is easiest to understand as two promises held together by one architecture.

The first promise is compatibility: existing PySpark code should be able to connect to Sail through Spark Connect and keep speaking the language of Spark SQL, DataFrames, functions, UDFs, and sessions. The second promise is performance and portability: the actual engine is Rust, Apache Arrow, and Apache DataFusion, with Sail adding Spark semantics, distributed planning, catalogs, Python interoperability, and cluster execution.

That means Sail is not “Spark implemented in Rust” in the narrow sense. It is a Spark-compatible front door over a DataFusion-centered query engine.

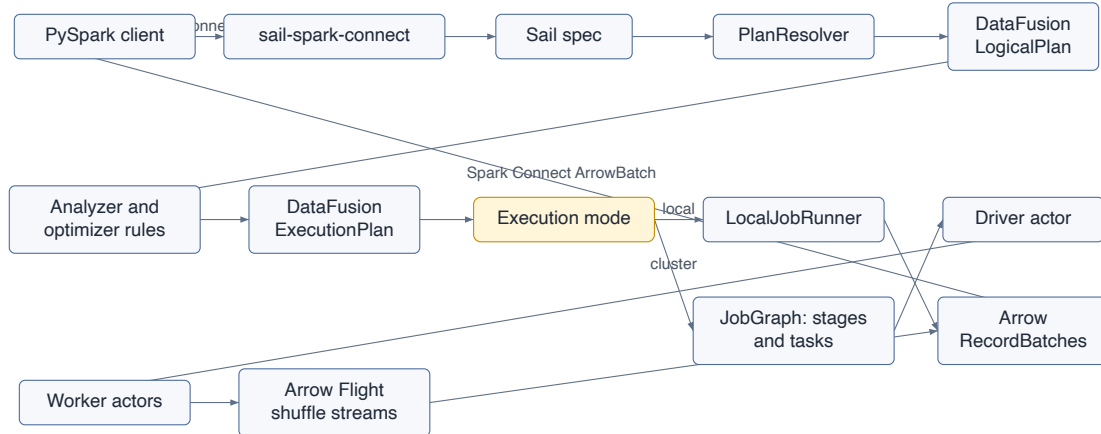


Figure 1: Flowchart 01.1

The rest of this book walks that diagram from left to right, then returns to the extension proposal in discussion #2001 and asks: where should a third-party DataFusion integration plug in so it works in both local and distributed execution?

1.1 The Big Pieces

Sail has a few major subsystems. Each one has a clean teaching role.

Subsystem	Main crates	What to learn there
Spark Connect front door	sail-spark-connect, sail-python, python/pysail	gRPC services, PySpark compatibility, Python-to-Rust server startup
Plan resolution	sail-plan, sail-sql-parser, sail-sql-analyzer	converting SQL and Spark relations into DataFusion logical plans
Session construction	sail-session	DataFusion SessionConfig, SessionState, custom planners, optimizer rules, and job runners
Query execution	sail-execution	local execution, cluster execution, job graphs, stages, tasks, drivers, workers, shuffles
Spark semantics	sail-function, sail-logical-plan, sail-physical-plan, sail-logical-optimizer, sail-physical-optimizer	custom functions, logical nodes, physical nodes, optimizer behavior
Data transport	Arrow, Arrow IPC, Arrow Flight	columnar batches across process and network boundaries
Catalogs and formats	sail-catalog-*, sail-data-source, sail-delta-lake, sail-iceberg	table discovery, scans, writes, system tables, lakehouse integration

The most important mental model is this:

PySpark API call

- > Spark Connect protobuf relation or command
- > Sail spec
- > DataFusion logical plan
- > optimized DataFusion logical plan
- > DataFusion physical execution plan, with Sail extension nodes
- > local stream or distributed job graph
- > Arrow record batches
- > Spark Connect response stream

This is also the extension story. If an integration only hooks one of these layers, it will work only until a query crosses into another layer. A scalar function registered at planning time still has to be recognized when a physical plan is decoded on a remote worker. A logical optimizer rule that

creates a custom extension node still needs a physical extension planner. A session option used by that rule has to be present in the `SessionConfig` that the planner reads.

That is exactly the problem described in discussion #2001.

1.2 Spark Connect Is the Front Door

The Spark Connect server is implemented by `SparkConnectServer` in `crates/sail-spark-connect/src/server.rs`. Its `execute_plan` method receives an `ExecutePlanRequest`, extracts the session ID and user ID, asks the `Sail SessionManager` for a `SessionContext`, and dispatches either a relation or a command.

The distinction matters:

- A relation is a query-producing tree: select, project, filter, join, aggregate, read, SQL relation, and so on.
- A command is an action or side effect: register a function, write data, create a view, start a stream, run a SQL command, merge into a table, and so on.

The central handoff is in `crates/sail-spark-connect/src/service/plan_executor.rs`. `handle_execute_relation` converts the Spark Connect relation into Sail's internal `spec::Plan`, then calls `handle_execute_plan`. That function asks `sail-plan` to resolve and plan the work, then asks the session's `JobService` to execute the resulting physical plan.

The output is a Spark Connect response stream. Sail reads `RecordBatch` values from `DataFusion` and serializes them into Spark Connect `ArrowBatch` messages using `Arrow IPC` in `crates/sail-spark-connect/src/executor.rs`.

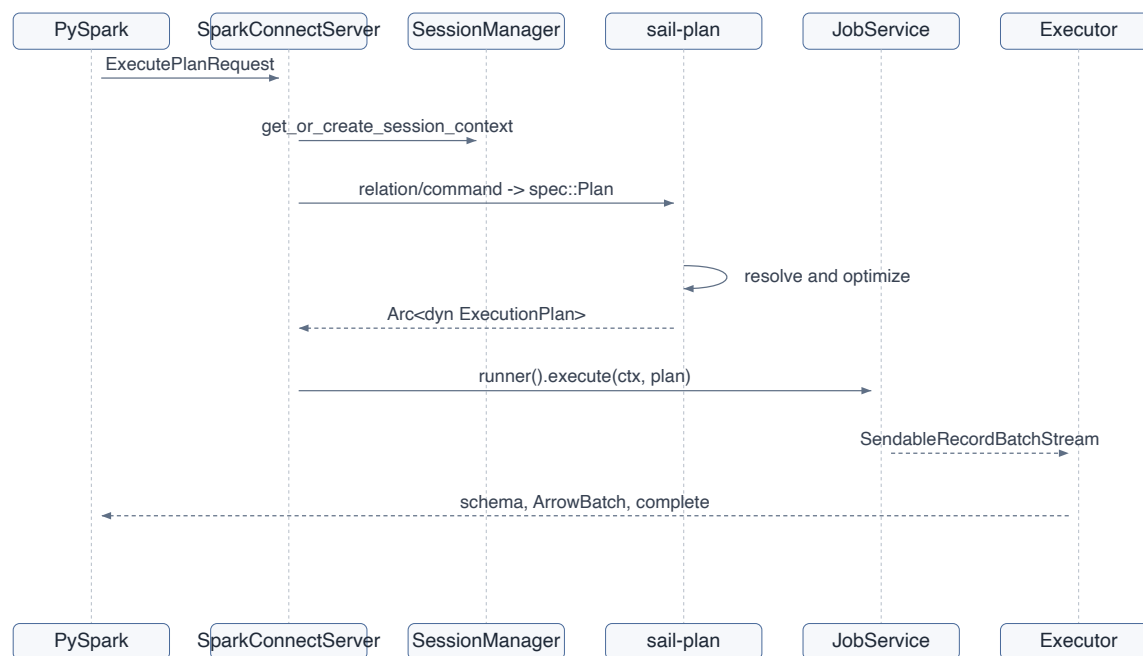


Figure 2: Sequence diagram 01.2

This is why Spark Connect deserves its own chapter. It is not just an API shim; it controls the session lifecycle, the response stream shape, the data type boundary, and the compatibility surface seen by PySpark.

1.3 pysail Starts the Rust Server

The Python package `pysail` is thin by design. The public Python class `python/pysail/spark/__init__.py::SparkConnectServer` delegates to a native PyO3 object.

The Rust side lives in `crates/sail-python/src/spark/server.rs`. It loads `AppConfig`, grabs the global Tokio runtime, binds a TCP listener, and starts the Spark Connect server in a background thread. The implementation explicitly releases the Python GIL while waiting for the server so Python UDFs are not blocked by the server thread.

This shape is important for the extension proposal. If third-party extensions are discovered from Python wheels, `pysail` startup is the natural discovery point. But the extension object has to cross from Python packaging into Rust planning and execution. The experimental branch implements opt-in Python entry points such as:

```
[project.entry-points."pysail.extensions"]
sedona = "sail_sedona:extension"
```

That works as user experience only if the registered extension can contribute to the same Rust-side machinery used by the CLI and by custom embedders.

1.4 Sail Uses DataFusion as the Query Kernel

Sail's query planning entry point is `resolve_and_execute_plan` in `crates/sail-plan/src/lib.rs`.

It performs the key transitions:

1. Build a `PlanResolver`.
2. Resolve a Sail spec into a named `DataFusion LogicalPlan`.
3. Ask `DataFusion's SessionState` to optimize the logical plan.
4. Rewrite streaming plans when needed.
5. Ask the session query planner to create a physical `ExecutionPlan`.
6. Record initial logical, final logical, and final physical plans for explain output.

The important architectural choice is that Sail does not use `DataFusion` as a black box. It uses `DataFusion's` abstractions as the spine, then installs its own semantics around them.

In `crates/sail-session/src/session_factory/server.rs`, Sail creates a `SessionConfig` with custom extensions:

- Table format registry.
- Catalog manager.
- Activity tracker.
- Job service.
- Repartition buffer configuration.

- System table service.
- Delta table cache.

Then it creates a `SessionStateBuilder` with Sail's analyzer rules, optimizer rules, physical optimizer rules, and custom query planner.

That custom query planner is in `crates/sail-session/src/planner.rs`. `ExtensionQueryPlanner` builds a `DataFusion DefaultPhysicalPlanner` with Sail's extension planners:

Delta and Iceberg planners

- > system table planner
- > listing, console, noop and Python planners
- > Sail `ExtensionPhysicalPlanner`

`ExtensionPhysicalPlanner` recognizes Sail logical extension nodes such as range, show string, map partitions, monotonic IDs, Spark partition IDs, streaming nodes, catalog commands, explicit repartition, checkpoints, and barriers. It turns them into physical `ExecutionPlan` implementations from `sail-physical-plan` and related crates.

This is where discussion #2001 finds one of its sharp edges. Today, if `ExtensionPhysicalPlanner` does not recognize a logical extension node, it returns an internal error. `DataFusion`'s extension planner convention is to return `Ok(None)` when a planner does not own a node, allowing later planners in the chain to try. For third-party planners, that difference controls whether composition works.

1.5 Local Execution Is Direct DataFusion Execution

When Sail is running locally, the `ServerSessionFactory` installs a `LocalJobRunner`. Its `execute` implementation in `crates/sail-execution/src/job_runner.rs` wraps the plan in tracing and then calls `DataFusion`'s `execute_stream`.

That is the simplest possible execution path:

`Arc<dyn ExecutionPlan>`

- > `execute_stream(plan, task_ctx)`
- > `SendableRecordBatchStream`
- > Spark Connect response stream

Local mode is ideal for learning `DataFusion` because all of `DataFusion`'s partitioned execution model is still present, but no distributed staging is needed. The physical plan executes in one process, and `DataFusion`'s operators recursively call their children through the `ExecutionPlan` trait.

1.6 Cluster Execution Adds a Driver, Workers, Stages, and Shuffles

Cluster mode swaps in `ClusterJobRunner`. Instead of executing the physical plan directly, it sends a `DriverMessage::ExecuteJob` to a driver actor. The driver builds a distributed job graph and schedules tasks on workers.

The core data structure is `JobGraph` in `crates/sail-execution/src/job_graph/mod.rs`. The code comments are wonderfully plain: a job has stages, each stage has partitions, and tasks execute individual stage partitions. Each task can produce output split into channels.

The graph is built in `crates/sail-execution/src/job_graph/planner.rs`. `JobGraph::try_new` starts from a `DataFusion` physical plan and recursively splits it at distributed boundaries:

- `RepartitionExec` becomes a shuffle boundary.
- `ExplicitRepartitionExec` becomes a shuffle boundary.
- `CoalescePartitionsExec` becomes a shuffle boundary.
- `SortPreservingMergeExec` creates a merge input.
- Sail's `CoalesceExec` creates a rescale input.
- System tables and catalog commands become driver stages.

The planner also contains two distributed-correctness rewrites:

- Global limits are forced to have a single input partition when a limit or offset is present.
- Certain collected hash joins are rewritten into partitioned hash joins when unmatched build-side rows would otherwise require shared row-match state across distributed partitions.

This is one of Sail's best teaching examples. `DataFusion` physical plans already know about partitioning, but a distributed engine must interpret that partitioning as data movement, task placement, materialization, and reuse.

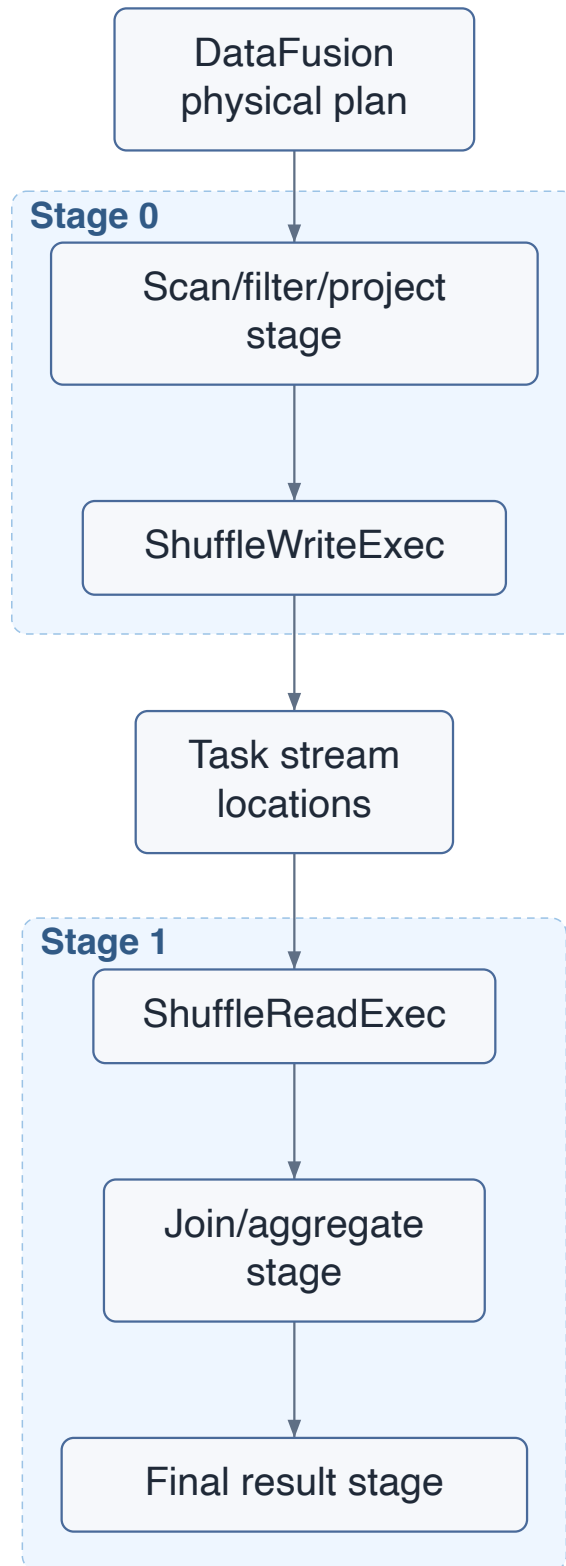


Figure 3: Flowchart 01.3

1.7 Shuffle Is Arrow Data Movement

Sail represents shuffle write and shuffle read as physical execution plan nodes.

`ShuffleWriteExec` in `crates/sail-execution/src/plan/shuffle_write.rs` executes its child for one input partition, partitions each `RecordBatch` into output channels using hash or round-robin partitioning, and writes those partitioned batches to task stream locations.

`ShuffleReadExec` in `crates/sail-execution/src/plan/shuffle_read.rs` has no children. For a given output partition, it opens the task stream locations it needs and merges the resulting record batch streams.

That design keeps the distributed runtime columnar all the way through:

`RecordBatch` stream

- > partition `RecordBatch` into channel batches
- > write channel batches
- > read channel batches from remote/local stream locations
- > merge streams
- > continue as `RecordBatch` stream

The public architecture docs describe Arrow Flight as Sail's data plane for shuffle exchange and result return. The code-level point is that the logical idea of "shuffle" becomes ordinary `DataFusion` `ExecutionPlan` nodes that read and write Arrow batch streams.

1.8 Functions Are Both Planning-Time and Execution-Time Concerns

Sail has a Spark-compatible function layer in `crates/sail-plan/src/function`. Built-in scalar, generator, table, aggregate, and window functions are stored in static registries. The resolver uses those registries to turn unresolved Spark functions into `DataFusion` expressions and UDF objects.

But distributed execution adds another requirement: workers must be able to decode the physical plan they receive. That is why `crates/sail-execution/src/proto/codec.rs` has explicit UDF and UDAF encode/decode logic. It can rebuild PySpark UDFs from serialized payloads, and it can re-resolve many built-in UDF names when decoding standard functions.

This is the most important extension lesson in the chapter:

Planning-time registry is necessary.

Distributed execution-time registry is also necessary.

If an extension contributes `ST_Intersects`, it is not enough for the planner to know the function. A remote worker decoding a physical plan also has to know how to reconstruct the same `ScalarUDF` or `AggregateUDF`. Discussion #2001 calls this out directly for Sedona-style extensions.

1.9 Where the Implemented Extensions Fit

The experimental branch separates client intent from native execution. Clients use registered relation payloads or ordinary function calls. Installed packages bind through Python bootstrap objects and `DataFusion` FFI capsules. They do not implement a public cross-library Rust `SailExtension` trait.

Sedona exercises scalar registration, geometry fields and worker decoding. Nutmeg adds explicit native graph staging, admission and ownership. Pecan demonstrates which graph work can remain ordinary Sail plans. Argentea adds worker-native state while reusing job stages, slot-sharing groups and existing shuffle streams. The algorithm code remains outside Sail's engine crates.

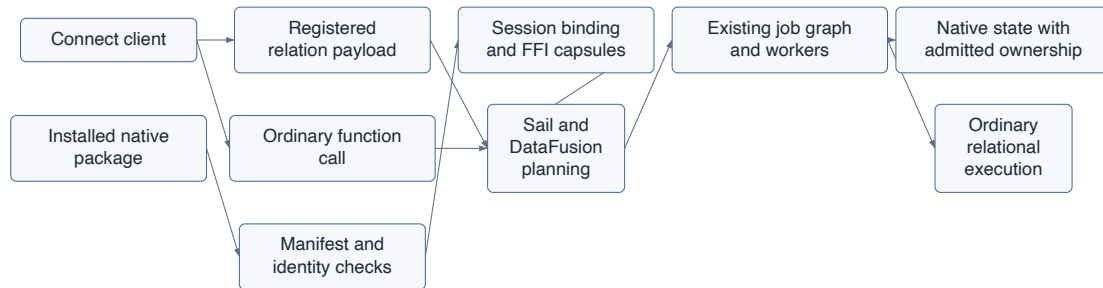


Figure 4: Flowchart 01.4

Raw Connect expression and command extension dispatch remain unsupported in this branch. An indexed spatial join is also outside the delivered Sedona scalar surface. Chapter 13 distinguishes implemented contracts, their artifacts and remaining qualification from the broader design possibilities.

1.10 A First Reading Path Through the Code

For this chapter, read these files in order:

1. docs/concepts/architecture/index.md
2. docs/concepts/query-planning/index.md
3. crates/sail-spark-connect/src/server.rs
4. crates/sail-spark-connect/src/service/plan_executor.rs
5. crates/sail-plan/src/lib.rs
6. crates/sail-session/src/session_factory/server.rs
7. crates/sail-session/src/planner.rs
8. crates/sail-execution/src/job_runner.rs
9. crates/sail-execution/src/job_graph/mod.rs
10. crates/sail-execution/src/job_graph/planner.rs
11. crates/sail-execution/src/plan/shuffle_write.rs
12. crates/sail-execution/src/plan/shuffle_read.rs
13. crates/sail-execution/src/proto/codec.rs

Do not try to understand every operator yet. Follow the type transitions:

```

ExecutePlanRequest
  -> SessionContext
  -> spec::Plan
  -> LogicalPlan
  
```

- > ExecutionPlan
- > SendableRecordBatchStream

Then follow the cluster-only transition:

ExecutionPlan

- > JobGraph
- > Stage
- > StageInput
- > Task
- > ShuffleWriteExec / ShuffleReadExec

Once those two paths feel familiar, the rest of the book can zoom into each layer without losing the whole shape.

1.11 Chapter Takeaways

Sail’s architecture is a layered translation pipeline. PySpark speaks Spark Connect. Spark Connect becomes Sail’s internal spec. The spec resolves into DataFusion logical plans. DataFusion optimizes and physical-plans the query, with Sail adding Spark semantics through custom functions, logical nodes, physical nodes, optimizer rules, and session extensions. Local mode executes the physical plan directly. Cluster mode decomposes it into stages and tasks, moving Arrow record batches through shuffle streams.

The implemented extension experiment tests these boundaries with scalar functions, relational graph plans and retained native graph state. Each path needs a specific combination of registration, planning, physical reconstruction and ownership guarantees. Chapter 13 distinguishes the package contract from internal extension points and explains the focused host changes needed for distributed state.

The next chapter should slow down and teach the Rust patterns that make this architecture possible: trait objects, Arc, async services, actor handles, DataFusion extension traits, and typed session extensions.

2 Rust Foundations in Sail

This chapter is not a full Rust tutorial. It is a map of the Rust ideas you need in order to read Sail without feeling like every file is speaking a private dialect.

Sail is an unusually good Rust learning project because it uses Rust for the things Rust is good at: explicit ownership, cheap shared references, trait-based interfaces, asynchronous services, structured errors, and safe concurrency. It is also a practical systems project, so these ideas show up under pressure. They are not decorative.

The core lesson is this:

Sail moves query plans and Arrow streams through a graph of typed interfaces. Rust makes those interfaces explicit.

When you see `Arc<dyn ExecutionPlan>`, `Box<dyn JobRunner>`, `SessionExtension`, or `ActorHandle<DriverActor>`, you are seeing the architecture in Rust form.

2.1 The Rust Shape of Sail

Chapter 1 described Sail as a pipeline:

Spark Connect request

- > Sail spec
- > DataFusion LogicalPlan
- > DataFusion ExecutionPlan
- > local stream or distributed job graph
- > Arrow RecordBatch stream

Rust gives each boundary a type. A few types appear again and again:

Rust pattern	Sail example	Why it matters
<code>Arc<T></code>	<code>Arc<AppConfig></code> , <code>Arc<dyn ExecutionPlan></code>	Shared ownership across async tasks, sessions, plans, and workers
<code>Box<dyn Trait></code>	<code>Box<dyn JobRunner></code> , <code>Box<dyn ServerSessionMutator></code>	Runtime choice among implementations
<code>Arc<dyn Trait></code>	<code>Arc<dyn ExecutionPlan></code> , <code>Arc<dyn QueryPlanner></code>	Shared polymorphic query operators
<code>async_trait</code>	<code>JobRunner</code> , <code>Actor</code> , gRPC service traits	Async methods in traits
<code>Result<T, E></code>	<code>PlanResult</code> , <code>ExecutionResult</code> , <code>SparkResult</code>	Explicit error paths across planning, execution, and protocol layers
Typed extensions	<code>SessionExtension</code>	Type-safe access to session services and configuration
Actor handles	<code>ActorHandle<DriverActor></code>	Message-passing control plane for distributed execution

These are the vocabulary words of the Sail codebase. The rest of the chapter explains each one through files you have already touched in the architecture overview.

2.2 Shared Ownership With Arc

`Arc<T>` means “atomically reference-counted pointer.” In practical terms, it lets multiple owners hold the same value safely across threads. Sail needs that constantly because sessions, runtimes, query plans, actors, and task contexts all outlive a single function call.

Look at `ServerSessionFactory` in `crates/sail-session/src/session_factory/server.rs`:

```
pub struct ServerSessionFactory {  
    config: Arc<AppConfig>,  
    ...  
}
```

```

runtime: RuntimeHandle,
system: Arc<Mutex<ActorSystem>>,
mutator: Box<dyn ServerSessionMutator>,
runtime_env: RuntimeEnvFactory,
catalog_cache_manager: Arc<CatalogCacheManager>,
}

```

The factory does not own the global application config in a lonely way. It shares it. The session factory, runtime environment factory, catalog manager, worker manager, and driver setup can all receive clones of the same `Arc<AppConfig>`.

Cloning an `Arc` does not clone the underlying config. It increments a reference count:

```
let runtime_env = RuntimeEnvFactory::new(config.clone(), runtime.clone());
```

That line is small, but it is one of Rust's most important performance habits. Large shared state can be passed cheaply while ownership stays explicit.

`DataFusion` plans use the same idea. A physical plan in `Sail` is usually:

```
Arc<dyn ExecutionPlan>
```

That reads as:

shared pointer to some concrete type that implements `DataFusion`'s `ExecutionPlan` trait

The concrete type might be a `DataFusion` operator, `ShuffleWriteExec`, `ShuffleReadExec`, `RangeExec`, `MapPartitionsExec`, `FileWriteExec`, or another `Sail` extension. The caller often does not need to know. It needs the `ExecutionPlan` interface.

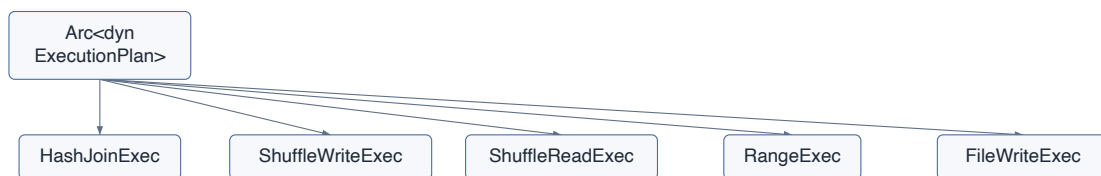


Figure 5: Flowchart 02.1

The `Arc` part lets the plan be shared. The `dyn ExecutionPlan` part lets the plan be polymorphic.

2.3 Trait Objects: `dyn Trait`

Traits define behavior. Trait objects let `Sail` pick an implementation at runtime.

The `JobRunner` trait in `crates/sail-common-datafusion/src/session/job.rs` is the cleanest example:

```

#[tonic::async_trait]
pub trait JobRunner: StateObservable<JobRunnerObserver> + Send + Sync + 'static {
    async fn execute(
        &self,
        ctx: &SessionContext,
    )
}

```

```

        plan: Arc<dyn ExecutionPlan>,
    ) -> Result<SendableRecordBatchStream>;

    async fn stop(&self, history: oneshot::Sender<JobRunnerHistory>);
}

```

A JobRunner takes a DataFusion physical plan and returns a stream of Arrow record batches. That is the interface. The implementation depends on execution mode.

In ServerSessionFactory::create_job_runner, Sail chooses:

```

let job_runner: Box<dyn JobRunner> = match self.config.mode {
    ExecutionMode::Local => Box::new(LocalJobRunner::new()),
    ExecutionMode::LocalCluster => { ... Box::new(ClusterJobRunner::new(...)) }
    ExecutionMode::KubernetesCluster => { ...
Box::new(ClusterJobRunner::new(...)) }
};

```

This is runtime polymorphism. The rest of the session does not need to branch on local versus cluster every time it executes a query. It just calls:

```
service.runner().execute(ctx, plan).await
```

The object behind runner() decides what that means.

The extension implementation also retains shared objects behind reference-counted handles. Its native package boundary uses the pinned FFI capsule contract described in chapter 13, rather than passing a public Rust SailExtension trait object across independently built libraries.

2.4 Local and Cluster Runners Share One Interface

LocalJobRunner and ClusterJobRunner are a small but powerful comparison.

The local runner executes the DataFusion physical plan directly:

```
Ok(execute_stream(plan, ctx.task_ctx()?))
```

The cluster runner sends the same plan to a driver actor:

```

self.driver
    .send(DriverMessage::ExecuteJob {
        plan,
        context: ctx.task_ctx(),
        result: tx,
    })
    .await?;

```

Same trait. Same method signature. Very different behavior.

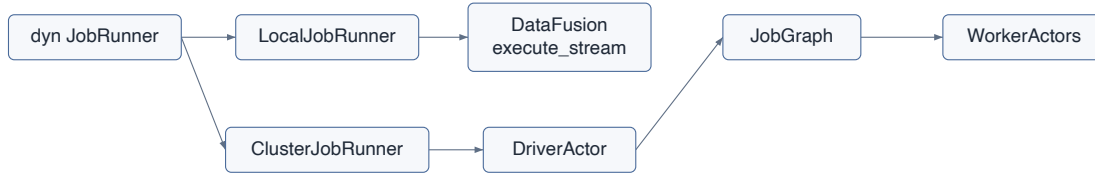


Figure 6: Flowchart 02.2

This is one of the most important Rust design moves in Sail: put the architectural decision behind a trait, then pass the trait object through the rest of the system.

2.5 Send, Sync, and 'static

You will often see trait bounds like this:

```
pub trait JobRunner: StateObservable<JobRunnerObserver> + Send + Sync + 'static
```

These are not noise.

Send means a value can be moved to another thread. Sync means references to it can be shared across threads. 'static means the value does not contain borrowed references that could expire while async tasks or background actors still need it.

Sail is full of async tasks, actor messages, gRPC handlers, and worker processes. If a service may be stored in a session, used by a task, or held across an `.await`, Rust needs to know it is safe to move and share.

These bounds also matter for extension-owned state that can outlive a planning call or be accessed by asynchronous tasks. Inside one compiled Rust program, trait objects express such contracts. Across separately built native wheels, Rust trait-object layout is not a stable binary interface; the implemented experiment uses explicit FFI capsules and lifetime contracts instead.

2.6 Async Traits

Rust traits do not natively support async methods in the most ergonomic way for this kind of code, so Sail uses `#[tonic::async_trait]` or `#[async_trait]`.

You see it in three central places:

- gRPC services, such as Spark Connect and worker services.
- JobRunner, where execution returns an async stream-producing result.
- Actor, where startup and shutdown may be async.

The Actor trait in `crates/sail-common/src/actor.rs` looks like this:

```
#[tonic::async_trait]
pub trait Actor: Sized + Send + 'static {
    type Message: Send + SpanAssociation + 'static;
    type Options;

    fn name() -> &'static str;
```

```

    fn new(options: Self::Options) -> Self;
    async fn start(&mut self, ctx: &mut ActorContext<Self>) {}
    fn receive(&mut self, ctx: &mut ActorContext<Self>, message: Self::Message) -
> ActorAction;
    async fn stop(self, ctx: &mut ActorContext<Self>) {}
}

```

Notice the split:

- start and stop are async because they may do setup or teardown work.
- receive is synchronous and should not block. If it needs async work, it spawns a task via the actor context.

That is a deliberate concurrency model. Actor message handling stays sequential, while longer async work is pushed into spawned tasks.

2.7 Actors: The Control Plane in Rust

Sail's distributed execution control plane uses actors. An actor owns state. Other code sends it messages through an `ActorHandle<T>`.

The generic actor system is in `crates/sail-common/src/actor.rs`:

```

pub struct ActorHandle<T: Actor> {
    sender: mpsc::Sender<MessageEnvelop<T::Message>>,
}

```

An `ActorHandle<DriverActor>` can send only `DriverActor` messages. An `ActorHandle<WorkerActor>` can send only `WorkerActor` messages. This gives the message-passing system compile-time shape.

The worker gRPC service shows the pattern. In `crates/sail-execution/src/worker/server.rs`, `run_task_batch` decodes the task definition, validates integer conversions for stages and attempts, and sends `TaskRunnerMessage::RunTaskBatch` through its task-runner handle. A oneshot channel returns the result to the RPC handler.

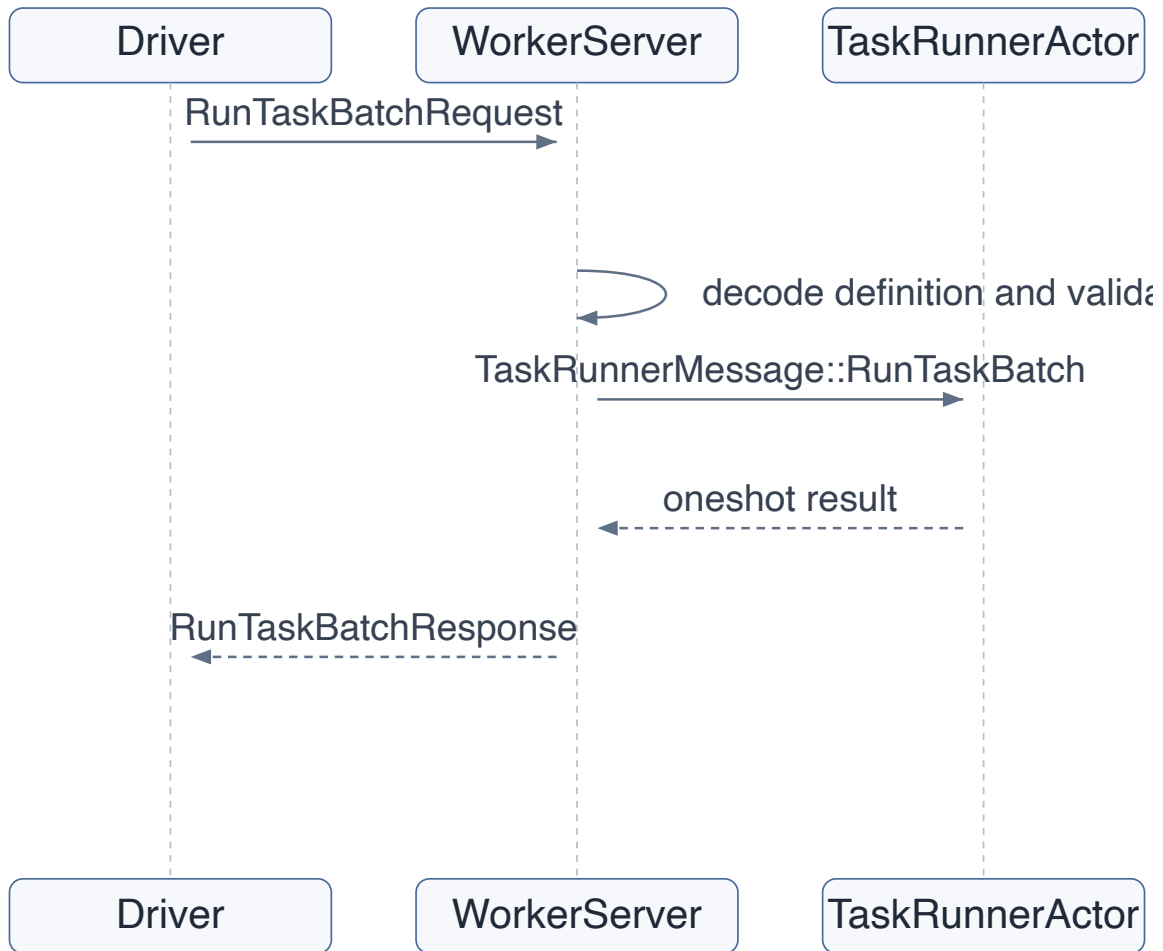


Figure 7: Sequence diagram 02.3

The worker lifecycle actor remains responsible for readiness, heartbeats and shutdown. Rust's typed handles distinguish it from the task runner: a message for the driver is not a valid message for `ActorHandle<TaskRunnerActor>`. Follow the current message types rather than older `WorkerEvent` examples when adding an RPC path.

2.8 Typed Session Extensions

DataFusion's `SessionConfig` can store extensions. Sail wraps that in a small trait:

```
pub trait SessionExtension: Send + Sync + 'static {
    fn name() -> &'static str;
}
```

Then `SessionExtensionAccessor` provides typed lookup from `SessionContext`, `SessionState`, `TaskContext`, and DataFusion's `Session` trait:

```
fn extension<T: SessionExtension>(&self) -> Result<Arc<T>>;
```

This turns session services into type-safe dependencies. For example, Spark Connect execution can ask the session for its `SparkSession` extension. Planning and physical execution code can ask for the catalog manager, data source registry, job service, activity tracker, repartition config, or system table service.

The pattern is:

```
register typed extension during session creation
-> retrieve typed extension where needed
-> fail clearly if missing
```

In `ServerSessionFactory::create_session_config`, Sail registers many extensions:

```
SessionConfig::new()
  .with_extension(create_data_source_registry()?)
  .with_extension(Arc::new(create_catalog_manager(...)?))
  .with_extension(Arc::new(ActivityTracker::new()))
  .with_extension(Arc::new(JobService::new(job_runner)))
  .with_extension(Arc::new(RepartitionBufferConfig::new(...)))
  .with_extension(Arc::new(self.create_system_table_service(info)?))
  .with_extension(Arc::new(DeltaTableCache::default()))
```

This matters for extensions because many third-party integrations need session state. Sedona-style spatial planning, for example, may need options that optimizer rules can read. The current `ServerSessionMutator` can mutate `SessionConfig`, `SessionStateBuilder`, and `RuntimeEnvBuilder`, but discussion #2001 argues that this is not enough because functions, codec re-resolution, and extension planner registration live elsewhere.

2.9 Builders and Mutators

Sail often uses builder-style APIs because DataFusion itself uses them. Session creation is the main example.

`ServerSessionFactory::create_session_state` builds a DataFusion session state:

```
let builder = SessionStateBuilder::new()
  .with_config(config)
  .with_runtime_env(runtime)
  .with_analyzer_rules(default_analyzer_rules())
  .with_optimizer_rules(default_optimizer_rules())
  .with_physical_optimizer_rules(get_physical_optimizers(...))
  .with_query_planner(new_query_planner());
let builder = self.mutator.mutate_state(builder, info)?;
Ok(builder.build())
```

The builder has two jobs:

- Accumulate configuration in a readable order.
- Give Sail one place to inject custom behavior before the immutable session state is built.

The mutator has a narrower purpose:

```
pub trait ServerSessionMutator: Send {
    fn mutate_config(...) -> Result<SessionConfig>;
    fn mutate_state(...) -> Result<SessionStateBuilder>;
    fn mutate_runtime_env(...) -> Result<RuntimeEnvBuilder>;
}
```

This is already an extension-like boundary. But it is embedder-oriented, not package/plugin-oriented. It does not solve plan-time function registries or worker-side UDF decoding. The experimental package contract in Chapter 13 addresses additional registration, serialization and lifetime requirements beyond this embedding hook.

2.10 Downcasting Extension Nodes

DataFusion has extension traits for custom logical and physical behavior. Sail uses them heavily.

In `crates/sail-session/src/planner.rs`, `ExtensionPhysicalPlanner` receives a generic `UserDefinedLogicalNode`:

```
async fn plan_extension(
    &self,
    planner: &dyn PhysicalPlanner,
    node: &dyn UserDefinedLogicalNode,
    logical_inputs: &[&LogicalPlan],
    physical_inputs: &[Arc<dyn ExecutionPlan>],
    session_state: &SessionState,
) -> Result<Option<Arc<dyn ExecutionPlan>>>
```

The planner then asks, one type at a time, whether the node is a Sail node:

```
if let Some(node) = node.as_any().downcast_ref::<RangeNode>() {
    ...
} else if let Some(node) = node.as_any().downcast_ref::<ShowStringNode>() {
    ...
} else if let Some(node) = node.as_any().downcast_ref::<MapPartitionsNode>() {
    ...
}
```

This is Rust’s way of combining an open interface with concrete behavior. The planner receives “some extension node.” It can only plan nodes it recognizes. Recognition happens through Any downcasting.

For readers, this explains a lot of Sail code:

```
trait object enters boundary
    -> as_any()
    -> downcast_ref::<ConcreteType>()
    -> concrete planning or execution logic
```

For extension authors, it explains why planner ordering matters. If one planner errors on unknown nodes instead of returning `Ok(None)`, later planners never get a chance.

2.11 Error Types

Sail has separate error layers:

- PlanError in sail-plan.
- ExecutionError in sail-execution.
- SparkError in sail-spark-connect.
- DataFusion's own DataFusionError.

The aliases are simple:

```
pub type PlanResult<T> = Result<T, PlanError>;
pub type ExecutionResult<T> = Result<T, ExecutionError>;
pub type SparkResult<T> = Result<T, SparkError>;
```

The point is not just style. Each layer needs to add context in its own vocabulary.

Planning errors talk about unsupported functions, invalid expressions, unresolved fields, and semantic analysis. Execution errors talk about task definitions, worker communication, job graphs, and DataFusion execution. Spark errors must become protocol-level statuses and Spark-compatible error responses.

When reading Sail, track where errors cross boundaries. A failed worker task should not leak as an arbitrary Rust panic. An unknown Spark function should become a planning error. A malformed protobuf request should become a Spark Connect status error.

2.12 A Mini Example: What Happens to a Query Plan Type

Here is the lifecycle of one important type:

```
Arc<dyn ExecutionPlan>
```

In local mode:

```
Arc<dyn ExecutionPlan>
-> LocalJobRunner::execute
-> datafusion::physical_plan::execute_stream
-> SendableRecordBatchStream
```

In cluster mode:

```
Arc<dyn ExecutionPlan>
-> ClusterJobRunner::execute
-> DriverMessage::ExecuteJob
-> JobGraph::try_new
-> Stage plans
-> serialized task definitions
-> worker execution
-> shuffle and result streams
```

Same Rust type, different execution strategy.

This is why `Arc<dyn ExecutionPlan>` is not just a pointer. It is the main currency between DataFusion and Sail's execution system.

2.13 How Rust Shapes the Implemented Extension Boundary

Within Sail, Arc, typed session services and trait objects organize ownership and dispatch. Across the native package boundary, the experiment uses named DataFusion FFI capsules plus an explicit resource-lease ABI. A Python entry point is discovery and bootstrap; it does not make an arbitrary Rust object safe to interpret using the host's crate layout.

The loader checks declared API, DataFusion and Arrow compatibility and validates package identity for distributed execution. The qualification matrix tests particular unchanged-wheel and host combinations. Those observations are more specific than a universal stable-ABI claim. Compiler versions and source SHAs remain artifact provenance, not substitutes for checking the actual compatibility contract.

The lifetime question is concrete: a native graph state, a returned Arrow buffer, and a session registry can each hold references. A host-funded memory lease must remain alive until the final admitted owner releases it, including output retained after the computation returns. Arc sharing helps implement that ownership; it does not by itself charge memory to DataFusion's pool.

For distributed native work, serialize a descriptor and reconstruct it on the worker. A driver-local pointer or plan identifier has no meaning in another process. Sail supplies authoritative task scope; the extension supplies its domain state and validates its messages within that scope.

2.14 Reading Exercises

Read these files with one question in mind: what interface is this code defining, and what concrete implementation sits behind it?

1. `crates/sail-common-datafusion/src/session/job.rs`
 - Find the `JobRunner` trait.
 - Compare it with `LocalJobRunner` and `ClusterJobRunner`.
2. `crates/sail-execution/src/job_runner.rs`
 - Follow local execution from `execute_stream`.
 - Follow cluster execution into `DriverMessage::ExecuteJob`.
3. `crates/sail-common/src/actor.rs`
 - Identify the actor message type.
 - Look at how `ActorHandle<T>` preserves message typing.
4. `crates/sail-common-datafusion/src/extension.rs`
 - Follow typed extension lookup from `SessionContext`, `SessionState`, and `TaskContext`.
5. `crates/sail-session/src/session_factory/server.rs`
 - List the session extensions registered in `create_session_config`.
 - Find where the query planner is installed.
6. `crates/sail-session/src/planner.rs`
 - Find the extension planner chain.
 - Trace one downcast from logical extension node to physical execution node.

2.15 Chapter Takeaways

Rust makes Sail’s architecture visible. `Arc` shows what is shared. `Box<dyn Trait>` and `Arc<dyn Trait>` show where implementations are chosen dynamically. `Send`, `Sync`, and `'static` show which objects must survive async and threaded execution. `SessionExtension` shows how Sail adds typed services to DataFusion sessions. `Actors` show how Sail keeps distributed control-plane state behind message boundaries.

These patterns are also the foundation for the extension proposal. A useful Sail extension API will not be a single callback. It will be a set of Rust trait-object contributions that can be registered, shared, ordered, used during planning, and reconstructed during distributed execution.

The next chapter moves back to the front door: Spark Connect. We will follow a PySpark request through Sail’s gRPC service, session manager, relation and command handlers, Arrow response stream, and error model.

2.16 Allocation and accounting in the current source

At upstream [b2470ea4b](#), the workspace uses Rust edition 2024 and requires Rust 1.97.1. That commit introduces `sail-mimalloc`, selected as the Rust global allocator by the CLI and Python extension. Its `GlobalAlloc` implementation uses `mimalloc`’s aligned allocation, zeroing, reallocation, and free functions. This is an allocation implementation, not a query-memory admission policy.

Keep three questions separate when reading extension code: which allocator owns a buffer; which query or native operation reserves its budget; and which owner releases both. A native wheel is a separately built library. The host’s global allocator declaration does not establish that every extension allocation uses the same allocator instance or participates in DataFusion’s memory pool. The extension experiment therefore carries explicit resource leases across its FFI boundary and retains them with the actual Arrow/native owners. Release callbacks must free through the allocation’s proper ownership path.

The retained extension benchmarks predate this upstream allocator change. Their results describe their recorded host/native binaries; they cannot be attributed to the newer allocator without a new qualified build and measurement. See `crates/sail-mimalloc/src/lib.rs`, `crates/sail-cli/src/main.rs`, and `crates/sail-python/src/lib.rs` in the pinned upstream tree.

3 Spark Connect

Spark Connect is Sail’s public front door for PySpark. When a PySpark user writes `spark.read.parquet(...).groupBy(...).count()`, Sail does not receive Python bytecode or a Spark JVM object. It receives Spark Connect protobuf messages over gRPC. The definitive starting point is Apache Spark’s own Spark Connect Overview, which describes Spark Connect as a decoupled client-server architecture using unresolved logical plans as the protocol.

That design gives Sail its “drop-in replacement” shape. The client can stay PySpark. The server can be Rust. The wire protocol between them is Spark Connect.

PySpark client

- > Spark Connect protobuf messages
- > Sail SparkConnectServer
- > Sail spec
- > DataFusion logical and physical plans
- > Arrow batches in Spark Connect responses

This chapter follows that front door in code. The goal is to understand what Spark Connect means architecturally, not just where the .proto files are generated.

3.1 Definitive Spark Connect References

Use these official references alongside this chapter:

- Spark Connect Overview: the Apache Spark documentation page for the client/server architecture, protobuf transport, gRPC, and Arrow result batches.
- Spark Connect architecture page: Apache Spark’s higher-level architecture explanation, including the connection flow, unresolved logical plans, Protocol Buffers, server-side optimization, and Arrow record batch result streaming.
- PySpark `SparkSession.builder.remote`: the official PySpark API for connecting to a Spark Connect server with an `sc://host:port` URL.
- PySpark Spark Session API reference: the Spark-session surface, including Spark Connect-only methods, artifacts, tags, interrupt methods, and the `client` attribute.
- PySpark API Reference: the official API index; it marks the Spark SQL, Pandas API on Spark, Structured Streaming, and DataFrame-based MLlib surfaces that support Spark Connect.
- Spark Connect protobuf definitions: the authoritative Spark repository location for `base.proto`, `relations.proto`, `expressions.proto`, `commands.proto`, `types.proto`, and related protocol files.

3.2 Choosing a session factory as an embedder

Merged upstream PR #2630 adds `serve_with_session_factory` in `crates/sail-spark-connect/src/entrypoint.rs`. The ordinary `serve` function calls it with `create_spark_session_factory`, preserving the default Spark session setup. An embedder can choose a different factory without replacing the gRPC service, session-manager actor, or shutdown sequence.

The shared type in `crates/sail-session/src/session_manager/mod.rs` is:

```
pub type ServerSessionFactoryFn =  
    fn(Arc<AppConfig>, RuntimeHandle) -> Box<dyn  
    SessionFactory<ServerSessionInfo>>;
```

This is a factory-construction function pointer, not a closure capturing arbitrary state. It receives application configuration and the runtime handle and returns the existing session-factory trait object. The Spark entry point forwards it to `create_spark_session_manager_with_factory`. `SparkSessionMutator` remains the place that installs Spark session configuration and planning services; an embedding mutator can wrap it while adding its own registrations.

This merged upstream capability is narrower than a portable extension loader. Selecting the server-session factory does not by itself install a package on workers, encode custom physical

plans, or account native graph allocations. The experimental extension branch builds those facilities on the existing session and execution paths. Keep the embedding entry point distinct from those additional branch changes when reading Chapter 13.

Source: upstream b2470ea4b, containing merged commit e976c8b317 (#2630).

3.3 The Main Files

The Spark Connect layer lives mainly in `crates/sail-spark-connect`.

File	Role
<code>src/server.rs</code>	Implements the Spark Connect gRPC service
<code>src/service/plan_executor.rs</code>	Executes relations, commands, streaming operations, interrupts, and reattach/release
<code>src/service/plan_analyzer.rs</code>	Handles schema, explain, tree string, version, DDL parsing, streaming checks
<code>src/service/config_manager.rs</code>	Handles runtime Spark config operations
<code>src/service/artifact_manager.rs</code>	Placeholder for artifact upload/status support
<code>src/proto/plan.rs</code>	Converts Spark Connect relations and commands into Sail specs
<code>src/proto/expression.rs</code>	Converts Spark Connect expressions
<code>src/proto/data_type*.rs</code>	Converts Spark, Arrow, JSON, and DDL data types
<code>src/executor.rs</code>	Converts DataFusion RecordBatch streams into Spark Connect response batches
<code>src/session.rs</code>	Stores Spark-session state inside DataFusion's SessionContext
<code>src/session_manager.rs</code>	Creates sessions with Spark-specific extensions
<code>src/error.rs</code>	Converts planning/execution errors into Spark-compatible gRPC statuses

The server implementation is generated-facing. The rest of the crate is translation-facing.

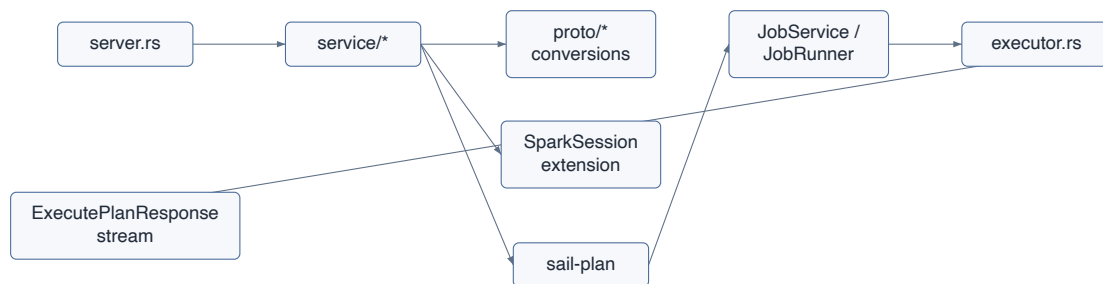


Figure 8: Flowchart 03.1

3.4 The gRPC Service Surface

SparkConnectServer in `crates/sail-spark-connect/src/server.rs` implements Spark’s generated SparkConnectService trait. The corresponding public protocol schema lives in Spark’s official Connect protobuf definitions, especially `base.proto` for the service and request/response envelope and `relations.proto/commands.proto` for plan payloads.

The service methods are the server’s public protocol surface:

- `execute_plan`
- `analyze_plan`
- `config`
- `add_artifacts`
- `artifact_status`
- `interrupt`
- `reattach_execute`
- `release_execute`
- `release_session`
- `fetch_error_details`
- `clone_session`

Some are fully implemented, some are partial, and some are explicit TODOs. That is normal for a compatibility server: Spark Connect is broad, and Sail implements the pieces needed by its Spark SQL/DataFrame compatibility goals first.

The official PySpark API exposes many of these protocol features through normal SparkSession methods. For example, the Spark Session API reference lists Spark Connect-only methods and client operations such as artifacts, progress handlers, tags, and operation interrupts.

The most important method is `execute_plan`.

```

async fn execute_plan(
    &self,
    request: Request<ExecutePlanRequest>,
) -> Result<Response<Self::ExecutePlanStream>, Status>

```

Its flow is simple and crucial:

1. Extract request metadata.
2. Get or create a session context.
3. Require that the request contains a plan.
4. Dispatch `Root(relation)` to relation execution.
5. Dispatch `Command(command)` to command execution.
6. Return an `ExecutePlanResponseStream`.

In code, the split is:

```
match op {
  plan::OpType::Root(relation) => {
    service::handle_execute_relation(&ctx, relation, metadata).await?
  }
  plan::OpType::Command(Command { command_type: command }) => {
    let command = command.required("command"?);
    handle_command(&ctx, command, metadata).await?
  }
  plan::OpType::CompressedOperation(_) => {
    return Err(Status::unimplemented("compressed operation plan"));
  }
}
```

Spark Connect calls the top-level operation a `Plan`, but Sail immediately separates it into the two categories that matter to a query engine:

Relation: produces a table-like result

Command: changes state, writes data, registers things, starts streams, or returns command metadata

3.5 Sessions: Spark State Inside DataFusion State

Every Spark Connect request carries a session ID and optional user context. On the client side, PySpark users create this remote session with `SparkSession.builder.remote("sc://host:port")`, documented in the official `builder.remote` API reference. Sail uses the request's session values to get a DataFusion `SessionContext`:

```
self.session_manager
    .get_or_create_session_context(session_id, user_id)
    .await
```

The Spark-specific session setup happens in `crates/sail-spark-connect/src/session_manager.rs`.

`SparkSessionMutator` implements `ServerSessionMutator`. During session creation, it adds two extensions to DataFusion's `SessionConfig`:

- `PlanService`, with Spark-style plan and catalog formatting.
- `SparkSession`, Sail's Spark-session state object.

```
Ok(config
  .with_extension(Arc::new(plan_service))
  .with_extension(Arc::new(spark)))
```

The SparkSession extension stores:

- session_id
- user_id
- Spark runtime config
- active executors for reattachable operations
- streaming query state

This is one of Sail's clever bridges. Spark Connect expects Spark session behavior. DataFusion expects a SessionContext. Sail installs Spark session semantics into DataFusion session state through typed extensions.

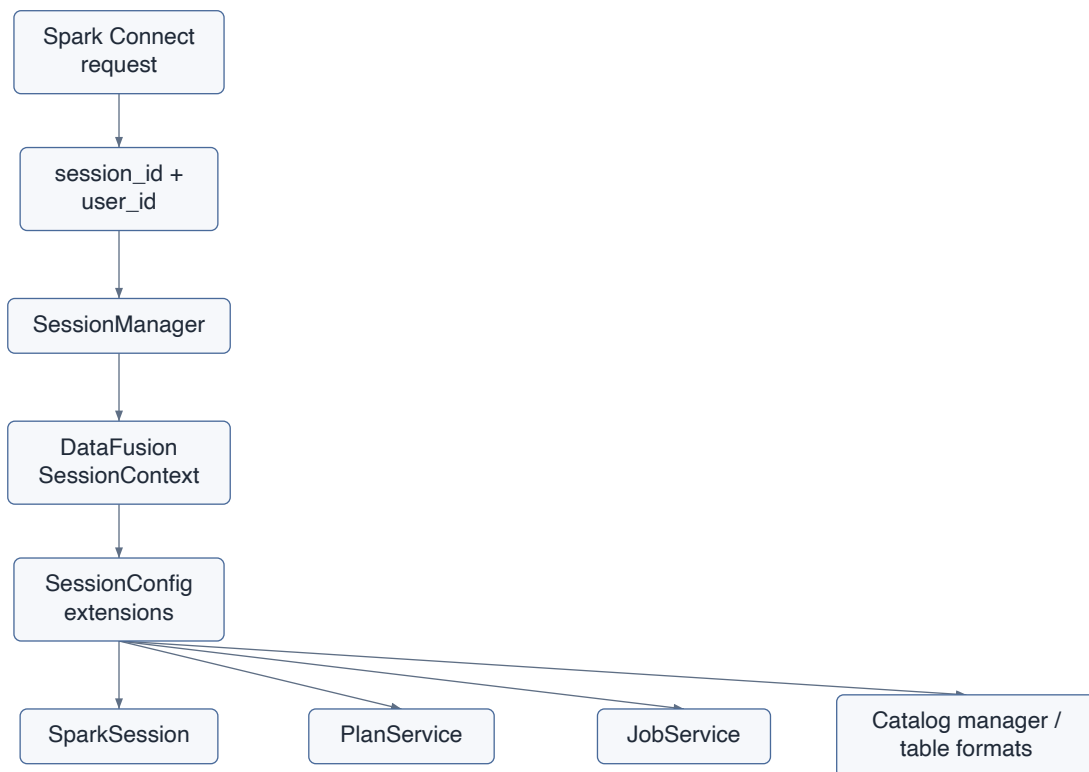


Figure 9: Flowchart 03.2

3.6 Relations Become Sail Specs

The key conversion file is `crates/sail-spark-connect/src/proto/plan.rs`.

Spark Connect relations are protobuf trees. Sail does not plan directly from those protobuf structs. It converts them into `sail_common::spec`. For the upstream protocol shape, read Spark's `relations.proto` in the official Spark Connect protobuf definitions.

The important implementation is:

```
impl TryFrom<Relation> for spec::Plan
```

It extracts:

- RelationCommon, including plan_id.
- RelType, the actual relation variant.
- A RelationNode, which is either a query node or a command node.

Then it returns:

```
spec::Plan::Query(spec::QueryPlan { ... })
```

or:

```
spec::Plan::Command(spec::CommandPlan { ... })
```

Examples of relation variants that become query nodes:

- Read
- Project
- Filter
- Join
- SetOp
- Sort
- Limit
- Aggregate
- Sql
- Range
- LocalRelation
- Repartition
- MapPartitions
- CommonInlineUserDefinedTableFunction

Some relation variants can become commands. This is why TryFrom<Relation> for spec::Plan returns a full spec::Plan, not only a QueryPlan.

The teaching point is that Spark Connect is not Sail's internal language. It is an external compatibility protocol. Sail's internal unresolved language is the Sail spec, because Sail also accepts SQL and needs one common representation for both.

Spark Connect Relation

- > RelationNode
- > spec::QueryNode or spec::CommandNode
- > PlanResolver
- > DataFusion LogicalPlan

3.7 SQL Is Also Routed Through the Front Door

Spark Connect can send a SQL relation. In `proto/plan.rs`, SQL text is parsed while converting the protobuf relation. This matches the official architecture description: the client sends unresolved intent, and the server analyzes and optimizes it, as described in Apache Spark's Spark Connect architecture page.

```
parse_one_statement(...)  
from_ast_statement(...)
```

This is an important compatibility detail. From the client's point of view:

```
spark.sql("select * from t")
```

is still a Spark Connect request. From Sail's point of view, it becomes a Sail spec through the same general conversion pipeline as DataFrame relations.

That gives Sail one downstream planning path:

```
Spark DataFrame relation  
  -> Sail spec  
SQL string  
  -> Sail SQL parser/analyzer  
  -> Sail spec  
Sail spec  
  -> DataFusion logical plan
```

The cost of this approach is compatibility work. Spark SQL has many grammar and semantic quirks. Sail has its own SQL parser and analyzer so the server can accept Spark-shaped SQL without embedding Spark itself.

3.8 Commands

The command dispatcher lives in `handle_command` in `server.rs`.

It routes Spark Connect command variants to service handlers. Examples:

- `RegisterFunction`
- `WriteOperation`
- `CreateDataframeView`
- `WriteOperationV2`
- `SqlCommand`
- `WriteStreamOperationStart`
- `StreamingQueryCommand`
- `GetResourcesCommand`
- `StreamingQueryManagerCommand`
- `RegisterTableFunction`
- `RegisterDataSource`
- `CheckpointCommand`
- `MergeIntoTableCommand`

The split matters because commands often execute eagerly. In `plan_executor.rs`, `ExecutePlanMode` has two variants:

```
enum ExecutePlanMode {  
    Lazy,  
    EagerSilent,  
}
```

Relations use `Lazy`: return a response stream and let the client consume result batches.

Commands often use `EagerSilent`: execute the plan immediately, drain the stream, and return no data unless Spark Connect expects a command result.

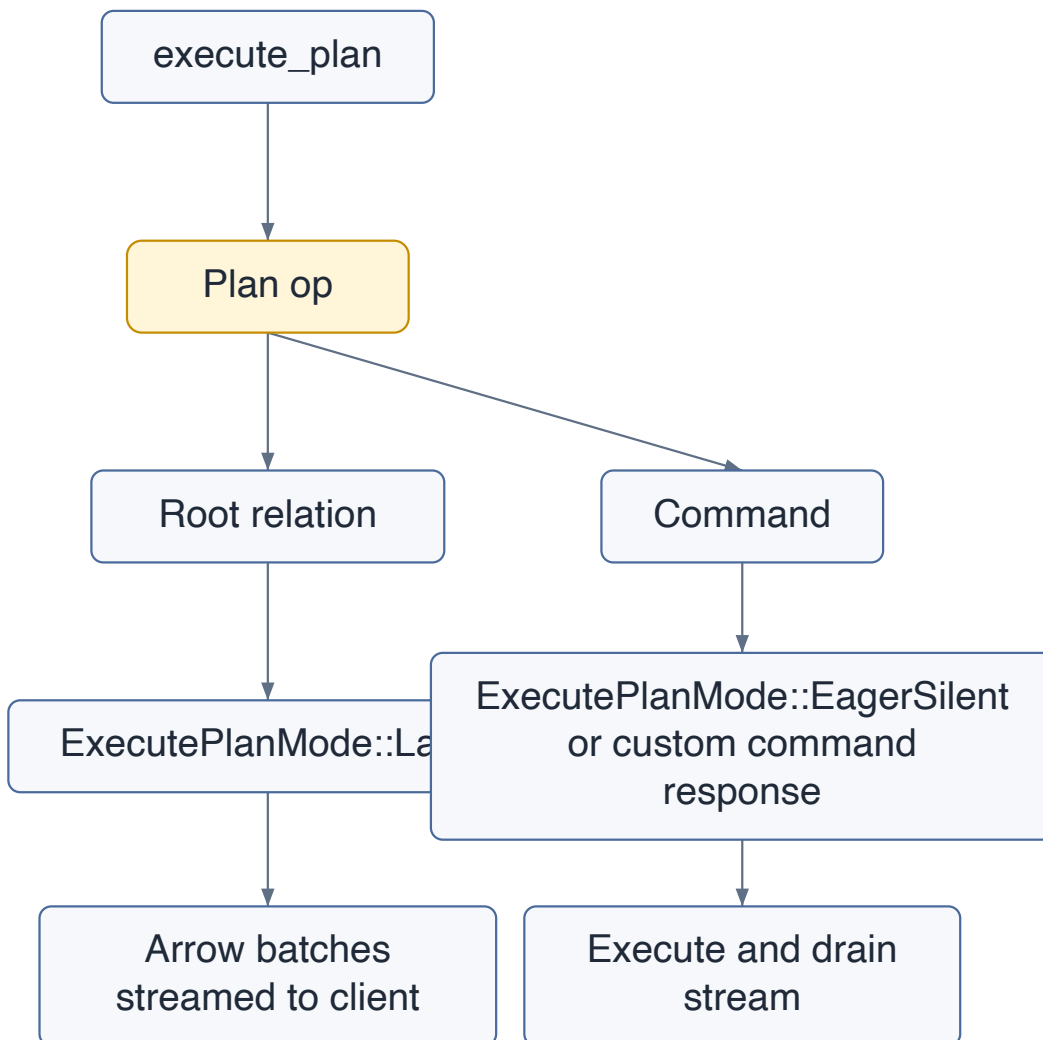


Figure 10: Flowchart 03.3

This is why a Spark statement like `CREATE TABLE` and a query like `SELECT * FROM t` use the same gRPC method but have different execution behavior inside Sail.

3.9 The Core Execution Path

The heart of execution is `handle_execute_plan` in `plan_executor.rs`. This is Sail's version of the official Spark Connect flow where a client sends an encoded unresolved logical plan and receives streamed Apache Arrow batches back over gRPC, described in the Spark Connect Overview.

It does four things:

1. Retrieve the `SparkSession` extension.
2. Retrieve the `JobService` extension.
3. Resolve and physical-plan the Sail spec.
4. Execute the physical plan through the session's job runner.

In code:

```
let spark = ctx.extension::<SparkSession>()?;
let service = ctx.extension::<JobService>()?;
let (plan, _) = resolve_and_execute_plan(ctx, spark.plan_config()?, plan).await?;
let stream = service.runner().execute(ctx, plan).await?;
```

That line `service.runner().execute(ctx, plan)` hides the local/cluster choice described in Chapter 2. Spark Connect does not care whether Sail is local, local-cluster, or Kubernetes-cluster. It receives a `SendableRecordBatchStream` either way.

```
spec::Plan
  -> resolve_and_execute_plan
  -> Arc<dyn ExecutionPlan>
  -> JobRunner::execute
  -> SendableRecordBatchStream
```

From there, Spark Connect's job is to translate a DataFusion/Arrow stream into Spark Connect response messages.

3.10 Response Streams and Arrow Batches

`crates/sail-spark-connect/src/executor.rs` owns the response-stream behavior. Apache Spark's Spark Connect docs call out this same result shape: query results are streamed to the client as Apache Arrow record batches rather than returned as one monolithic response.

The important output enum is:

```
pub enum ExecutorBatch {
    ArrowBatch(ArrowBatch),
    SqlCommandResult(Box<SqlCommandResult>),
    WriteStreamOperationStartResult(Box<WriteStreamOperationStartResult>),
    StreamingQueryCommandResult(Box<StreamingQueryCommandResult>),
    StreamingQueryManagerCommandResult(Box<StreamingQueryManagerCommandResult>),
    CheckpointCommandResult(Box<CheckpointCommandResult>),
    Schema(Box<DataType>),
}
```

```

    Complete,
}

```

A running executor first sends the schema, then Arrow batches, then a completion marker:

```

Schema
-> ArrowBatch
-> ArrowBatch
-> ...
-> Complete

```

The Arrow conversion uses Arrow IPC:

```

let cursor = Cursor::new(&mut output.data);
let mut writer = StreamWriter::try_new(cursor, batch.schema().as_ref())?;
writer.write(batch)?;
writer.finish()?;

```

That means Spark Connect sees result data as serialized Arrow streams, not row-by-row JSON or Python objects.

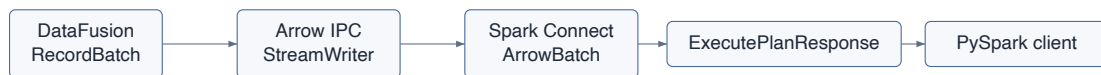


Figure 11: Flowchart 03.4

The row-count field is populated from `batch.num_rows()`, and empty result streams still emit an empty Arrow batch so the client receives schema-consistent output.

3.11 Reattachable Operations

Spark Connect supports reattachable execution. A client can disconnect and later resume reading from an operation. On the Python surface, related operation controls such as tags and interrupts are listed in the official Spark Session API reference.

Sail tracks this with:

- `operation_id`
- response IDs
- an executor buffer
- `SparkSession`'s executor map

When `execute_plan` receives request options, it checks whether the operation is reattachable:

```

reattachable: is_reattachable(&request.request_options)

```

If the operation is lazy, `handle_execute_plan` creates an `Executor`, starts it, and registers it in `SparkSession`:

```

let executor = Executor::new(metadata, stream, heartbeat_interval);
let rx = executor.start()?;
spark.add_executor(executor)?;

```

The executor saves outputs in a bounded buffer. `reattach_execute` pauses the running executor if necessary, releases already acknowledged responses, and starts the executor again:

```

executor.pause_if_running().await?;
executor.release(response_id)?;
let rx = executor.start()?;

```

`release_execute` lets the client tell the server which buffered responses can be dropped.

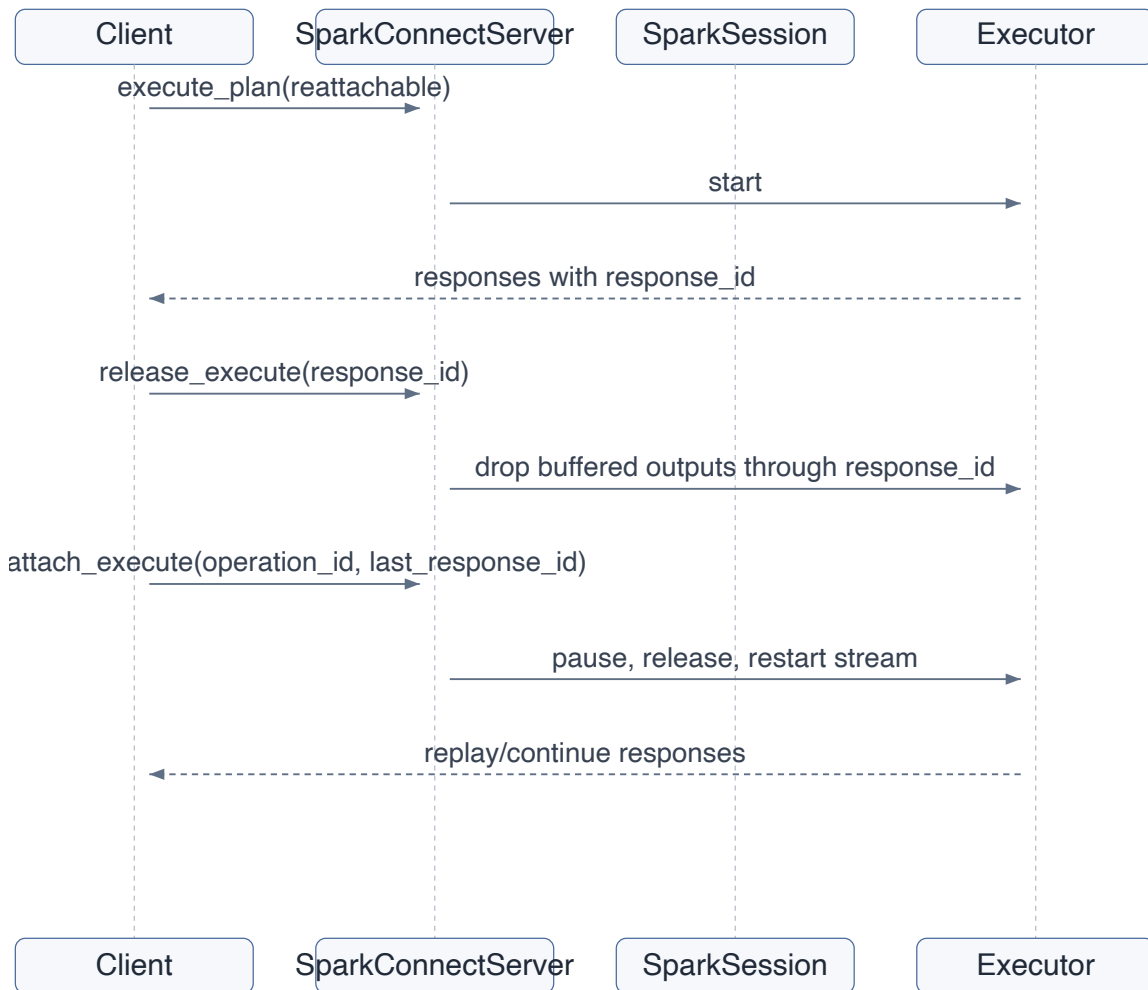


Figure 12: Sequence diagram 03.5

This is a protocol-level feature, but it shapes execution internals. Sail cannot simply return an anonymous stream and forget it. It must store enough operation state in the Spark session to pause, replay, release, or interrupt it.

3.12 Interrupts

The interrupt endpoint supports three modes, corresponding to Spark-session operation controls exposed in PySpark:

- Interrupt all operations in the session.
- Interrupt operations with a tag.
- Interrupt one operation ID.

The service functions remove matching executors from SparkSession, pause them if running, and return interrupted operation IDs.

This is another reason SparkSession is not just a bag of configuration. It is operational state for Spark Connect behavior.

```
InterruptRequest
-> find executor(s)
-> remove from session state
-> pause if running
-> return operation IDs
```

3.13 Analyze Plan

analyze_plan serves Spark client introspection calls. It does not usually execute data. It answers questions about a plan. These calls back the ordinary PySpark APIs whose Spark Connect support is documented throughout the PySpark API reference.

Implemented or partially implemented analysis handlers include:

- Schema
- Explain
- TreeString
- IsLocal
- IsStreaming
- SparkVersion
- DdlParse
- Persist
- Unpersist
- GetStorageLevel
- JsonToDdl

The schema path is worth reading:

```
let resolver = PlanResolver::new(ctx, spark.plan_config()?);
let NamedPlan { plan, fields } = resolver
  .resolve_named_plan(spec::Plan::Query(plan.try_into()?))
  .await?;
let schema = ...
to_spark_schema(schema)
```

This uses Sail's normal plan resolver, but stops at schema. That means analysis is semantically meaningful: schema answers come from the same resolution machinery that execution uses.

Explain also routes through Sail planning:

```
explain_string(  
  ctx,  
  spark.plan_config()?,  
  spec::Plan::Query(plan.try_into()),  
  options,  
)  
.await
```

The Spark Connect layer therefore has two major plan paths:

```
execute_plan: convert -> resolve -> optimize -> physical plan -> execute  
analyze_plan: convert -> resolve/explain/schema -> response
```

Extensions must work in both. If an extension function works during execution but schema analysis cannot resolve it, PySpark users will still see failures because clients often ask for schema before collecting data.

3.14 Config

The config endpoint manipulates Spark runtime configuration stored in SparkSession. The user-facing API is SparkSession.conf, documented as part of the official Spark Session reference.

Handlers in config_manager.rs implement:

- Get
- Set
- GetWithDefault
- GetOption
- GetAll
- Unset
- IsModifiable

All of these retrieve the typed SparkSession extension:

```
let spark = ctx.extension::<SparkSession>()?;
```

Then they delegate to SparkSession methods such as get_config, set_config, and unset_config.

This is separate from DataFusion's SessionConfig options. That distinction matters:

Spark runtime config:
 stored in SparkSession state
 visible through Spark Connect config API
 used to create PlanConfig during planning

DataFusion SessionConfig:
 created during session construction

stores DataFusion options and typed extensions
read by DataFusion planning/execution

Extensions often need both. A Spark-facing option might arrive through `spark.conf.set(...)`, but a DataFusion optimizer rule may need to read a typed extension or session option during planning.

3.15 Data Types and Spark Compatibility

Spark Connect forces Sail to translate types carefully. The official protocol definitions for types live in Spark's `types.proto` under the Connect protobuf definitions.

`proto/data_type_arrow.rs` maps Arrow fields and data types back into Spark Connect `DataType` messages. It handles ordinary Arrow types and extension cases such as:

- Spark UDT metadata.
- GeoArrow WKB extension types mapped to Spark geometry/geography.
- Variant extension types.

This conversion sits on the output side of planning and execution. DataFusion and Arrow may represent a type one way, but PySpark expects Spark Connect's data type model.

One subtle example is timestamps:

```
Arrow Timestamp(Microsecond, None)
  -> Spark TimestampNtz
```

```
Arrow Timestamp(Microsecond, Some(_))
  -> Spark Timestamp
```

The front door therefore constrains internal semantics. Sail can use Arrow/DataFusion internally, but it must preserve enough Spark meaning for the client.

3.16 Errors Become Spark Exceptions

Spark Connect clients expect Spark-shaped errors, not arbitrary Rust error strings.

`crates/sail-spark-connect/src/error.rs` converts many internal errors into `SparkError`, then into `tonic::Status`.

The mapping eventually produces a `SparkThrowable` with Spark/Java class names such as:

- `org.apache.spark.sql.AnalysisException`
- `org.apache.spark.sql.execution.QueryExecutionException`
- `java.lang.IllegalArgumentException`
- `java.lang.ArithmeticException`
- `org.apache.spark.api.python.PythonException`
- `java.time.DateTimeException`
- `java.lang.UnsupportedOperationException`

The status conversion intentionally uses Spark-compatible error details:

```
details.set_error_info(class, "org.apache.spark", metadata);
Status::with_error_details(Code::Internal, message, details)
```

It also truncates long gRPC messages to stay below metadata limits. That sounds mundane until you remember Python tracebacks can be long. Protocol compatibility includes boring survival details like this.

3.17 Artifacts and Python Distribution

Spark Connect includes artifact upload and status endpoints. In Sail, `artifact_manager.rs` currently returns TODO errors for add/status handling. The user-facing artifact APIs are listed in the official Spark Session API reference as `addArtifact` and `addArtifacts`.

This matters for the extension story. Spark’s artifact mechanism is one way clients distribute files, Python dependencies, or resources. Discussion #2001, however, focuses more directly on Python entry-point based extension discovery:

```
[project.entry_points."pysail.extensions"]
sedona = "pysail_sedona:register"
```

Those are different mechanisms:

```
Spark Connect artifacts:
  client sends files/resources to a session
```

```
Python entry-point extensions:
  installed Python package contributes Sail extension behavior
```

A complete extension system may eventually touch both, but they solve different problems.

3.18 Registering Functions and Data Sources

Spark Connect commands include function and data source registration.

`RegisterFunction` becomes a Sail command plan. The broader PySpark function registration APIs, including UDF and UDTF registration, are indexed in the official PySpark SQL API reference:

```
spec::CommandNode::RegisterFunction(udf.try_into()?)
```

Then it runs through normal planning and command execution.

`RegisterDataSource` has a more direct session-scoped path. The handler extracts the pickled Python data source class and registers a `PythonDataSourceAdapter` in the session’s `DataSourceRegistry`:

```
let source = Arc::new(PythonDataSourceAdapter::with_pickled_class(
  name.clone(),
  command,
));
registry.register_data_source(source)?;
```

This is a small preview of extension behavior. A client can contribute behavior to a session, but the contribution is still routed through Sail’s typed session services and DataFusion planning interfaces.

3.19 What Spark Connect Means for Extensions

Discussion #2001 is not only about Rust-side plugin ergonomics. Spark Connect adds several extra requirements.

First, extensions must be visible during analysis as well as execution.

PySpark frequently asks for schema, explain output, local/streaming status, or type conversion before actual execution. An extension function has to resolve in `analyze_plan`, not just in `execute_plan`.

Second, extension configuration may arrive through Spark config.

If a user writes:

```
spark.conf.set("sail.sedona.join.index", "rtree")
```

then the extension must decide how that value becomes available to the optimizer or physical planner. Storing it only in `SparkSession` may not be enough if `DataFusion` rules expect `SessionConfig` extensions.

Third, extensions must preserve Spark Connect type compatibility.

A spatial extension may expose geometry/geography types. Sail already maps `GeoArrow` metadata in the Arrow-to-Spark conversion path. A third-party extension must either reuse those conventions or provide a compatible type conversion story.

Fourth, distributed execution still matters.

Spark Connect receives one logical operation from PySpark, but Sail may execute it on remote workers. A function registered through Spark Connect must be available when the worker decodes and executes the physical plan.

Fifth, Spark Connect itself provides extension hooks.

Spark Connect defines `Relation.extension`, `Command.extension`, and `Expression.extension`, each carrying `google.protobuf.Any`. In the experimental branch, relation messages dispatch through the registered type URL and bounded input envelope. Raw command and expression dispatch remain unsupported: scalar extensions use ordinary unresolved function calls, and command-shaped operations execute relations that emit receipts. Chapter 13 describes the implemented wire contract rather than treating every protocol extension point as already usable.

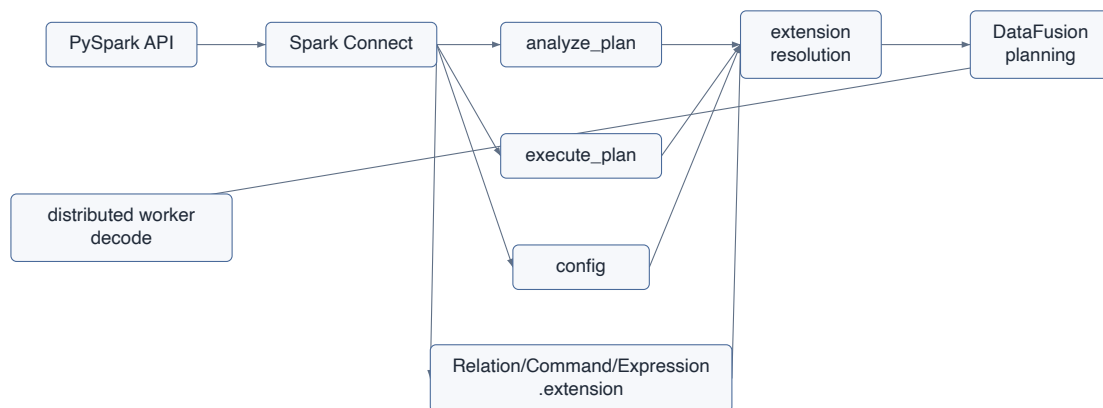


Figure 13: Flowchart 03.6

A good extension API must therefore cross the Spark Connect boundary, not sit behind it.

3.20 Reading Exercises

1. Read `crates/sail-spark-connect/src/server.rs`.
 - Find each gRPC method.
 - For `execute_plan`, identify where session lookup happens and where relation/command dispatch happens.
2. Read `crates/sail-spark-connect/src/service/plan_executor.rs`.
 - Follow `handle_execute_relation`.
 - Follow `handle_execute_plan`.
 - Compare Lazy and EagerSilent.
 - Find reattach and release handling.
3. Read `crates/sail-spark-connect/src/executor.rs`.
 - Find where schema is emitted.
 - Find where `RecordBatch` becomes `ArrowBatch`.
 - Find the executor buffer used for reattachable operations.
4. Read `crates/sail-spark-connect/src/proto/plan.rs`.
 - Follow `TryFrom<Relation>` for `spec::Plan`.
 - Pick one relation variant, such as `Project` or `Filter`, and trace the conversion into `spec::QueryNode`.
5. Read `crates/sail-spark-connect/src/service/plan_analyzer.rs`.
 - Follow schema analysis.
 - Follow explain analysis.
 - Notice which analysis requests are TODOs or no-ops.
6. Read `crates/sail-spark-connect/src/error.rs`.
 - Find how `PlanError` and `ExecutionError` become `SparkError`.
 - Find how `SparkError` becomes a gRPC Status.

3.21 Chapter Takeaways

Spark Connect is the compatibility contract between PySpark and Sail. It gives Sail a Spark-shaped protocol while letting the engine be Rust, Arrow, and DataFusion.

Inside Sail, Spark Connect requests are translated into Sail specs, resolved into DataFusion plans, executed through a JobRunner, and streamed back as Arrow batches. Sessions carry Spark-specific state through typed DataFusion extensions. Analysis, configuration, reattach/release, interrupts, and errors are all part of the compatibility surface.

For extensions, Spark Connect raises the bar. An extension must work during analysis, execution, configuration, output type conversion, error handling, and distributed worker execution. That is why the extension proposal cannot be just “let users register a UDF.” Spark Connect makes extension behavior user-visible before, during, and after query execution.

The next chapter moves from the protocol front door to the Python experience: `pysail`, PySpark, Python UDFs, Python data sources, and how Python packaging could become the extension discovery mechanism.

4 PySpark and pysail

PySpark is the user experience Sail tries to preserve. `pysail` is the Python package that makes Sail feel like something a Python developer can install, start, test, and use from ordinary PySpark code.

The design is intentionally asymmetric:

```
PySpark remains the client API.  
pysail starts and packages the Rust engine.  
Spark Connect is the wire protocol between them.
```

That is why Sail can claim that no PySpark code rewrites are needed once the user connects to a Sail server. A PySpark program still imports `pyspark.sql.Session`; the difference is the remote URL:

```
from pyspark.sql import Session  
  
spark = Session.builder.remote("sc://localhost:50051").getOrCreate()  
spark.sql("SELECT 1 + 1").show()
```

The official PySpark entry point for this is `Session.builder.remote`. Sail’s job is to provide a compatible Spark Connect server at that address.

4.1 The Main Files

File	Role
<code>pyproject.toml</code>	Python package metadata, build backend, optional dependencies, test matrices
<code>python/pysail/spark/__init__.py</code>	Public Python wrapper for SparkConnectServer
<code>python/pysail/cli.py</code> and <code>python/pysail/__main__.py</code>	Python entry points into the Sail CLI
<code>crates/sail-python/src/lib.rs</code>	PyO3 <code>_native</code> module registration
<code>crates/sail-python/src/spark/server.rs</code>	Native Python class that starts the Spark Connect server
<code>crates/sail-python/src/globals.rs</code>	Global runtime, config, telemetry, and environment snapshot
<code>crates/sail-python-udf/*</code>	Python UDF, UDAF, UDTF, Pandas, and Arrow execution support
<code>crates/sail-plan/src/resolver/expression/udf.rs</code>	Converts Spark Connect inline Python UDFs into DataFusion UDF expressions
<code>python/pysail/tests/spark/conftest.py</code>	How Sail's own tests create a PySpark client connected to Sail

The code divides into two worlds:

User-facing Python package:

`pysail`, `pysail.spark`, `sail` CLI

Engine-facing Rust/PyO3 bindings:

`_native` module, `SparkConnectServer`, Python UDF runtime

4.2 Package Shape

The Python package is defined in `pyproject.toml`.

It is named `pysail`, supports Python `>=3.10,<3.15`, and is built with `maturin`. That tells you the package is not pure Python. It ships a compiled Rust extension module:

```
[build-system]
requires = ["maturin>=1.0,<2.0"]
build-backend = "maturin"
```

The package entry point is:

```
[project.scripts]
sail = "pysail.cli:main"
```

So the installed `sail` command is a Python console script, but the Python script immediately delegates to Rust:

```
from pysail import _native

def main():
    _native.main(sys.argv)
```

The native module is built by `crates/sail-python/src/lib.rs`:

```
#[pymodule]
fn _native(m: &Bound<'_, PyModule>) -> PyResult<()> {
    flight::register_module(m)?;
    spark::register_module(m)?;
    m.add_function(wrap_pyfunction!(cli::main, m))?;
    m.add("_SAIL_VERSION", env!("CARGO_PKG_VERSION"))?;
    Ok(())
}
```

That module exposes:

- `pysail._native.main`
- `pysail._native.spark.SparkConnectServer`
- Flight-related bindings
- `_SAIL_VERSION`

The package layout is a useful lesson in Rust/Python hybrid projects:

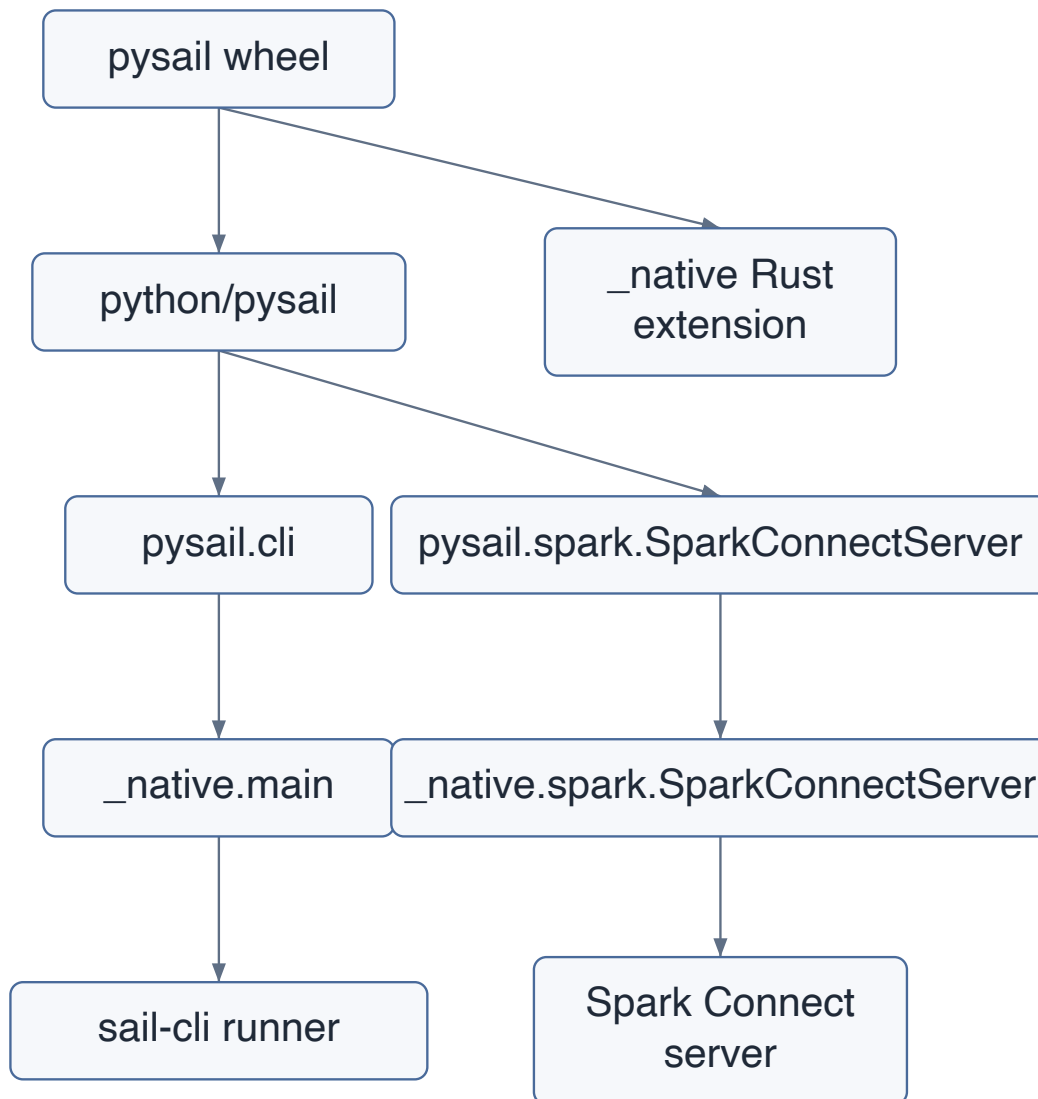


Figure 14: Flowchart 04.1

4.3 Starting Sail From Python

The public Python wrapper is tiny:

```

class SparkConnectServer:
    def __init__(self, ip: str = "127.0.0.1", port: int = 0) -> None:
        self._inner = _native.spark.SparkConnectServer(ip, port)

    def start(self, *, background=True) -> None:
        self._inner.start(background=background)
  
```

```

def stop(self) -> None:
    self._inner.stop()

@property
def listening_address(self) -> tuple[str, int] | None:
    return self._inner.listening_address

```

The real work happens in Rust, in `crates/sail-python/src/spark/server.rs`.

The PyO3 class:

- loads AppConfig
- initializes or retrieves global runtime state
- binds a TCP listener
- starts the Spark Connect server
- records the actual listening address
- can run in the background or block the calling thread
- can shut down through a one-shot channel

The most important method is `start`:

```

let listener = self
    .runtime
    .primary()
    .block_on(TcpListener::bind(address))?;
self.state = Some(self.run(listener)?);

```

If the user passes port 0, the OS chooses an available port. The actual address is exposed through `listening_address`. Sail's tests use exactly that:

```

server = SparkConnectServer("127.0.0.1", 0)
server.start(background=True)
_, port = server.listening_address
yield f"sc://localhost:{port}"
server.stop()

```

That is the local development loop in one picture:

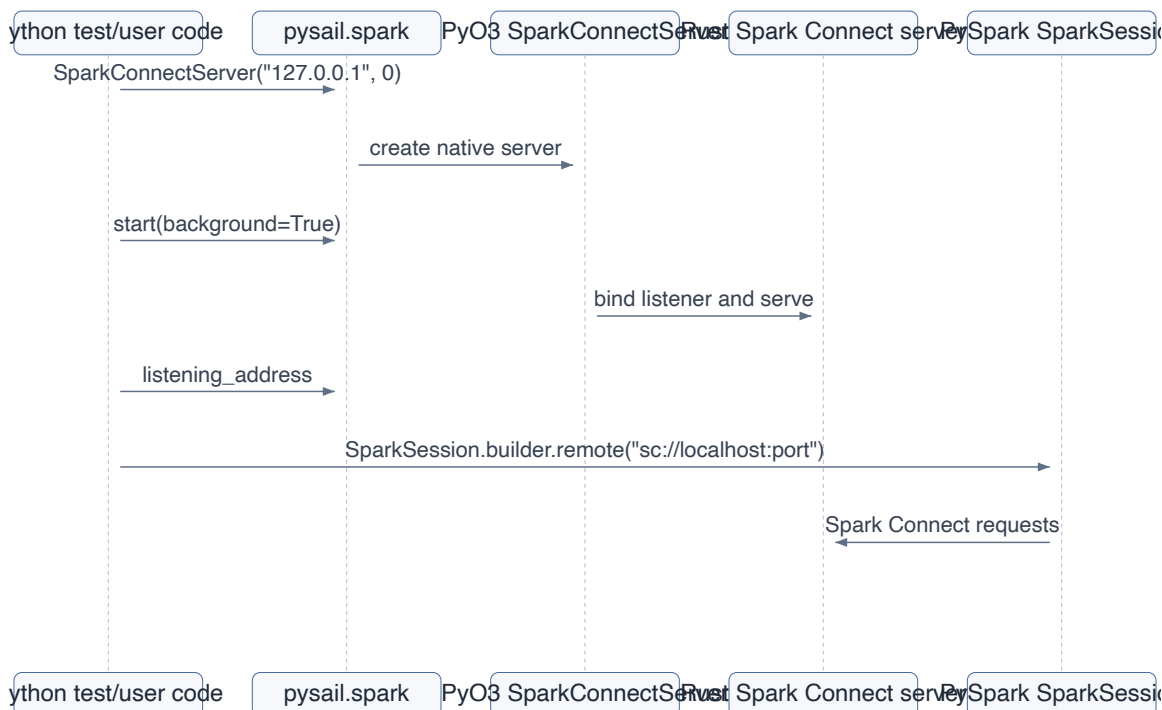


Figure 15: Sequence diagram 04.2

4.4 The Global Runtime

crates/sail-python/src/globals.rs contains GlobalState.

This is where pysail creates a global Sail runtime and initializes telemetry. It uses PyOnceLock so initialization happens once per Python interpreter:

```
static GLOBALS: PyOnceLock<GlobalState> = PyOnceLock::new();
```

GlobalState contains:

- a RuntimeManager
- an EnvironmentSnapshot

The environment snapshot matters because Sail configuration is environment-variable driven. Some environment variables are effectively static once the runtime and telemetry have been initialized. If they change afterward, pysail warns that the changes are ignored.

This is one of those systems details that looks small but saves debugging time. Python users often set environment variables inside notebooks or test processes. Sail has to explain when that is too late.

```
import pysail._native
-> load AppConfig
-> create runtime
-> initialize telemetry
-> snapshot Sail environment variables
```

4.5 Releasing the GIL

When Python calls into Rust and Rust blocks, Python's global interpreter lock can prevent other Python code from running. That is dangerous for Sail because Python UDFs may need to run while the server is active.

The server code explicitly uses `Python::detach`.

In `SparkConnectServerState::wait`, the comment says the method should be called within `Python::detach`; otherwise, the GIL is not released and Python UDFs will be blocked when the server handles client requests.

The blocking CLI path does the same:

```
py.detach(move || {  
    sail_cli::runner::main(args)  
})
```

This is an important Rust/Python boundary rule:

Long-running Rust server work should not hold the Python GIL.

Without that, Sail could start fine and then mysteriously deadlock or starve Python UDF execution.

4.6 Connecting With PySpark

Sail's own tests show the intended user pattern in `python/pysail/tests/spark/conftest.py`:

```
spark = SparkSession.builder.remote(remote).getOrCreate()
```

Then the test fixture configures the session:

```
session.conf.set("spark.sql.session.timeZone", "UTC")  
session.conf.set("spark.sql.ansi.enabled", "true")  
session.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")
```

These are ordinary PySpark calls. They go through Spark Connect and reach Sail's config/session machinery. The fixture then tests Sail through the normal PySpark surface: SQL, DataFrames, functions, catalog calls, writes, UDFs, streaming, and lakehouse features.

The official PySpark reference documents the broader Spark SQL API at `pyspark.sql`, and the main API index notes that Spark SQL, Structured Streaming, and DataFrame-based MLlib support Spark Connect through the Python API surface.

The user sees:

```
spark.range(10).where("id % 2 = 0").count()
```

Sail sees:

```
Spark Connect relation tree  
-> Sail spec  
-> DataFusion logical plan
```

- > DataFusion physical plan
- > Arrow result batches

4.7 pysail Is Not a PySpark Fork

This is subtle but central. pysail does not replace PySpark classes like DataFrame, Column, or SparkSession. It starts a server that PySpark can talk to.

That means compatibility is mostly tested at the protocol/API behavior level:

- Does PySpark emit a Spark Connect plan Sail can understand?
- Does Sail return the schema PySpark expects?
- Does Sail return Arrow batches PySpark can decode?
- Do errors look like Spark errors?
- Do config, UDF, catalog, write, and streaming operations behave like Spark?

This is why the test dependencies include pyspark[connect] in development and multiple Spark versions in test matrices:

```
[[tool.hatch.envs.test.matrix]]
spark = ["3.5.7", "4.0.1", "4.1.1"]
```

The engine is Sail. The client is still PySpark.

4.8 Python UDFs Enter Through Spark Connect

PySpark UDFs are user-provided Python functions. In Spark Connect, the function is serialized into the request and sent to the server.

Sail resolves those inline Python UDFs in crates/sail-plan/src/resolver/expression/udf.rs.

The resolver receives a spec::CommonInlineUserDefinedFunction, extracts:

- function name
- determinism
- distinct flag
- arguments
- serialized function payload

Then it builds a PySparkUdfPayload and wraps it in a DataFusion ScalarUDF or AggregateUDF.

For scalar UDFs:

```
let udf = PySparkUDF::new(
    PySparkUdfKind::Batch,
    get_udf_name(name, &payload),
    payload,
    deterministic,
    input_types,
    function.output_type,
    self.config.pyspark_udf_config.clone(),
);
```

```
Ok(Expr::ScalarFunction(expr::ScalarFunction {
  func: Arc::new(ScalarUDF::from(udf)),
  args: arguments,
}))
```

For grouped aggregate UDFs, Sail creates a `PySparkGroupAggregateUDF` and returns a `DataFusion` aggregate expression.

The key idea is:

```
Python function payload
-> Sail UDF payload
-> DataFusion ScalarUDF/AggregateUDF
-> executable physical plan
```

The official PySpark UDF APIs are:

- `pyspark.sql.functions.udf`
- `pyspark.sql.functions.pandas_udf`
- `pyspark.sql.functions.udtf`
- `pyspark.sql.DataFrame.mapInArrow`

4.9 UDF Kinds

`crates/sail-python-udf/src/udf/pyspark_udf.rs` defines the scalar UDF kinds Sail supports:

```
pub enum PySparkUdfKind {
  Batch,
  ArrowBatch,
  ScalarPandas,
  ScalarPandasIter,
  ScalarArrow,
  ScalarArrowIter,
}
```

The resolver maps Spark eval types to these internal UDF kinds:

Spark/PySpark style	Sail internal kind	Python-side data shape
regular row-oriented UDF	Batch	Python values
Arrow-optimized batch UDF	ArrowBatch	Arrow-backed batches
Pandas scalar UDF	ScalarPandas	<code>pandas.Series</code>
Pandas scalar iterator UDF	ScalarPandasIter	iterator of <code>pandas.Series</code>
Arrow scalar UDF	ScalarArrow	<code>pyarrow.Array</code>
Arrow scalar iterator UDF	ScalarArrowIter	iterator of <code>pyarrow.Array</code>

The official PySpark docs describe these APIs from the user's point of view. Sail's code answers the engine question: what kind of object should `DataFusion` execute when such a function appears in a query plan?

4.10 Loading a PySpark UDF Payload

`crates/sail-python-udf/src/cereal/pyspark_udf.rs` handles the serialized UDF payload format.

The payload builder writes:

- eval type
- selected Spark/PySpark config values
- input type metadata for some PySpark versions
- profiling flag
- argument offsets
- keyword argument names when supported
- the serialized Python command bytes

The payload loader calls into PySpark internals:

```
let serializer = PyModule::import(py, intern!(py, "pyspark.serializers"))?
    .getattr(intern!(py, "CPickleSerializer"))?
    .call0()?;
let tuple = PyModule::import(py, intern!(py, "pyspark.worker"))?
    .getattr(intern!(py, "read_udfs"))?
    .call1((serializer, infile, eval_type))?;
```

This is not an accident. To be compatible with PySpark UDF behavior, Sail reuses PySpark's own worker deserialization conventions. It wants the same Python wrapper behavior Spark users expect.

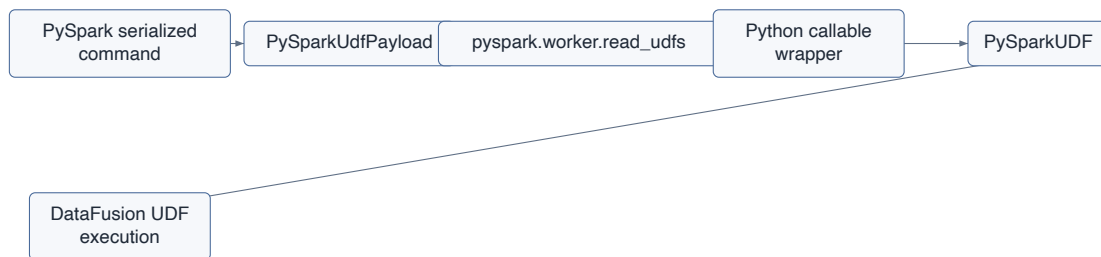


Figure 16: Flowchart 04.3

4.11 Executing Python UDFs in Process

`PySparkUDF` implements DataFusion's `ScalarUDFImpl`.

When DataFusion invokes it, Sail:

1. Converts DataFusion `ColumnarValue` arguments into Arrow arrays.
2. Attaches to Python.
3. Lazily loads or reuses the Python UDF wrapper.
4. Converts Arrow arrays to Python objects using PyArrow bridges.
5. Calls the Python function wrapper.
6. Converts the result back to Arrow `ArrayData`.

7. Casts it to the declared output type.

The core execution path is:

```
let args: Vec<ArrayRef> = ColumnarValue::values_to_arrays(&args)?;
let udf = Python::attach(|py| self.udf(py))?;
let data = Python::attach(|py| -> PyUdfResult<_> {
    let output = udf.call1(py, (args.try_to_py(py)?, number_rows))?;
    Ok(ArrayData::try_from_py(py, &output)?)
})?;
let array = cast(&make_array(data), &self.output_type)?;
Ok(ColumnarValue::Array(array))
```

That differs from JVM Spark. In JVM Spark, Python UDF execution typically involves a Python worker process and serialization between JVM and Python. Sail's Python UDF runs in the same process as the Rust execution engine, and Arrow memory can be shared through PyArrow bindings.

The Sail UDF performance docs summarize the motivation: use Pandas or Arrow UDFs when possible so wrapper overhead is amortized over batches, and use Arrow-native UDFs for the most direct Arrow sharing.

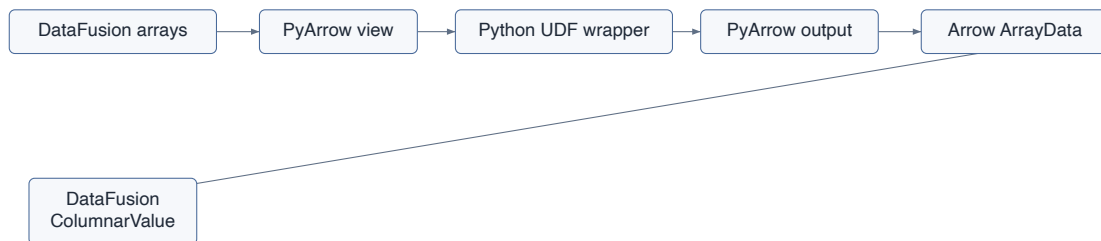


Figure 17: Flowchart 04.4

4.12 Python Conversion Code

The Python helper module embedded in Rust is `crates/sail-python-udf/src/python/spark.py`.

It contains conversion wrappers for:

- scalar Python values
- Pandas UDFs
- Arrow UDFs
- grouped map functions
- co-grouped map functions
- table functions
- Arrow table functions
- UDTF analysis

The Rust side loads that Python code from an embedded string:

```
const MODULE_SOURCE_CODE: &str = include_str!("spark.py");
```

Then `PySpark::module` initializes it once through a `PyOnceLock`.

This is a nice pattern: Sail can ship its Python-side UDF helpers inside the Rust extension module, so it does not need to locate a separate Python file at runtime.

4.13 Config for Python UDF Behavior

`PySparkUdfConfig` captures the Spark/PySpark settings that affect Python UDF behavior:

- session time zone
- Pandas grouped map column assignment
- safe Arrow conversion
- Arrow max records per batch
- Pandas conversion toggles
- int-to-decimal coercion
- binary-as-bytes behavior

It can also emit key-value pairs that PySpark's worker code understands:

```
"spark.sql.session.timeZone"  
"spark.sql.execution.arrow.maxRecordsPerBatch"  
"spark.sql.execution.pyspark.binaryAsBytes"
```

This shows another compatibility layer. The same Python function may behave differently depending on Spark configuration. Sail has to carry those settings from Spark Connect session state into the UDF payload and wrapper.

4.14 UDTFs and Python-Side Analysis

PySpark UDTFs can have an `analyze` static method. The official `udtf` documentation describes this as Python-side analysis that can return a dynamic schema.

Sail has hooks for this in `crates/sail-python-udf/src/python/spark.rs`:

```
pub fn analyze_udtf<'py>(  
    py: Python<'py>,   
    handler: Bound<'py, PyAny>,   
    arguments: Bound<'py, PyAny>,   
) -> PyResult<Bound<'py, PyAny>> {  
    Self::module(py)?  
        .getattr(intern!(py, "analyze_udtf"))?  
        .call1((handler, arguments))  
}
```

This matters because analysis happens before physical execution. A UDTF may determine its output schema from argument types or literal values. That means Python code can participate in planning, not just execution.

For the extension proposal, this is a preview of a broader rule:

Extensions may need hooks before execution starts.

4.15 Python Data Sources

Spark Connect can register Python data sources. Sail handles `RegisterDataSource` in `crates/sail-spark-connect/src/service/plan_executor.rs`.

The handler extracts the pickled Python data source class and registers a session-scoped `PythonDataSourceAdapter` in the `DataSourceRegistry`:

```
let source = Arc::new(PythonDataSourceAdapter::with_pickled_class(
    name.clone(),
    command,
));
registry.register_data_source(source)?;
```

This is parallel to Python UDF registration:

Python behavior arrives through Spark Connect
-> Sail stores it in session-scoped registry
-> later scans can resolve and execute it

The important architectural point is session isolation. A registered Python data source belongs to that session's table format registry, not a global singleton shared by all users.

4.16 Testing PySpark Compatibility

Sail's Python tests are themselves a guide to compatibility.

The fixture in `python/pysail/tests/spark/conftest.py` either uses `SPARK_REMOTE` or starts a local Sail Spark Connect server. Then it creates a normal PySpark session:

```
SparkSession.builder.remote(remote).getOrCreate()
```

The tests cover:

- DataFrame behavior
- SQL behavior
- catalog behavior
- joins, aggregation, ordering, sampling, repartitioning
- functions
- Python UDFs, Pandas UDFs, Arrow UDFs, UDTFs
- data sources
- writes
- streaming
- Delta and Iceberg behavior
- TPC-H, TPC-DS, ClickBench plans/results

The test matrix explicitly checks different PySpark versions. That is because Spark Connect is a moving protocol and PySpark's UDF behavior evolves. Sail has to track both API surface and wire behavior.

4.17 Why PySpark Versions Matter

The UDF payload builder contains version-specific logic:

```

let pyspark_version = get_pyspark_version()?;
...
if matches!(pyspark_version, PySparkVersion::V4_1)
    && matches!(eval_type, spec::PySparkUdfType::ArrowBatched)
{
    let schema_json = build_input_types_json(input_types)?;
    ...
}

```

That is a concrete example of why “Spark compatible” is not a single target. Spark 3.5, Spark 4.0, and Spark 4.1 differ in function support, UDF payload details, UDTF behavior, Arrow APIs, and type handling.

`pyproject.toml` reflects this with test dependencies and test matrices for multiple Spark versions.

4.18 What pysail Means for Extensions

The experimental branch implements opt-in Python entry points such as:

```

[project.entry-points."pysail.extensions"]
sedona = "sail_sedona:extension"

```

This is a natural Python packaging experience:

```
bash examples/extensions/scripts/build.sh
```

On the experimental branch, `SAIL_EXPERIMENTAL_EXTENSIONS=1` enables discovery and manifest validation. Use its source-distribution tutorial and built wheels; this is not a promise that the named example packages are published on PyPI.

But this chapter should make the hard parts clear.

First, discovery is Python-level, but most extension hooks are Rust/DataFusion-level:

```

Python entry point
-> Rust extension registration
-> DataFusion UDFs, optimizer rules, extension planners, codecs

```

Second, version coupling is strict. A Python wheel that exposes Rust extension objects must match Sail’s arrow, datafusion, pyo3, and pysail versions. Rust trait objects are not a stable plugin ABI across arbitrary crate versions.

Third, worker execution must see the same extension behavior. Installing an extension in the client Python environment is not enough if cluster workers cannot decode the physical plan or reconstruct extension UDFs.

Fourth, analysis must work too. If a PySpark client asks for schema or explain output before execution, the extension must be registered before `analyze_plan` resolves the query.

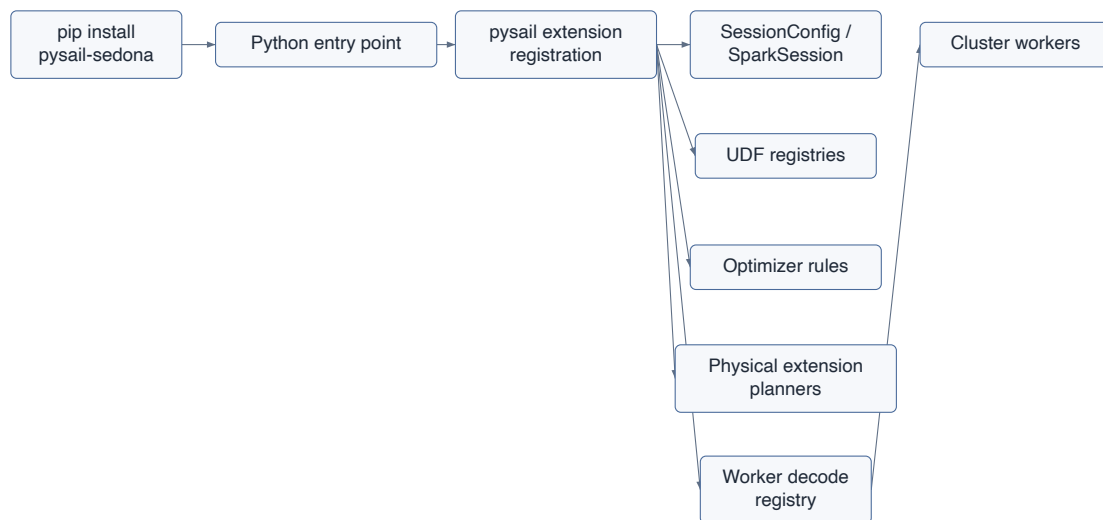


Figure 18: Flowchart 04.5

The pleasant user story is Pythonic. The engine story is Rust and distributed.

4.19 A Small End-to-End Example

Suppose the user writes:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

spark = SparkSession.builder.remote("sc://localhost:50051").getOrCreate()

@udf(returnType=IntegerType())
def plus_one(x):
    return None if x is None else x + 1

spark.range(3).select(plus_one("id")).show()

```

Sail's path is:

PySpark creates Spark Connect relation containing inline Python UDF

- > sail-spark-connect converts relation to Sail spec
- > PlanResolver resolves CommonInlineUserDefinedFunction
- > PySparkUdfPayload is built
- > PySparkUDF becomes a DataFusion ScalarUDF
- > DataFusion physical plan executes
- > PySparkUDF invokes Python wrapper in process
- > output Arrow array returns to DataFusion
- > Spark Connect streams ArrowBatch results to PySpark

This is the whole Sail philosophy in miniature: keep the PySpark surface, translate through Spark Connect, execute in Rust/DataFusion/Arrow, and invoke Python only where Python semantics are actually needed.

4.20 Reading Exercises

1. Read `python/pysail/spark/__init__.py`.
 - Notice how small the public wrapper is.
 - Find the `listening_address` property.
2. Read `crates/sail-python/src/spark/server.rs`.
 - Follow `new`, `start`, `run`, and `stop`.
 - Find where `Python::detach` is used.
3. Read `crates/sail-python/src/globals.rs`.
 - Follow global runtime initialization.
 - Find environment-variable warning behavior.
4. Read `python/pysail/tests/spark/conftest.py`.
 - Follow the remote fixture.
 - Find `SparkSession.builder.remote`.
 - Note which Spark config values tests set by default.
5. Read `crates/sail-plan/src/resolver/expression/udf.rs`.
 - Trace a scalar UDF from Spark Connect spec to DataFusion `ScalarUDF`.
 - Trace a grouped aggregate UDF to `AggregateUDF`.
6. Read `crates/sail-python-udf/src/udf/pyspark_udf.rs`.
 - Find `PySparkUdfKind`.
 - Follow `invoke_with_args`.
7. Read `crates/sail-python-udf/src/cereal/pyspark_udf.rs`.
 - Find payload build and load.
 - Notice the PySpark version-specific logic.
8. Read `crates/sail-python-udf/src/python/spark.py`.
 - Skim the converter classes.
 - Find how Pandas and Arrow wrappers shape Python execution.

4.21 Chapter Takeaways

`pysail` is the Python package that makes Sail usable from Python, but PySpark remains the primary user API. `pysail` starts and packages a Rust Spark Connect server. PySpark connects to that server using `SparkSession.builder.remote`.

Python UDF support is where the layers meet most dramatically. PySpark serializes Python functions into Spark Connect plans. Sail turns those payloads into DataFusion UDFs. Execution invokes Python in process and exchanges Arrow memory through PyArrow bridges. Pandas and Arrow UDFs amortize Python overhead over batches, while Arrow-native functions can share Arrow data most directly.

For extensions, Python packaging gives an attractive discovery story, but the actual extension hooks must reach Rust planning, DataFusion execution, and distributed worker decoding. The final extension architecture has to make that Python-to-Rust bridge explicit.

The next chapter moves into Apache Arrow itself: arrays, schemas, record batches, IPC, PyArrow bridges, Arrow Flight, and why columnar memory is the common currency between Spark Connect, DataFusion, Python UDFs, and Sail’s distributed shuffle.

5 Apache Arrow

Apache Arrow is the data plane hiding in plain sight throughout Sail.

Spark Connect gives Sail a protocol for receiving unresolved Spark plans and returning results to Spark clients. DataFusion gives Sail an optimizer and an execution engine. PySpark compatibility gives Sail a Python surface area. The distributed runtime gives Sail a way to split work across workers. Arrow is the format that lets those pieces hand data to each other without constantly reinterpreting rows.

In Sail, Arrow is not merely an output serialization format. It is the shape of execution itself:

- DataFusion physical plans produce `RecordBatch` streams.
- Spark Connect responses carry Arrow IPC payloads.
- Python UDFs convert Rust Arrow arrays to PyArrow arrays and back.
- Shuffle partitions batches into per-task streams.
- Flight SQL encodes `SendableRecordBatchStream` as Arrow Flight data.
- Extension types preserve domain semantics such as geometry, geography, and variants.

This chapter is about learning Arrow through Sail’s code. We will use the official Arrow documentation as the reference vocabulary, then map that vocabulary to the concrete Rust modules that make Sail work.

5.1 Definitive References

Keep these open while reading the chapter:

- Apache Arrow Columnar Format
- Arrow Flight RPC protocol
- PyArrow documentation
- PyArrow `RecordBatch` API
- DataFusion gentle Arrow introduction

The Arrow columnar format specification is the most important one. It defines the memory layout, data types, schemas, record batches, and IPC messages. The Flight specification explains the network layer built on Arrow IPC and gRPC. The PyArrow documentation matters because Sail crosses the Rust/Python boundary for PySpark UDFs.

5.2 Where Arrow Lives In Sail

Here are the main code paths for this chapter:

Area	Files	Arrow role
Spark Connect result streaming	<code>crates/sail-spark-connect/src/executor.rs</code>	Converts RecordBatch values into Spark Connect ArrowBatch messages using Arrow IPC
Spark schema conversion	<code>crates/sail-spark-connect/src/proto/data_type_arrow.rs</code>	Maps Arrow fields and data types back to Spark Connect data types
Python UDF conversion	<code>crates/sail-python-udf/src/conversion.rs</code>	Converts Rust Arrow objects to/from PyArrow objects
PySpark UDF execution	<code>crates/sail-python-udf/src/udf/pyspark_udf.rs</code>	Invokes Python functions with Arrow arrays and receives Arrow data
Distributed shuffle write	<code>crates/sail-execution/src/plan/shuffle_write.rs</code>	Partitions RecordBatch streams into shuffle outputs
Distributed shuffle read	<code>crates/sail-execution/src/plan/shuffle_read.rs</code>	Opens shuffle locations and merges RecordBatch streams
Arrow Flight SQL	<code>crates/sail-flight/src/service.rs</code>	Serves DataFusion output streams over Flight SQL
Physical plan examples	<code>crates/sail-physical-plan/src/range.rs</code>	Builds batches from Arrow arrays
Row round-robin repartition	<code>crates/sail-physical-plan/src/repartition.rs</code>	Uses Arrow compute kernels to split batches
GeoArrow extension type	<code>crates/sail-common/src/geoarrow/extension.rs</code>	Defines Arrow extension metadata for WKB geometry/geography

The short version is:

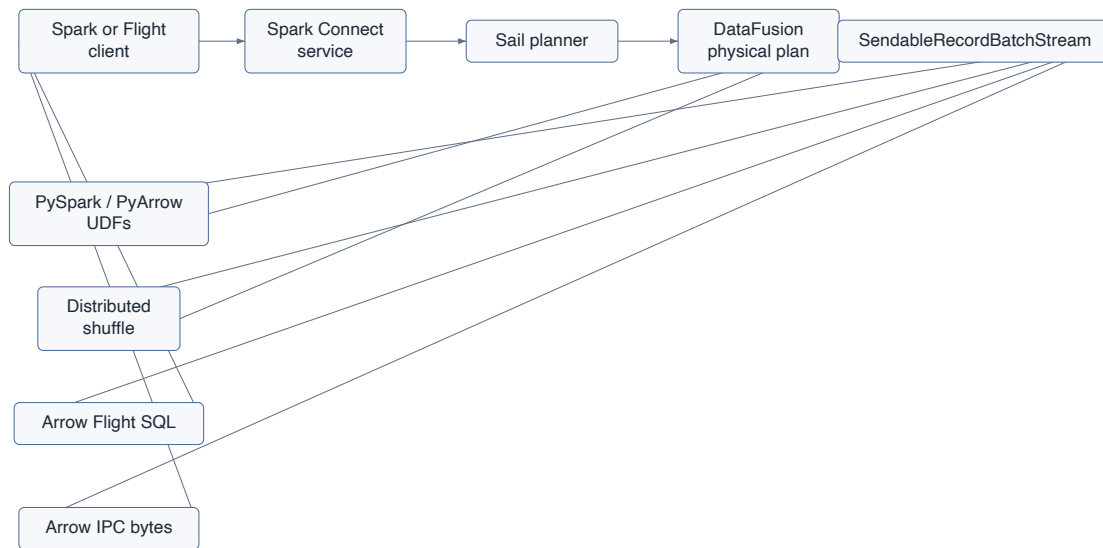


Figure 19: Flowchart 05.1

Arrow is the contract between almost every box in that diagram.

5.3 The Arrow Mental Model

Arrow is a columnar memory format. A table is not stored as a sequence of row objects. It is stored as arrays, one array per column, with a schema describing the column names, logical types, nullability, metadata, and nested structure.

The key types you see in Sail are:

Rust type	Meaning in Sail
<code>ArrayRef</code>	Shared reference to an immutable Arrow array
<code>ArrayData</code>	Low-level array buffers and metadata, useful at FFI boundaries
<code>DataType</code>	Arrow logical type such as <code>Int64</code> , <code>Utf8</code> , <code>Struct</code> , <code>Map</code> , <code>Timestamp</code>
<code>Field</code>	One named column, including data type, nullability, and metadata
<code>Schema</code>	Ordered collection of fields plus schema metadata
<code>SchemaRef</code>	Shared <code>Arc<Schema></code>
<code>RecordBatch</code>	Equal-length arrays plus a schema; the basic execution unit
<code>RecordBatchStream</code>	Async stream of <code>RecordBatch</code> values
<code>SendableRecordBatchStream</code>	Boxed, pinned, sendable <code>DataFusion</code> batch stream

The official Arrow format documentation describes a record batch as an ordered collection of arrays with the same length, described by a schema. `DataFusion`'s Arrow introduction adds the execution intuition: a `RecordBatch` is columnar inside, but externally it behaves like a row chunk that can be streamed, partitioned, and scheduled.

That dual nature explains why Arrow fits distributed query processing so well:

- Vectorized operators can process contiguous column arrays.
- The scheduler can move work in batch-sized units.
- Network protocols can serialize batches without inventing a new row format.
- Python UDFs can receive arrays instead of per-row Python objects.
- Metadata can carry logical semantics beyond the physical storage type.

5.4 Why Arc Shows Up Everywhere

In Sail's Rust code, Arrow objects are usually wrapped in `Arc`.

For example, `SchemaRef` is an `Arc<Schema>`, and `ArrayRef` is an `Arc<dyn Array>`. This reflects the Arrow memory model: arrays are immutable once built, so it is cheap and safe to share them across operators, streams, and async tasks.

When a physical operator needs to produce a new batch, it usually creates new array references and a shared schema:

```
let id_array: ArrayRef = Arc::new(Int64Array::from(x));
let batch = RecordBatch::try_new(projected_schema.clone(), vec![id_array])?;
```

That pattern appears in `RangeExec` in `crates/sail-physical-plan/src/range.rs`. The range source partitions a numeric range, builds an `Int64Array` for each chunk, then wraps the chunks in `RecordBatch` values.

The important lesson is that Sail does not need an internal row object for this operator. The execution unit is already Arrow-native:

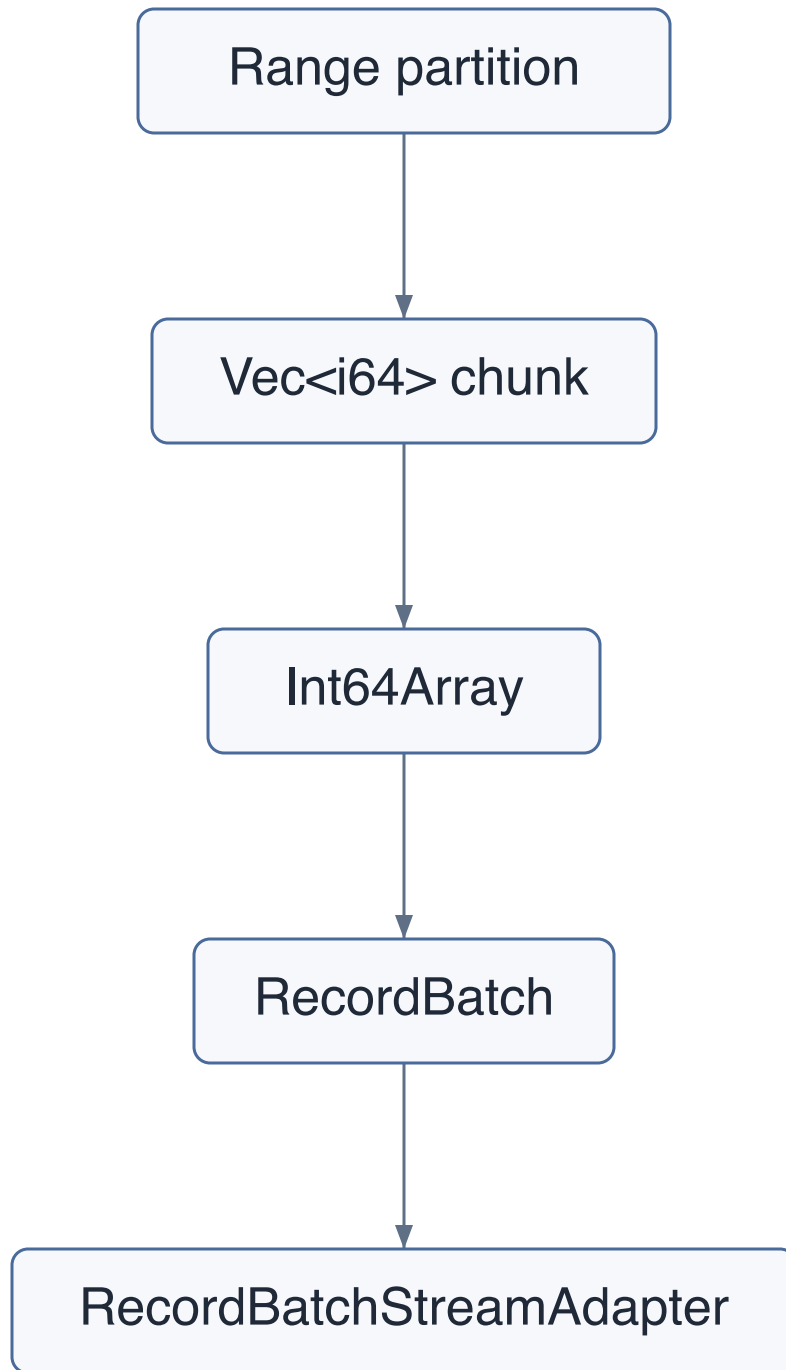


Figure 20: Flowchart 05.2

5.5 RecordBatch As The Unit Of Execution

Most query engines have a concept like “a batch of rows.” In Sail, that concept is concretely Arrow’s RecordBatch.

Look at the signature of DataFusion physical execution:

```
fn execute(  
    &self,  
    partition: usize,  
    context: Arc<TaskContext>,  
) -> Result<SendableRecordBatchStream>
```

This shape appears across Sail physical plan nodes. A plan node is executed for one output partition, and the result is a stream of Arrow batches. Sail can then compose operators by chaining streams.

RangeExec is a good first example:

1. Validate the requested partition.
2. Compute that partition’s range values.
3. Chunk values into RANGE_BATCH_SIZE.
4. Build Arrow arrays.
5. Build RecordBatch values.
6. Return a RecordBatchStreamAdapter.

The same type shows up at much larger boundaries:

- Spark Connect’s ExecutorTaskContext owns a SendableRecordBatchStream.
- Shuffle write consumes a SendableRecordBatchStream.
- Shuffle read returns a SendableRecordBatchStream.
- Flight SQL stores and later encodes a SendableRecordBatchStream.
- Python UDF execution turns ColumnarValue values into Arrow arrays.

Once you learn to recognize SendableRecordBatchStream, you can follow data through Sail.

5.6 Spark Connect Output: Arrow IPC In Protobuf Clothing

Spark Connect uses Protobuf messages for the control protocol, but tabular results are encoded as Arrow batches.

In `crates/sail-spark-connect/src/executor.rs`, the executor has this batch enum:

```
pub enum ExecutorBatch {  
    ArrowBatch(ArrowBatch),  
    SqlCommandResult(Box<SqlCommandResult>),  
    ...  
    Schema(Box<DataType>),  
    Complete,  
}
```

For query output, the important variants are:

- Schema(Box<DataType>)
- ArrowBatch(ArrowBatch)
- Complete

The executor first converts the DataFusion stream schema into a Spark schema:

```
let schema = to_spark_schema(context.stream.schema());
let out = ExecutorOutput::new(ExecutorBatch::Schema(Box::new(schema)));
```

Then it repeatedly reads Arrow RecordBatch values from the stream:

```
while let Some(batch) = context.next().await? {
    let batch = to_arrow_batch(&batch)?;
    let out = ExecutorOutput::new(ExecutorBatch::ArrowBatch(batch));
    tx.send(out).await?;
}
```

The conversion to Spark Connect's ArrowBatch is compact:

```
pub(crate) fn to_arrow_batch(batch: &RecordBatch) -> SparkResult<ArrowBatch> {
    let mut output = ArrowBatch::default();
    {
        let cursor = Cursor::new(&mut output.data);
        let mut writer = StreamWriter::try_new(cursor, batch.schema().as_ref())?;
        writer.write(batch)?;
        output.row_count += batch.num_rows() as i64;
        writer.finish()?;
    }
    Ok(output)
}
```

This is Arrow IPC streaming format. The official Arrow columnar documentation describes IPC streams as schema followed by record batch messages and optional dictionary batches. Sail uses StreamWriter to produce that payload and stores the bytes inside Spark Connect's ArrowBatch.data.

The relationship looks like this:

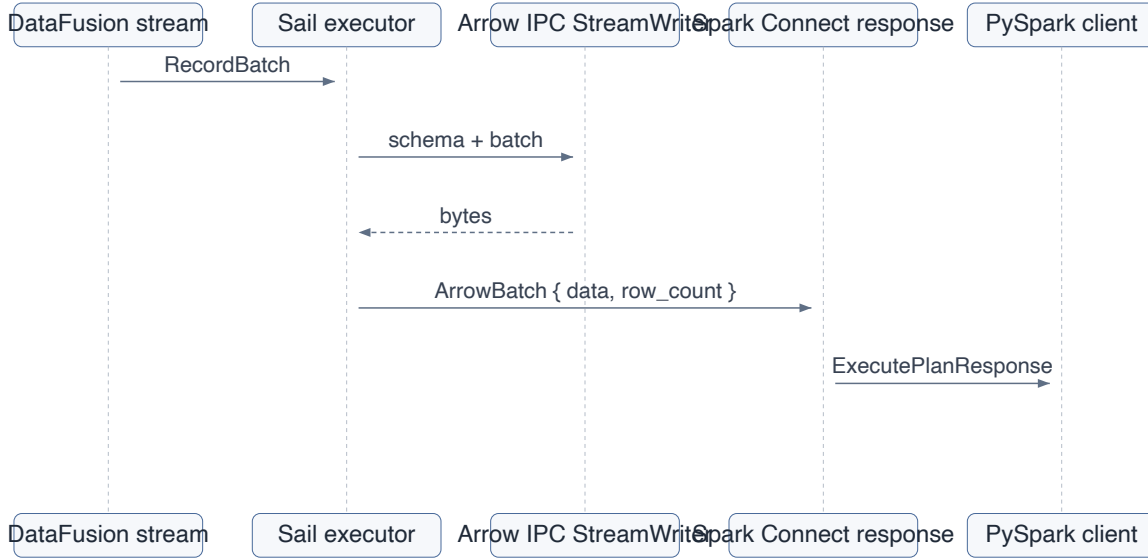


Figure 21: Sequence diagram 05.3

The executor also sends empty batches in two cases:

- As a heartbeat when no batch arrives before the configured interval.
- As an output placeholder when a query produces no rows.

That is why `ExecutorTaskContext::next` can return `RecordBatch::new_empty(self.stream.schema())`. The stream contract remains Arrow-shaped even when the batch contains zero rows.

5.7 Schema Conversion: Arrow Back To Spark Types

Sail must speak Arrow internally and Spark externally. The conversion from Arrow types to Spark Connect types lives in `crates/sail-spark-connect/src/proto/data_type_arrow.rs`.

The central conversion is:

```
impl TryFrom<adt::DataType> for DataType
```

It maps Arrow types such as:

- `adt::DataType::Boolean` -> Spark Boolean
- `adt::DataType::Int32` -> Spark Integer
- `adt::DataType::Int64` -> Spark Long
- `adt::DataType::Utf8` -> Spark String
- `adt::DataType::Date32` -> Spark Date
- `adt::DataType::Struct(fields)` -> Spark Struct
- `adt::DataType::List(field)` -> Spark Array
- `adt::DataType::Map(field, _)` -> Spark Map

The code also documents where the mapping is lossy or constrained. For example, Spark Char or VarChar may become Arrow string-like data and then come back as Spark String. Timestamp

precision is another important compatibility choice: the conversion accepts microsecond timestamps and rejects other timestamp units in several branches.

This is a crucial extension lesson. Arrow is a strong physical and logical format, but it is not identical to Spark's type system. If an extension needs Spark-specific semantics, it must preserve them deliberately, often through:

- logical plan metadata,
- Arrow field metadata,
- Arrow extension types,
- or explicit conversion rules.

5.8 Extension Types: GeoArrow And Variant

Arrow supports extension types: a field can have a physical storage type plus metadata that names a higher-level logical meaning.

Sail already uses this idea in `crates/sail-common/src/geoarrow/extension.rs`:

```
pub struct GeoArrowWkbType {
    pub metadata: GeoArrowMetadata,
}

impl GeoArrowWkbType {
    pub const NAME: &'static str = "geoarrow.wkb";
}
```

The extension stores geometry/geography values as binary WKB, while metadata captures CRS and edge semantics:

```
pub struct GeoArrowMetadata {
    pub edges: Option<GeoArrowEdges>,
    pub crs: Option<GeoArrowCrs>,
}
```

The storage type check is strict:

```
match data_type {
    DataType::Binary | DataType::LargeBinary | DataType::BinaryView => Ok(()),
    data_type => Err(...),
}
```

Then the Spark Connect type conversion recognizes this extension:

```
} else if extension_type_name == Some(GeoArrowWkbType::NAME) {
    let ext = field.try_extension_type::<GeoArrowWkbType>()?;
    let meta: SparkGeoMetadata = ext.metadata.try_into()?;
    ...
}
```

Sail maps GeoArrow metadata to Spark geometry or geography. If edges is present, Sail treats the value as geography. If edges is absent, it treats the value as geometry. CRS metadata becomes Spark SRID values for supported CRS strings.

The same file recognizes `parquet_variant_compute::VariantType`, mapping that Arrow extension to Spark Variant.

This is the most concrete preview of the final extensions chapter. An extension does not have to invent a new internal data plane. It can often use Arrow's existing storage plus extension metadata:

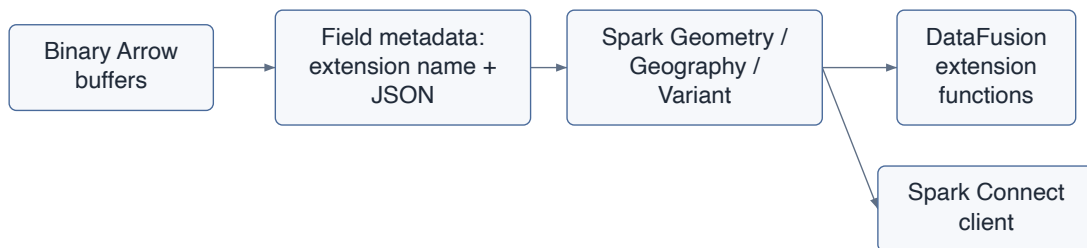


Figure 22: Flowchart 05.4

That architecture keeps data compatible with Arrow tooling while preserving domain meaning.

5.9 PyArrow: The Python Boundary

The Python side of Sail depends heavily on Arrow because PySpark UDFs should not have to receive rows one Python object at a time.

The conversion layer in `crates/sail-python-udf/src/conversion.rs` uses the `arrow_pyarrow` crate:

```
use arrow_pyarrow::{FromPyArrow, ToPyArrow};
```

Sail implements `TryToPy` for:

- `&DataType`
- slices of `DataType`
- slices of `ArrayRef`
- `Vec<ArrayRef>`
- `&Schema`
- `SchemaRef`
- `RecordBatch`

And it implements `TryFromPy` for:

- `ArrayData`
- `RecordBatch`

For arrays, the conversion uses the underlying Arrow `ArrayData`:

```
self.iter()
  .map(|x| x.into_data().to_pyarrow(py))
  .collect:::<PyResult<Vec<_>>>()
```

That is a big deal. It means Sail is intentionally crossing the language boundary with Arrow arrays, not with ad hoc serialized Python values.

The UDF invocation path in `crates/sail-python-udf/src/udf/pyspark_udf.rs` makes that practical:

```
let args: Vec<ArrayRef> = ColumnarValue::values_to_arrays(&args)?;
let output = udf.call1(py, (args.try_to_py(py)?, number_rows))?;
let data = ArrayData::try_from_py(py, &output)?;
let array = cast(&make_array(data), &self.output_type)?;
```

The steps are:

1. Convert DataFusion ColumnarValue arguments to Arrow arrays.
2. Convert those Arrow arrays into PyArrow objects.
3. Call the Python function.
4. Convert the returned PyArrow object back into Arrow ArrayData.
5. Wrap it as an Arrow array.
6. Cast it to the declared output type.

The official PyArrow RecordBatch API is useful for understanding the Python objects Sail is inter-operating with. PyArrow exposes schemas, columns, row counts, zero-copy slices, filters, take, conversion from arrays, and IPC serialization methods. Sail's Rust side uses the same conceptual model, but with Rust ownership and type checks.

5.10 UDF Kinds And Arrow-Native Execution

The PySpark UDF kind enum includes several execution modes:

```
pub enum PySparkUdfKind {
    Batch,
    ArrowBatch,
    ScalarPandas,
    ScalarPandasIter,
    ScalarArrow,
    ScalarArrowIter,
}
```

The comment in the code calls out Spark 4.0 Arrow-native scalar UDF types. The important distinction is:

- Pandas UDFs use Arrow as an efficient transport to and from Pandas.
- Arrow-native UDFs can pass Arrow arrays more directly.

From an engine architecture perspective, this is exactly the direction Sail wants. Every conversion into Python objects costs time and memory. Every operator that can remain Arrow-native preserves vectorization and avoids unnecessary row materialization.

The performance guide at `docs/guide/udf/performance.md` makes the same point: Arrow UDFs avoid copying data into row-oriented Python objects and let the Rust engine and Python function share Arrow data through the Arrow/PyArrow boundary.

5.11 Distributed Shuffle: Arrow As The Exchange Unit

Distributed query processing needs exchanges. A join, aggregation, sort, or repartition may require data from one set of workers to move to another set of workers.

In Sail, the exchange unit is still `RecordBatch`.

The write side is `crates/sail-execution/src/plan/shuffle_write.rs`. `ShuffleWriteExec` wraps an input physical plan and a desired output partitioning. When `execute` is called for an input partition, it:

1. Opens one sink per shuffle output location.
2. Executes the child plan for the input partition.
3. Reads `RecordBatch` values from the child stream.
4. Partitions each batch by hash or row round-robin.
5. Writes partitioned batches to the corresponding sinks.
6. Closes the sinks.
7. Returns an empty batch stream as the execution result.

The core loop is:

```
while let Some(batch) = stream.next().await {
    let batch = batch?;
    let mut partitions: Vec<Option<RecordBatch>> = vec![None;
partition_sinks.len()];
    partitioner.partition(batch, |p, batch| {
        partitions[p] = Some(batch);
        Ok(())
    })?;
    ...
}
```

The read side is `crates/sail-execution/src/plan/shuffle_read.rs`. `ShuffleReadExec` has a set of read locations for each output partition. When executed, it opens all relevant task streams and merges them:

```
let futures = locations
    .iter()
    .map(|location| reader.open(location, schema.clone()));
let streams = try_join_all(futures).await?;
Ok(Box::pin(MergedRecordBatchStream::new(schema, streams)))
```

That gives downstream operators a normal `SendableRecordBatchStream` again. The shuffle itself is an implementation detail hidden between write and read nodes.

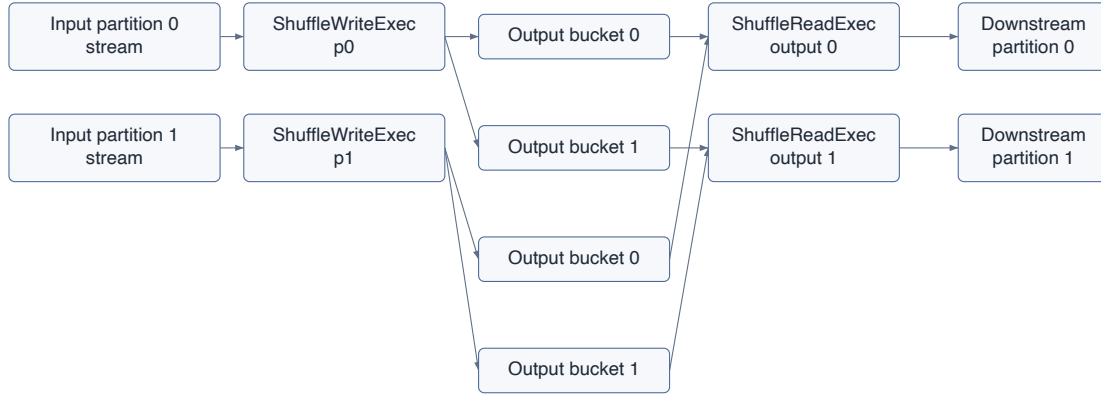


Figure 23: Flowchart 05.5

Notice the absence of a Sail-specific row format. The shuffle API talks in task stream locations, but the payload remains Arrow batches.

5.12 Row Round-Robin Repartition

Hash partitioning can delegate to DataFusion’s BatchPartitioner. Sail also has a row round-robin partitioner in `crates/sail-physical-plan/src/repartition.rs`.

The logic is a useful Arrow lesson:

```

let schema = batch.schema();
let mut indices = vec![Vec::new(); self.num_partitions];
for row_index in 0..batch.num_rows() {
    let partition = (self.next_idx + row_index) % self.num_partitions;
    indices[partition].push(row_index as u32);
}
...
let indices_array: PrimitiveArray<UInt32Type> = partition_indices.into();
let columns = take_arrays(batch.columns(), &indices_array, None)?;
let partition_batch =
    RecordBatch::try_new_with_options(schema.clone(), columns, &options)?;
  
```

Sail does not loop over cells and rebuild rows. It builds an Arrow array of row indices and uses Arrow compute’s `take_arrays` to select rows from every column. The output is one `RecordBatch` per non-empty destination partition.

This is the practical meaning of vectorized execution: even operations that are row-directed can often be implemented as array operations.

5.13 Flight SQL: Arrow On The Network

Sail also exposes a Flight SQL service in `crates/sail-flight/src/service.rs`. The official Arrow Flight protocol describes Flight as an RPC framework for high-performance Arrow data services, built on gRPC and Arrow IPC. It is organized around streams of Arrow record batches, with metadata methods for discovering and retrieving streams.

That maps cleanly to Sail's service.

For a SQL statement, `get_flight_info_statement`:

1. Parses the SQL string.
2. Converts it to Sail's plan representation.
3. Resolves and executes the plan.
4. Gets a `SendableRecordBatchStream`.
5. Stores the stream under a query handle.
6. Returns `FlightInfo` with a ticket.

Then `do_get_statement`:

1. Decodes the ticket into a query handle.
2. Removes the stored stream from state.
3. Reads the stream schema.
4. Encodes the stream with `FlightDataEncoderBuilder`.
5. Returns the encoded Flight data stream.

The encoding step is:

```
let output = FlightDataEncoderBuilder::new()
  .with_schema(schema)
  .build(output)
  .map(|result| result.map_err(|e| Status::internal(format!("encoding error:
{e}"))));
```

The same DataFusion output can therefore leave Sail through two different protocol doors:

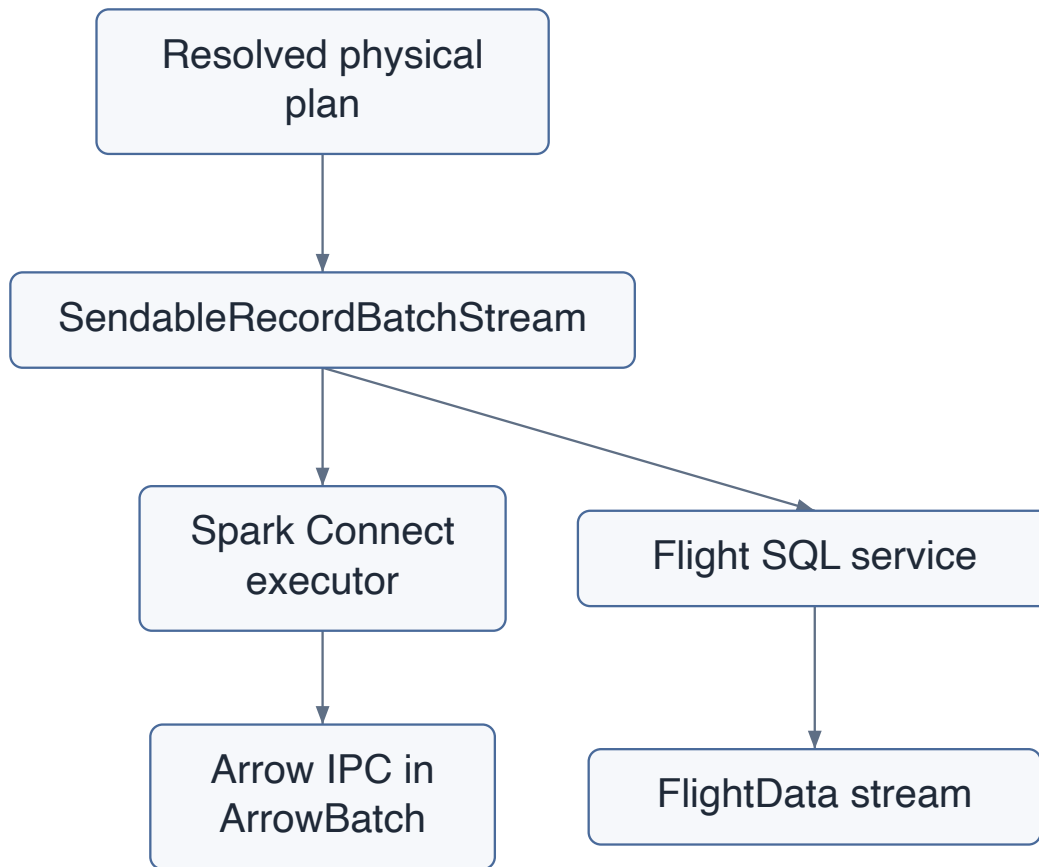


Figure 24: Flowchart 05.6

This is a powerful architectural property. Sail's execution engine does not need separate data representations for Spark Connect and Flight SQL. Protocols wrap the same Arrow stream abstraction.

5.14 Arrow IPC Versus Flight

It is easy to blur Arrow IPC and Arrow Flight. Sail uses both, so it helps to separate them:

Concept	In Arrow	In Sail
In-memory format	Column buffers, validity bitmaps, offsets, schemas	ArrayRef, SchemaRef, RecordBatch
IPC stream/file format	Serialized schema, dictionaries, record batches	Spark Connect ArrowBatch.data via StreamWriter
Flight	gRPC service protocol carrying Arrow data streams	SailFlightSqlService with FlightDataEncoderBuilder
PyArrow bridge	Python implementation of Arrow data structures	arrow_pyarrow conversions for UDFs

Think of it this way:

- Arrow memory format is how data exists while operators work.
- Arrow IPC is how batches are serialized.
- Arrow Flight is a service protocol for discovering and transferring streams.
- PyArrow is Python's Arrow implementation and API surface.

Sail benefits because these layers are designed to compose.

5.15 A Small End-To-End Example

Imagine a PySpark client runs:

```
spark.range(0, 5).selectExpr("id + 10 as value").collect()
```

The high-level flow is:

1. PySpark builds an unresolved Spark Connect plan.
2. Spark Connect sends that plan to Sail.
3. Sail resolves it into its internal plan representation.
4. DataFusion physical execution creates a SendableRecordBatchStream.
5. A source such as RangeExec produces an Arrow Int64Array.
6. Projection produces another Arrow array for value.
7. The executor reads each RecordBatch.
8. StreamWriter encodes each batch as Arrow IPC bytes.
9. Spark Connect returns ArrowBatch messages to the client.
10. The client decodes Arrow batches into Spark result rows.

The data path can be described without a single row class:

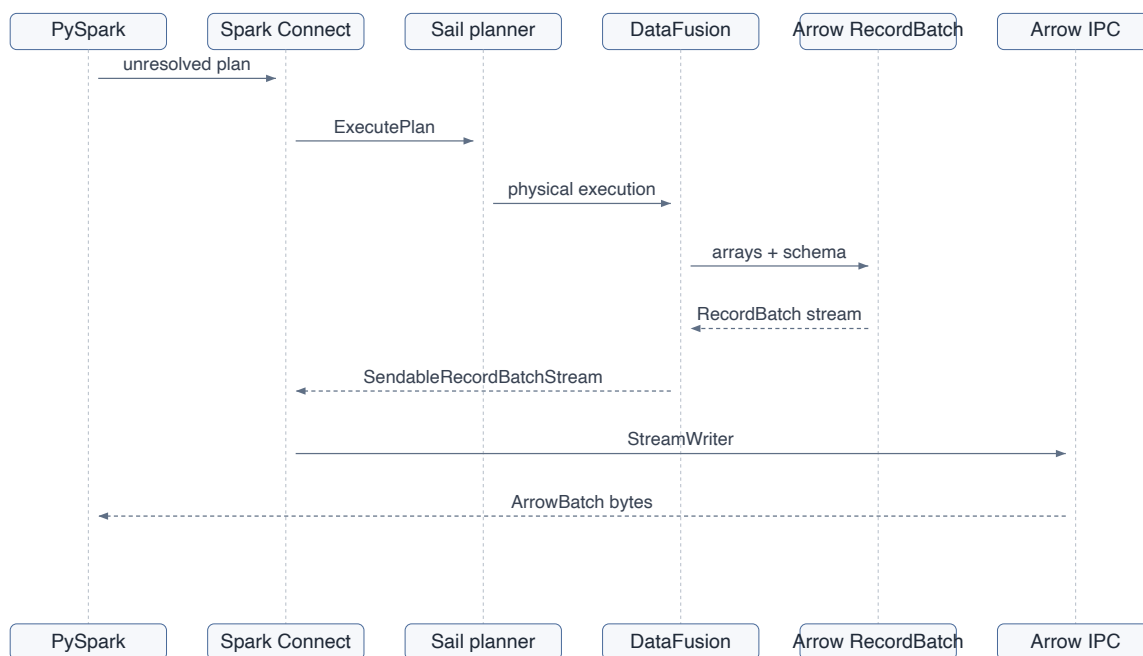


Figure 25: Sequence diagram 05.7

That is the central architecture of Sail result execution.

5.16 What Arrow Does Not Solve By Itself

Arrow gives Sail a powerful data representation, but it does not solve every problem.

Arrow does not decide:

- how Spark logical plans should map to DataFusion plans,
- how distributed tasks should be scheduled,
- how shuffle locations should be assigned,
- how Python worker lifetimes should be managed,
- how Spark-only semantics should survive type conversion,
- how extension packages should be loaded,
- how security and sandboxing should work for user code.

Those are Sail architecture problems. Arrow is the shared data language that makes the solutions simpler and faster.

This distinction matters for extensions. An extension proposal should not merely say “use Arrow.” It should identify:

- the Arrow storage type,
- the Arrow metadata or extension name,
- the DataFusion logical and physical expressions,
- the Spark Connect type mapping,
- the PyArrow/Python behavior,

- the distributed shuffle compatibility story,
- and the protocol boundary behavior.

5.17 Extension Design Pattern: Arrow-First Semantics

The GeoArrow code suggests a reusable pattern for future extensions:

1. Choose an Arrow physical representation.
2. Define metadata that captures the logical semantics.
3. Implement validation for allowed storage types.
4. Teach Spark Connect schema conversion to recognize the extension.
5. Add DataFusion expressions and functions that operate on the Arrow layout.
6. Preserve metadata through projections, UDFs, shuffle, and output protocols.
7. Add Python wrappers that expose the same semantics through PyArrow.

For example, a geospatial extension can store WKB as Binary and use `geoarrow.wkb` metadata for CRS and edge semantics. A variant extension can store variant-encoded bytes and preserve logical type identity with an Arrow extension name. A machine-learning vector extension might store `FixedSizeList<Float32>` plus metadata for dimension and metric assumptions.

The best extensions feel native to Arrow rather than bolted onto it.

5.18 Common Mistakes When Learning Arrow In Sail

5.18.1 Mistake 1: Thinking RecordBatch Means Row-Oriented

A `RecordBatch` is a batch of rows from the outside, but internally it is columnar. Operators should usually work column-by-column.

5.18.2 Mistake 2: Treating Schema Metadata As Decoration

Schema and field metadata can carry compatibility-critical information. Sail's UDT and GeoArrow conversions depend on metadata to recover Spark semantics.

5.18.3 Mistake 3: Copying Data At Language Boundaries

Python integration should use PyArrow whenever possible. Converting every row into Python objects defeats the point of using a columnar engine.

5.18.4 Mistake 4: Forgetting Empty Batches

Empty batches are still meaningful. They carry schema, heartbeats, and protocol state. Sail's Spark Connect executor intentionally emits empty batches in some cases.

5.18.5 Mistake 5: Ignoring Unsupported Type Mappings

Arrow and Spark type systems overlap but are not identical. Unsupported units, dictionary encodings, unions, and lossy string-like conversions must be handled explicitly.

5.19 Reading Exercise: Follow One Batch

To build intuition, trace one `RecordBatch` through the code:

1. Start in `crates/sail-physical-plan/src/range.rs`.

2. Find the `Int64Array::from(x)` call.
3. Follow the `RecordBatch::try_new` call.
4. Find where plan execution returns `SendableRecordBatchStream`.
5. Jump to `crates/sail-spark-connect/src/executor.rs`.
6. Find `while let Some(batch) = context.next().await?`.
7. Follow to `_arrow_batch`.
8. Look at `StreamWriter::try_new`, `writer.write`, and `writer.finish`.

Then repeat the exercise for Flight SQL:

1. Start in `crates/sail-flight/src/service.rs`.
2. Find `service.runner().execute(&ctx, plan)`.
3. Follow the stored stream into `SailFlightSqlState`.
4. Find `do_get_statement`.
5. Follow `FlightDataEncoderBuilder`.

After these two traces, you will understand the two biggest Arrow output paths in Sail.

5.20 Reading Exercise: Follow One Python UDF

For Python UDFs:

1. Start in `crates/sail-python-udf/src/udf/pyspark_udf.rs`.
2. Find `ColumnarValue::values_to_arrays`.
3. Follow `args.try_to_py(py)`.
4. Jump to `crates/sail-python-udf/src/conversion.rs`.
5. Find the `TryToPy` implementation for `&[ArrayRef]`.
6. Follow `into_data().to_pyarrow(py)`.
7. Return to `invoke_with_args`.
8. Follow `ArrayData::try_from_py`.
9. Observe the final `cast(&make_array(data), &self.output_type)`.

This is the Arrow/PyArrow bridge in miniature.

5.21 How This Prepares Us For DataFusion

The next chapter can now talk about DataFusion with the right foundation. DataFusion is not just a planner that happens to use Arrow. Its physical operators, stream interfaces, expression evaluation, partitioning, and UDF APIs are designed around Arrow batches.

When Sail adds Spark compatibility on top of DataFusion, it repeatedly answers one question:

How do we preserve Spark semantics while keeping the data path Arrow-native?

That question appears in:

- schema conversion,
- physical plan construction,
- Spark-specific expressions,

- Python UDF execution,
- shuffle exchanges,
- and future extension APIs.

5.22 Native Extension Ownership Across FFI

In the experimental branch, native packages export DataFusion objects through named FFI capsules. Arrow buffers can outlive the function or algorithm call that produced them. A zero-copy handoff therefore transfers or shares an owner; it does not make the lifetime or allocation cost disappear.

The host's native resource lease must remain reachable from every retained graph state or output buffer that consumes its admitted quota. Closing a session-owned registry entry can remove one reference while a consumer still holds a batch. The final release path, rather than the moment a query returns, determines when that reservation can be returned. Release callbacks must free using the correct allocation owner across the library boundary.

The Sedona example also demonstrates why a `Field` matters alongside an array. Geometry WKB remains binary at the storage level, while compatible GeoArrow metadata gives it logical meaning. Metadata must survive the supported expression and shuffle paths without being copied onto incompatible ordinary binary or geography values. Correct bytes with an incorrect field are still an incorrect result.

Arrow interoperability does not prove resource admission, spillability or a shared allocator across wheels. The branch tests native lease ownership separately from RSS and records wheel/platform identity. See Chapter 13 for the package boundary and Chapter 17 for retained-output and same-worker quota-reuse tests.

5.23 Takeaways

Apache Arrow is Sail's data plane. The most important concrete type is `RecordBatch`, and the most important execution abstraction is `SendableRecordBatchStream`.

Spark Connect wraps Arrow IPC bytes in Protobuf responses. Flight SQL wraps Arrow streams in the Flight protocol. PySpark UDFs cross the Rust/Python boundary through PyArrow. Distributed shuffle partitions and merges Arrow batch streams. Extension types use Arrow metadata to carry domain-specific logical meaning without abandoning Arrow compatibility.

Once you see Sail as a system that plans Spark-compatible queries but executes Arrow-native streams, the architecture becomes much easier to reason about.

6 Apache DataFusion

DataFusion is Sail's execution kernel.

Spark Connect gives Sail the client protocol. PySpark gives Sail a familiar Python API. Arrow gives Sail the in-memory format. DataFusion gives Sail the query engine: logical plans, expressions, optimizers, physical planning, partitioned execution, vectorized functions, file format integration, and `RecordBatch` streams.

Sail is not a thin wrapper around DataFusion, though. Sail is a Spark-compatible system built on DataFusion. That distinction matters. Spark compatibility requires Spark-shaped plans, Spark function semantics, Spark catalog behavior, Spark error behavior, Spark UDF behavior, Spark streaming concepts, and a distributed execution model. DataFusion supplies the engine primitives; Sail supplies the Spark interpretation and the distributed control plane.

The question for this chapter is:

How does Sail turn Spark-compatible plans into DataFusion execution without losing Spark semantics?

6.1 Definitive References

Keep these references nearby:

- DataFusion introduction
- DataFusion library user guide
- Building logical plans
- Using the DataFrame API
- Adding user-defined functions
- Extending operators
- Query optimizer
- ExecutionPlan API docs
- QueryPlanner API docs

The DataFusion introduction describes DataFusion as an extensible Rust query engine using Arrow's in-memory format. That one sentence captures why Sail can exist: DataFusion is fast enough and flexible enough to serve as the execution core for a Spark-compatible system.

Sail currently depends on DataFusion 53.1.0 in the workspace `Cargo.toml`. The exact APIs will evolve, but the architectural ideas in this chapter are stable: session state, logical plans, optimizer rules, physical plans, extension planners, and Arrow batch streams.

6.2 Where DataFusion Lives In Sail

The main files for this chapter are:

Area	Files	DataFusion role
Plan resolution and execution	<code>crates/sail-plan/src/lib.rs</code>	Converts Sail specs to DataFusion logical plans, optimizes them, and creates physical plans
Session construction	<code>crates/sail-session/src/session_factory/server.rs</code>	Builds <code>SessionConfig</code> , <code>SessionState</code> , runtime env, rules, query planner, and Sail session extensions
Custom query planner	<code>crates/sail-session/src/planner.rs</code>	Installs extension physical planners and maps Sail logical extension nodes to physical operators
Session extension helper	<code>crates/sail-common-datafusion/src/extension.rs</code>	Provides typed access to DataFusion session extensions from <code>SessionContext</code> , <code>SessionState</code> , and <code>TaskContext</code>
Logical extension nodes	<code>crates/sail-logical-plan/src/*.rs</code>	Defines Sail-specific logical nodes such as range, repartition, barrier, map partitions, and streaming nodes
Physical extension nodes	<code>crates/sail-physical-plan/src/*.rs</code>	Implements custom <code>ExecutionPlan</code> nodes
Physical optimizer rules	<code>crates/sail-physical-optimizer/src/*.rs</code>	Adds or adjusts DataFusion physical optimization behavior
Spark function mapping	<code>crates/sail-plan/src/function/*.rs</code>	Maps Spark function names and semantics to DataFusion expressions and UDFs
Function kernels	<code>crates/sail-function/src/*.rs</code>	Implements vectorized DataFusion UDFs for Spark-compatible functions
Table functions	<code>crates/sail-plan/src/function/table/range.rs</code>	Implements DataFusion <code>TableFunctionImpl</code> and <code>TableProvider</code> patterns

The short version:

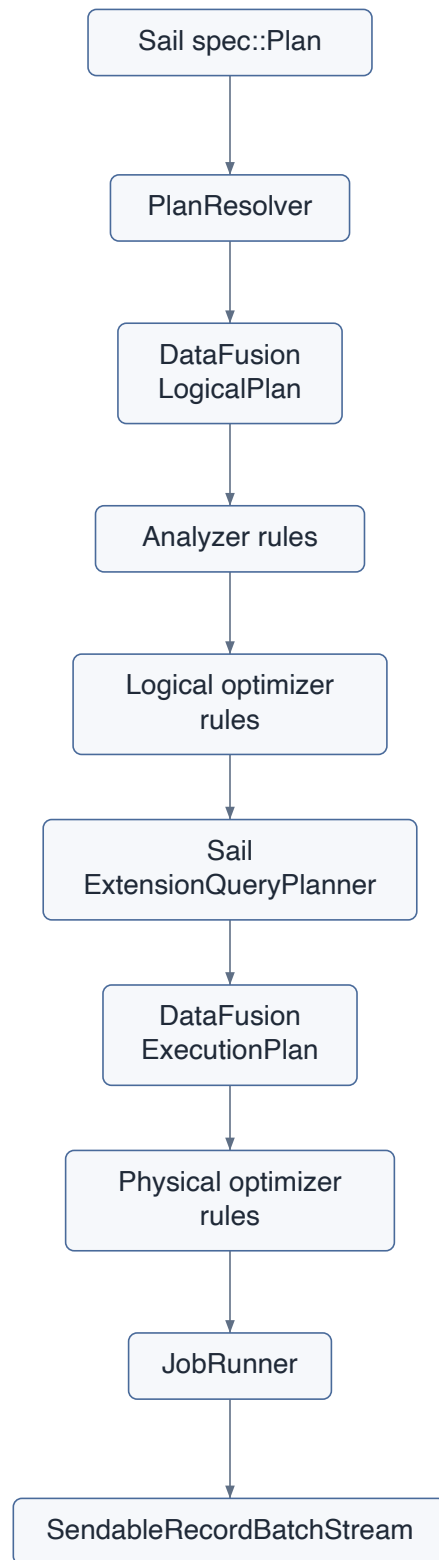


Figure 26: Flowchart 06.1

DataFusion owns the core plan abstractions in the middle. Sail owns the translation into those abstractions and the custom nodes around the edges.

6.3 The Critical Function: `resolve_and_execute_plan`

The most compact tour of Sail's DataFusion integration is `crates/sail-plan/src/lib.rs`.

The key function is:

```
pub async fn resolve_and_execute_plan(
    ctx: &SessionContext,
    config: Arc<PlanConfig>,
    plan: spec::Plan,
) -> PlanResult<(Arc<dyn ExecutionPlan>, Vec<StringifiedPlan>)>
```

This is the bridge from Sail's Spark-compatible plan spec to a DataFusion physical plan. It performs these steps:

1. Create a `PlanResolver`.
2. Resolve the Sail `spec::Plan` into a named DataFusion `LogicalPlan`.
3. Store the initial logical plan for explain output.
4. Ask the `SessionContext` to execute the logical plan into a `DataFrame`.
5. Pull the `SessionState` and logical plan back out of the `DataFrame`.
6. Run DataFusion logical optimization.
7. Rewrite streaming plans if necessary.
8. Use the session's query planner to create a physical plan.
9. Rename physical output fields if Spark field names require it.
10. Store the final physical plan for explain output.

In code:

```
let resolver = PlanResolver::new(ctx, config);
let NamedPlan { plan, fields } = resolver.resolve_named_plan(plan).await?;
let df = execute_logical_plan(ctx, plan).await?;
let (session_state, plan) = df.into_parts();
let plan = session_state.optimize(&plan)?;
let plan = session_state
    .query_planner()
    .create_physical_plan(&plan, &session_state)
    .await?;
```

This is a beautiful little funnel:

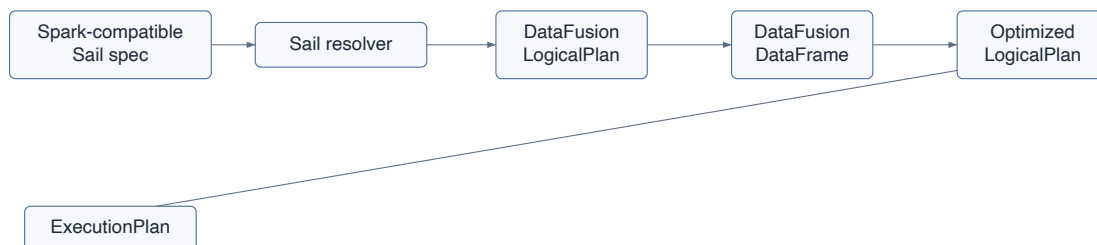


Figure 27: Flowchart 06.2

The `DataFrame` step may look surprising. Sail already has a logical plan, so why ask the context to execute it into a `DataFrame` and then split it apart? Because `SessionContext::execute_logical_plan` lets DataFusion apply its normal session machinery: analyzer behavior, catalog/function binding, and context state. Sail then takes back the plan and drives the rest of the pipeline.

6.4 SessionContext, SessionState, And SessionConfig

DataFusion’s session types are the engine’s ambient context:

- `SessionConfig` stores configuration options and extension objects.
- `SessionState` stores the config, runtime env, optimizer rules, function registries, catalog state, and query planner.
- `SessionContext` is the user-facing handle around session state.
- `TaskContext` is the execution-time context available to physical operators.

Sail constructs these deliberately in `crates/sail-session/src/session_factory/server.rs`.

The server session factory creates a `SessionContext` like this:

```
fn create(&mut self, mut info: ServerSessionInfo) -> Result<SessionContext> {
    let state = self.create_session_state(&mut info)?;
    let context = SessionContext::new_with_state(state);
    context.state_ref().write().register_udaf(first_value_udaf());
    Ok(context)
}
```

The registration of `first_value_udaf` is a telling detail. Sail does not simply enable all DataFusion defaults. It builds a session with selected behavior and then patches in assumptions required by its chosen optimizer rules.

The session config contains Sail services as DataFusion extensions:

```
SessionConfig::new()
    .with_create_default_catalog_and_schema(false)
    .with_information_schema(false)
    .with_extension(create_data_source_registry())
    .with_extension(Arc::new(create_catalog_manager(...)))
    .with_extension(Arc::new(ActivityTracker::new()))
    .with_extension(Arc::new(JobService::new(job_runner)))
```

```

.with_extension(Arc::new(RemoteCheckpointRegistry::new(
    self.config.execution.checkpoint.path.clone(),
    info.session_id.clone(),
)))
.with_extension(Arc::new(RepartitionBufferConfig::new(...)))
.with_extension(Arc::new(self.create_system_table_service(info)?))
.with_extension(Arc::new(DeltaTableCache::default()));

```

That list is a capsule summary of Sail's architecture:

- Sail manages catalogs itself.
- Sail registers data sources and their optional lake capabilities.
- Sail tracks session activity.
- Sail installs a job service and a session-scoped remote checkpoint registry.
- Sail controls repartition buffering.
- Sail exposes system tables.
- Sail caches Delta table state.

DataFusion supplies the typed extension slot. Sail uses it as session-local dependency injection.

The experimental branch adds package registration in `create_session_state`, after creating the runtime environment and before constructing the state builder. The runtime creation also receives the session resource domain. This ordering lets extension bindings use the runtime and participate in the session's native resource accounting. Upstream's ordinary session factory does not contain that package-registration call; merged #2630 provides factory selection at the Spark server boundary. Keep those two changes distinct when reviewing adoption.

6.5 Typed Session Extensions

The helper in `crates/sail-common-datafusion/src/extension.rs` makes DataFusion extensions feel typed:

```

pub trait SessionExtension: Send + Sync + 'static {
    fn name() -> &'static str;
}

pub trait SessionExtensionAccessor {
    fn extension<T: SessionExtension>(&self) -> Result<Arc<T>>;
    fn runtime_env(&self) -> Arc<RuntimeEnv>;
}

```

Sail implements `SessionExtensionAccessor` for:

- `SessionContext`
- `SessionState`
- `&dyn Session`
- `TaskContext`

That matters because different parts of the engine are at different layers:

- Planning code often has `SessionContext` or `SessionState`.

- Table providers receive `&dyn Session`.
- Physical operators receive `TaskContext`.

The same pattern works everywhere:

```
let service = ctx.extension::<JobService>()?;
let registry = session_state.extension::<DataSourceRegistry>()?;
let config = task_context.extension::<RepartitionBufferConfig>()?;
```

This is one of Sail's cleanest Rust patterns. DataFusion gives an untyped extension store; Sail wraps it with a trait that produces a useful error and a typed `Arc<T>`.

For the extension proposal later in the book, this pattern is a key precedent. Third-party integrations will need a way to store per-session services and configuration. Sail already has the mechanism; the open question is how to make registration public, ordered, discoverable, and distributed-safe.

6.6 Sail's Query Planner

DataFusion lets a session install a custom query planner. Sail does this in `ServerSessionFactory::create_session_state`:

```
let builder = SessionStateBuilder::new()
    .with_config(config)
    .with_runtime_env(runtime)
    .with_analyzer_rules(default_analyzer_rules())
    .with_optimizer_rules(default_optimizer_rules())
    .with_physical_optimizer_rules(get_physical_optimizers(...))
    .with_query_planner(new_query_planner());
```

`new_query_planner` returns `ExtensionQueryPlanner` from `crates/sail-session/src/planner.rs`.

The current `QueryPlanner::create_physical_plan` receives a logical plan and session: `&dyn Session`. Its sequence is:

1. Apply the Delta and Iceberg metadata aggregate logical rewriters.
2. Construct `DefaultPhysicalPlanner` with Delta, Iceberg, system-table, listing, console, noop, Python and Sail extension planners, in that order.
3. Create the physical plan with DataFusion.
4. Apply `ensure_scalar_subquery_nullability` to the resulting plan.

DataFusion handles ordinary projections, filters, aggregates, joins, sorts and scans. The extension planners translate custom logical nodes that the default planner cannot handle. Listing and Python writes are part of this chain, not just lakehouse-specific operations.

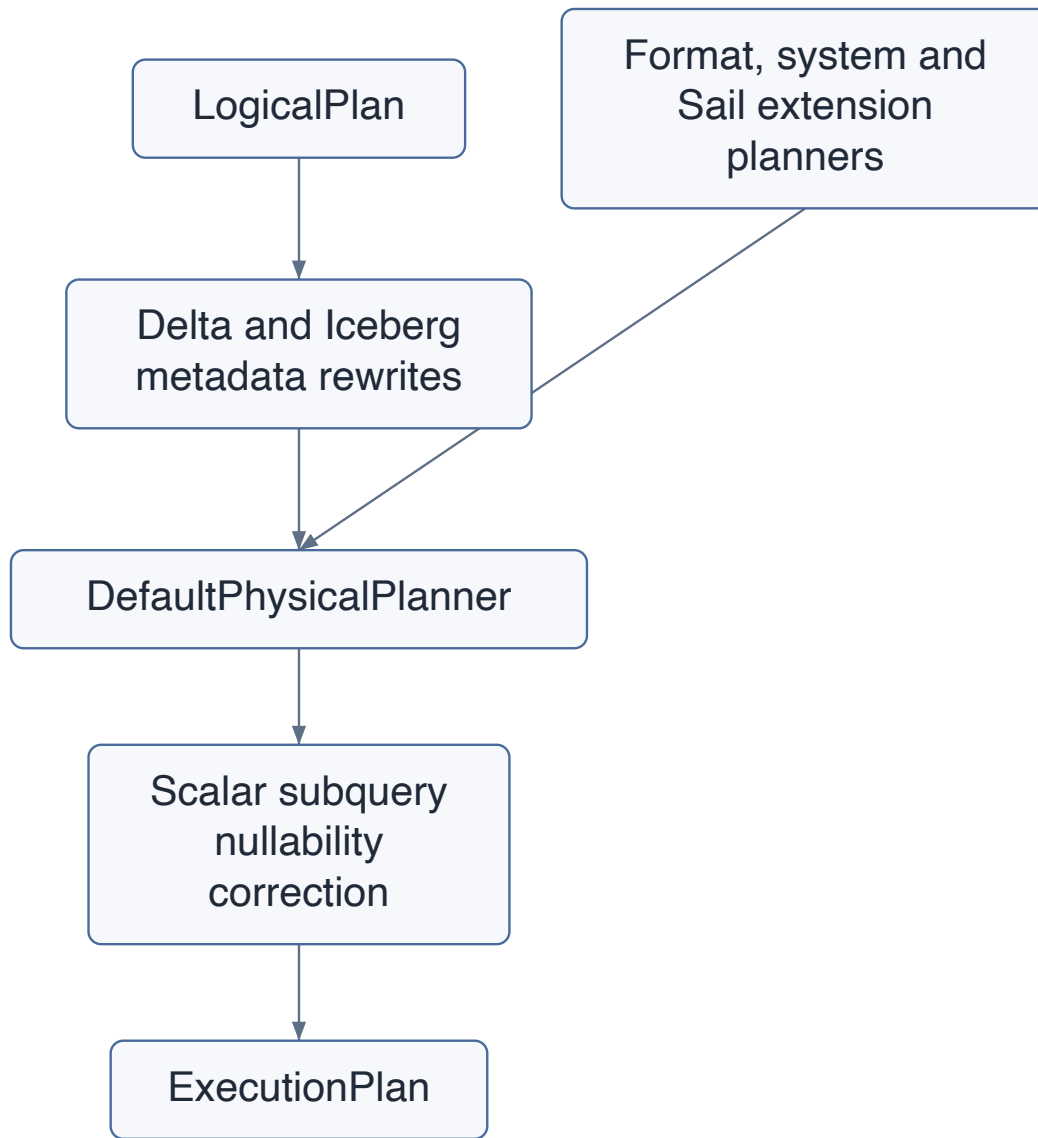


Figure 28: Flowchart 06.3

This planner list remains explicit host wiring. The packaged extension contract in chapter 13 does not expose an arbitrary optimizer/planner registration API. Its relation bindings return supported native objects or compose existing operators; adding another public hook needs its own use case and compatibility contract.

6.7 Logical Extension Nodes

DataFusion has built-in logical plan nodes, but it also supports user-defined logical nodes. Sail uses those for Spark concepts that do not map directly to a single built-in DataFusion logical plan node.

RangeNode in `crates/sail-logical-plan/src/range.rs` is the friendliest example:

```
pub struct RangeNode {
    range: Range,
    num_partitions: usize,
    schema: DFSchemaRef,
}
```

It implements `UserDefinedLogicalNodeCore`:

```
impl UserDefinedLogicalNodeCore for RangeNode {
    fn name(&self) -> &str {
        "Range"
    }

    fn inputs(&self) -> Vec<&LogicalPlan> {
        vec![]
    }

    fn schema(&self) -> &DFSchemaRef {
        &self.schema
    }
}
```

The node carries:

- Spark range parameters,
- target partition count,
- and a DataFusion schema.

It is logical because it says what should happen, not yet how to produce the Arrow batches.

Other Sail logical extension nodes include:

- `ExplicitRepartitionNode`
- `BarrierNode`
- `MapPartitionsNode`
- `ShowStringNode`
- `SchemaPivotNode`
- `SortWithinPartitionsNode`
- `SparkPartitionIdNode`
- streaming source/filter/limit/collector nodes
- file write/delete and row-level write nodes

These nodes are Spark compatibility pressure made visible. When Spark semantics fit DataFusion's built-in logical plan, Sail uses DataFusion's built-in logical plan. When they do not, Sail introduces a logical extension node and teaches the physical planner what to do with it.

6.8 Physical Extension Planning

ExtensionPhysicalPlanner is the function table from Sail logical extension nodes to physical execution nodes.

For RangeNode, it builds RangeExec:

```
if let Some(node) = node.as_any().downcast_ref::<RangeNode>() {
    let schema = UserDefinedLogicalNode::schema(node).inner().clone();
    let projection = (0..schema.fields().len()).collect();
    Arc::new(RangeExec::try_new(
        node.range().clone(),
        node.num_partitions(),
        schema,
        projection,
    )?)
}
```

For MapPartitionsNode, it uses the physical input and wraps it in MapPartitionsExec:

```
let [input] = physical_inputs else {
    return internal_err!("MapPartitionsExec requires exactly one physical input");
};
Arc::new(MapPartitionsExec::new(
    input.clone(),
    node.udf().clone(),
    UserDefinedLogicalNode::schema(node).inner().clone(),
))
```

For SortWithinPartitionsNode, it does not need a custom physical operator. It uses DataFusion's SortExec with preserve_partitioning:

```
let sort = SortExec::new(ordering, input.clone())
    .with_fetch(node.fetch())
    .with_preserve_partitioning(true);
Arc::new(sort)
```

This is the architectural sweet spot:

- Use custom logical nodes to preserve Spark intent.
- Use built-in DataFusion physical operators whenever they already match.
- Use custom Sail physical operators only when necessary.

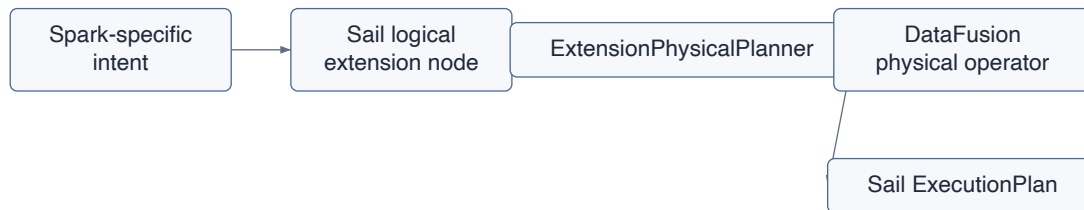


Figure 29: Flowchart 06.4

The result is less code than a from-scratch engine, but more semantic control than a simple SQL translation layer.

6.9 Physical Plans And ExecutionPlan

DataFusion physical operators implement `ExecutionPlan`. The most important method is:

```

fn execute(
    &self,
    partition: usize,
    context: Arc<TaskContext>,
) -> Result<SendableRecordBatchStream>
  
```

You saw this in Chapter 5, but now we can place it in DataFusion’s architecture. An `ExecutionPlan` describes a partitioned physical computation. It knows its schema, children, properties, partitioning, boundedness, and how to execute one partition.

Sail custom physical nodes follow the same contract. `RangeExec`:

1. Checks the partition number.
2. Computes a partition-specific range.
3. Builds Arrow arrays and batches.
4. Returns a `RecordBatchStreamAdapter`.

`ExplicitRepartitionExec`:

1. Wraps an input plan.
2. Creates output channels per target partition.
3. Executes all input partitions cooperatively.
4. Sends partitioned `RecordBatch` values to receivers.
5. Returns a stream for the requested output partition.

`ShuffleWriteExec` and `ShuffleReadExec` in `sail-execution` also implement the same contract, which is why the distributed runtime can insert them into a DataFusion physical plan. DataFusion’s abstraction is local and partitioned; Sail’s distributed planner can split it into stages and reconnect it with shuffle nodes.

6.10 Plan Properties: Partitioning, Boundedness, Emission

DataFusion physical plans carry `PlanProperties`. Sail uses those properties carefully because distributed execution depends on them.

For example, RangeExec creates:

```
PlanProperties::new(
  EquivalenceProperties::new(projected_schema.clone()),
  Partitioning::RoundRobinBatch(num_partitions),
  EmissionType::Both,
  Boundedness::Bounded,
)
```

That tells the optimizer and scheduler:

- the output schema’s equivalence properties,
- the number and kind of output partitions,
- whether the operator emits incrementally or finally,
- and whether it is bounded.

ShuffleWriteExec uses different properties. It returns an empty stream from execute, but the side effect is writing shuffle data. Its properties reflect the input partition count rather than the shuffle output count, because each input partition execution writes many shuffle output streams.

These details matter. A distributed engine cannot safely insert exchanges, coalesce partitions, re-order joins, or execute streaming plans unless physical operators accurately describe themselves.

6.11 Logical Optimizers

Sail installs analyzer and optimizer rules in `crates/sail-session/src/optimizer.rs`:

```
pub fn default_analyzer_rules() -> Vec<Arc<dyn AnalyzerRule + Send + Sync>> {
  sail_logical_optimizer::default_analyzer_rules()
}

pub fn default_optimizer_rules() -> Vec<Arc<dyn OptimizerRule + Send + Sync>> {
  let rules = sail_logical_optimizer::default_optimizer_rules();
  rules
    .into_iter()
    .filter(|r| r.name() != "push_down_leaf_projections")
    .collect()
}
```

Sail delegates rule construction to `sail_logical_optimizer`, then removes leaf projection push-down by name. The session-level test checks that exclusion. Lakehouse-specific metadata rewrites also appear in the query planner described above; they are not supplied here through the earlier edition’s `sail_plan_lakehouse::lakehouse_optimizer_rules` helper.

This is an important DataFusion lesson: optimizer rule order is part of engine semantics. For extensions, “add my optimizer rule” is not sufficient. The API must answer where it runs, what it can assume, and which rules it must precede or follow.

6.12 Physical Optimizers

Sail also customizes physical optimization in `crates/sail-physical-optimizer/src/lib.rs`.

The rule list preserves the relative order of DataFusion's standard rules and inserts Sail-specific behavior at selected points. Optional JoinReorder runs before JoinSelection; upstream #2676 adds SelectSemiJoinBuildSide immediately after it. The extension snapshot predates that latter rule. Near the end of the pipeline, Sail applies:

```
rules.push(Arc::new(RewriteExplicitRepartition::new()));
rules.push(Arc::new(RewriteCollectLeftHashJoin::new()));
rules.push(Arc::new(EnforceBarrierPartitioning::new()));
rules.push(Arc::new(SanityCheckPlan::new()));
```

The test compares Sail's rule order with DataFusion's default physical optimizer to ensure Sail does not accidentally reorder DataFusion defaults.

RewriteExplicitRepartition is a good example. During physical planning, Sail creates an ExplicitRepartitionExec placeholder. Later, the physical optimizer rewrites it:

- hash partitioning -> DataFusion RepartitionExec
- unknown partitioning to one partition -> CoalescePartitionsExec
- unknown partitioning to fewer partitions -> Sail CoalesceExec
- unknown partitioning to at least the input partition count -> remove the node
- round-robin -> keep Sail's explicit repartition node

Why not decide all of that immediately in the physical planner? Because the optimizer sees the larger physical plan and can make a more context-aware choice. This keeps planning declarative and lets rewrite rules simplify the physical plan after DataFusion has done its own work.

EnforceBarrierPartitioning is another distributed-execution hint. It rewrites BarrierExec preconditions so all precondition partitions complete before the actual plan begins. That behavior matters for Spark-like commands where one operation must finish globally before another starts.

6.13 Function Mapping: Spark Names To DataFusion Expressions

Spark has a large function surface. DataFusion has its own function surface. Sail sits in between.

The function registry in `crates/sail-plan/src/function/mod.rs` collects built-in scalar, generator, aggregate, table, and window functions:

```
lazy_static! {
    pub static ref BUILT_IN_SCALAR_FUNCTIONS: HashMap<'static str,
ScalarFunction> =
        HashMap::from_iter(scalar::list_built_in_scalar_functions());
    pub static ref BUILT_IN_GENERATOR_FUNCTIONS: HashMap<'static str,
ScalarFunction> =
        HashMap::from_iter(generator::list_built_in_generator_functions());
    pub static ref BUILT_IN_TABLE_FUNCTIONS: HashMap<'static str,
Arc<TableFunction>> =
        HashMap::from_iter(table::list_built_in_table_functions());
}
```

The type alias for scalar planning functions is:

```
pub(crate) type ScalarFunction =
    Arc<dyn Fn(ScalarFunctionInput) -> PlanResult<expr::Expr> + Send + Sync>;
```

This means Sail’s “function registry” is not only a map from name to UDF. It is a map from Spark function name to a small planning function that can:

- validate argument count,
- inspect Spark planning config,
- use the current schema,
- rewrite arguments,
- call a DataFusion built-in,
- call a Sail UDF,
- or construct a larger expression tree.

For simple cases, `ScalarFunctionBuilder::udf` wraps a `ScalarUDFImpl`:

```
pub fn udf<F>(f: F) -> ScalarFunction
where
    F: ScalarUDFImpl + Send + Sync + 'static,
{
    let func = ScalarUDF::from(f);
    Arc::new(move |input| Ok(func.call(input.arguments)))
}
```

For more complex Spark functions, Sail uses custom builders that produce DataFusion expressions directly. This planning-time layer is one of the main places Spark semantics are preserved.

6.14 Vectorized UDFs

The DataFusion UDF documentation explains that scalar UDFs are vectorized: they receive Arrow arrays and return Arrow arrays. Sail’s function kernels follow that model.

`SparkMask` in `crates/sail-function/src/scalar/string/spark_mask.rs` implements `ScalarUDFImpl`:

```
impl ScalarUDFImpl for SparkMask {
    fn name(&self) -> &str {
        "spark_mask"
    }

    fn signature(&self) -> &Signature {
        &self.signature
    }

    fn return_type(&self, arg_types: &[DataType]) -> Result<DataType> {
        ...
    }

    fn invoke_with_args(&self, args: ScalarFunctionArgs) -> Result<ColumnarValue> {
        ...
    }
}
```

```
    }
}
```

The return type function is a planning/type-checking hook. The `invoke_with_args` function is the execution hook. It receives `ColumnarValue` arguments, which may be scalar values or Arrow arrays.

The `Explode` function in `crates/sail-function/src/scalar/explode.rs` shows a different pattern:

```
fn invoke_with_args(&self, _: ScalarFunctionArgs) -> Result<ColumnarValue> {
    plan_err!(
        "{} should be rewritten during logical plan analysis",
        self.name()
    )
}
```

`explode` is represented like a function for analysis, but it should not execute as a normal scalar UDF. It must be rewritten into a plan shape that can expand rows. This is a good example of Spark semantics not fitting a plain expression kernel.

6.15 Table Functions And Providers

DataFusion table functions return `TableProvider` objects. Sail's range table function in `crates/sail-plan/src/function/table/range.rs` is a small complete example.

`RangeTableFunction` implements `TableFunctionImpl`:

```
impl TableFunctionImpl for RangeTableFunction {
    fn call(&self, args: &[Expr]) -> Result<Arc<dyn TableProvider>> {
        ...
        let node = RangeNode::try_new("id".to_string(), start, end, step,
num_partitions)?;
        Ok(Arc::new(RangeTableProvider {
            node: Arc::new(node),
        }))
    }
}
```

The provider exposes both a logical and physical path:

```
fn get_logical_plan(&self) -> Option<Cow<'_, LogicalPlan>> {
    Some(Cow::Owned(LogicalPlan::Extension(
        logical_plan::Extension {
            node: self.node.clone(),
        },
    )))
}

async fn scan(...) -> Result<Arc<dyn ExecutionPlan>> {
    Ok(Arc::new(RangeExec::try_new(...)?))
}
```

This is a useful pattern for extension authors:

- If the planner wants a logical plan, provide a logical extension node.
- If DataFusion asks for a scan directly, provide an `ExecutionPlan`.
- Keep the shared semantic payload in a small node object.

Range is simple, but the shape generalizes to custom table-valued functions, specialized data sources, and extension-backed virtual tables.

6.16 DataFusion As Local Engine, Sail As Distributed Engine

DataFusion's `ExecutionPlan` API is partitioned and asynchronous, but DataFusion itself is not Sail's whole distributed runtime. Sail builds a distributed layer around DataFusion physical plans.

The boundary is visible in `ServerSessionFactory::create_job_runner`:

```
let job_runner: Box<dyn JobRunner> = match self.config.mode {  
    ExecutionMode::Local => Box::new(LocalJobRunner::new()),  
    ExecutionMode::LocalCluster => Box::new(ClusterJobRunner::new(...)),  
    ExecutionMode::KubernetesCluster => Box::new(ClusterJobRunner::new(...)),  
};
```

`JobService` is placed in `SessionConfig` as an extension. Later, Spark Connect and Flight SQL can retrieve it from the session and execute a physical plan.

This separation is one of Sail's strongest design choices:

- DataFusion plans and executes partitioned operators.
- Sail decides whether those operators run locally or as a distributed job.
- The same logical and physical plan pipeline feeds both modes.

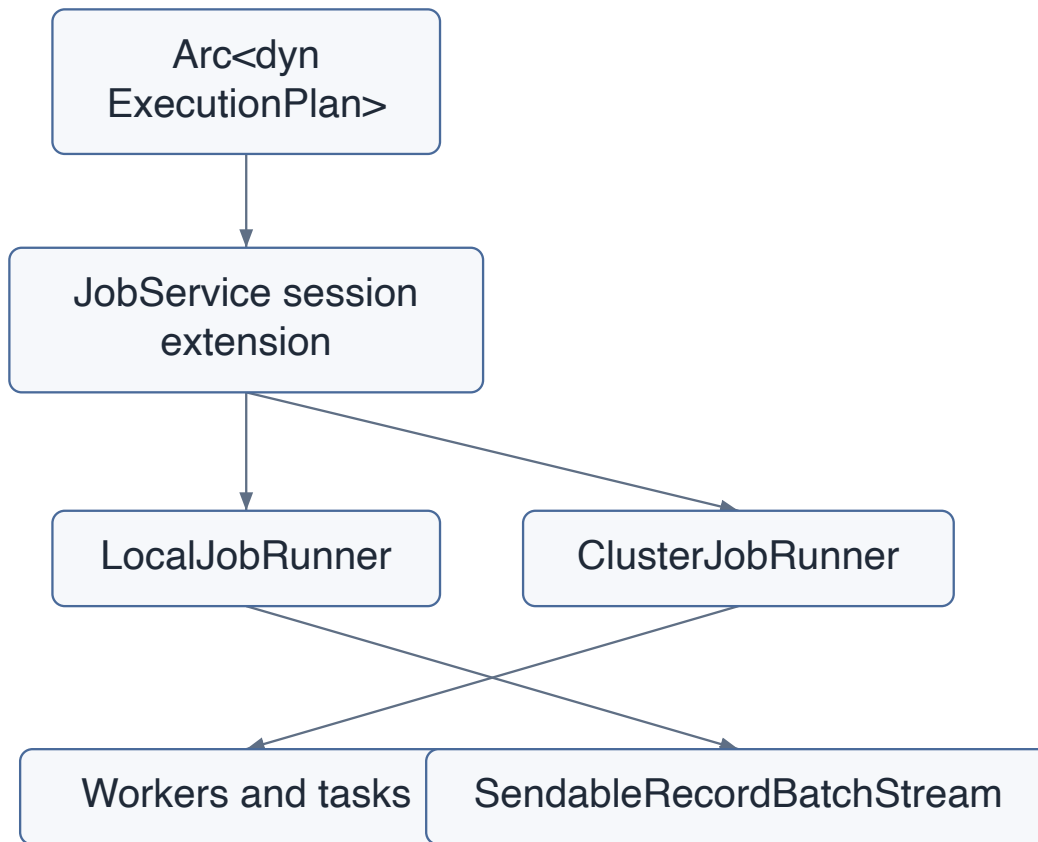


Figure 30: Flowchart 06.5

The next chapters will zoom into that distributed layer. For now, remember that DataFusion’s physical plan is the unit Sail distributes.

6.17 Spark Semantics On A DataFusion Kernel

Sail’s DataFusion integration repeatedly follows this pattern:

1. Resolve Spark/Sail input into a DataFusion-compatible plan.
2. Preserve Spark-only semantics in extension nodes or metadata.
3. Let DataFusion optimize and plan what it understands.
4. Intercept extension nodes with custom planners.
5. Implement missing execution behavior with custom `ExecutionPlan` nodes.
6. Return Arrow batch streams to the protocol layer.

Some examples:

Spark/Sail concept	DataFusion integration
Spark range	RangeNode -> RangeExec
Spark repartition/coalesce	ExplicitRepartitionNode -> ExplicitRepartitionExec -> physical optimizer rewrite
Sort within partitions	logical extension -> DataFusion SortExec with preserved partitioning
Spark built-in functions	planning registry -> DataFusion expressions or Sail ScalarUDFImpl
Spark generator functions	analyzed as functions, then rewritten into plan structure
Catalog commands	logical command nodes -> CatalogCommandExec
Lakehouse row-level operations	lakehouse optimizer rules before generic optimizer rules
System tables	system table extension planner and TableProvider

That table is the heart of the chapter. Sail is not asking DataFusion to become Spark. Sail is using DataFusion as a powerful substrate and adding Spark compatibility at well-defined seams.

6.18 Extension Implications

Sail already uses typed session extensions, function maps, logical and physical optimizer rules, DataFusion extension planners, custom plan nodes, table providers and physical codecs. These internal seams are not all public package APIs.

The implemented package boundary in chapter 13 exposes relation bindings and native scalar objects, with manifest validation and physical reconstruction. Sedona exercises scalar functions and Arrow geometry field semantics. Pecan expresses graph computation through existing relational operators; Grenada executes DataFusion plans inside the extension; Banda uses native graph kernels; Argentea distributes retained native graph partitions through Sail's existing execution machinery.

Those paths test different requirements. Registering a scalar function does not prove support for a packaged aggregate, spatial-join planner or optimizer rule. Similarly, a native plan's opaque internals are not automatically visible to DataFusion's relational optimizer. Keep each added host contract tied to the behavior it enables and test planning, worker decoding and execution separately.

6.19 Reading Exercise: Trace range

Follow Spark's range from function to execution:

1. Start in `crates/sail-plan/src/function/table/mod.rs`.
2. See `range` registered as a built-in table function.
3. Open `crates/sail-plan/src/function/table/range.rs`.
4. Read `RangeTableFunction::call`.
5. Follow `RangeNode::try_new` in `crates/sail-logical-plan/src/range.rs`.
6. Jump to `ExtensionPhysicalPlanner` in `crates/sail-session/src/planner.rs`.
7. Find the `RangeNode` downcast.
8. Follow `RangeExec::try_new` and `RangeExec::execute`.
9. Observe the final `SendableRecordBatchStream`.

You have now traced a Spark-compatible table function through DataFusion's logical and physical extension APIs.

6.20 Reading Exercise: Trace repartition

Follow explicit repartitioning:

1. Start in `crates/sail-logical-plan/src/repartition.rs`.
2. Find `ExplicitRepartitionNode`.
3. Jump to `ExtensionPhysicalPlanner`.
4. Find the `ExplicitRepartitionNode` case.
5. Read `plan_explicit_partitioning`.
6. Open `crates/sail-physical-plan/src/repartition.rs`.
7. Read `ExplicitRepartitionExec`.
8. Open `crates/sail-physical-optimizer/src/explicit_repartition.rs`.
9. Observe how the placeholder is rewritten to DataFusion or Sail physical operators depending on partitioning.

This trace teaches the difference between preserving user intent and choosing the final execution shape.

6.21 Reading Exercise: Trace A Spark Function

Pick a function such as `mask`:

1. Start in `crates/sail-plan/src/function/scalar/mod.rs`.
2. Find where string functions are added.
3. Open `crates/sail-plan/src/function/scalar/string.rs`.
4. Find the mapping to `SparkMask`.
5. Open `crates/sail-function/src/scalar/string/spark_mask.rs`.
6. Read `return_type` and `invoke_with_args`.
7. Notice how planning-time Spark semantics and execution-time Arrow kernels live in different crates.

This is the pattern most third-party scalar functions will want to follow.

6.22 Takeaways

DataFusion gives Sail its query engine, but Sail decides how Spark semantics enter and leave that engine.

The central pipeline is: Sail spec -> DataFusion logical plan -> logical optimization -> Sail/DataFusion physical planning -> physical optimization -> ExecutionPlan -> Arrow RecordBatch stream.

Sail customizes DataFusion through session extensions, analyzer and optimizer rules, function planning registries, extension logical nodes, extension physical planners, custom physical operators, and physical optimizer rules. Those are the same seams the extension architecture must eventually expose to third-party packages.

The next chapter moves from “how does Sail get a physical plan?” to “how does Sail split that physical plan into a distributed job graph?”

7 From Physical Plan To Job Graph

The previous chapter ended with a DataFusion physical plan:

```
Arc<dyn ExecutionPlan>
```

That is enough for local execution. DataFusion can call `execute(partition, task_context)` on the plan and return a stream of Arrow RecordBatch values.

But a distributed engine needs one more transformation. It has to decide:

- which parts of the physical plan can run together,
- where data has to be materialized,
- where shuffle boundaries exist,
- how partitions map to tasks,
- which stage outputs are consumed by which later stages,
- and which tasks run on workers versus the driver.

In Sail, that transformation is the job graph.

This chapter follows the path:

```
DataFusion ExecutionPlan -> Sail JobGraph -> JobTopology -> task regions -> task definitions
```

The job graph is where DataFusion’s local, partitioned execution model becomes Sail’s distributed execution model.

7.1 The Core Files

The main files for this chapter are:

Area	Files	Role
Job graph data model	<code>crates/sail-execution/src/job_graph/mod.rs</code>	Defines jobs, stages, inputs, distributions, placement, and input modes
Job graph planner	<code>crates/sail-execution/src/job_graph/planner.rs</code>	Splits a DataFusion physical plan into stages
Job runner boundary	<code>crates/sail-execution/src/job_runner.rs</code>	Chooses local execution or cluster execution
Driver job acceptance	<code>crates/sail-execution/src/driver/job_scheduler/core.rs</code>	Builds the job graph and creates job output streams
Job topology	<code>crates/sail-execution/src/driver/job_scheduler/topology.rs</code>	Groups stages into task regions and dependencies
Job state	<code>crates/sail-execution/src/driver/job_scheduler/state.rs</code>	Tracks jobs, stages, tasks, attempts, and task states
Task definition	<code>crates/sail-execution/src/task/definition.rs</code>	Defines serialized worker task inputs and outputs
Stage input placeholder	<code>crates/sail-execution/src/plan/stage_input.rs</code>	Placeholder <code>ExecutionPlan</code> node for cross-stage inputs
Shuffle plans	<code>crates/sail-execution/src/plan/shuffle_write.rs</code> , <code>shuffle_read.rs</code>	Physical data movement at stage boundaries
Job output	<code>crates/sail-execution/src/driver/output.rs</code>	Merges final task output streams into the client-facing stream

The first file to read is `job_graph/mod.rs`. It defines the vocabulary.

7.2 Local Runner Versus Cluster Runner

Sail can execute the same DataFusion physical plan locally or through the cluster runtime.

`crates/sail-execution/src/job_runner.rs` has two implementations of `JobRunner`:

- `LocalJobRunner`
- `ClusterJobRunner`

The local runner is intentionally simple:

```
let plan = trace_execution_plan(plan, options)?;  
Ok(execute_stream(plan, ctx.task_ctx()?))
```

It hands the plan directly to DataFusion's `execute_stream`.

The cluster runner sends the plan to the driver actor:

```
self.driver  
  .send(DriverMessage::ExecuteJob {  
    plan,  
    context: ctx.task_ctx(),  
    result: tx,  
  })  
  .await?;
```

That event is handled by the driver:

```
let out = self.job_scheduler.accept_job(ctx, plan, context);  
if let Ok((job_id, _)) = &out {  
  self.refresh_job(ctx, *job_id);  
  self.run_tasks(ctx);  
  self.scale_up_workers(ctx);  
}
```

So the split is:

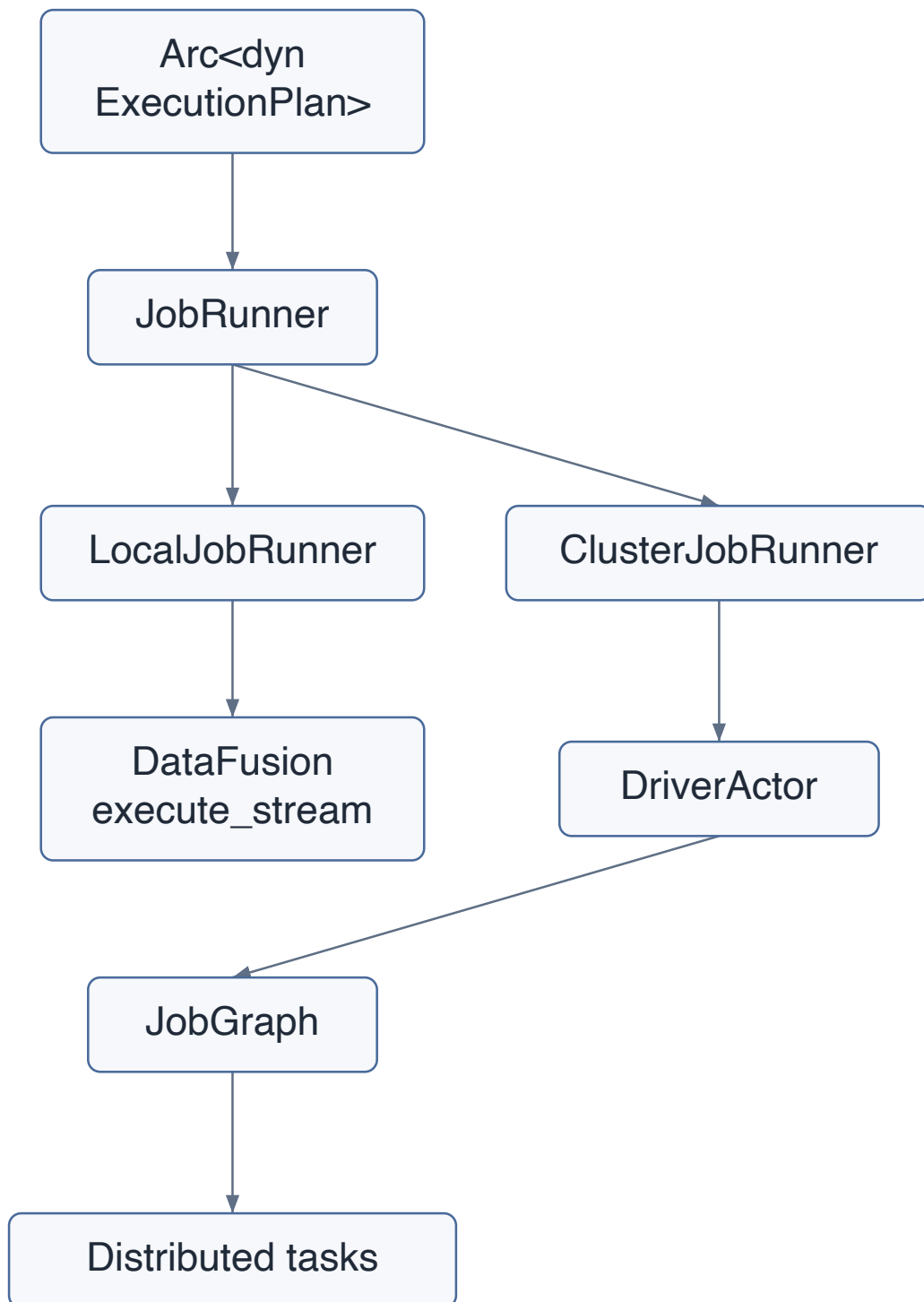


Figure 31: Flowchart 07.1

This chapter is about the cluster branch.

7.3 What A Job Graph Represents

JobGraph is defined in `crates/sail-execution/src/job_graph/mod.rs`:

```
pub struct JobGraph {  
    stages: Vec<Stage>,  
    schema: SchemaRef,  
}
```

The code comment gives the mental model:

- a job has stages,
- a stage has partitions,
- a task executes one partition of one stage,
- a task can have multiple attempts,
- each task produces output split into channels.

That last point is essential. A task does not merely produce one stream. It can produce multiple channels so downstream tasks can read:

- the same partition,
- all partitions,
- a shuffle channel from all partitions,
- all channels for broadcast,
- or a contiguous subset for rescale.

The Stage struct carries:

```
pub struct Stage {  
    pub inputs: Vec<StageInput>,  
    pub plan: Arc<dyn ExecutionPlan>,  
    pub group: String,  
    pub mode: OutputMode,  
    pub distribution: OutputDistribution,  
    pub placement: TaskPlacement,  
}
```

Each stage is still an `ExecutionPlan`. Sail does not compile to a separate mini-language for stages. Instead, it cuts a `DataFusion` physical plan into smaller `DataFusion` physical plans and links them with placeholders.

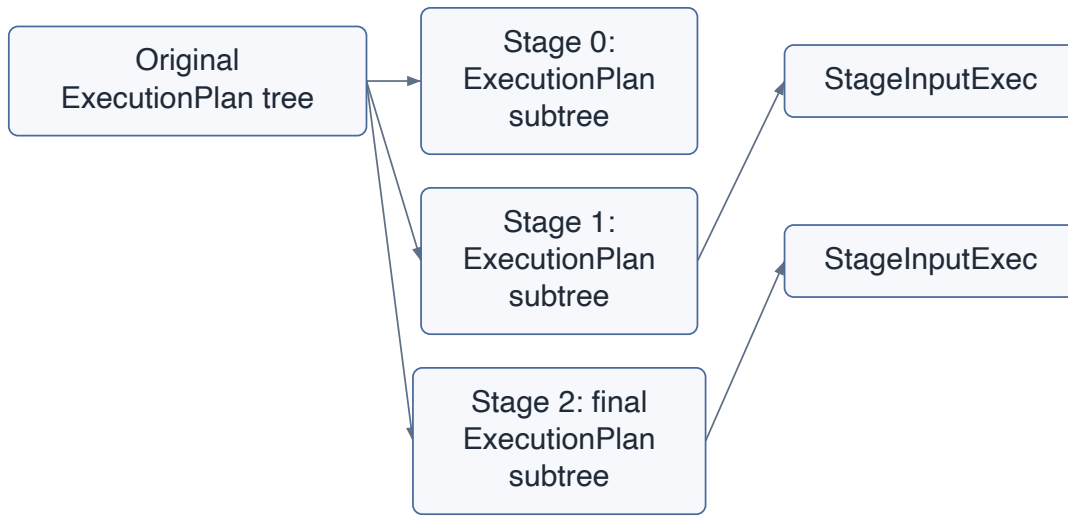


Figure 32: Flowchart 07.2

StageInputExec is the placeholder that marks “read another stage’s output here.”

7.4 StageInputExec: A Placeholder, Not An Operator

crates/sail-execution/src/plan/stage_input.rs defines StageInputExec<I>.

It implements DataFusion’s ExecutionPlan, but its execute method errors:

```

fn execute(
    &self,
    _partition: usize,
    _context: Arc<TaskContext>,
) -> Result<SendableRecordBatchStream> {
    internal_err!("{ } should be resolved before execution", self.name())
}

```

That is deliberate. StageInputExec is not meant to run as-is. It is a marker inserted during job graph construction. Later, when a worker task is prepared, the task runner resolves the placeholder into a real ShuffleReadExec or stream input.

The generic parameter I changes meaning during planning:

- StageInputExec<StageInput> records a logical dependency on another stage.
- StageInputExec<usize> records an index into the current stage’s input list.

The rewrite_inputs function performs that conversion:

```

if let Some(placeholder) =
node.as_any().downcast_ref::<StageInputExec<StageInput>>() {
    let index = inputs.len();
    inputs.push(placeholder.input().clone());
    let placeholder = StageInputExec::new(index,

```

```
placeholder.properties().clone());
    Ok(Transformed::yes(Arc::new(placeholder)))
}
```

That gives every stage:

- a plan containing numbered input placeholders,
- and a separate `inputs: Vec<StageInput>` that describes where those inputs come from.

This separation is neat. The plan stays serializable as a DataFusion physical plan, while stage dependencies stay explicit in the job graph.

7.5 Stage Inputs And Input Modes

StageInput has two fields:

```
pub struct StageInput {
    pub stage: usize,
    pub mode: InputMode,
}
```

InputMode is the distributed execution contract:

```
pub enum InputMode {
    Forward,
    Merge,
    Shuffle,
    Broadcast,
    Rescale,
}
```

The code comments are worth translating into a table:

Mode	Current partition reads	Used for
Forward	Same partition from the input stage, all channels	Pipelined one-to-one dependencies
Merge	All partitions from input stage, all channels	Sort-preserving merge and global merge-style inputs
Shuffle	Same output channel from all input partitions	Hash or round-robin repartition
Broadcast	All partitions and all channels	Shared build-side or repeated consumption
Rescale	A contiguous subset of input partitions, all channels	Coalescing from many partitions to fewer partitions

The scheduler later converts each mode into concrete `TaskInputKey` groups in `build_task_input_keys`.

For shuffle, the comment says it plainly:

```
// Enumerate channels in the outer loop and partitions in the inner loop.
// This is the whole point of shuffle!
```

If an upstream stage has P partitions and C channels, then shuffle input groups are:

```
output partition 0 reads: (input p0, channel 0), (input p1, channel 0), ...
output partition 1 reads: (input p0, channel 1), (input p1, channel 1), ...
...
```

That is the exchange pattern for repartitioning.

7.6 Output Distribution

A stage's output distribution describes how each task splits its output into channels:

```
pub enum OutputDistribution {
  Hash {
    keys: Vec<Arc<dyn PhysicalExpr>>,
    channels: usize,
  },
  RoundRobin {
    channels: usize,
  },
}
```

Hash means evaluate physical expressions on each row and send the row to the corresponding channel. RoundRobin means distribute rows across channels without hash keys.

This becomes a TaskOutputDistribution when the scheduler creates a task definition:

```
TaskOutputDistribution::Hash {
  keys,
  channels,
}
```

The hash keys are serialized physical expressions, which is one reason the extensions chapter will need to care about physical-plan codecs. If an extension introduces a custom physical expression, distributed workers must be able to deserialize it.

7.7 How JobGraph::try_new Starts

The entry point is:

```
pub fn try_new(plan: Arc<dyn ExecutionPlan>) -> ExecutionResult<Self>
```

The first two lines are important rewrites:

```
let plan = ensure_single_input_partition_for_global_limit(plan)?;
let plan =
  ensure_partitioned_hash_join_if_build_side_emits_unmatched_rows(plan)?;
```

Then Sail builds an empty graph and recursively splits the plan:

```
let mut graph = Self {
  stages: vec![],
```

```

        schema: plan.schema(),
    };
    let last = build_job_graph(plan, PartitionUsage::Once, &mut graph)?;
    let (last, inputs) = rewrite_inputs(last)?;
    graph.stages.push(Stage {
        inputs,
        plan: last,
        mode: OutputMode::Pipelined,
        distribution: OutputDistribution::RoundRobin { channels: 1 },
        placement: TaskPlacement::Worker,
    });

```

The final stage is always added after recursive splitting. Its output schema is the job schema. If no later stage consumes it, the driver will expose its task streams as job output.

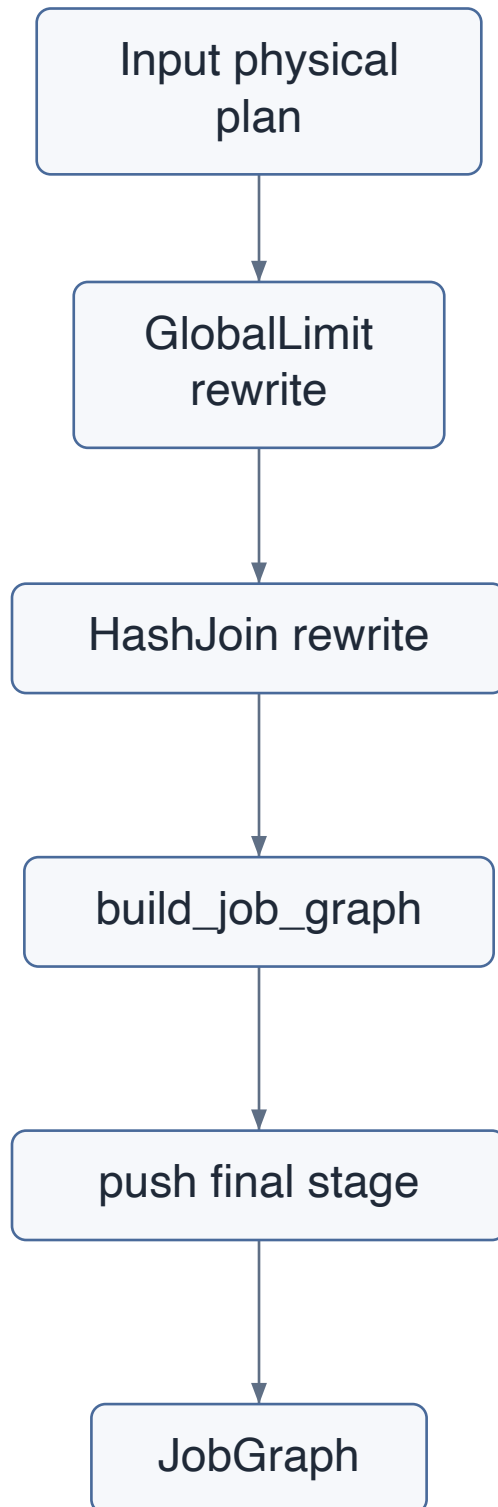


Figure 33: Flowchart 07.3

7.8 Rewrite 1: Global Limit Needs One Input Partition

`ensure_single_input_partition_for_global_limit` rewrites every `GlobalLimitExec`.

If a global limit has a real `LIMIT` or `OFFSET` and its input has more than one partition, Sail wraps the input in `CoalescePartitionsExec`:

```
let input = Arc::new(CoalescePartitionsExec::new(input.clone()));
Arc::new(GlobalLimitExec::new(input, skip, fetch))
```

Why?

A global limit is not the same as a per-partition limit. If each worker applies `LIMIT 10` locally, the whole job may return far more than 10 rows. Sail keeps any local limit optimization that `DataFusion` created, but ensures the final global limit sees a single partition.

This is an example of distributed correctness requiring a physical rewrite after `DataFusion` planning.

7.9 Rewrite 2: Collect-Left Hash Joins And Unmatched Rows

`ensure_partitioned_hash_join_if_build_side_emits_unmatched_rows` handles a more subtle distributed problem.

`DataFusion` can use `PartitionMode::CollectLeft` for hash joins. In that mode, one side is collected and reused. This is fine in local execution. In a distributed engine, it becomes tricky for join types that need to emit unmatched rows from the build side, because row-match state is not shared across all distributed partitions.

For join types such as `Left`, `LeftAnti`, `LeftSemi`, `LeftMark`, and `Full`, Sail rewrites the join to `PartitionMode::Partitioned` with explicit repartitioning on both sides:

```
HashJoinExec::try_new(
    repartition(join.left, left_exprs, partition_count)?,
    repartition(join.right, right_exprs, partition_count)?,
    join.on.clone(),
    ...
    PartitionMode::Partitioned,
    ...
)
```

This makes each output partition independently executable. It may be less clever than a local collect-left plan, but it is correct for Sail's distributed execution model.

The lesson is that physical plans produced by a single-node optimizer sometimes need a distribution-aware fixup before being split into stages.

7.10 Recursive Stage Splitting

The heart of the planner is `build_job_graph`.

It walks the physical plan tree from the leaves upward. First it recursively processes children. Then it decides whether the current node introduces a stage boundary.

These nodes introduce boundaries:

- RepartitionExec
- ExplicitRepartitionExec
- CoalescePartitionsExec
- SortPreservingMergeExec
- Sail CoalesceExec
- driver-only plans such as SystemTableExec and CatalogCommandExec

The broad shape is:

```
let children = ... build_job_graph(child, usage, graph) ...;
let plan = with_new_children_if_necessary(plan, children)?;

let plan = if let Some(repartition) =
plan.as_any().downcast_ref::<RepartitionExec>() {
    create_shuffle(child, graph, properties, consumption)?
} else if let Some(coalesce) =
plan.as_any().downcast_ref::<CoalescePartitionsExec>() {
    create_shuffle(child, graph, properties, consumption)?
} else if plan.as_any().is::<SortPreservingMergeExec>() {
    plan.with_new_children(vec![create_merge_input(child, graph)?])?
} else if let Some(coalesce) = plan.as_any().downcast_ref::<CoalesceExec>() {
    create_rescale_input(child, coalesce.output_partitions(), graph)?
} else {
    plan
};
```

The planner preserves ordinary operators inside a stage. It cuts only when the execution pattern changes across partitions.

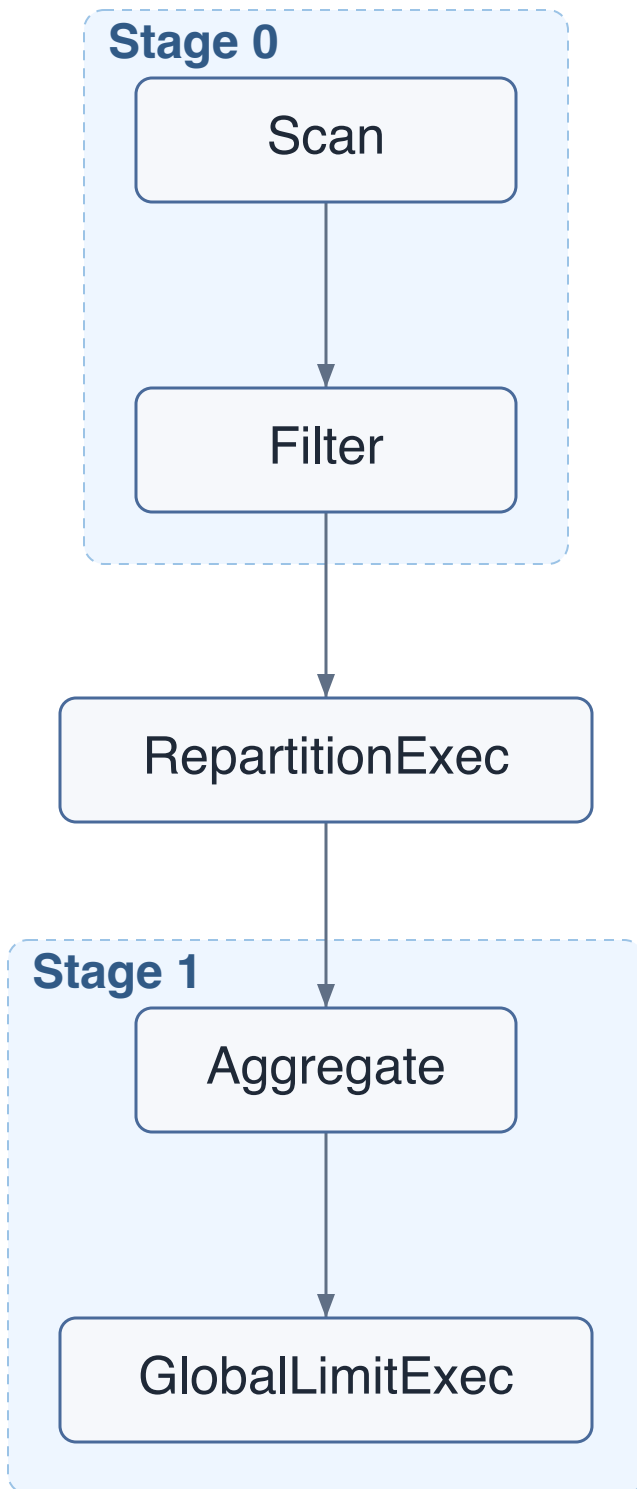


Figure 34: Flowchart 07.4

In the job graph, RepartitionExec itself is replaced by a stage boundary: stage 0 writes channels; stage 1 reads those channels.

7.11 Partition Usage: Single Versus Shared

The PartitionUsage enum has two variants:

```
enum PartitionUsage {  
    Once,  
    Shared,  
}
```

Most plan inputs are used once. But some joins reuse one side across many partitions. For example, a collect-left style join may gather the build-side data through `execute(0)` for each probe-side partition.

In local DataFusion, helper machinery can make that efficient inside one process. In distributed execution, reused data must be materialized so multiple downstream tasks can consume it safely.

Sail maps usage to shuffle consumption:

```
let consumption = match usage {  
    PartitionUsage::Once => ShuffleConsumption::Single,  
    PartitionUsage::Shared => ShuffleConsumption::Multiple,  
};
```

Then `create_shuffle` chooses the input mode:

```
let mode = match consumption {  
    ShuffleConsumption::Single => InputMode::Shuffle,  
    ShuffleConsumption::Multiple => InputMode::Broadcast,  
};
```

So a shared input becomes broadcast-like downstream consumption.

This is a nice example of a local execution property becoming a distributed data movement decision.

7.12 create_shuffle

`create_shuffle` is used for repartition and coalesce-style boundaries.

It converts DataFusion partitioning into Sail output distribution:

```
let distribution = match properties.partitioning.clone() {  
    Partitioning::RoundRobinBatch(channels)  
    | Partitioning::UnknownPartitioning(channels) => {  
        OutputDistribution::RoundRobin { channels }  
    }  
    Partitioning::Hash(keys, channels) => OutputDistribution::Hash { keys,  
channels },  
};
```

Then it turns the child into a stage:

```

let (plan, inputs) = rewrite_inputs(plan.clone())?;
let stage = Stage {
  inputs,
  plan,
  mode: OutputMode::Pipelined,
  distribution,
  placement: TaskPlacement::Worker,
};
graph.stages.push(stage);

```

Finally it returns a StageInputExec placeholder for the parent plan:

```

StageInputExec::new(
  StageInput { stage: s, mode },
  properties,
)

```

The parent sees an execution plan input with the right schema and partitioning. The job graph sees a dependency on a previous stage.



Figure 35: Flowchart 07.5

7.13 Merge And Rescale Boundaries

SortPreservingMergeExec uses `create_merge_input`.

That creates a worker stage for the child and returns a StageInputExec with `InputMode::Merge`. A merge input reads all partitions from the input stage. It is how a later operator can see globally merged streams.

Sail's custom CoalesceExec uses `create_rescale_input`.

Rescale is different from broadcast and shuffle. It divides input partitions into contiguous ranges:

```

let start = output_partition * input_partitions / output_partitions;
let end = (output_partition + 1) * input_partitions / output_partitions;

```

Each output partition consumes only its assigned range. This is the distributed form of reducing partition count without fully merging everything into one partition.

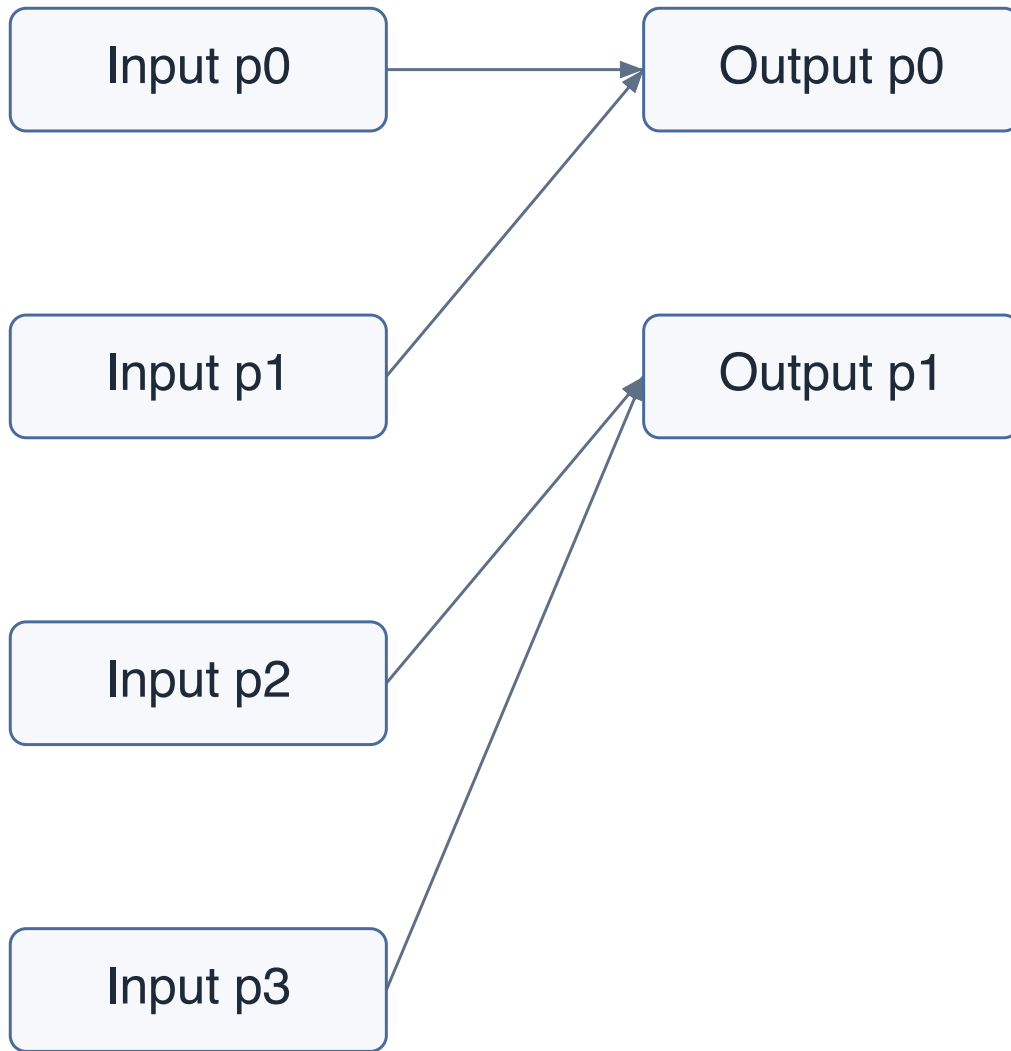


Figure 36: Flowchart 07.6

7.14 Driver Stages

Most stages run on workers:

placement: TaskPlacement::Worker

But some plans must run on the driver. The job graph planner recognizes:

- SystemTableExec
- CatalogCommandExec

and creates a driver stage:

```
Stage {
  inputs: vec![],
  plan: plan.clone(),
  distribution: OutputDistribution::RoundRobin { channels: 1 },
  placement: TaskPlacement::Driver,
}
```

The TODO says driver stages with inputs are not supported yet. That is an important limitation for extension design. If a future extension introduces a driver-only physical operator that consumes distributed inputs, Sail would need to extend this part of the planner.

7.15 Topological Order

JobGraph stores stages in topological order:

```
/// For any stage, all its input stages are guaranteed to
/// appear before it in the list.
stages: Vec<Stage>
```

The recursive construction naturally creates earlier stages before later stages. When the final stage is pushed, all its dependencies have already been added.

This matters because stage indices become part of task stream keys:

```
TaskStreamKey {
  job_id,
  stage,
  partition,
  attempt,
  channel,
}
```

Once a stage is inserted into the graph, its index is the stable identity used by the scheduler, task runner, stream manager, and job output system.

7.16 Replicas And Repeated Consumption

JobGraph::replicas(stage) computes how many replicas of a stage's output are needed:

```
match input.mode {
  InputMode::Forward | InputMode::Shuffle | InputMode::Rescale => 1,
  InputMode::Merge | InputMode::Broadcast => {
    x.plan.output_partitioning().partition_count()
  }
}
```

The result is at least one, because final stages need output for the client even if no later stage consumes them.

Why do merge and broadcast need more replicas? Because multiple downstream partitions may read the same upstream streams. If output is pipelined and stored locally, Sail must keep enough stream replicas available for all consumers.

This is the data-plane cost of repeated consumption.

7.17 From Job Graph To Job Topology

After `JobGraph::try_new`, the scheduler creates a `JobDescriptor`:

```
let graph = JobGraph::try_new(plan)?;  
let (output, stream) = build_job_output(ctx, job_id, graph.schema().clone());  
let descriptor = JobDescriptor::try_new(graph, JobState::Running { output,  
context })?;
```

`JobDescriptor::try_new` creates:

- one `StageDescriptor` per stage,
- one `TaskDescriptor` per stage partition,
- a `JobTopology`,
- one `TaskRegionDescriptor` per topology region.

The topology builder groups pipelined stages into task regions. This is the transition from “stages” to “what should be scheduled together.”

`JobTopology::try_new` first records stage consumers and pipelined adjacency. Then it finds connected components of pipelined stages.

If every in-component input is Forward, the component can be sliced by partition:

```
region 0: stage A partition 0, stage B partition 0  
region 1: stage A partition 1, stage B partition 1  
...
```

If the component has non-forward inputs, Sail creates one region containing all partitions of the component.

This captures a practical scheduling idea:

- one-to-one pipelined dependencies can run partition by partition,
- shuffle/merge/broadcast-style dependencies need broader coordination.

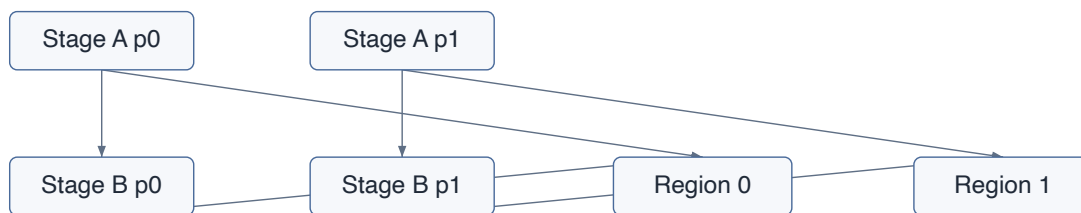


Figure 37: Flowchart 07.7

7.18 Region Dependencies

After regions are created, the topology builder adds dependencies.

For Forward input, a task depends on the corresponding partition of the input stage:

```
TaskTopology {
    stage: input.stage,
    partition: task.partition,
}
```

For all other input modes, the task depends on all partitions of the input stage:

```
for p in 0..partitions {
    TaskTopology {
        stage: input.stage,
        partition: p,
    }
}
```

This is conservative and correct. Shuffle, broadcast, merge, and rescale may need data from multiple upstream partitions, so downstream regions wait until the relevant upstream regions are complete.

7.19 Task Definitions

A worker does not receive a Rust Stage object. It receives a serialized TaskDefinition:

```
pub struct TaskDefinition {
    pub plan: Arc<[u8]>,
    pub inputs: Vec<TaskInput>,
    pub output: TaskOutput,
}
```

The plan is serialized bytes. Inputs describe where to read upstream streams. Output describes how to publish this task's result streams.

Inputs can be:

```
pub enum TaskInputLocator {
    Driver { stage, keys },
    Worker { stage, keys },
    Remote { uri, stage, keys },
}
```

Outputs can be:

```
pub enum TaskOutputLocator {
    Local { replicas },
    Remote { uri },
}
```

The current pipelined path uses local stream storage and worker/driver stream locations. Blocking remote output is present in the type system but not fully implemented in the code paths shown here.

This is another extension lesson: physical plans and expressions must be serializable if they are going to run on workers. A local-only extension is much easier than a distributed-safe extension.

7.20 Job Output

The final stage's output becomes the stream returned to Spark Connect or Flight SQL.

`build_job_output` creates:

- a `JobOutputManager`, used by the scheduler to add task streams,
- and a `SendableRecordBatchStream`, returned to the query caller.

The stream is a `RecordBatchStreamAdapter` around a receiver. Internally, `JobOutputStream` keeps a `SelectAll` of task streams.

When final stage tasks start running or succeed, `extend_job_output` adds their channels:

```
for c in 0..channels {
  let key = TaskStreamKey { job_id, stage: s, partition: p, attempt, channel:
c };
  actions.push(JobAction::ExtendJobOutput {
    handle: output.handle(),
    key,
    schema: schema.clone(),
  });
}
```

The job output stream can therefore begin returning batches while final tasks are still running. That is why the output mode is currently `Pipelined`.

The stream also handles task attempts carefully. If a later attempt supersedes an earlier attempt for the same task stream, the wrapper can mute the older stream so the client does not see duplicate output.

7.21 A Worked Example: Hash Aggregate

Imagine a query that scans a table and groups by `customer_id`:

```
SELECT customer_id, count(*)
FROM orders
GROUP BY customer_id
```

A simplified physical plan might look like:

```
AggregateExec final
  RepartitionExec Hash(customer_id, 4)
    AggregateExec partial
      TableScanExec
```

The job graph planner sees `RepartitionExec` and cuts the plan:

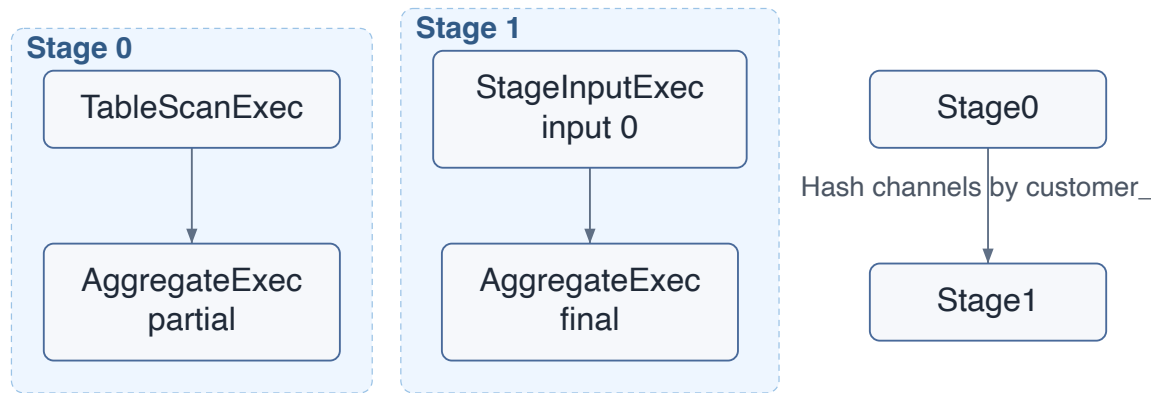


Figure 38: Flowchart 07.8

Stage 0 output distribution is Hash { keys: [customer_id], channels: 4 }. Stage 1 input mode is Shuffle. Therefore:

- Stage 0 partition 0 writes channels 0..3.
- Stage 0 partition 1 writes channels 0..3.
- Stage 1 partition 0 reads channel 0 from every Stage 0 partition.
- Stage 1 partition 1 reads channel 1 from every Stage 0 partition.
- and so on.

That is a distributed hash exchange.

7.22 A Worked Example: Global Limit

Consider:

```
SELECT *
FROM events
LIMIT 10
```

If the scan has many partitions, a global limit must not independently return 10 rows from every partition. Sail rewrites:

```
GlobalLimitExec
  multi-partition input
```

into:

```
GlobalLimitExec
  CoalescePartitionsExec
    multi-partition input
```

Then CoalescePartitionsExec becomes a stage boundary. The upstream stage produces partitioned output, and the final stage reads it as a coalesced input before applying the global limit.

Correctness beats parallelism here. The global decision has to be made in one place.

7.23 A Worked Example: Broadcast-Like Shared Input

Some joins reuse one side. In Sail's planner, that shows up as `PartitionUsage::Shared`. Shared usage turns into `ShuffleConsumption::Multiple`, which turns into `InputMode::Broadcast`.

The upstream stage is materialized once. Downstream partitions can all read it.

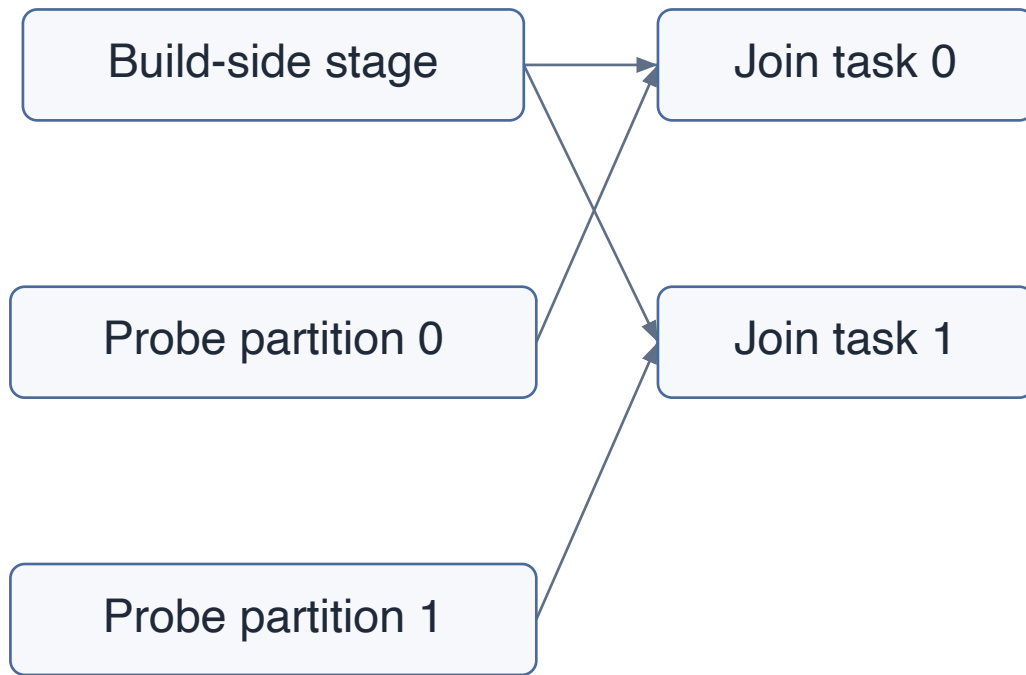


Figure 39: Flowchart 07.9

The key point is not that Sail necessarily implements every broadcast join optimization you might imagine. The key point is that the job graph has an explicit mode for repeated consumption of upstream data.

7.24 Why This Design Works

Sail does not throw away the DataFusion plan and invent a separate distributed IR. Instead, it:

1. Uses DataFusion physical plans as stage bodies.
2. Rewrites a few single-node assumptions for distributed correctness.
3. Cuts stage boundaries at exchange-like operators.
4. Replaces cross-stage edges with `StageInputExec`.
5. Tracks how each stage output is distributed into channels.
6. Lets the scheduler turn stages into task regions and attempts.

This has several benefits:

- Sail inherits DataFusion physical operators and optimizer behavior.

- Custom Sail physical nodes can participate as normal `ExecutionPlan` nodes.
- The job graph can be displayed in terms of familiar physical plans.
- Distributed correctness is concentrated in stage splitting and scheduling.
- The final output is still a normal `Arrow RecordBatch` stream.

7.25 Extension Implications

For discussion #2001, this chapter is the warning label on the box.

It is not enough for an extension to register a `DataFusion` function or physical planner. If the extension participates in distributed execution, it must also fit this job graph transformation.

An extension author needs to ask:

- Does my physical operator preserve partitioning?
- Does it require all input partitions at once?
- Can it run independently per partition?
- Does it need driver placement?
- Does it produce a custom physical expression used for hash distribution?
- Can its physical plan be serialized to workers?
- If it creates a new exchange-like node, how should the job graph planner cut it into stages?
- If it creates a shared input, should downstream consumption be broadcast?
- If it introduces a driver-only command, can it have inputs?

The current planner hard-codes the known Sail and `DataFusion` nodes. A general extension API will need a way for extensions to declare distributed planning behavior, or at least a fallback policy that rejects unsupported distributed plans clearly.

7.26 Reading Exercise: Trace A Repartition Boundary

Follow a repartition from `DataFusion` physical plan to job graph:

1. Open `crates/sail-execution/src/job_graph/planner.rs`.
2. Find `build_job_graph`.
3. Find the `RepartitionExec` branch.
4. Follow the call to `create_shuffle`.
5. Observe how `Partitioning::Hash` becomes `OutputDistribution::Hash`.
6. Observe how `ShuffleConsumption::Single` becomes `InputMode::Shuffle`.
7. Open `crates/sail-execution/src/driver/job_scheduler/core.rs`.
8. Find `build_task_input_keys`.
9. Read the `InputMode::Shuffle` branch.

At the end, you should be able to explain which upstream task streams a downstream shuffle partition reads.

7.27 Reading Exercise: Trace Job Acceptance

Follow a cluster query:

1. Start in `crates/sail-execution/src/job_runner.rs`.

2. Find `ClusterJobRunner::execute`.
3. Follow `DriverMessage::ExecuteJob`.
4. Open `crates/sail-execution/src/driver/actor/handler.rs`.
5. Find `handle_execute_job`.
6. Follow `job_scheduler.accept_job`.
7. Open `crates/sail-execution/src/driver/job_scheduler/core.rs`.
8. Find `JobGraph::try_new(plan)`.
9. Follow `build_job_output`.

That is the control path from a physical plan to a client-visible output stream.

7.28 Worker-Native Regions in the Extension Branch

The following additions belong to the pinned experimental branch, not upstream main. They reuse job stages and slot-sharing groups rather than adding a graph scheduler. `WorkerExtensionExec` transports a bounded descriptor and ordinary host child plans. It does not serialize a driver pointer or a process-local plan registry identifier.

`WorkerDescriptor` in `crates/sail-common-datafusion/src/worker_extension.rs` records package identity, relation type URL, an operation label, opaque payload, input names and schemas, input routing, output schema and partition count. The host validates descriptor bounds, input cardinalities and unique names. Integer-range routing requires an `Int64` column and strictly increasing split points with one interval per partition. The worker's task scope is supplied by Sail; a payload cannot grant itself a host job or worker identity.

`crates/sail-execution/src/job_graph/worker_groups.rs` finds native operations in stages and assigns their existing `Stage.group` values. It groups by package and operation identity and joins groups transitively when operations share a stage. The earliest stage supplies a deterministic job-local group name. Ordinary source widths must not shift the task-set buckets of stateful stages. This is why grouping is more than attaching an arbitrary string to one operator.

The scheduler treats jobs containing these stateful regions as fail-fast. A lost acknowledgement cannot prove whether native mutation committed. Retrying an upstream ordinary region can also deliver its contribution twice, so the boundary covers the whole job rather than only the immediately native stage. Recovery would require an additional state/replay protocol; it is not obtained from ordinary task retry.

This placement contract is exercised with empty owners, skewed ownership, multiple phases, cancellation and worker loss. A functional answer alone is not sufficient: qualification checks the complete native stage/partition matrix, actual worker assignments and attempt numbers. See Chapter 17 for the distinction between process-cluster and physical two-host evidence.

7.29 Takeaways

The job graph is Sail's distributed version of a DataFusion physical plan. It keeps each stage as an `ExecutionPlan`, but replaces cross-stage edges with `StageInputExec` placeholders and records explicit input modes.

The most important enum in this chapter is `InputMode`: `Forward`, `Merge`, `Shuffle`, `Broadcast`, and `Rescale`. Those five modes describe how downstream partitions consume upstream task streams.

Before splitting the plan, Sail rewrites global limits and certain collect-left hash joins for distributed correctness. During splitting, it cuts at repartition, coalesce, merge, rescale, and driver-only nodes. After splitting, the scheduler groups stages into task regions, builds task definitions, and connects final task streams into a single Arrow output stream.

The next chapter will follow those task regions into the driver, workers, task assigner, and stream manager.

8 Drivers, Workers, Tasks, And Streams

Chapter 7 explained how Sail turns a DataFusion physical plan into a distributed job graph. This chapter follows that graph into motion.

The job graph says what should run:

- stages,
- partitions,
- input modes,
- output distributions,
- driver or worker placement,
- and task stream dependencies.

The driver, workers, task assigner, task runner, and stream managers decide how that work actually happens.

If Chapter 7 was the map, Chapter 8 is the traffic system.

8.1 The Core Files

Area	Files	Role
Actor runtime	<code>crates/sail-common/src/actor.rs</code>	Small async actor system used by the driver and workers
Driver actor	<code>crates/sail-execution/src/driver/actor/*.rs</code>	Accepts jobs, workers, task updates, stream requests, and shutdown
Driver events	<code>crates/sail-execution/src/driver/actor/message.rs</code>	Message protocol for the driver actor
Worker actor	<code>crates/sail-execution/src/worker/actor/*.rs</code>	Registers with driver, receives tasks, reports status, serves/fetches streams
Worker events	<code>crates/sail-execution/src/worker/actor/message.rs</code>	Message protocol for worker actor
Worker pool	<code>crates/sail-execution/src/driver/worker_pool/*.rs</code>	Launches, registers, monitors, and talks to workers
Task assigner	<code>crates/sail-execution/src/driver/task_assigner/*.rs</code>	Maps task regions to driver or worker task slots
Job scheduler	<code>crates/sail-execution/src/driver/job_scheduler/*.rs</code>	Tracks job state, creates attempts, schedules regions, builds task definitions
Task runner	<code>crates/sail-execution/src/task_runner/*.rs</code>	Executes serialized DataFusion physical plans on driver or worker
Stream manager	<code>crates/sail-execution/src/stream/local/</code>	Owens local task streams and pending stream fetches
Stream accessor	<code>crates/sail-execution/src/stream/accessor.rs</code>	Implements task stream reader/writer by sending actor messages
Stream service	<code>crates/sail-execution/src/stream/service/</code>	Uses Arrow Flight to fetch task streams across processes
Worker managers	<code>crates/sail-execution/src/worker_manager/*.rs</code>	Launches local or Kubernetes workers

The chapter will follow a single distributed query from the moment the cluster runner sends it to the driver until final Arrow batches are returned.

8.2 The Actor Runtime

Sail's driver and workers are actors. The actor runtime lives in `crates/sail-common/src/actor.rs`.

The trait is small:

```
pub trait Actor: Sized + Send + 'static {
    type Message: Send + SpanAssociation + 'static;
    type Options;

    fn name() -> &'static str;
    fn new(options: Self::Options) -> Self;
    async fn start(&mut self, ctx: &mut ActorContext<Self>) {}
    fn receive(&mut self, ctx: &mut ActorContext<Self>, message: Self::Message) -
> ActorAction;
    async fn stop(self, ctx: &mut ActorContext<Self>) {}
}
```

Messages are processed sequentially:

```
while let Some(MessageEnvelop { message, context }) = self.receiver.recv().await
{
    let action = self.actor.receive(&mut self.ctx, message);
    ...
    self.ctx.reap();
}
```

That gives actor state a simple programming model: the actor mutates its own fields without locks because only one message is handled at a time.

But the actor must not block. The trait comment says blocking work should be spawned through `ActorContext::spawn`. That pattern appears everywhere in driver and worker code:

- launch a worker asynchronously,
- send an RPC to a worker,
- fetch a remote stream,
- report task status,
- run a task monitor,
- merge job output streams.

The actor system gives Sail a clean split:

- state transitions happen synchronously in `receive`,
- slow IO happens in spawned futures,
- spawned futures report back by sending another actor message.

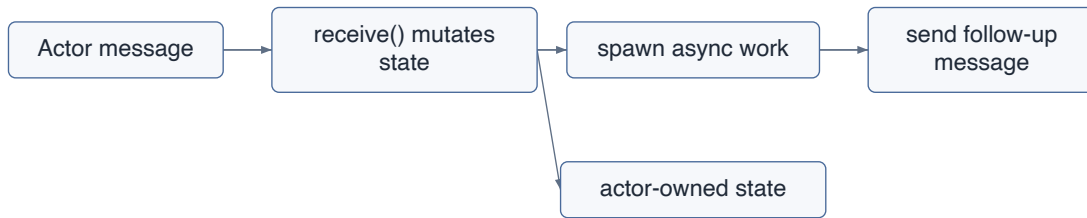


Figure 40: Flowchart 08.1

This is the control-plane style behind Sail’s distributed runtime.

8.3 Driver Actor: The Coordinator

`DriverActor` is defined across `crates/sail-execution/src/driver/actor`.

Its constructor builds `WorkerPool`, `JobScheduler`, `TaskAssigner` and `WorkerScaler`. The task runner starts separately, with local streams and the configured storage or Celeborn support assembled during actor startup. The current actor does not contain the older monolithic `StreamManager` shown in previous editions.

These components divide responsibility: the worker pool tracks workers and their clients; the job scheduler tracks jobs and attempts; the task assigner manages assignment; the scaler manages worker demand. The driver also tracks task-status sequence numbers so stale reports do not overwrite newer observations.

The driver creates its task-runner child and stream extensions during startup. Follow `driver/actor/core.rs` for construction and `driver/actor/handler.rs` for message handling. Server/gateway integration is separate from the worker’s `ServerMonitor`; an old `self.server` field sketch is not the current driver constructor.

8.4 Worker Actor: Lifecycle and Services

`WorkerActor::new` creates its driver clients, metrics sender and server monitor. At startup it creates `LocalStreamManager`, optional storage or Celeborn support, and a `TaskRunnerActor` child. `TaskRunnerPlacement` distinguishes driver execution from worker execution while reusing the task-runner implementation.

The worker server combines the worker RPC service and an Arrow Flight service. Its Flight fetcher sends `TaskRunnerMessage::FetchLocalStream` to the task runner. `WorkerMessage::ServerReady` reports the actual listening port and supplies the shutdown signal. Heartbeat startup and shutdown are worker lifecycle messages; task batches are handled by the task-runner path described below.

8.5 Worker Launch And Registration

The worker lifecycle starts in the driver `WorkerPool`.

`start_worker`:

1. Allocates a new `WorkerId`.

2. Inserts a `WorkerDescriptor` in Pending state.
3. Schedules a pending-worker probe for timeout handling.
4. Calls the configured `WorkerManager` to launch the worker.

The launch options include:

- TLS setting,
- driver external host and port,
- heartbeat interval,
- task stream buffer,
- stream creation timeout,
- RPC retry strategy.

For local cluster mode, `LocalWorkerManager` spawns a `WorkerActor` in a local actor system:

```
let options = WorkerOptions::local(id, options, self.runtime.clone(),
self.session.clone());
system.spawn::<WorkerActor>(options);
```

For Kubernetes mode, the worker manager uses the Kubernetes worker manager implementation. The driver side does not care which launch strategy is used; it just waits for a `RegisterWorker` event.

When registration arrives:

```
worker.state = WorkerState::Running {
  host,
  port,
  updated_at: Instant::now(),
  heartbeat_at: Instant::now(),
  client: None,
};
```

Then the driver schedules:

- a lost-worker probe,
- an idle-worker probe,
- and activates the worker in the task assigner.

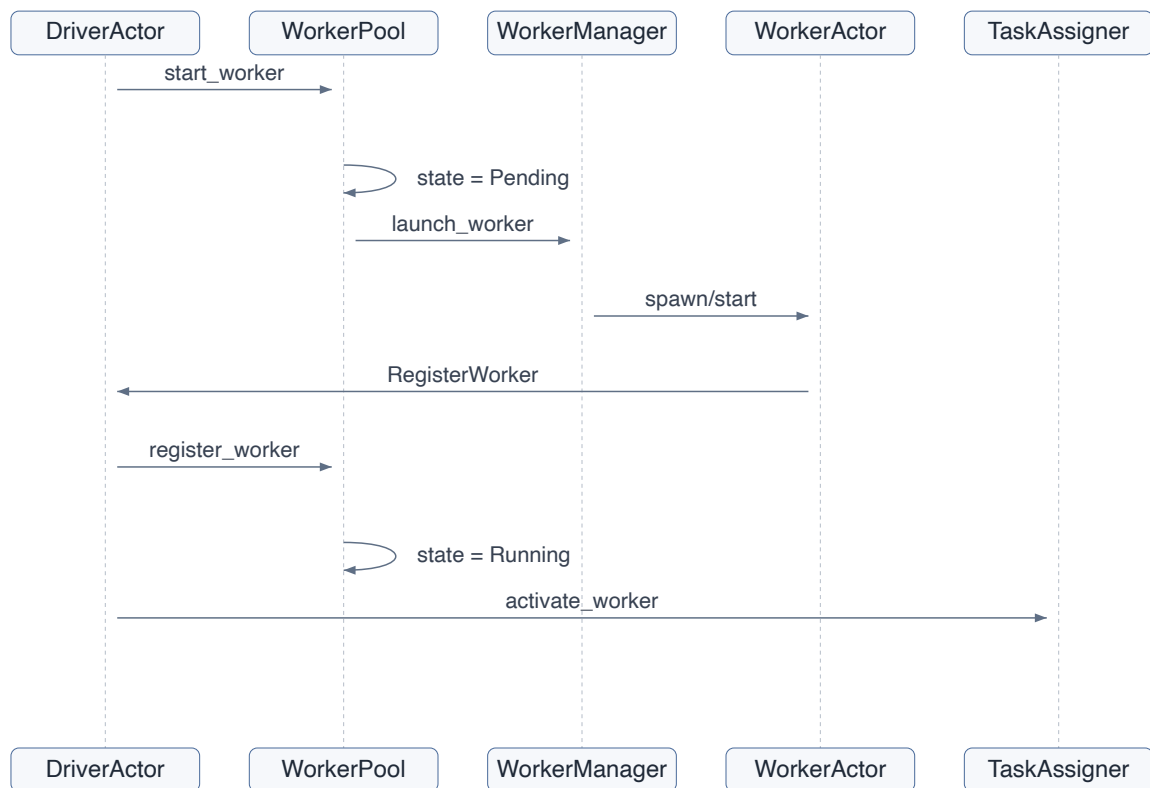


Figure 41: Sequence diagram 08.2

8.6 Worker Health

Workers send heartbeats to the driver. The driver records the latest heartbeat:

```

if let WorkerState::Running { heartbeat_at, .. } = &mut worker.state {
    *heartbeat_at = Instant::now();
    Self::schedule_lost_worker_probe(ctx, worker_id, worker, &self.options);
}

```

If the lost-worker probe fires and the heartbeat is stale, the driver:

1. Stops the worker.
2. Finds tasks assigned to that worker.
3. Marks those task attempts failed.
4. Refreshes affected jobs.
5. Tries to run tasks again.
6. Scales up workers if needed.

The handler in `driver/actor/handler.rs` makes that explicit:

```

let keys = self.task_assigner.find_worker_tasks(worker_id);
self.task_assigner.deactivate_worker(worker_id);
for key in keys.iter() {
    self.job_scheduler.update_task(

```

```

    key,
    TaskState::Failed,
    Some(message.clone()),
    Some(CommonErrorCause::Execution(message.clone())),
  );
}

```

This is the retry story at the worker level. Worker loss becomes task attempt failure. Ordinary retryable jobs may reschedule the region up to their attempt limit. Jobs retaining worker-native graph state instead fail as a whole, as described below.

8.7 From Job To Task Regions

The job scheduler accepts a job in `crates/sail-execution/src/driver/job_scheduler/core.rs`:

```

let graph = JobGraph::try_new(plan)?;
let (output, stream) = build_job_output(ctx, job_id, graph.schema().clone());
let descriptor = JobDescriptor::try_new(graph, JobState::Running { output,
context });
self.jobs.insert(job_id, descriptor);

```

After acceptance, the driver calls `refresh_job`.

`refresh_job` is the scheduler's main decision function. Its comment lists the steps:

1. Cancel task attempts in a region if any task attempt fails.
2. Add final-stage running/succeeded task streams to job output.
3. Clean up stage output streams when all consumers have succeeded.
4. Fail a job if any task exceeds max attempts.
5. Mark the job succeeded when final regions succeed.
6. Schedule regions whose dependencies have succeeded.

The important point is that the scheduler does not immediately schedule every task. It schedules task regions when dependencies allow them.

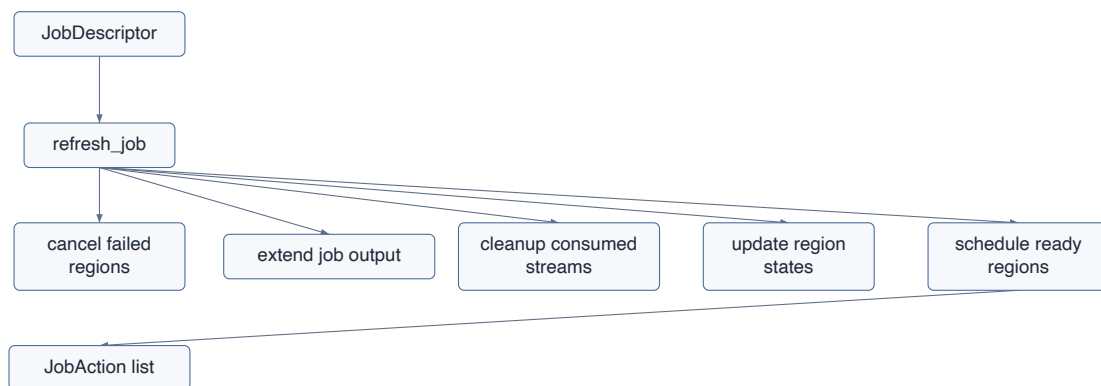


Figure 42: Flowchart 08.3

The driver then executes the returned `JobAction` values.

8.8 Task Attempts

Each stage has tasks. Each task can have multiple attempts:

```
pub struct TaskDescriptor {  
    pub attempts: Vec<TaskAttemptDescriptor>,  
}
```

An attempt has:

- state,
- messages,
- error cause,
- job_output_fetched,
- creation time,
- stop time.

When a task region becomes schedulable, the scheduler pushes a new attempt for each task:

```
attempts.push(TaskAttemptDescriptor {  
    state: TaskState::Created,  
    messages: vec![],  
    cause: None,  
    job_output_fetched: false,  
    created_at: Utc::now(),  
    stopped_at: None,  
});
```

Task states are:

- Created
- Scheduled
- Running
- Succeeded
- Failed
- Canceled

The driver receives status updates from workers as `DriverMessage::UpdateTask`. Those updates include an optional sequence number. The driver ignores stale updates:

```
if sequence <= *s {  
    warn!("{sequence} is stale", TaskKeyDisplay(&key));  
    return ActorAction::Continue;  
}
```

This protects the control plane from delayed or duplicate worker status messages.

8.9 Task Regions And Cascading Cancellation

Task regions are important because Sail schedules and retries them as units. If any task in a region fails, the scheduler cancels other active attempts in that region:

```

if failed {
  for t in &region.tasks {
    for (a, attempt) in task.attempts.iter_mut().enumerate() {
      if !attempt.state.is_terminal() {
        attempt.state = TaskState::Canceled;
        actions.push(JobAction::CancelTask { key });
      }
    }
  }
}

```

Why cancel the whole region?

Because a region represents a group of tasks that are pipelined or otherwise scheduled together. If one attempt fails, its peers may be producing or consuming streams that are no longer valid for that attempt set. Canceling the region keeps attempt boundaries consistent.

This is the distributed version of “do not mix outputs from different attempts unless the system has explicitly decided to do so.”

8.10 Task Assignment

The scheduler emits `JobAction::ScheduleTaskRegion`. The driver gives the region to `TaskAssigner`:

```
self.task_assigner.enqueue_tasks(region);
```

Then `run_tasks` asks for assignments:

```
let assignments = self.task_assigner.assign_tasks();
self.task_assigner.track_streams(&assignments);
```

`TaskAssigner` tracks:

- active workers,
- worker task slots,
- driver task slots,
- queued task regions,
- task assignments,
- local stream ownership,
- remote stream ownership.

Worker task slots are limited:

```
task_slots: Vec![TaskSlot::default(); self.options.worker_task_slots]
```

Driver task slots can grow:

```
/// The number of task slot can grow indefinitely.
task_slots: Vec<TaskSlot>
```

Assignment is region-aware. `TaskSlotAssigner::try_assign_task_region` only succeeds if the entire region can be assigned:

```

for (placement, set) in &region.tasks {
    match placement {
        TaskPlacement::Driver => ...
        TaskPlacement::Worker => {
            if let Some((worker_id, slot)) = self.next() {
                ...
            } else {
                return Err(region);
            }
        }
    }
}
}

```

If a region cannot fit, it goes back to the front of the queue. This can cause head-of-line blocking, but it preserves scheduling order.

8.11 Scaling Workers

The task assigner also tells the driver how many workers to request:

```

let required_slots = enqueued_slots.saturating_sub(vacant_slots);
let required_workers = required_slots
    .div_ceil(self.options.worker_task_slots)
    .min(allowed_workers);

```

The driver passes that demand to a separate WorkerScaler:

```

self.worker_scaler
    .reconcile(self.task_assigner.count_worker_demands())

```

The scaler returns launch requests. The driver starts workers through `WorkerPool::start_worker` and binds each worker ID to its demand. Registration fulfills the demand; a failed launch can schedule a retry under the configured launch retry strategy. The demand identity survives that retry, so reconciliation does not repeatedly create fresh demand for the same failed request.

Initial worker requests count toward the task target. The task assigner accounts for vacant slots and the configured worker limit; the scaler manages the lifecycle of the resulting launch demands. This separates capacity calculation from retry policy. See `crates/sail-execution/src/driver/worker_scaler/core.rs`.

8.12 Building A Task Definition

After assignment, the driver asks the scheduler for each task definition:

```

let (definition, context) =
    self.job_scheduler.get_task_definition(&entry.key, &self.task_assigner)?;

```

A `TaskDefinition` contains:

```

pub struct TaskDefinition {
    pub plan: Arc<[u8]>,
    pub inputs: Vec<TaskInput>,
}

```

```

    pub output: TaskOutput,
}

```

The plan is serialized with DataFusion’s physical plan protobuf support and Sail’s extension codec:

```
let plan = encode_remote_physical_plan(self.codec.as_ref(), stage.plan.clone());
```

Inputs come from `stage.inputs`, using `InputMode` and current task assignments to decide locations.

For pipelined worker outputs, input keys become:

```

TaskInput {
    stage: input.stage,
    locator: Arc::new(TaskInputLocator::Worker { keys }),
}

```

Each key includes:

- upstream partition,
- upstream attempt,
- channel.

The task output includes:

- distribution,
- pipelined or blocking locator,
- replica count for pipelined output.

This object is the portable description of one task attempt.

8.13 Dispatching Tasks

`DriverActor::run_tasks` first reserves task assignments and tracks their streams. It groups work by job, region, stage and worker. Within that scheduling snapshot, it constructs one shared definition per job/stage, marks the tasks scheduled, and sends their partition/attempt identities as a batch.

Worker batches go through `WorkerPool::run_task_batch`; driver batches go to the local task runner. The worker-pool path supplies peer locations, sends the batch over gRPC and reports dispatch failures for the affected tasks. Reusing a stage definition avoids repeating its physical-plan serialization for every partition. The batch still preserves task-region and worker boundaries.

The peer list is optimized by remembering known peers:

```

let peers = running_workers
    .into_iter()
    .filter(|x| !worker.peers.contains(&x.worker_id))
    .collect();

```

Workers report back which peers they now know, so the driver avoids sending the same location information repeatedly.

8.14 Running a Task on a Worker

Current upstream separates the worker lifecycle actor from task execution. `WorkerMessage` covers server readiness, heartbeat startup and shutdown; there is no `WorkerEvent::RunTask` in this source. Worker RPC services route task work to the `TaskRunnerActor`, whose `TaskRunnerMessage::RunTaskBatch` carries the job, stage, tasks, definition, task context and peer information.

`TaskPreparation` in `crates/sail-execution/src/task_runner/preparation.rs` reconstructs and prepares the physical plan. The central sequence is:

```
proto_to_physical_plan(context, RemoteExecutionCodec, proto)
  -> rewrite_file_scans
  -> rewrite_shuffle
  -> trace_execution_plan
  -> cancellation check
  -> execute(task partition, context)
```

This is a flow summary, not a compilable replacement for the implementation. The returned preparation stream has the completion schema of `ShuffleWriteExec`, not the schema of the stage's data batches.

The file-scan rewrite has an important distributed correctness purpose. A `DataFusion` scan may share a queue among sibling partitions in one process. `Sail` reconstructs a separate plan for each distributed task; recreating that shared queue in every task could make every task scan every file. The rewrite sets `preserve-order` on the file configuration to keep each task on its assigned file group. For `Parquet` without an expression adapter, it installs `Sail`'s schema evolution adapter.

`Shuffle` preparation replaces `StageInputExec` placeholders with concrete readers from `TaskStreamFactory`. It validates input indexes against the task definition. The writer side connects the task's output to the selected stream backend. The task runner owns execution and reporting; the worker lifecycle actor owns readiness and shutdown coordination.

On the experimental extension branch, decoding must additionally reconstruct worker-native regions using the installed package and host-issued task scope. That scope supplies job/owner identity; extension payloads do not authorize their own worker identity. Native state cleanup follows the job lifecycle, while retained buffers keep their resource owners until final release.

8.15 Why The Task Monitor Drains The Stream

The task runner does not simply obtain a stream and report success. It supervises a `TaskMonitor` that polls the stream to completion.

The monitor first reports `Running`:

```
let _ = handle.send(Self::running(key.clone())).await;
```

Then it races execution against cancellation:

```
tokio::select! {
  x = Self::execute(key.clone(), stream) => x,
```

```

    x = Self::cancel(key.clone(), signal) => x,
}

```

execute drains the stream:

```

while let Some(batch) = stream.next().await {
    if let Err(e) = batch {
        return Failed;
    }
}
return Succeeded;

```

This matters because in DataFusion, executing a plan returns a stream. Work may not happen until the stream is polled. If Sail reported success immediately after obtaining the stream, it would be lying. Draining the stream ensures the task really ran and all shuffle writes were closed.

8.16 Stream Accessor: Actors As Readers And Writers

`TaskStreamFactory` in `crates/sail-execution/src/stream/accessor.rs` constructs readers and writers from the task definition, schema and task-runner handle. Its private `TaskStreamAccessor` sends requests to that actor and awaits their replies.

The reader implements `open(partition)` and selects the appropriate fetch method:

Input locator	Accessor operation
Driver	<code>fetch_driver_stream</code>
Worker	<code>fetch_worker_stream</code>
Storage	<code>fetch_storage_stream</code>
ShuffleService	<code>fetch_celeborn_stream</code>

Writers likewise create local, storage or Celeborn streams. The physical operators use `TaskStreamReader` and `TaskStreamWriter`; they do not contain actor-message dispatch or construct network clients. Chapter 9 follows the multi-channel sink's write, commit and abort lifecycle.

8.17 Stream Identity

A `TaskStreamKey` identifies one stream:

```
job_id, stage, partition, attempt, channel
```

That key is the identity that ties together:

- task output,
- stream ownership,
- stream fetches,
- job output,
- cleanup,
- retry attempts.

The inclusion of `attempt` is especially important. If a task is retried, the new attempt writes a different stream key. Consumers can avoid accidentally mixing data from failed and replacement attempts.

8.18 Local Stream Ownership

`LocalStreamManager` is provided to the task runner through its extensions. It tracks streams by `TaskStreamKey`, including consumers waiting for a producer. Its states distinguish pending, created and failed streams. Creating an already created stream is an error; a producer failure is propagated to waiting senders.

`create_stream` publishes a sink over the appropriate replicas. `fetch_stream` resolves a consumer against this tracked state. Pending-stream probes are `TaskRunnerMessage` values, so timeout handling returns through the actor that owns the stream manager. This is not a separate global stream-manager actor.

Storage and Celeborn streams have their own managers. Local stream removal, object-store cleanup and remote shuffle cleanup must be followed through their respective paths rather than assuming a dropped local channel deletes all data.

8.19 Memory Streams And Replicas

The current local stream implementation is `MemoryStream`.

Its comment explains the design:

```
/// A memory stream that can be read multiple times.
/// It maintains multiple replicas of the stream internally.
/// Since [Arc] is used inside the record batch, it is relatively cheap
/// to clone the data in multiple replicas.
```

A memory stream has one publisher and multiple receivers:

```
sender: Option<MemoryStreamReplicaSender>,
receivers: Vec<mpsc::Receiver<TaskStreamResult<RecordBatch>>>,
```

When a batch is written, `MemoryStreamReplicaSender` tries to send it to every active replica. If a receiver is full, it uses an overflow buffer. If a receiver is closed, it drops that replica:

```
Err(mpsc::error::TrySendError::Closed(_)) => {
    dropped = true;
}
```

A closed receiver is not necessarily an error. A downstream LIMIT may stop reading early. The sink returns `Closed` only when all replicas are gone.

This replica design supports `JobGraph::replicas(stage)`: stages consumed by merge or broadcast may need multiple readers for the same output stream.

8.20 Arrow Flight For Task Streams

When a task needs a stream from another process, Sail uses Arrow Flight.

The server is `TaskStreamFlightServer` in `crates/sail-execution/src/stream/service/server.rs`. Its important method is `do_get`:

1. Decode a `TaskStreamTicket`.
2. Convert it to `TaskStreamKey`.
3. Ask a `TaskStreamFetcher` for the stream.
4. Encode the stream with `FlightDataEncoderBuilder`.
5. Return Flight data.

```
let stream = rx.await?;
let stream = stream.map_err(|e| FlightError::Tonic(Box::new(e.into())));
let stream = FlightDataEncoderBuilder::new()
    .build(stream)
    .map_err(Status::from);
```

The client is `TaskStreamFlightClient`:

```
let response = self.inner.get().await?.do_get(request).await?;
let stream = response.into_inner().map_err(|e| e.into());
let stream = FlightRecordBatchStream::new_from_flight_data(stream)?;
```

Again, the data plane is Arrow batches. The control plane moves task keys and locations; Flight moves the batch stream.

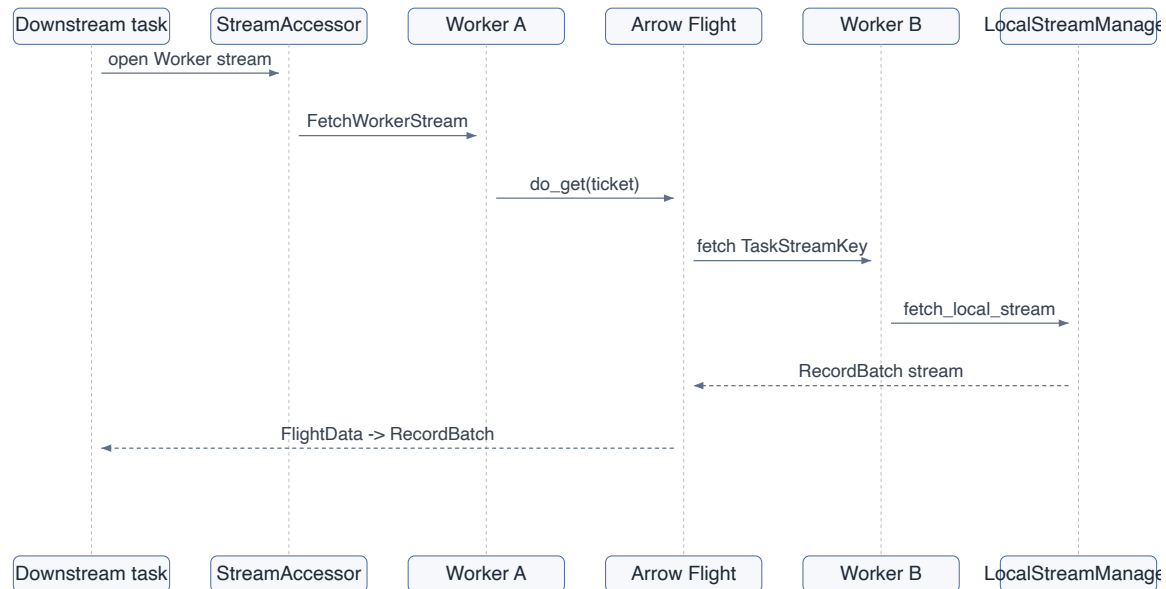


Figure 43: Sequence diagram 08.4

8.21 Peer Tracking

`PeerTracker` stores worker locations received with task dispatch. It ignores an empty update and inserts newly observed peers. `get_client_set` lazily constructs a `WorkerClientSet` from the stored location and TLS setting, then returns a clone. Requesting a client for the worker itself is rejected; local streams should use the local path. An unknown worker is also an explicit error.

This cache supplies connectivity, not graph ownership. In Argentea, the execution scope and job placement establish which worker owns a native partition. Knowing a peer's address is insufficient evidence that it holds the correct snapshot or phase of graph state.

8.22 Cleanup and Stream Tracking

In upstream `TaskRunnerActor::handle_close_job`, the runner closes the job's task registry, removes its local streams and drops its signals. Stage-specific local cleanup calls the same manager with an optional stage selector. Storage cleanup uses the storage manager and task context asynchronously; cleanup failures are reported rather than silently treated as deletion success.

The experimental branch extends this lifecycle to job-owned native state. Closing tasks is not enough if an extension registry still retains graph partitions. The host closes the registered native owners; active readers and output buffers retain their admitted resources until their final references disappear.

The branch also repairs scheduler task bookkeeping on failed/canceled jobs: nonterminal task records become canceled before unassignment. Success ordering and existing terminal states are preserved. Validate the original failed job, all task attempts, native close receipts and final staging independently. No single observation substitutes for the others.

8.23 Job Output

The job output path begins when final-stage tasks are running or succeeded.

`extend_job_output` finds final stages and adds their task streams:

```
actions.push(JobAction::ExtendJobOutput {  
    handle: output.handle(),  
    key,  
    schema: schema.clone(),  
});
```

The driver resolves the stream location from task assignment:

```
Some(TaskAssignment::Driver) =>  
    self.stream_manager.fetch_local_stream(ctx, &key)  
Some(TaskAssignment::Worker { worker_id, .. }) =>  
    self.worker_pool.fetch_task_stream(ctx, *worker_id, &key, schema.clone())
```

Then it sends the stream to the `JobOutputHandle`.

`JobOutputStream` merges all added streams using `SelectAll`. It stays active while new streams may arrive, then drains remaining streams once the output manager is dropped.

This is how a distributed job becomes one `SendableRecordBatchStream` for the caller.

8.24 One Query Lifecycle

Here is the complete lifecycle in one diagram:

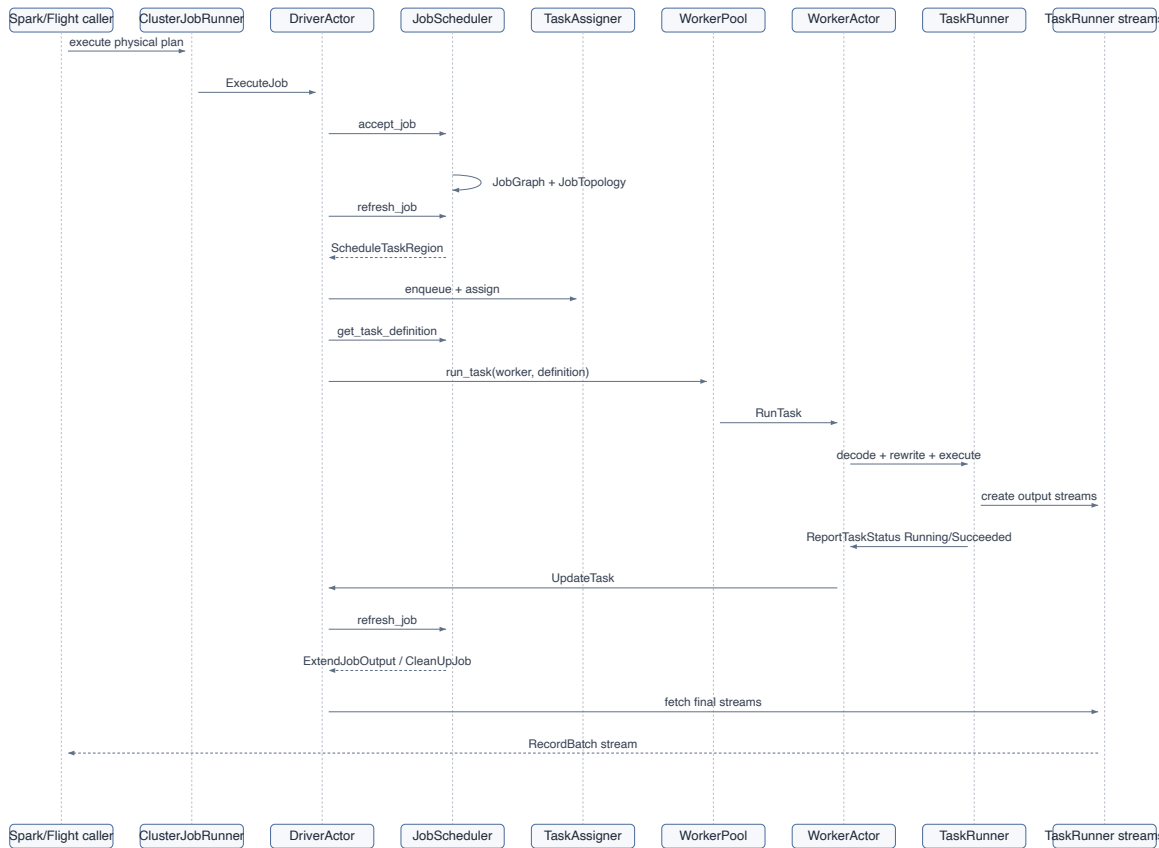


Figure 44: Sequence diagram 08.5

It is a lot of machinery, but each piece has a bounded job.

8.25 Failure And Retry Story

Sail's retry model is attempt-based:

- A task attempt fails if its monitor sees a stream error.
- A task attempt can be canceled explicitly.
- A lost worker causes all assigned task attempts to fail.
- A failed attempt causes the whole task region to cancel active peers.
- A region can be rescheduled by creating new attempts.
- If attempts exceed the configured max, the region and job fail.

The job output stream standardizes data-plane and control-plane errors through `CommonErrorCause`, so the client sees coherent failures whether the error comes from:

- a task stream,
- a task status update,
- job output failure,
- or cleanup/shutdown.

For ordinary retryable jobs, region failure can therefore lead to a new attempt. The extension branch adds a stricter rule for jobs containing worker-native state: failure terminates the whole job, including its ordinary relational regions. Retrying a task cannot recover a graph partition lost with its worker. This fail-fast boundary is part of Argentea’s current contract, not transparent recovery of native state.

8.26 Why This Design Fits Rust

This part of Sail shows several Rust strengths:

- actor-owned mutable state avoids large shared locks,
- Arc shares immutable plans, schemas, and clients,
- trait objects abstract workers, streams, actors, and job runners,
- async tasks isolate slow IO from actor message handling,
- enums make control-plane states explicit,
- typed IDs prevent accidental confusion between jobs, tasks, streams, and workers,
- oneshot channels turn actor messages into request/response APIs.

The design is not “just async Rust.” It is a careful layering:

Actor messages -> scheduler state -> task definitions -> physical plan execution
-> stream IO

Each layer is explicit enough to inspect and test.

8.27 Extension Implications

For the final extension chapter, this control plane raises several requirements.

A distributed-safe extension must consider:

- Can its physical plan be serialized into a `TaskDefinition`?
- Does `RemoteExecutionCodec` know how to decode it on workers?
- Does it require worker-local state?
- Does it require driver-only coordination?
- Does it produce task streams that can be retried safely?
- Does it rely on external resources that must be available in worker pods?
- Does it need peer-to-peer stream access?
- Does it need cleanup hooks when a job or stage finishes?
- Does it need custom task placement?

This is where a simple plugin API becomes a distributed systems API. A scalar UDF that uses Arrow arrays is easy. A custom physical operator with new stream semantics is much more serious.

Sail’s existing control plane gives us the vocabulary to design those capabilities precisely.

8.28 Reading Exercise: Follow A Task To A Worker

Trace a task from scheduling to worker execution:

1. Open `crates/sail-execution/src/driver/actor/handler.rs`.

2. Find `run_tasks`.
3. Follow `task_assigner.assign_tasks`.
4. Follow `job_scheduler.get_task_definition`.
5. Follow `worker_pool.run_task`.
6. Open `crates/sail-execution/src/worker/actor/handler.rs`.
7. Find `handle_run_task`.
8. Follow `task_runner.run_task`.
9. Open `crates/sail-execution/src/task_runner/actor/core.rs`.
10. Read `execute_plan`.

At the end, you should be able to say how a stage partition becomes `plan.execute(key.partition, context)`.

8.29 Reading Exercise: Follow A Stream Fetch

Trace a downstream task reading an upstream worker stream:

1. Start in `TaskPreparation::rewrite_shuffle`.
2. Find where `StageInputExec<usize>` becomes `ShuffleReadExec`.
3. Follow `StreamAccessor::new(handle.clone())`.
4. Open `crates/sail-execution/src/stream/accessor.rs`.
5. Read `TaskStreamReader::open`.
6. Follow `fetch_worker_stream`.
7. On the worker, open `worker/actor/handler.rs`.
8. Find `handle_fetch_worker_stream`.
9. If the stream is remote, follow `TaskStreamFlightClient`.
10. If the stream is local, follow `LocalStreamManager::fetch_stream`.

This trace connects the control-plane location lookup to the Arrow Flight data plane.

8.30 Reading Exercise: Follow Worker Loss

Trace worker failure handling:

1. Open `crates/sail-execution/src/driver/actor/handler.rs`.
2. Find `handle_probe_lost_worker`.
3. Follow `worker_pool.stop_worker`.
4. Follow `task_assigner.find_worker_tasks`.
5. Follow `job_scheduler.update_task(... Failed ...)`.
6. Follow `refresh_job`.
7. Find `cascade_cancel_task_attempts`.
8. Find `schedule_task_regions`.

This shows how infrastructure failure becomes task attempt retry.

8.31 Takeaways

Sail's distributed runtime is actor-driven. The driver actor coordinates jobs, workers, task assignment, stream ownership, and cleanup. Worker actors register with the driver, heartbeat, run serialized task definitions, serve local streams, and report task status.

The task runner turns a serialized DataFusion physical plan back into an executable plan, rewrites stage inputs into `ShuffleReadExec`, wraps outputs in `ShuffleWriteExec`, and drains the resulting stream through a task monitor.

Streams are identified by (job, stage, partition, attempt, channel). Stream managers handle pending, created, failed, replicated, and cleaned-up local streams. Arrow Flight carries streams between processes.

The next chapter zooms in on shuffle and data movement, using the stream and task machinery from this chapter as the foundation.

9 Shuffle And Data Movement

Shuffle is where a distributed query engine proves that it is actually distributed.

Up to this point, the book has followed Sail from the front door through logical plans, DataFusion physical plans, stage graphs, drivers, workers, and task execution. This chapter zooms in on the data plane: how a `RecordBatch` produced by one task becomes input to another task, possibly on another worker, under a distribution chosen by the query plan.

In Sail, this movement is expressed in a compact set of ideas:

- Data moves as Arrow `RecordBatch` streams.
- A stage boundary is represented in the physical plan by `StageInputExec`.
- At task runtime, `StageInputExec` is rewritten into `ShuffleReadExec`.
- The task's final physical plan is wrapped in `ShuffleWriteExec`.
- The driver assigns stream keys, channel numbers, and read/write locations.
- The stream subsystem moves batches through local memory today, with Arrow Flight as the remote transport shape.

That last phrase is important: Sail already has the architecture of a networked shuffle service, but some remote and blocking pieces are intentionally not finished. This makes the codebase unusually good for learning. You can see the shape of a distributed engine without getting lost in years of accumulated production machinery.

9.1 Current transport and ownership map

In the pinned upstream tree, stream code is consolidated under `crates/sail-execution/src/stream/`. `TaskStreamFactory` in `accessor.rs` creates readers and writers using a task context and the task-runner actor handle. The actor receives `TaskRunnerMessage` values; local stream storage lives under `stream/local/`, storage-backed streams under `stream/storage/`, and Celeborn support under `stream/celeborn/`. These are distinct execution choices, not interchangeable names for an in-memory channel.

For blocking outputs, the factory can select `CelebornTaskStreamWriter` when Celeborn is configured; otherwise it builds the multi-channel writer. A graph extension reusing ordinary shuffles inherits the selected stream machinery. It does not need to implement a separate peer-to-peer graph network.

Internal Flight transport is in `stream/service/transport/mod.rs`. Current upstream uses Hyper's HTTP/2 client directly so it can control the reset budget needed when many shuffle streams terminate early, such as with LIMIT. Transport clones share connection establishment and reconnection while dispatching requests concurrently. Reconnecting a transport does not establish that a stateful native algorithm may safely replay a task. Argentea's whole-job fail-fast policy remains a separate scheduling and state-lifetime contract.

9.2 Code Map

The core shuffle code lives in these files:

Concern	File
Physical shuffle writer	<code>crates/sail-execution/src/ plan/shuffle_write.rs</code>
Physical shuffle reader	<code>crates/sail-execution/src/ plan/shuffle_read.rs</code>
Merging task streams	<code>crates/sail-execution/src/stream/ merge.rs</code>
Round-robin partitioning	<code>crates/sail-physical-plan/ src/repartition.rs</code>
Task input/output definitions	<code>crates/sail-execution/src/ task/definition.rs</code>
Scheduler input/output placement	<code>crates/sail-execution/src/driver/ job_scheduler/core.rs</code>
Runtime shuffle rewrite	<code>crates/sail-execution/src/task_runner/ actor/core.rs</code>
Stream reader and writer traits	<code>crates/sail-execution/src/stream/ reader.rs, crates/sail-execution/src/ stream/writer.rs</code>
Actor bridge to stream manager	<code>crates/sail-execution/src/stream/ accessor.rs</code>
Local stream manager	<code>crates/sail-execution/src/stream/ local/core.rs</code>
In-memory stream replicas	<code>crates/sail-execution/src/stream/ local/memory.rs</code>
Arrow Flight stream service	<code>crates/sail-execution/src/stream/ service/server.rs, crates/sail- execution/src/stream/service/client.rs</code>

If Chapter 8 was about who runs the work, this chapter is about how the work's bytes find the next consumer.

9.3 The Vocabulary Of Movement

Sail's shuffle layer uses a small vocabulary. Once these terms are clear, the rest of the code becomes much easier to read.

Term	Meaning
Stage	A group of physical operators that can run without needing data from a later exchange.
Task	One partition of a stage, executed as one attempt.
Partition	A task-level unit of parallelism within a stage.
Channel	A logical output lane from a producer stage to a consumer stage.
Attempt	A retry number for a task. Attempts are part of stream keys.
Task input	A set of stream locations that a task should read.
Task output	A distribution and locator that describe where a task should write.
Stream key	The stable identity of a task stream: job, stage, partition, attempt, channel.
Location	Where the stream lives: driver, worker, or remote URI for reads; local or remote for writes.

The central identity is TaskStreamKey. Conceptually, it looks like this:

```
TaskStreamKey {
    job_id,
    stage,
    partition,
    attempt,
    channel,
}
```

The channel field is the part that turns one task output into many possible inputs. If a producer stage has four upstream partitions and eight shuffle channels, each producer task may write eight streams. A downstream task then reads the channel or channels assigned to it from all relevant producer partitions.

That gives Sail the basic distributed exchange shape:

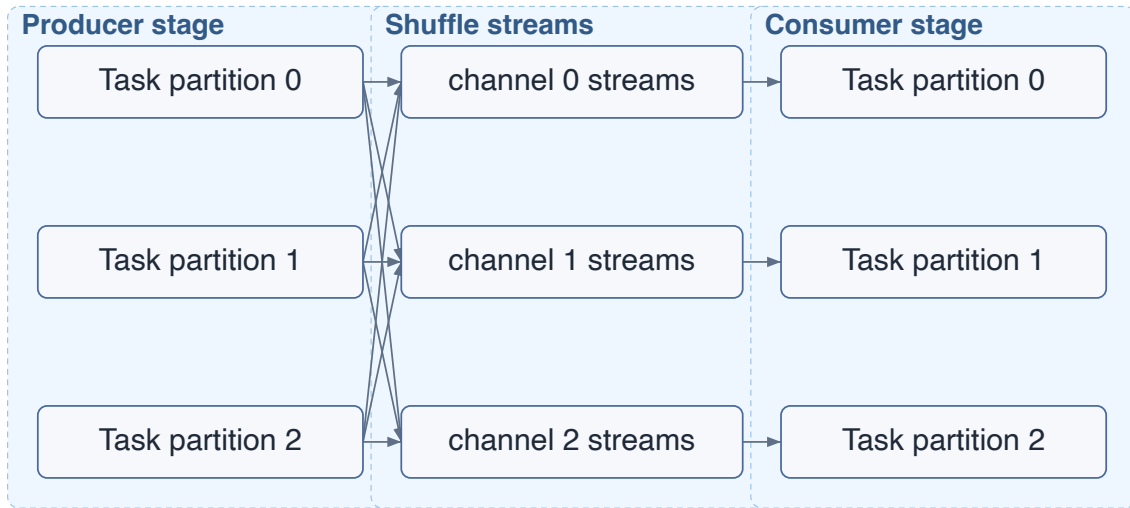


Figure 45: Flowchart 09.1

For a hash shuffle, “channel 1” means “rows whose hash maps to bucket 1.” For a round-robin shuffle, it means “the next row assigned to lane 1.” For a broadcast-like movement, the scheduler may arrange the input keys so multiple consumers can read the same producer output.

9.4 From Job Graph To Task Definition

The job graph knows that one stage depends on another. It does not directly contain open streams. Before a worker can execute a task, the driver must turn graph edges into concrete task inputs and outputs.

That happens in `TaskInputBuilder::build()` and `TaskOutputBuilder::build()` in `crates/sail-execution/src/driver/job_scheduler/core.rs`.

For task inputs, the scheduler:

1. Finds the producer stage for the input.
2. Determines the producer partition count and channel count.
3. Computes the `TaskInputKey` values that this consumer task should read.
4. Finds the latest successful or assigned attempt for each producer partition.
5. Turns those keys into a `TaskInputLocator`.

The input locator records where the consumer should fetch streams from:

```

TaskInputLocator::Driver { keys }
TaskInputLocator::Worker { keys }
TaskInputLocator::Storage { keys }
TaskInputLocator::ShuffleService { channels }
  
```

For task outputs, the scheduler:

1. Reads the output distribution from the job graph.
2. Serializes hash expressions if the output is hash-partitioned.
3. Chooses the output locator.

4. Returns a `TaskOutput`.

Output locators distinguish `Pipelined { replicas }` from `Blocking`. For pipelined input, producer placement selects a driver or worker locator. Blocking input uses a storage locator for the Storage and Flight backends, or a shuffle-service locator for Celeborn. The stream layer implements the selected backend; the scheduler does not open its sockets or files.

The result is not an open socket or a live stream. It is a serializable task definition: inputs, outputs, partition numbers, attempts, and encoded expressions. That definition can be sent to a worker.

9.5 The Runtime Rewrite

The most important shuffle transition happens inside `TaskPreparation::rewrite_shuffle()` in `crates/sail-execution/src/task_runner/preparation.rs`.

The stage-level physical plan contains placeholders:

```
StageInputExec<usize>
```

Those placeholders are not executable by themselves. They say, “this is where this stage reads from an upstream stage.” When the task runner receives the concrete `TaskInput` values from the driver, it rewrites each placeholder into a real reader:

```
StageInputExec<usize> -> ShuffleReadExec
```

Then the task runner wraps the whole plan with a writer:

```
plan -> ShuffleWriteExec(plan)
```

The shape is:

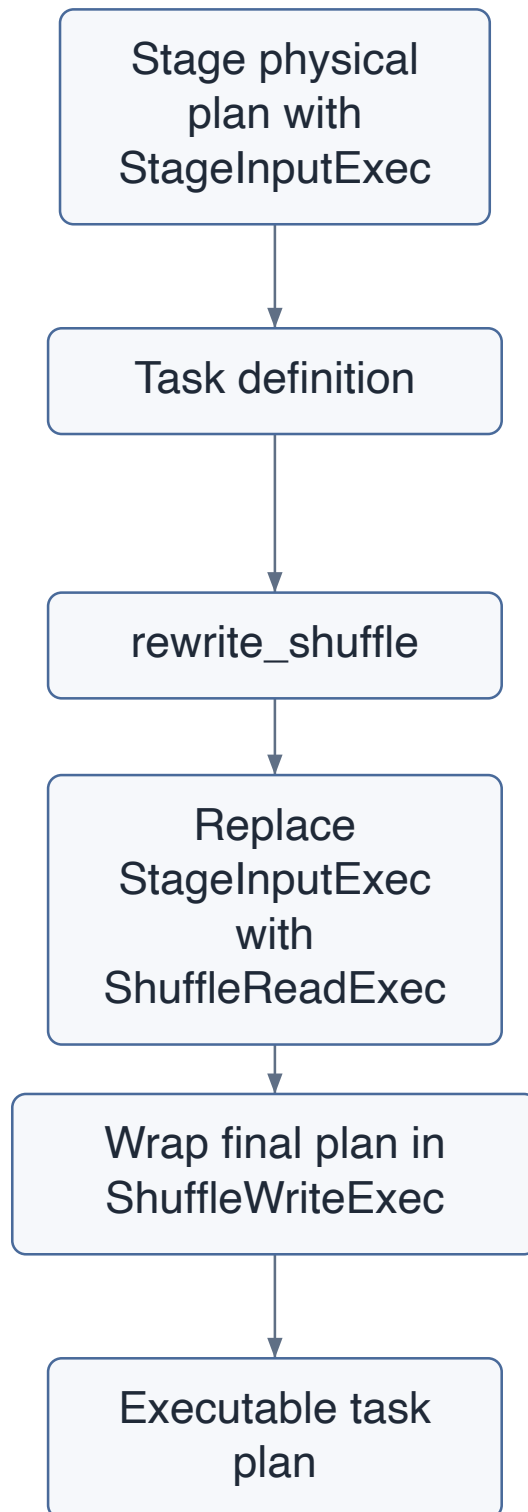


Figure 46: Flowchart 09.2

That rewrite is the bridge between planning and execution:

- Planning says which stages depend on which other stages.
- Scheduling says where the task’s input and output streams live.
- Runtime rewriting turns those locations into physical execution nodes.

This is a powerful Rust pattern in miniature. The stage plan is generic and reusable. The task plan is concrete and contextual. Sail uses a tree transform to keep those concerns separated until the last responsible moment.

9.6 Writing Shuffle Output

ShuffleWriteExec is a DataFusion ExecutionPlan implementation, but it behaves a little differently from ordinary relational operators. It does not produce meaningful rows downstream. Its job is to consume its child plan, partition the child batches, and write the resulting batches into task streams.

Its main fields are:

```
pub struct ShuffleWriteExec {  
    plan: Arc<dyn ExecutionPlan>,  
    partitioning: ShufflePartitioning,  
    properties: Arc<PlanProperties>,  
    writer: Arc<dyn TaskStreamWriter>,  
}
```

The operator runs its child, partitions batches according to partitioning, and passes the resulting channel batches to its writer. It does not retain a matrix of physical write locations. TaskStreamFactory constructs the writer from the task key, output definition and schema during preparation.

9.6.1 Partitioning

Sail supports two writer-side partitioning modes here:

```
Partitioning::Hash(keys, channels)  
Partitioning::RoundRobinBatch(channels)
```

There is also UnknownPartitioning, which Sail treats like round-robin for write purposes.

Hash partitioning uses DataFusion’s BatchPartitioner. This is the natural choice because DataFusion already knows how to evaluate physical expressions against Arrow batches and assign rows to hash buckets.

Round-robin partitioning uses Sail’s own RowRoundRobinPartitioner in crates/sail-physical-plan/src/repartition.rs. It is intentionally Arrow-native:

1. Build row-index arrays for each destination partition.
2. Use Arrow’s take_arrays() compute kernel to select the rows for each partition.
3. Construct new RecordBatch values with the same schema.

That avoids converting rows into Rust structs or ad hoc values. The shuffle layer stays columnar.

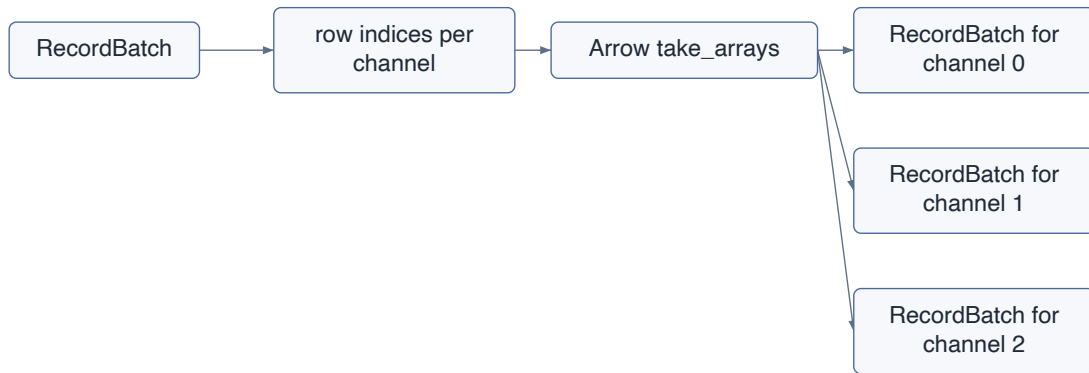


Figure 47: Flowchart 09.3

The start index for round-robin is derived from the input partition:

```
start = (input_partition * num_partitions) / num_input_partitions
```

That small detail helps distribute initial rows across output channels when multiple input partitions are writing at once.

9.6.2 Sinks And Side Effects

The `shuffle_write()` helper opens one task sink with `writer.open(partition)`. It skips empty child batches, partitions each remaining batch into a `Vec<Option<RecordBatch>>`, and passes the whole vector to `sink.write`. The sink handles its physical channels.

Completion has three paths:

- Exhausted input commits the sink.
- A `TaskStreamWriteState::Closed` result stops reading and aborts the sink.
- A read, partition or write error triggers best-effort abort and returns the original error.

The current code still uses `abort` for successful early termination; it has a TODO to model that separately. Do not equate every abort with query failure.

This matters because a downstream consumer may stop early. A `LIMIT` query is the classic example: once the driver has enough rows, some readers may close. Sail treats closed sinks differently from failed sinks so that early termination does not necessarily become a query failure.

`ShuffleWriteExec::execute()` returns a stream, because `DataFusion` expects every physical operator to return a `SendableRecordBatchStream`. But the useful work happens as a side effect: writing to task streams. After writing, the operator emits an empty `RecordBatch`.

That makes `ShuffleWriteExec` a boundary operator. It turns a normal `DataFusion` stream into Sail task output.

9.7 Reading Shuffle Input

`ShuffleReadExec` is the mirror image. Its fields are:

```
pub struct ShuffleReadExec {
    properties: Arc<PlanProperties>,
    reader: Arc<dyn TaskStreamReader>,
}
```

`execute(partition, context)` opens `reader.open(partition)` and adapts the returned `TaskStreamSource` to a `DataFusion record-batch stream`. The reader owns the routing information and merges the producers for that partition. For bounded input, the operator then coalesces batches using one `LimitedBatchCoalescer` shared across producers. Unbounded input bypasses that coalescer so partial batches can be emitted promptly.

The merge is handled by `MergedRecordBatchStream` in `crates/sail-execution/src/stream/merge.rs`. Internally, it uses a `SelectAll` over the task streams. That means batches are yielded as upstream streams become ready, not by fully draining one producer before reading the next.

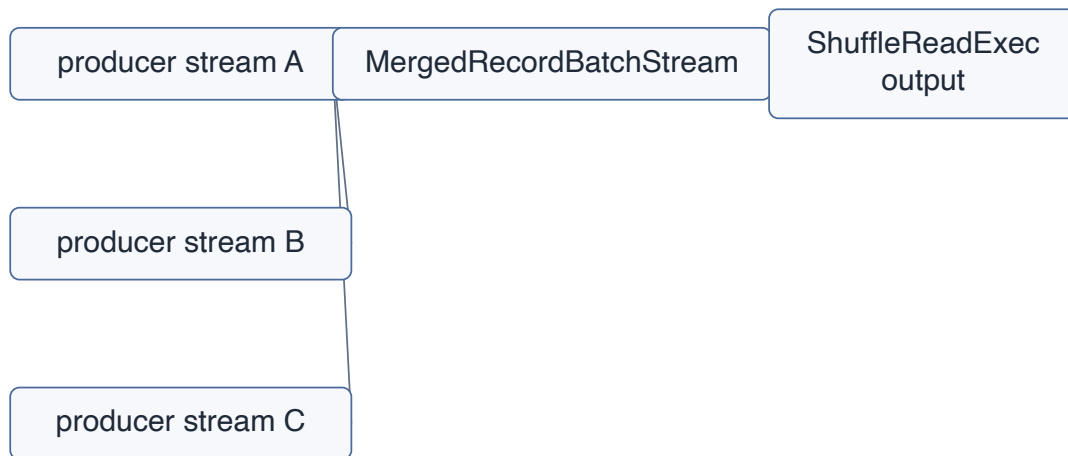


Figure 48: Flowchart 09.4

This is why a consumer task can start processing a pipelined shuffle before every producer has finished, as long as its input streams are available.

9.8 Stream Readers, Writers and Sinks

The current traits in `stream/reader.rs` and `stream/writer.rs` open one task partition. The reader returns a `TaskStreamSource`; the writer returns a sink. Schema and locator information are supplied when `TaskStreamFactory` constructs the concrete objects, rather than passed into each open call.

```
async fn open(&self, partition: usize) -> Result<TaskStreamSource>;
```

The writer's corresponding method returns `Result<Box<dyn TaskStreamSink>>`. A task sink handles all output channels. Its `write` accepts `Vec<Option<RecordBatch>>`, with at most one batch per channel per call, and returns `TaskStreamWriteState`. `commit` and `abort` consume the boxed sink. A `TaskStreamChannelSink` instead represents exactly one physical channel.

The multi-channel implementation can write independent channels concurrently, while calls preserve order within each channel. It retains sinks so a failed write can be followed by abort. Closed channels are distinct from active ones; early consumer termination is part of the stream contract, not necessarily a query error.

Concrete readers and writers communicate with the task runner through the accessor layer. That actor owns local streams and delegates storage or Celeborn operations to the configured managers. The DataFusion operators depend on the small stream traits, while task definitions determine the actual data movement.

9.9 Local Memory Streams

The local stream manager has three visible states for local streams:

Pending
Created
Failed

This solves a real scheduling race. A consumer may ask for a stream before the producer has created it. Rather than fail immediately, the manager can register that the stream is pending and wake the reader when the producer creates it. If creation never happens, the pending stream eventually times out.

For in-memory streams, Sail uses replicas. The task output locator records `TaskOutputLocator::Pipelined { replicas }`; the local manager creates the corresponding memory stream.

The producer writes each batch to the active replica senders. This supports multiple readers for the same produced stream, which is useful for broadcast-like movement and for cases where more than one consumer needs the same task output.

The memory stream implementation also handles closed receivers. If a receiver is closed, the producer can keep writing to remaining active replicas. Once no active replicas remain, the sink can report Closed.

That behavior is one of the quiet but important pieces of distributed query execution: the data plane must distinguish “nobody needs this anymore” from “the query is broken.”

9.10 Arrow Flight As The Remote Shape

Sail’s stream service transports task streams between processes using Arrow Flight. This is an implemented path, including the two-host extension tests.

On the server side, `do_get`:

1. Decodes a `TaskStreamTicket`.
2. Fetches the requested task stream.
3. Encodes `RecordBatch` values as Flight data.
4. Returns a Flight stream.

On the client side, `TaskStreamFlightClient::fetch_task_stream()`:

1. Builds a ticket for the requested task stream.
2. Calls Flight do_get.
3. Wraps the returned Flight data as a RecordBatch stream.

The important architecture point is that Flight does not replace Arrow batches. It transports them. Sail's task operators still speak in RecordBatch streams on both sides of the network boundary.

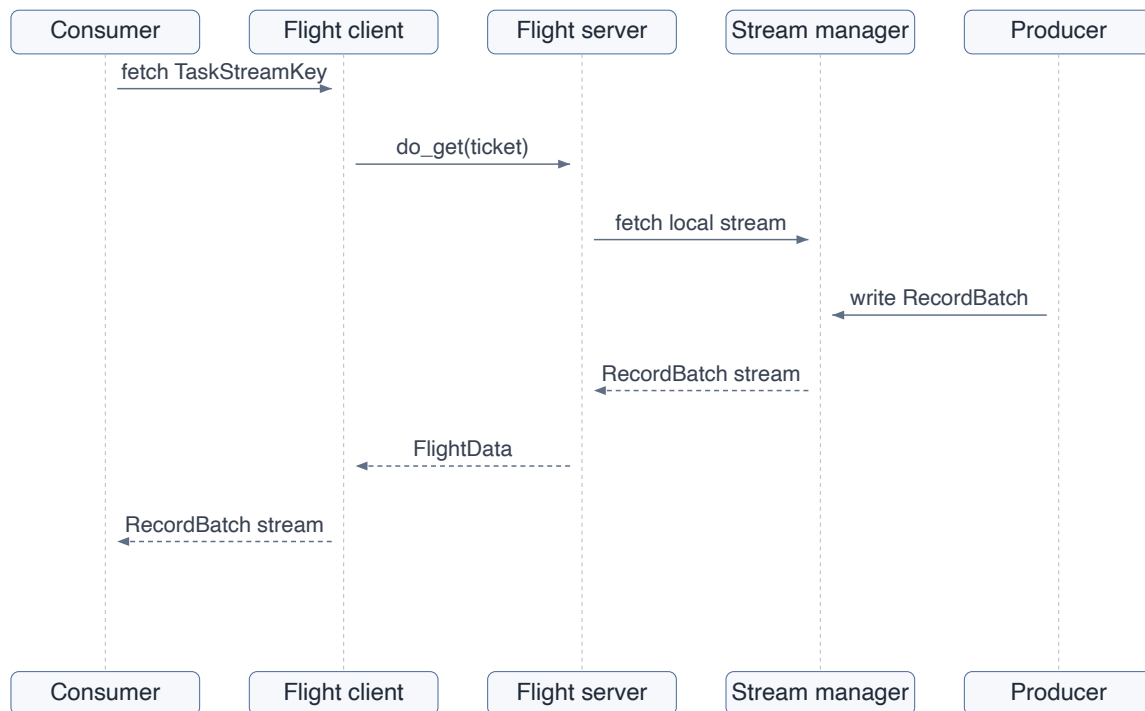


Figure 49: Sequence diagram 09.5

This is the main reason Arrow Flight fits a system like Sail so well. It lets the engine preserve its columnar execution model while crossing process and machine boundaries.

9.11 A Hash Shuffle Walkthrough

Imagine a query like:

```

SELECT customer_id, COUNT(*)
FROM orders
GROUP BY customer_id
  
```

At scale, each worker can scan a subset of orders, but final groups must be brought together by `customer_id`. Rows with the same `customer_id` need to land in the same downstream partition.

The high-level plan is:

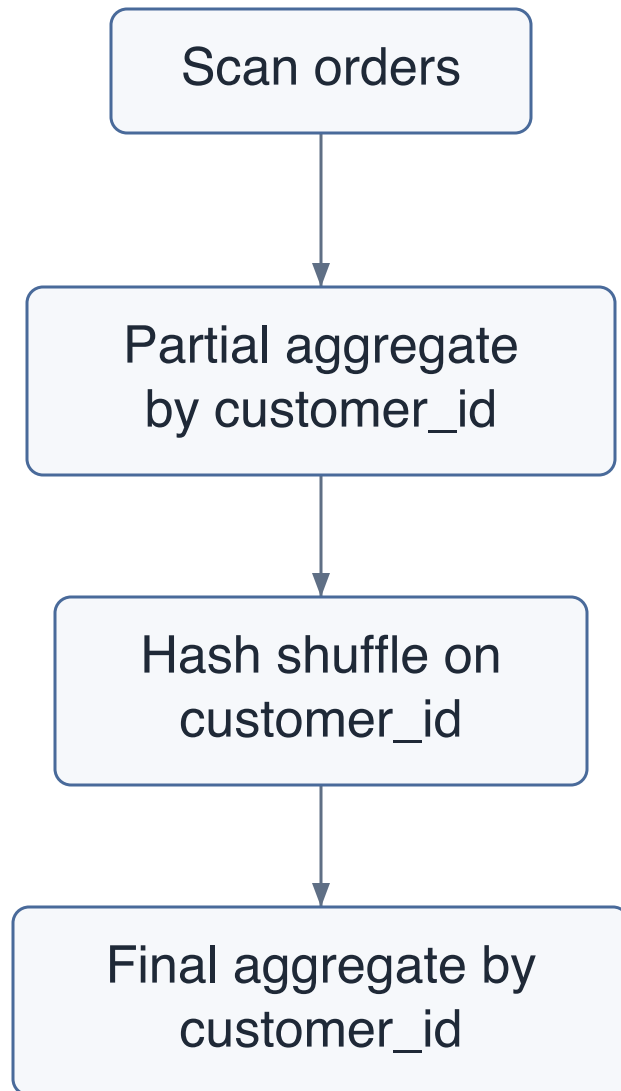


Figure 50: Flowchart 09.6

In Sail terms:

1. The planner inserts a stage boundary at the exchange.
2. The producer stage output distribution is Hash.
3. The scheduler serializes the hash expression into `TaskOutputDistribution::Hash`.
4. The task runner decodes that expression back into DataFusion physical expressions.
5. `ShuffleWriteExec` uses DataFusion's `BatchPartitioner`.
6. Each producer task writes one stream per hash channel.
7. Each consumer task opens the producer streams for its assigned channel.

8. ShuffleReadExec merges those streams.
9. The final aggregate sees a normal input stream of Arrow batches.

The row movement looks like this:

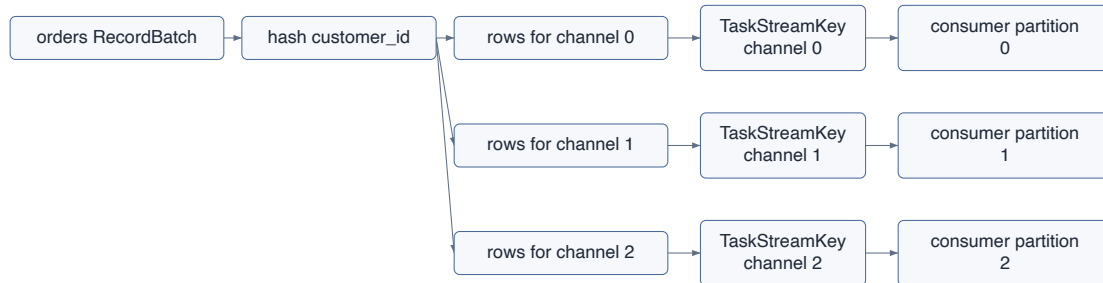


Figure 51: Flowchart 09.7

Notice what does not happen:

- Sail does not serialize rows into a custom row format at the shuffle boundary.
- Sail does not make ShuffleReadExec understand hash expressions.
- Sail does not make the scheduler evaluate data.

Each layer keeps a narrow job.

9.12 Other Movement Patterns

Hash shuffle is the easiest to visualize, but Sail's input-key construction can represent several movement patterns.

9.12.1 One-To-One

A downstream partition reads the corresponding upstream partition. This is the cheapest movement pattern and is useful when partitioning is already compatible.

```

producer partition 0 -> consumer partition 0
producer partition 1 -> consumer partition 1
  
```

9.12.2 Hash

Every producer may write every channel. Each consumer reads the channel or channels assigned to it.

```

producer partition N, channel C -> consumer partition C
  
```

9.12.3 Round Robin

Rows are spread across output channels without using data values as keys. Sail's round-robin partitioner uses Arrow take kernels to construct the destination batches.

9.12.4 Broadcast

The same upstream output is made readable by multiple downstream consumers. Local memory stream replicas are the stream-level mechanism that makes this possible.

9.12.5 Merge

Many upstream streams are merged into one downstream stream. `MergedRecordBatchStream` is the simple core abstraction here.

9.12.6 Rescale

Producer and consumer partition counts may differ. The scheduler's input-key builder can assign groups of producer streams to consumer partitions.

The exact key-building logic belongs to the scheduler, not the stream operators. This is another example of Sail's separation between control plane and data plane.

9.13 Failure, Attempts, And Early Termination

Distributed data movement needs identity. Without identity, retries are dangerous: a consumer might accidentally read data from an old failed attempt.

Sail includes attempt in every task stream key:

```
job_id / stage / partition / attempt / channel
```

The scheduler chooses the latest attempt when building task input keys. That lets the system distinguish replacement work from old work.

There are also several important runtime behaviors:

- Pending streams allow consumers and producers to start in either order.
- Pending stream timeouts prevent consumers from waiting forever.
- Closed sinks can be normal if downstream no longer needs the data.
- Failed streams are distinct from closed streams.
- Stream errors are mapped back into `DataFusion` errors at the merge boundary.

These are small details, but they are the difference between a toy exchange and an engine that can tolerate real distributed timing.

9.14 Extension Use and Qualification Boundaries

Argentea uses existing stage dependencies, partition distributions and stream transport to exchange graph messages. Its worker-local native state adds an ownership constraint: a later phase must reach the worker that owns its graph partition. That constraint belongs in placement and lifecycle handling, not a second graph transport. Chapter 13 describes the focused host additions.

The code includes blocking storage and Celeborn paths; their presence is not proof that every extension workload has been qualified on each backend. Record the selected backend with each test. The two-host graph evidence establishes the configuration actually run, not interchangeable behavior under every shuffle service or storage failure.

Attempt identities distinguish streams from different task attempts. They do not reconstruct lost native graph state. The current native graph job therefore fails as a whole on worker loss instead of replaying one task against missing state. Reusing Sail's transport preserves its data movement contracts while leaving this extension-specific recovery limitation explicit.

9.15 Reading Exercise

Trace one hash-shuffled row through the code:

1. Start in `TaskPreparation::rewrite_shuffle()`.
2. Find where `TaskOutput::partitioning()` converts task output metadata into `Partitioning::Hash`.
3. Open `ShuffleWriteExec::execute()` and follow the creation of the partitioner.
4. Follow `shuffle_write()` until it calls `sink.write(batch)`.
5. Open `StreamAccessor` and see how the write location becomes a stream-manager message.
6. Then reverse direction through `ShuffleReadExec::execute()`.
7. End in `MergedRecordBatchStream::poll_next()`.

The important question to ask at every step is: “Is this layer deciding where data should go, or only carrying out a decision made earlier?”

That question is the key to reading distributed query engines.

9.16 Takeaways

Sail’s shuffle layer is small enough to study and rich enough to teach the real ideas:

- Shuffle is expressed as Arrow `RecordBatch` streams, not row objects.
- The scheduler chooses keys, attempts, channels, and locations.
- The task runner rewrites stage placeholders into concrete shuffle operators.
- `ShuffleWriteExec` partitions and writes batches as a side effect.
- `ShuffleReadExec` opens task streams and merges them.
- Local memory streams support pending readers and replicas.
- Arrow Flight provides the natural shape for remote batch transport.
- The open areas around remote, blocking, disk, and adaptive shuffle are prime extension points.

The next chapter moves from movement to memory and execution behavior: how Arrow batch size, streaming, boundedness, and operator properties influence distributed execution.

10 The Sail Spec And Plan Resolver

Spark Connect sends unresolved intent. DataFusion executes resolved logical and physical plans. Sail’s plan layer is the translation space between those two worlds.

This chapter is about that translation space.

In previous chapters, we followed query execution once DataFusion had a physical plan. Now we step earlier in the life of a query, into the code that decides what a Spark Connect relation, SQL statement, unresolved column, function call, catalog command, or UDF registration actually means inside Sail.

The central idea is simple but powerful:

Spark Connect proto or SQL
-> Sail spec
-> DataFusion LogicalPlan

- > DataFusion optimizer
- > Sail/DataFusion physical planning
- > distributed execution

The Sail spec is not DataFusion's logical plan. It is also not exactly Spark Connect's protobuf model. It is Sail's own unresolved intermediate representation, designed to be easy to parse, serialize, inspect, and resolve.

That makes it one of the most important extension points in the whole architecture.

10.1 Code Map

The main files for this chapter are:

Concern	File
Plan execution entry point	<code>crates/sail-plan/src/lib.rs</code>
Resolver entry point	<code>crates/sail-plan/src/resolver/plan.rs</code>
Resolver state	<code>crates/sail-plan/src/resolver/state.rs</code>
Query resolver dispatch	<code>crates/sail-plan/src/resolver/query/mod.rs</code>
Expression resolver dispatch	<code>crates/sail-plan/src/resolver/expression/mod.rs</code>
Command resolver dispatch	<code>crates/sail-plan/src/resolver/command/mod.rs</code>
Attribute resolution	<code>crates/sail-plan/src/resolver/expression/attribute.rs</code>
Function resolution	<code>crates/sail-plan/src/resolver/expression/function.rs</code>
Table and data source reads	<code>crates/sail-plan/src/resolver/query/read.rs</code>
Repartition nodes	<code>crates/sail-plan/src/resolver/query/repartition.rs</code>
WithRelations support	<code>crates/sail-plan/src/resolver/query/with_relations.rs</code>
Sail spec plan model	<code>crates/sail-common/src/spec/plan.rs</code>
Sail spec expression model	<code>crates/sail-common/src/spec/expression.rs</code>
Sail spec data types	<code>crates/sail-common/src/spec/data_type.rs</code>
Spark Connect relation conversion	<code>crates/sail-spark-connect/src/proto/plan.rs</code>
Spark Connect expression conversion	<code>crates/sail-spark-connect/src/proto/expression.rs</code>
Spark Connect execution handler	<code>crates/sail-spark-connect/src/service/plan_executor.rs</code>
DataFusion extension physical planner	<code>crates/sail-session/src/planner.rs</code>

Useful external references:

- [Spark Connect Overview](#)
- [Apache Spark Connect architecture page](#)
- [Application Development with Spark Connect](#)
- [Spark Connect base.proto](#)

- Spark Connect `relations.proto`
- Spark Connect `expressions.proto`

Spark’s own documentation describes Spark Connect as a client-server architecture where clients send unresolved logical plans over gRPC and receive Arrow-encoded batches back. Sail follows that same shape, but swaps Spark’s analyzer and engine for Sail’s resolver, DataFusion’s optimizer, and Sail’s distributed runtime.

10.2 Why Sail Has A Spec Layer

The spec layer lives in `sail_common::spec`.

At first glance, it may look like a reimplementation of Spark Connect relations and expressions. That is only partly true. The comment above `spec::Plan` explains the design: the starting point is Spark Connect’s Relation model, but Sail makes intentional changes.

The spec layer:

- separates query plans from command plans,
- avoids raw SQL strings as unresolved plan nodes,
- uses parsed schemas rather than schema strings,
- prefers Rust naming and serde-friendly enum shapes,
- adds nodes needed by SQL and Sail that are not direct Spark Connect relation nodes,
- stores data types, literals, expressions, and plans in one common model.

That gives Sail a stable internal contract:

```
pub enum Plan {
    Query(QueryPlan),
    Command(CommandPlan),
}
```

Both query and command plans carry optional `plan_id` values:

```
pub struct QueryPlan {
    pub node: QueryNode,
    pub plan_id: Option<i64>,
}
```

```
pub struct CommandPlan {
    pub node: CommandNode,
    pub plan_id: Option<i64>,
}
```

Those plan IDs matter because Spark Connect clients often reference subplans and attributes by ID. Sail preserves that identity through conversion and uses it during resolution.

10.3 The Full Planning Pipeline

The public planning entry point is `resolve_and_execute_plan()` in `crates/sail-plan/src/lib.rs`.

Despite the name, it does more than execute. It performs the whole plan path up to an executable physical plan:

```
pub async fn resolve_and_execute_plan(
    ctx: &SessionContext,
    config: Arc<PlanConfig>,
    plan: spec::Plan,
) -> PlanResult<(Arc<dyn ExecutionPlan>, Vec<StringifiedPlan>)>
```

The steps are:

1. Build a PlanResolver.
2. Resolve a `spec::Plan` into a DataFusion LogicalPlan.
3. Record the initial logical plan for explain output.
4. Ask DataFusion to create a DataFrame from the logical plan.
5. Optimize the logical plan with DataFusion's session state.
6. Rewrite streaming plans if needed.
7. Ask the session query planner to create a physical plan.
8. Rename physical output fields back to user-facing names when needed.
9. Return the physical plan and plan strings.

In diagram form:

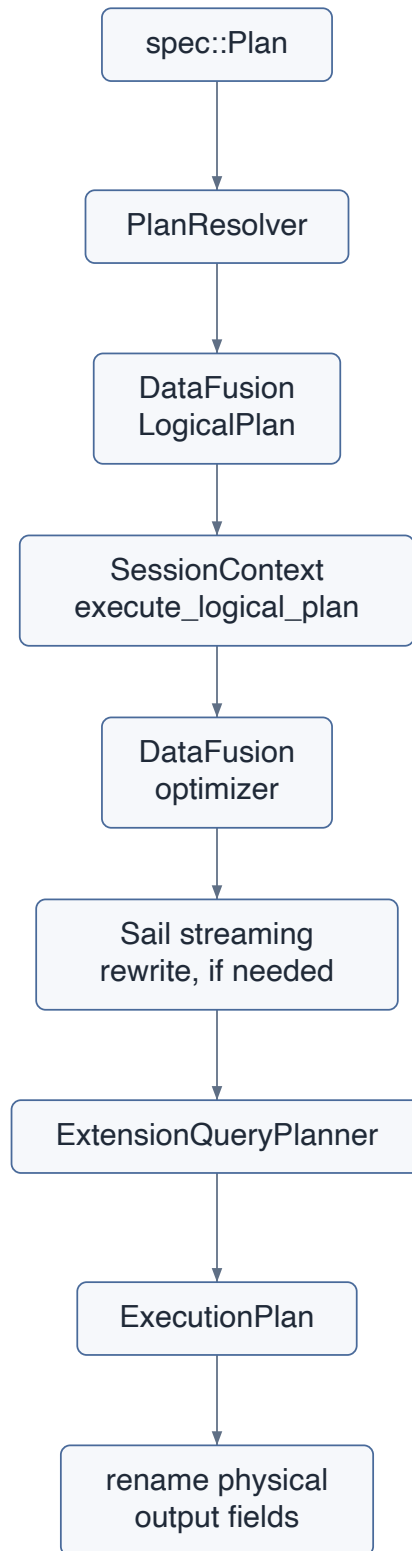


Figure 52: Flowchart 10.1

This is the same boundary we saw from the other side in earlier chapters:

- Chapter 6 focused on DataFusion plans.
- Chapter 7 split physical plans into a job graph.
- This chapter explains how Sail gets to the logical plan in the first place.

10.4 Spark Connect To Sail Spec

Spark Connect requests arrive as generated protobuf Rust types in `crates/sail-spark-connect`. The conversion layer turns those types into Sail spec values.

The important conversion file is:

```
crates/sail-spark-connect/src/proto/plan.rs
```

For relations:

```
impl TryFrom<Relation> for spec::Plan
impl TryFrom<Relation> for spec::QueryPlan
impl TryFrom<Relation> for spec::CommandPlan
impl TryFrom<RelType> for RelationNode
```

This conversion has to make a decision: is a Spark Connect relation a query node or a command node?

Sail uses an internal helper enum:

```
enum RelationNode {
    Query(spec::QueryNode),
    Command(spec::CommandNode),
}
```

That matters because Spark Connect has several operation shapes. Some produce relations; others execute side effects or return command results. Sail normalizes them into `spec::Plan::Query` or `spec::Plan::Command`.

For expressions, the conversion file is:

```
crates/sail-spark-connect/src/proto/expression.rs
```

There, Spark Connect expressions become `spec::Expr`:

```
impl TryFrom<Expression> for spec::Expr
```

Examples:

Spark Connect expression	Sail spec expression
Literal	<code>spec::Expr::Literal</code>
UnresolvedAttribute	<code>spec::Expr::UnresolvedAttribute</code>
UnresolvedFunction	<code>spec::Expr::UnresolvedFunction</code>
ExpressionString	parsed SQL expression, then Sail spec
UnresolvedStar	<code>spec::Expr::UnresolvedStar</code>
Alias	<code>spec::Expr::Alias</code>
Cast	<code>spec::Expr::Cast</code>
Window	<code>spec::Expr::Window</code>
CommonInlineUserDefinedFunction	<code>spec::Expr::CommonInlineUserDefinedFunction</code>
SubqueryExpression	<code>spec::Expr::Subquery</code>

This is the first important lesson: Sail does not resolve columns or functions while converting protobuf messages. It only parses the client's intent into Sail's own unresolved representation.

That keeps Spark Connect compatibility code separate from analysis.

10.5 SQL To Sail Spec

Spark Connect is not the only source of spec plans. SQL also enters the same model.

The conversion layer uses `sail_sql_analyzer` helpers such as:

```

parse_one_statement
from_ast_statement
parse_expression
from_ast_expression
parse_object_name
from_ast_object_name

```

For example, Spark Connect may send an `ExpressionString`. Sail parses that expression string into an AST, then converts the AST into `spec::Expr`.

The same pattern appears for SQL commands in `handle_execute_sql_command()`:

```

SQL string
-> Spark Connect relation shape
-> Sail spec plan
-> resolver
-> DataFusion logical plan

```

This unification is important. It means that SQL and DataFrame APIs meet before DataFusion planning, not after.

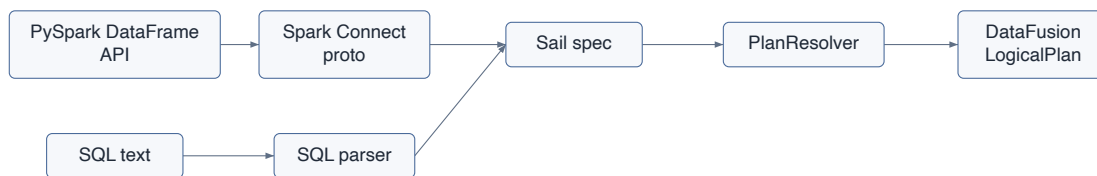


Figure 53: Flowchart 10.2

For extension authors, this is a clue: if an extension is only available through SQL, it is not really integrated with the Spark Connect-style architecture. A good extension should be expressible in the spec layer or reachable through a spec-producing parser.

10.6 The Resolver Entry Point

The resolver itself is tiny at the top:

```
pub struct PlanResolver<'a> {
    ctx: &'a SessionContext,
    config: Arc<PlanConfig>,
}
```

It holds:

- a DataFusion SessionContext,
- a PlanConfig.

The main entry point is `resolve_named_plan()` in `crates/sail-plan/src/resolver/plan.rs`.

```
pub async fn resolve_named_plan(&self, plan: spec::Plan) -> PlanResult<NamedPlan>
```

It returns a `NamedPlan`:

```
pub struct NamedPlan {
    pub plan: LogicalPlan,
    pub fields: Option<Vec<String>>,
}
```

The `fields` value is subtle. Sail often gives internal columns opaque field IDs during analysis to avoid name collisions. But the user still expects Spark-like output names. For query plans, `resolve_named_plan()` captures the user-facing field names so the physical plan can later be renamed back.

Commands do not get output field renaming in the same way:

```
spec::Plan::Query    -> LogicalPlan plus user-facing fields
spec::Plan::Command -> LogicalPlan with fields = None
```

10.7 Resolver State

Most of the interesting resolver behavior is not in `PlanResolver` itself. It is in `PlanResolverState`.

The state tracks:

- generated internal field IDs,
- user-facing field names,
- hidden fields,
- plan IDs attached to fields,
- the outer query schema for correlated subqueries,
- aggregate resolution mode,
- CTEs,
- WithRelations subquery references,
- temporary config flags,
- positional and named parameter values.

The most important field-resolution trick is this:

```
user name: "customer_id"
internal field ID: "#7"
```

The DataFusion logical plan may carry #7, but Sail remembers that the user-facing name is customer_id.

Why do this? Because DataFrame plans can easily contain duplicate column names:

```
SELECT left.id, right.id
FROM left
JOIN right ON left.id = right.id
```

If both columns were simply named id, subsequent resolution would become ambiguous too early or in the wrong way. Sail uses opaque internal names to keep the logical plan well-formed while preserving Spark-facing names for display and final output.



Figure 54: Flowchart 10.3

The state also records hidden fields. Hidden fields are temporary columns needed for analysis or execution but not intended to appear in the final output. For example, a join or sort rewrite may need to carry a helper field through a subplan.

After resolving a query, `resolve_query_plan()` calls `remove_hidden_fields()` so the public logical plan does not expose those helper fields.

10.8 Query Resolution

The query resolver dispatch lives in:

`crates/sail-plan/src/resolver/query/mod.rs`

It is an async recursive dispatcher over `spec::QueryNode`.

The main structure is:

```
match plan.node {
  QueryNode::Read { .. } => ...
  QueryNode::Project { .. } => ...
  QueryNode::Filter { .. } => ...
  QueryNode::Join(join) => ...
  QueryNode::Aggregate(aggregate) => ...
  QueryNode::Repartition { .. } => ...
  QueryNode::WithRelations { .. } => ...
  ...
}
```

Each variant delegates to a focused resolver file:

Query shape	Resolver file
Read	query/read.rs
Project	query/project.rs
Filter	query/filter.rs
Join	query/join.rs
Aggregate	query/aggregate.rs
Sort	query/sort.rs
Limit	query/limit.rs
Repartition	query/repartition.rs
WithRelations	query/with_relations.rs
UDF and UDTF plan forms	query/udf.rs, query/udtf.rs

After each query node is resolved, the dispatcher does two things:

```
verify_query_plan(plan, state)
register_schema_with_plan_id(plan, plan_id, state)
```

That means every resolved plan must use fields known to the resolver state. If a new resolver creates a field and forgets to register it, Sail fails with an internal resolver error.

That is a useful invariant. It catches bugs at the boundary where Spark-compatible names become DataFusion columns.

10.9 Reading Tables And Data Sources

query/read.rs is a rich file because reading is where catalogs, file formats, temporary views, CTEs, and dynamic table names meet.

For a named table, Sail handles several cases:

1. If the name looks like <format>.<path> and the format is registered, treat it as a direct data source read.
2. If the name matches a CTE, use the CTE plan.
3. Otherwise, ask the CatalogManager for a table or view.
4. For a table, build SourceInfo and ask DataSourceRegistry to create a source.
5. For a persistent view, parse the stored SQL definition and resolve it.
6. For a temporary view, clone and rename the stored logical plan.

The rough flow:

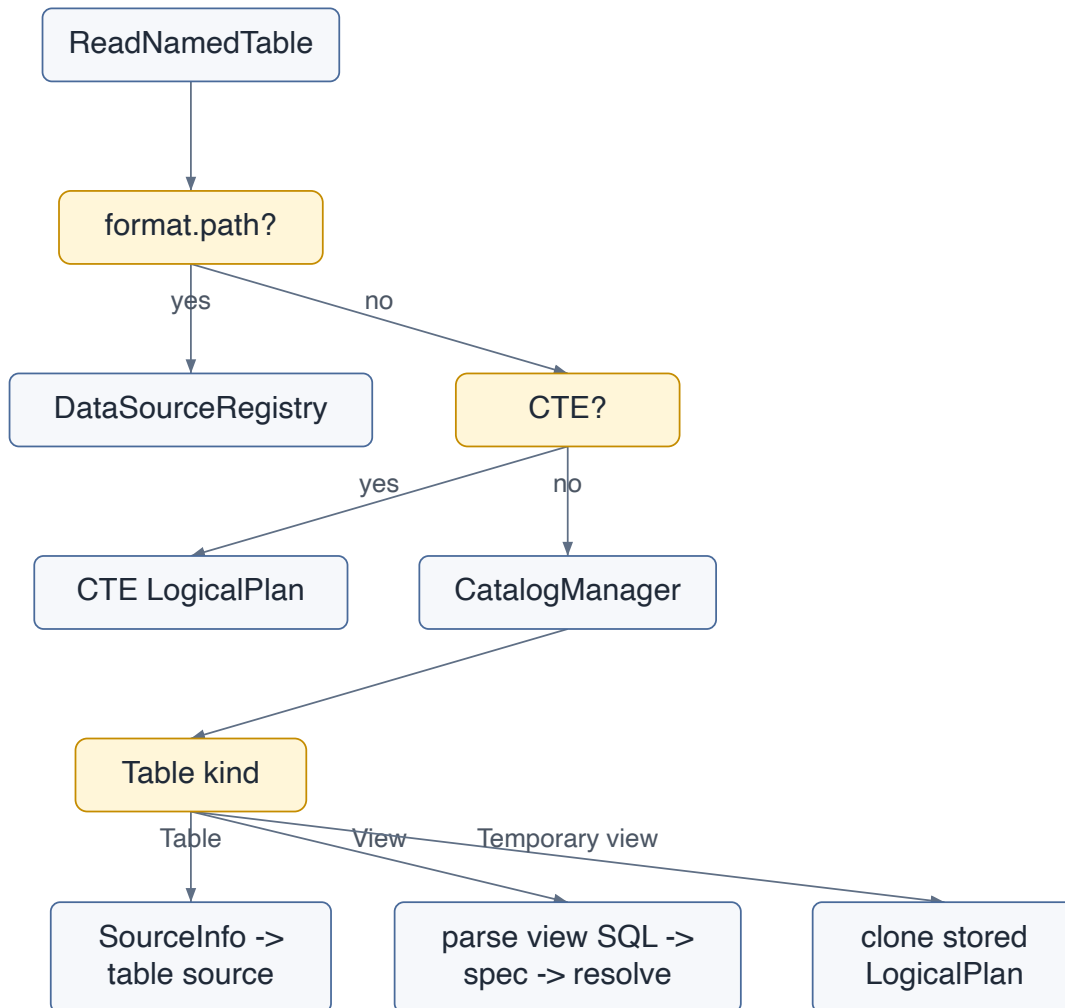


Figure 55: Flowchart 10.4

This is where Sail's session extensions begin to matter:

- CatalogManager supplies table and view metadata.
- DataSourceRegistry turns format-specific metadata into table sources.
- PlanService provides display and formatting helpers elsewhere in resolution.

Those are not global singletons. They are DataFusion session extensions. That design lets Sail attach Spark-compatible services to a normal DataFusion SessionContext.

10.10 Project Resolution And Rewriters

Projection is deceptively complex. The Project node has to handle ordinary expressions, wildcard expansion, aliases, generators, windows, aggregate shortcuts, and Spark-specific functions like `spark_partition_id()`.

`resolve_query_project()` follows this pattern:

1. Resolve the input plan, or create a one-row empty input when the project has no input.
2. Resolve each spec expression into a `NamedExpr`.
3. Expand wildcards.
4. Run projection rewriters.
5. Rewrite multi-expression functions.
6. If aggregate functions are present, rewrite the projection as an aggregate.
7. Otherwise build a `DataFusion Projection`.

The projection rewriters are especially instructive:

`MonotonicIdRewriter`
`SparkPartitionIdRewriter`
`ExplodeRewriter`
`WindowRewriter`

They transform expressions that cannot remain as plain scalar expressions into plan shapes that `DataFusion` can execute.

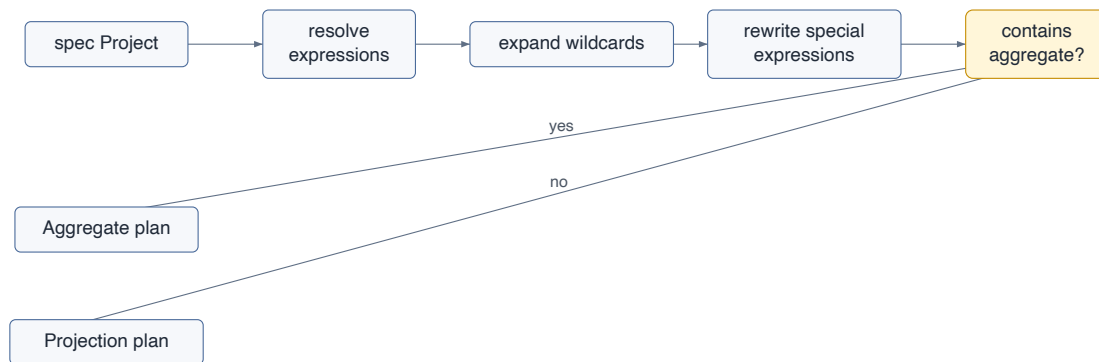


Figure 56: Flowchart 10.5

This pattern appears across `Sail`: the spec layer preserves Spark intent, then the resolver reshapes that intent into `DataFusion`-compatible logical plans.

10.11 Attribute Resolution

Attributes are where users feel analysis quality most sharply.

The resolver handles:

- case-insensitive matching,
- qualified names,
- nested struct fields,
- plan ID filtering,
- aggregate aliases,
- hidden fields,
- outer references for correlated subqueries.

The central method is:

```
resolve_expression_attribute(...)
```

It tries candidates in a careful order:

1. Aggregate fields visible in HAVING.
2. Normal fields and nested fields.
3. Aggregate grouping fields.
4. Hidden fields.
5. Outer query fields.

Nested-field resolution is Arrow-aware. For a struct field, Sail builds a DataFusion `get_field` scalar function expression rather than inventing a custom row accessor.

For example:

```
SELECT address.city FROM customers
```

becomes conceptually:

```
column(address)
  -> get_field("city")
```

Qualified matching supports forms like:

```
column
table.column
schema.table.column
catalog.schema.table.column
```

The helper `qualifier_matches()` performs case-insensitive comparison against DataFusion `TableReference` values.

This is one of the places where the resolver has to act more like Spark than vanilla DataFusion. The user's unresolved expression is not just a name; it carries Spark resolution expectations.

10.12 Function Resolution

Function resolution lives mainly in:

```
crates/sail-plan/src/resolver/expression/function.rs
crates/sail-plan/src/function/mod.rs
```

The function resolver follows a layered lookup:

1. Normalize the function name.
2. Extract named arguments.
3. Check the catalog for registered functions, including PySpark UDFs.
4. Check Sail built-in scalar and generator functions.
5. Check Sail built-in aggregate functions.
6. Build a DataFusion expression.
7. Format a Spark-like display name.

The catalog lookup comes before built-ins because Spark Connect does not reliably mark all UDF calls in a way Sail can trust. The code even notes this:

`is_user_defined_function` is always false, so we need to check UDFs before built-in functions.

For built-ins, Sail has registries:

```
BUILT_IN_SCALAR_FUNCTIONS
BUILT_IN_GENERATOR_FUNCTIONS
BUILT_IN_TABLE_FUNCTIONS
```

For aggregate functions, the resolver also handles clauses like:

- DISTINCT,
- FILTER,
- ORDER BY,
- IGNORE NULLS.

For PySpark UDFs, Sail carries enough information to later execute Python code:

```
pub(super) struct PythonUdf {
    pub python_version: String,
    pub eval_type: spec::PySparkUdfType,
    pub command: Vec<u8>,
    pub output_type: DataType,
}
```

This is the same pattern we saw in the PySpark chapter: the Python function is not run in the resolver. It is represented as a plan expression that later physical execution can evaluate.

10.13 Commands Become Logical Plans Too

Commands enter through `spec::CommandPlan` and are resolved in:

```
crates/sail-plan/src/resolver/command/mod.rs
```

The command resolver handles catalog operations, writes, streaming writes, explains, inserts, merge, deletes, variables, and view/table/database DDL.

Many catalog commands become DataFusion extension logical plans:

```
LogicalPlan::Extension(Extension {
    node: Arc::new(CatalogCommandNode::try_new(self.ctx, command)?),
})
```

Later, the physical planner recognizes `CatalogCommandNode` and turns it into:

```
CatalogCommandExec
```

This is the key pattern for command execution:

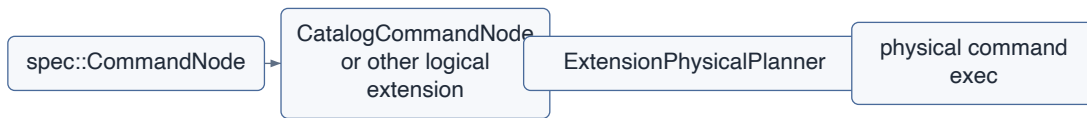


Figure 57: Flowchart 10.6

This lets Sail preserve DataFusion’s plan pipeline even for operations that are not ordinary relational queries.

10.14 Logical Extension Nodes

Sail uses DataFusion logical extension nodes for Spark-specific plan concepts that DataFusion does not natively model.

Examples include:

- RangeNode,
- ShowStringNode,
- MapPartitionsNode,
- MonotonicIdNode,
- SparkPartitionIdNode,
- SortWithinPartitionsNode,
- SchemaPivotNode,
- FileWriteNode,
- FileDeleteNode,
- MergeIntoNode,
- ExplicitRepartitionNode,
- streaming source/filter/limit/collector nodes,
- CatalogCommandNode,
- BarrierNode.

These nodes are planned in `crates/sail-session/src/planner.rs` by `ExtensionPhysicalPlanner`.

That planner is installed through `ExtensionQueryPlanner`, which builds a `DataFusion DefaultPhysicalPlanner` with extension planners:

```
lakehouse extension planners
system table physical planner
Sail extension physical planner
```

This is a crucial architectural point: Sail does not fork DataFusion’s planner. It uses DataFusion’s extension hooks.

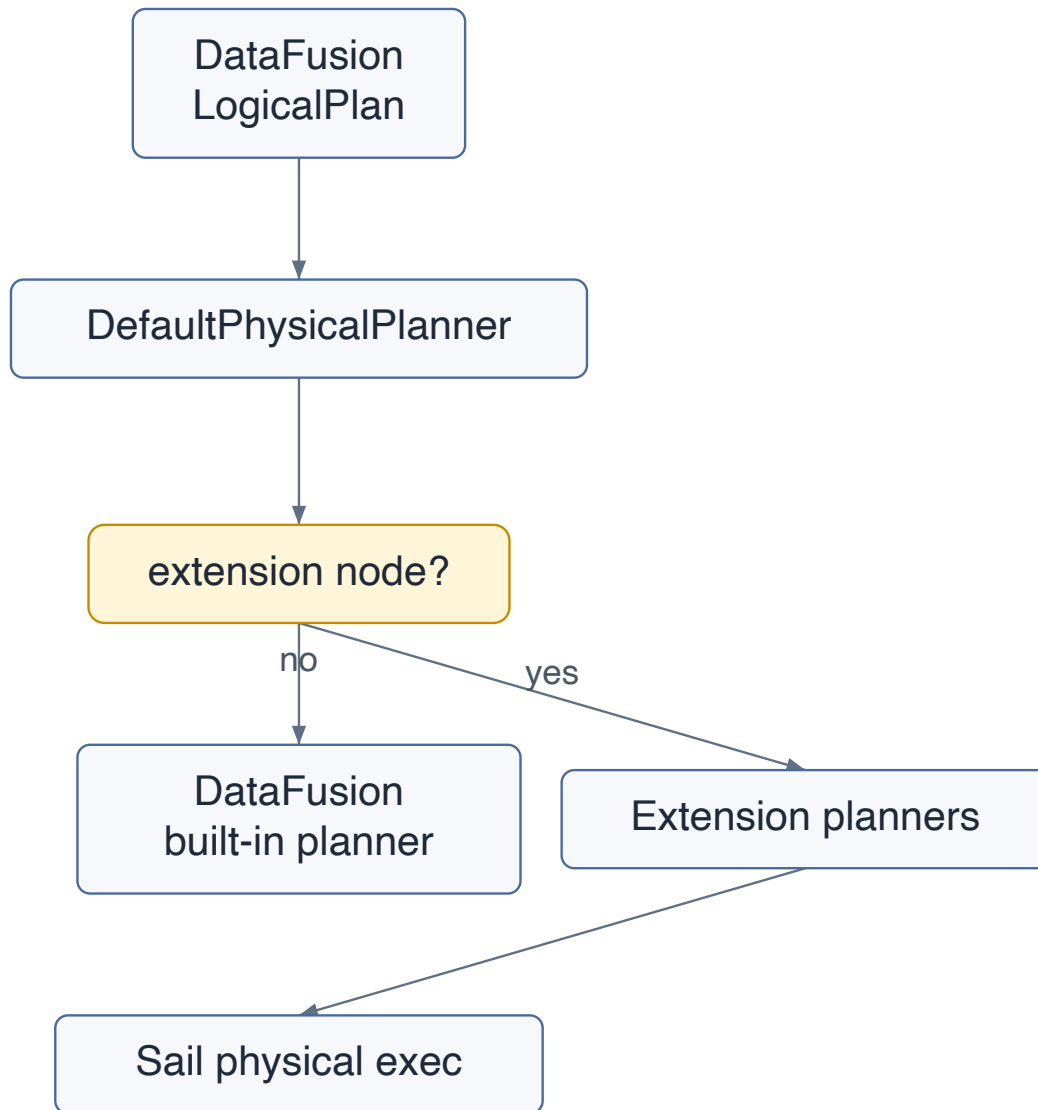


Figure 58: Flowchart 10.7

For discussion #2001, this pattern is already half of the answer. Third-party integrations need a disciplined way to register logical and physical extension behavior without hard-coding every integration into `sail-session/src/planner.rs`.

10.15 Repartition As A Bridge To Distributed Execution

`query/repartition.rs` is a compact example that connects this chapter to the shuffle chapters.

Spark-facing repartition intent becomes a Sail logical extension node:

```
ExplicitRepartitionNode::new(
  Arc::new(input),
  Some(num_partitions),
  ExplicitRepartitionKind::RoundRobin,
  vec![],
)
```

For `repartitionByExpression`, Sail resolves the partition expressions and creates:

```
ExplicitRepartitionKind::Hash
```

Later, `ExtensionPhysicalPlanner` turns `ExplicitRepartitionNode` into `ExplicitRepartitionExec`, using `DataFusion` physical expressions and partitioning:

```
RoundRobin -> Partitioning::RoundRobinBatch
Hash       -> Partitioning::Hash
Coalesce   -> UnknownPartitioning with fewer partitions
```

Then Chapter 7's job graph planner and Chapter 9's shuffle operators take over.

This path is worth memorizing:

```
Spark repartition call
-> spec::QueryNode::Repartition
-> ExplicitRepartitionNode
-> ExplicitRepartitionExec
-> distributed stage boundary
-> ShuffleWriteExec / ShuffleReadExec
```

That is how a user-level API becomes data movement.

10.16 WithRelations And Plan IDs

Spark Connect can send a root relation plus referenced relations. Sail models this as:

```
QueryNode::WithRelations { root, references }
```

The resolver stores the references in `PlanResolverState` by `plan_id`.

It also handles a useful PySpark pattern: SQL strings can refer to DataFrames passed as arguments. The conversion may wrap those DataFrames in `SubqueryAlias` nodes inside `WithRelations`. Sail resolves those references and registers them as CTE-like table names for the root query.

The flow is:

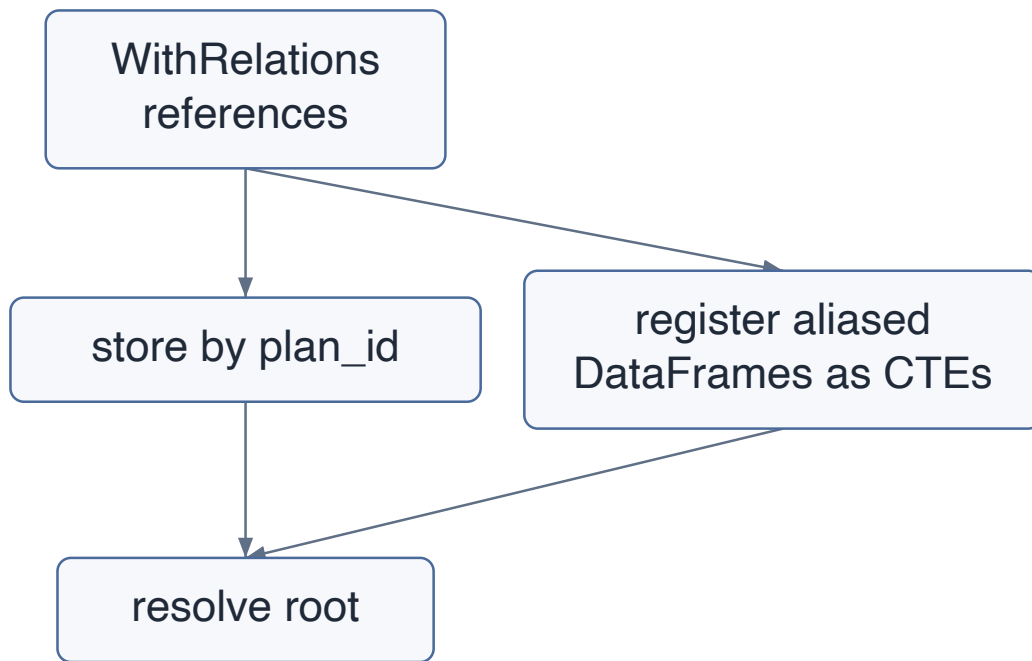


Figure 59: Flowchart 10.8

The scoping helpers in `PlanResolverState` make this safe:

- `enter_with_relations_scope()`,
- `enter_cte_scope()`.

Each helper restores the previous state on drop. This is a very Rust-flavored design: scope cleanup is tied to ownership and `Drop`, so temporary resolver state does not leak into the outer query.

10.17 Commands In Spark Connect Execution

The Spark Connect plan executor in:

`crates/sail-spark-connect/src/service/plan_executor.rs`

uses different modes for different operations.

Normal relations are lazy:

```

handle_execute_relation
  -> relation.try_into()
  -> handle_execute_plan(..., Lazy)
  
```

Commands such as UDF registration and writes are eager and silent:

```

handle_execute_register_function
handle_execute_write_operation
  
```

```
handle_execute_create_dataframe_view
handle_execute_write_operation_v2
```

They build a `spec::Plan::Command`, resolve it, execute it, drain the stream, and return completion metadata rather than a normal relation stream.

SQL command handling has one extra twist. If a SQL string resolves to a command, Sail executes the command and returns a local relation containing the command result. That matches Spark Connect's expectation that a SQL command can return an opaque relation for the client to use.

10.18 A Worked Example: DataFrame Filter And Project

Consider a PySpark call:

```
df = spark.table("orders").where("amount > 100").select("customer_id", "amount")
```

Spark Connect sends an unresolved relation tree roughly like:

```
Project(customer_id, amount)
  Filter(amount > 100)
    Read(NamedTable orders)
```

Sail first converts that into spec:

```
spec::QueryNode::Project
  spec::QueryNode::Filter
    spec::QueryNode::Read
```

Then the resolver walks bottom-up:

1. ReadNamedTable asks the catalog for orders.
2. The table source is created and fields are registered with internal IDs.
3. Filter resolves amount against the input schema.
4. Project resolves customer_id and amount.
5. The final schema is verified.
6. User-facing field names are captured for later physical renaming.

The interesting part is the field mapping:

```
customer_id -> #0
amount      -> #1
```

DataFusion sees stable internal columns. The Spark client eventually sees the expected names.

10.19 A Worked Example: Repartition By Customer

Now consider:

```
df.repartition(16, "customer_id")
```

At the spec level:

```
QueryNode::RepartitionByExpression {
  partition_expressions: [UnresolvedAttribute(customer_id)],
```

```

    num_partitions: Some(16),
  }

```

The resolver:

1. Resolves the input plan.
2. Resolves `customer_id` into a DataFusion column expression.
3. Builds an `ExplicitRepartitionNode` with `ExplicitRepartitionKind::Hash`.

The physical planner:

1. Converts the logical expression into a physical expression.
2. Builds `Partitioning::Hash(expressions, 16)`.
3. Creates `ExplicitRepartitionExec`.

The distributed planner:

1. Sees the repartition boundary.
2. Creates producer and consumer stages.
3. Schedules shuffle channels.
4. Uses the runtime shuffle read/write path from Chapter 9.

One user-level method call has passed through four layers:

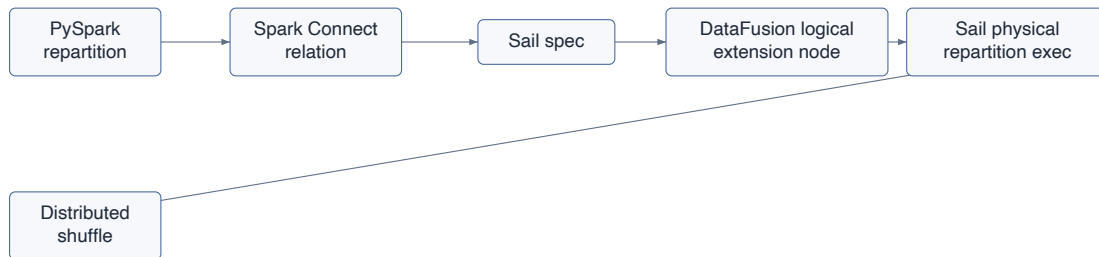


Figure 60: Flowchart 10.9

10.20 A Worked Example: Registered Python UDF

A registered Python UDF follows a different path.

First, Spark Connect sends a register-function command. Sail builds:

```

spec::Plan::Command(
  spec::CommandNode::RegisterFunction(...)
)

```

The command resolver stores the function in the catalog.

Later, a query calls the function:

```

spark.sql("SELECT my_udf(x) FROM t")

```

The expression resolver sees:

```

spec::Expr::UnresolvedFunction("my_udf", [x])

```

It checks the catalog before built-ins, finds the PySpark unresolved UDF, resolves the argument expressions, and builds a Python UDF expression that can be planned and executed later.

The UDF command bytes stay as bytes. The resolver does not deserialize Python logic or execute Python code. It only creates a typed plan representation.

That distinction is vital for distributed execution. Workers need a serializable plan and enough metadata to run the UDF in the right execution context.

10.21 Extension Implications

The experimental branch implements a focused relation dispatch path. In `crates/sail-plan/src/resolver/query/mod.rs`, `QueryNode::Extension` routes to `resolve_query_extension` in `crates/sail-plan/src/resolver/query/extension.rs`.

The resolver selects the registered handler using payload type URL, envelope status and input count. It resolves input plans and restores their field names; duplicate input column names are rejected with a request to alias them. The resolved physical inputs are wrapped as `HostInputExec` objects before the handler plans its provider.

`Connect Relation.extension`

- > validated envelope and `Sail QueryNode::Extension`
- > registered relation handler
- > resolved, named input plans
- > provider returned by `handler.plan`
- > ordinary `Sail/DataFusion` planning and execution

`Raw Expression.extension` and `Command.extension` dispatch are unsupported. Native scalar expressions use registered function names instead. Mutating graph operations use relations whose execution produces a receipt; merely resolving their schema must not perform the mutation.

10.22 Resolver Contract and Package Boundary

The branch does not provide a public `SailPlanExtension` trait granting packages arbitrary SQL grammar, optimizer or physical-planner hooks. Its manifest, relation envelope and native bindings form the implemented contract. Chapter 13 specifies that boundary and the client/runtime versions used in the examples.

An extension that expands into existing relational operators can reuse their planning and distributed codecs. An extension retaining a custom physical object needs a supported reconstruction path and a compatible package on the executing process. These are separate obligations: accepting a payload does not prove that its eventual plan can execute remotely.

10.23 Design Rules For Future Extensions

The resolver code suggests several design rules.

First, preserve unresolved intent until enough context exists. The protobuf conversion layer should parse and normalize, but not bind names too early.

Second, keep Spark-facing names separate from engine-facing names. Any extension that creates fields should register them through resolver state or an equivalent API.

Third, make side effects explicit. The implemented command-shaped relations return receipts and perform work only when executed; schema resolution must remain free of graph mutations.

Fourth, make worker compatibility explicit. If an extension creates physical operators, workers must have the same extension and codec registrations.

Fifth, use DataFusion extension hooks where possible. Sail's strength is that it extends DataFusion rather than replacing it.

Sixth, expose ordering and collision rules. Function names, optimizer rules, and physical planners all need deterministic registration behavior.

10.24 Reading Exercise

Trace a simple query:

```
SELECT customer_id, count(*)  
FROM orders  
GROUP BY customer_id
```

Suggested path:

1. Start in `crates/sail-spark-connect/src/proto/plan.rs` if the query arrives through Spark Connect, or in the SQL analyzer path if it starts as SQL.
2. Find the resulting `spec::QueryNode` and `spec::Expr` values.
3. Open `crates/sail-plan/src/resolver/plan.rs`.
4. Follow `resolve_named_plan()` into `resolve_query_plan()`.
5. In `query/read.rs`, follow table resolution for orders.
6. In `expression/attribute.rs`, follow `customer_id`.
7. In `expression/function.rs`, follow `count`.
8. In `query/aggregate.rs`, follow group-by planning.
9. Return to `resolve_and_execute_plan()` and see how DataFusion optimization begins.

The core question is:

At this line, are we still describing user intent, or have we bound that intent to a DataFusion plan object?

Once you can answer that, the resolver stops feeling like a forest and starts feeling like a set of well-marked trails.

10.25 Takeaways

The Sail spec and resolver form the semantic center of the engine:

- Spark Connect protobufs and SQL both become Sail spec plans.
- The Sail spec is an unresolved, serializable representation of Spark-compatible query and command intent.
- `PlanResolver` turns spec plans into DataFusion logical plans.

- PlanResolverState tracks internal field IDs, user-facing names, hidden fields, plan IDs, CTEs, subqueries, and temporary scopes.
- Query, expression, and command resolution are split into focused modules.
- Sail uses DataFusion logical extension nodes for Spark-specific behavior.
- The session physical planner turns those extension nodes into Sail physical execution plans.
- Extension proposal #2001 should build on these existing boundaries rather than bypassing them.

The next chapter turns from plans to callable behavior: functions, UDFs, UDAFs, UDTFs, codecs, and why distributed execution makes serialization and worker-side registration non-negotiable.

11 Functions, UDFs, And Codecs

A function call looks small in a query:

```
SELECT lower(name), my_python_udf(amount)
FROM orders
```

Inside a distributed engine, that little expression is a contract.

The driver must resolve the function name. The logical plan must carry the right DataFusion expression. The physical plan must know how to execute it. If the query runs on workers, the function implementation and all of its parameters must survive serialization. If the function is a Python UDF, the worker must also reconstruct a PySpark-compatible payload and call Python with Arrow data in the right shape.

This chapter is about that contract.

Sail's function architecture spans four layers:

```
spec::Expr / spec::CommandNode
  -> PlanResolver function logic
  -> DataFusion UDF/UDAF/window/stream objects
  -> RemoteExecutionCodec for worker execution
```

That makes functions one of the best places to understand why extension proposal #2001 is not just about registering names. Distributed extensions must be resolvable, plannable, serializable, decodable, and executable everywhere the query can run.

11.1 Code Map

The main files for this chapter are:

Concern	File
Spec representation of UDFs and UDTFs	crates/sail-common/src/spec/expression.rs
Built-in function registry	crates/sail-plan/src/function/mod.rs
Built-in scalar groups	crates/sail-plan/src/function/scalar/mod.rs
Built-in aggregate functions	crates/sail-plan/src/function/aggregate.rs
Built-in table functions	crates/sail-plan/src/function/table/mod.rs
Function expression resolution	crates/sail-plan/src/resolver/expression/function.rs
Inline Python UDF resolution	crates/sail-plan/src/resolver/expression/udf.rs
Function registration commands	crates/sail-plan/src/resolver/command/function.rs
UDTF resolution	crates/sail-plan/src/resolver/query/udtf.rs
Catalog function storage	crates/sail-catalog/src/manager/function.rs
Stream UDF trait	crates/sail-common-datafusion/src/udf.rs
Map partitions physical operator	crates/sail-physical-plan/src/map_partitions.rs
PySpark scalar UDF implementation	crates/sail-python-udf/src/udf/pyspark_udf.rs
PySpark aggregate UDF implementation	crates/sail-python-udf/src/udf/pyspark_udaf.rs
PySpark UDTF implementation	crates/sail-python-udf/src/udf/pyspark_udtf.rs
PySpark payload building	crates/sail-python-udf/src/cereal/pyspark_udf.rs
PySpark stream bridge	crates/sail-python-udf/src/stream.rs
Remote execution codec	crates/sail-execution/src/proto/codec.rs
Codec protobuf schema	crates/sail-execution/proto/sail/plan/physical.proto
Server session setup	crates/sail-session/src/session_factory/server.rs
Worker session setup	crates/sail-session/src/session_factory/worker.rs

11.2 The Function Lifecycle

A function in Sail can enter from several front doors:

- a Spark Connect UnresolvedFunction,
- a SQL function call,
- a Spark Connect inline Python UDF expression,
- a function registration command,
- a table-valued function relation,
- an internal rewrite that inserts a helper UDF.

All of those eventually need to become DataFusion expressions or Sail physical operators.

The lifecycle looks like this:

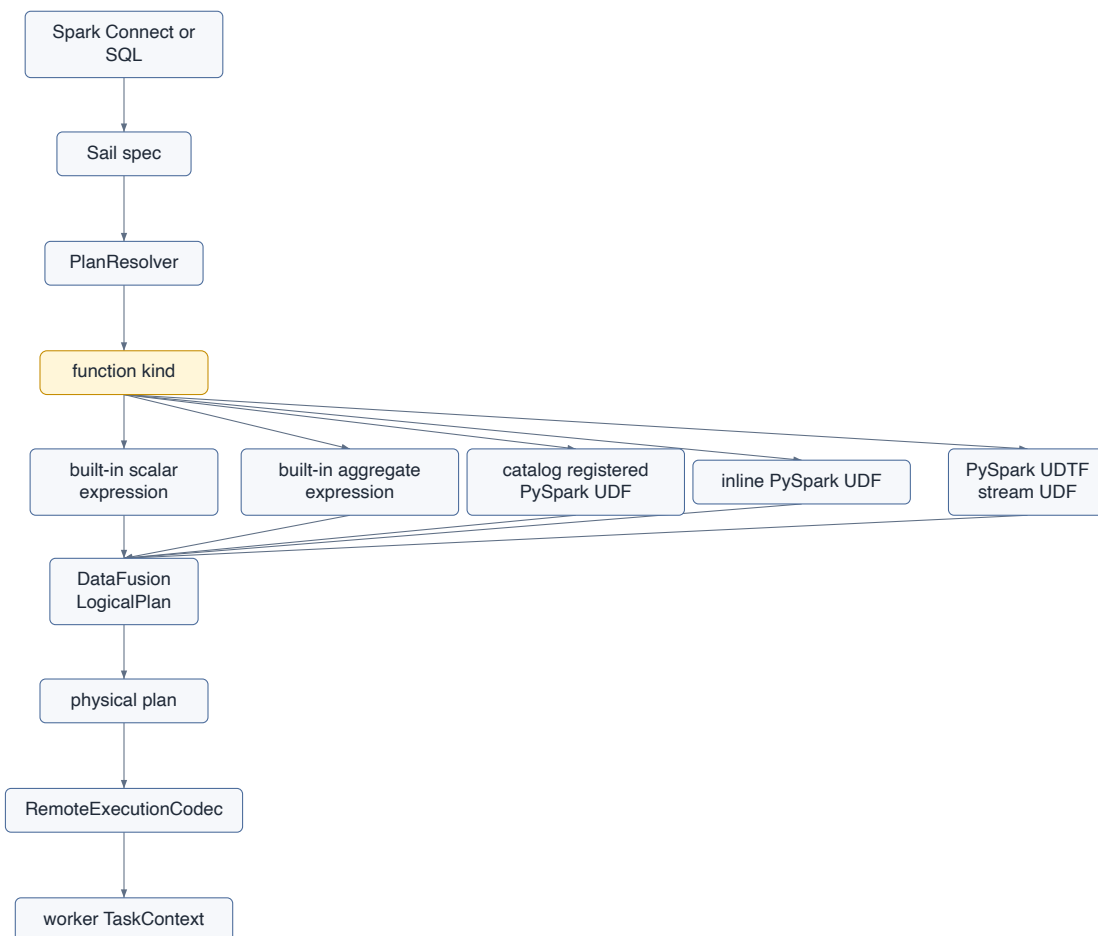


Figure 61: Flowchart 11.1

The key point is that the resolver does not execute functions. It builds objects that DataFusion and Sail can execute later.

11.3 Spec Representation

The spec layer models inline user-defined functions in `crates/sail-common/src/spec/expression.rs`.

Scalar UDFs use:

```
pub struct CommonInlineUserDefinedFunction {
    pub function_name: Identifier,
    pub deterministic: bool,
    pub is_distinct: bool,
    pub arguments: Vec<Expr>,
    pub function: FunctionDefinition,
}
```

The function definition can be:

```
pub enum FunctionDefinition {
    PythonUdf {
        output_type: DataType,
        eval_type: PySparkUdfType,
        command: Vec<u8>,
        python_version: String,
        additional_includes: Vec<String>,
    },
    ScalarScalaUdf { ... },
    JavaUdf { ... },
}
```

Table functions use:

```
pub struct CommonInlineUserDefinedTableFunction {
    pub function_name: Identifier,
    pub deterministic: bool,
    pub arguments: Vec<Expr>,
    pub function: TableFunctionDefinition,
}
```

Today, the table function definition is Python-specific:

```
pub enum TableFunctionDefinition {
    PythonUdtf {
        return_type: Option<DataType>,
        eval_type: PySparkUdfType,
        command: Vec<u8>,
        python_version: String,
    },
}
```

The `command: Vec<u8>` field is the serialized Python payload from PySpark. Sail treats it as opaque bytes until it is time to construct a PySpark worker-compatible payload.

The `PySparkUdfType` enum mirrors PySpark evaluation modes:

```

Batched
ArrowBatched
ScalarPandas
GroupedAggPandas
ScalarPandasIter
ScalarArrow
ScalarArrowIter
GroupedAggArrow
Table
ArrowTable
ArrowUdtf
...

```

Different evaluation types imply different execution shapes:

Eval type family	Sail execution shape
Scalar batch UDF	DataFusion ScalarUDFImpl
Scalar Pandas or Arrow UDF	DataFusion ScalarUDFImpl with Python bridge
Grouped aggregate UDF	DataFusion AggregateUDFImpl
Map iterator UDF	Sail StreamUDF through MapPartitionsExec
UDTF	Sail StreamUDF through MapPartitionsExec

This is why the function type must be explicit in the spec. The same surface concept, “a Python function,” can require very different execution machinery.

11.4 Built-In Function Registry

Sail’s built-in Spark-compatible functions are registered in static maps in `crates/sail-plan/src/function/mod.rs`.

```

lazy_static! {
    pub static ref BUILT_IN_SCALAR_FUNCTIONS: HashMap<&'static str,
ScalarFunction> =
        HashMap::from_iter(scalar::list_built_in_scalar_functions());

    pub static ref BUILT_IN_GENERATOR_FUNCTIONS: HashMap<&'static str,
ScalarFunction> =
        HashMap::from_iter(generator::list_built_in_generator_functions());

    pub static ref BUILT_IN_TABLE_FUNCTIONS: HashMap<&'static str,
Arc<TableFunction>> =
        HashMap::from_iter(table::list_built_in_table_functions());
}

```

The scalar registry is assembled from many focused modules:

```
array
bitwise
collection
conditional
conversion
csv
datetime
geo
hash
json
lambda
map
math
misc
predicate
string
struct
url
variant
xml
```

This layout is worth copying in your mental model. There is no single monstrous function resolver where every implementation lives. Instead:

- `sail-plan` maps Spark-compatible names to expression builders.
- `sail-function` implements many custom DataFusion UDFs and UDAFs.
- `datafusion-spark` and DataFusion built-ins provide additional behavior.
- `sail-python-udf` implements Python-backed functions.

The aggregate registry works similarly, but aggregate functions often have to adapt Spark syntax to DataFusion's expected function shape.

For example, Spark's ordered-set syntax:

```
percentile_cont(0.5) WITHIN GROUP (ORDER BY col)
```

does not arrive in the same argument order DataFusion expects. Sail's aggregate resolver extracts the ordered column and percentile argument, then builds the DataFusion aggregate expression.

That pattern is common: Sail preserves Spark semantics while targeting DataFusion's execution model.

11.5 Function Resolution Order

Function calls are resolved in:

```
crates/sail-plan/src/resolver/expression/function.rs
```

The core method is:

```
resolve_expression_function(...)
```

The resolution order is deliberate:

1. Normalize the function name.
2. Extract keyword arguments from `NamedArgument` expressions.
3. Merge named arguments from SQL analyzer paths.
4. Reject duplicate keyword arguments.
5. Check the `CatalogManager` for registered functions.
6. Resolve and type-check arguments.
7. If a registered PySpark UDF exists, build a Python UDF expression.
8. Otherwise try Sail built-in scalar or generator functions.
9. Otherwise try Sail built-in aggregate functions.
10. Format the output expression name with `PlanService`.

The catalog check happens before built-ins because Spark Connect does not reliably set the `is_user_defined_function` flag. Sail chooses the behavior users expect: if a function was registered by name, it should be found.

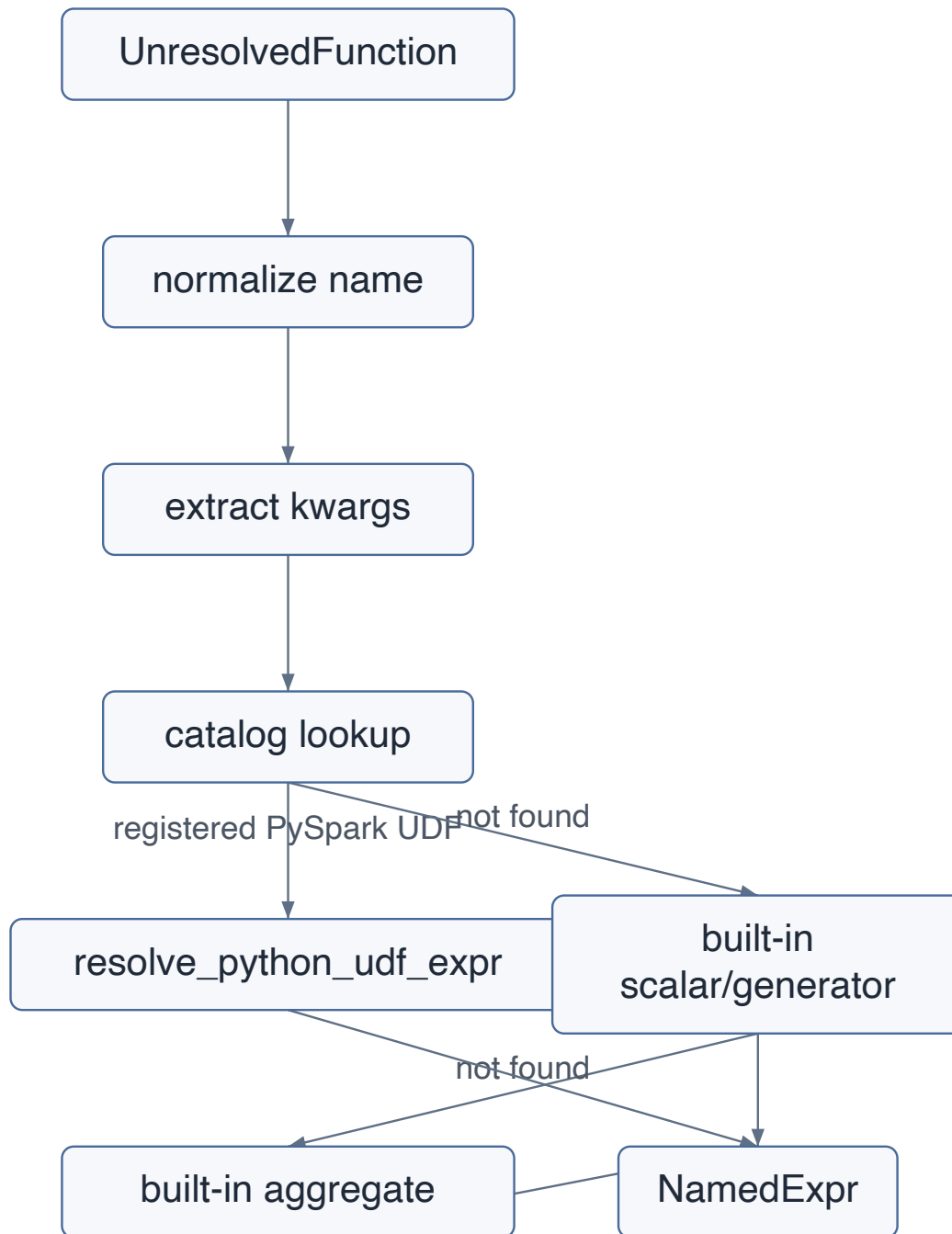


Figure 62: Flowchart 11.2

This is also where Sail handles Spark-specific display names. The expression object is DataFusion-compatible, but the output name should still look like Spark.

11.6 Registered PySpark UDFs

Registration commands are handled in:

```
crates/sail-plan/src/resolver/command/function.rs
```

For a scalar Python UDF registration, Sail:

1. Enters a resolver config scope.
2. Enables large Arrow variable types for UDF resolution.
3. Resolves the Python UDF definition and output type.
4. Wraps the unresolved Python function in `PySparkUnresolvedUDF`.
5. Tracks the function in `CatalogManager`.
6. Creates a catalog command logical plan.

Conceptually:

```
RegisterFunction command
-> PySparkUnresolvedUDF
-> CatalogManager::track_function
-> CatalogCommand::RegisterFunction
```

`CatalogManager` stores functions case-insensitively:

```
fn canonical_function_name(name: &str) -> Arc<str> {
    name.to_ascii_lowercase().into()
}
```

When the query later calls that name, the expression resolver finds it in the catalog and creates the real executable UDF expression with the correct input types.

That two-step model matters:

- Registration knows the Python command bytes and declared return type.
- Call-site resolution knows the actual argument expressions and input types.

You need both to build the runtime payload.

11.7 Inline PySpark UDFs

Spark Connect can also carry a UDF inline inside an expression:

```
spec::Expr::CommonInlineUserDefinedFunction
```

That path is handled by:

```
crates/sail-plan/src/resolver/expression/udf.rs
```

The resolver:

1. Extracts positional and keyword arguments.
2. Rejects duplicate kwargs.
3. Rejects positional arguments after keyword arguments.
4. Resolves argument expressions and display names.
5. Resolves the Python UDF definition.

6. Computes input Arrow data types from the resolved arguments.
7. Builds a PySpark worker-compatible payload.
8. Creates a DataFusion scalar or aggregate UDF expression.

The payload build step is the heart of the integration:

```
let payload = PySparkUdfPayload::build(
  &function.python_version,
  &function.command,
  function.eval_type,
  &arg_offsets,
  &input_types,
  kwarg_names,
  &self.config.pyspark_udf_config,
)?;
```

The output depends on the PySpark eval type.

PySpark eval type	Sail object
Batched	PySparkUDF with PySparkUdfKind::Batch
ArrowBatched	PySparkUDF with PySparkUdfKind::ArrowBatch
ScalarPandas	PySparkUDF with PySparkUdfKind::ScalarPandas
ScalarPandasIter	PySparkUDF with PySparkUdfKind::ScalarPandasIter
ScalarArrow	PySparkUDF with PySparkUdfKind::ScalarArrow
ScalarArrowIter	PySparkUDF with PySparkUdfKind::ScalarArrowIter
GroupedAggPandas	PySparkGroupAggregateUDF with Pandas mode
GroupedAggArrow	PySparkGroupAggregateUDF with Arrow mode

Unsupported eval types are rejected early if they do not make sense for a scalar inline UDF.

11.8 Scalar Python Execution

The executable scalar UDF object is PySparkUDF in:

crates/sail-python-udf/src/udf/pyspark_udf.rs

It implements DataFusion's ScalarUDFImpl.

Its fields include:

```

kind: PySparkUdfKind,
name: String,
payload: Vec<u8>,
deterministic: bool,
input_types: Vec<DataType>,
output_type: DataType,
config: Arc<PySparkUdfConfig>,
udf: LazyPyObject,

```

The LazyPyObject is important. The Python callable is not eagerly loaded when the plan is built. It is loaded when the UDF is invoked:

payload bytes

```

-> PySparkUdfPayload::load
-> PySpark read_udfs
-> PySpark wrapper function

```

At execution time, invoke_with_args():

1. Converts DataFusion ColumnarValue inputs into Arrow arrays.
2. Converts those arrays into Python/PyArrow values.
3. Calls the Python wrapper.
4. Converts returned Python data back into Arrow ArrayData.
5. Casts the result to the declared output type.
6. Returns a DataFusion ColumnarValue::Array.

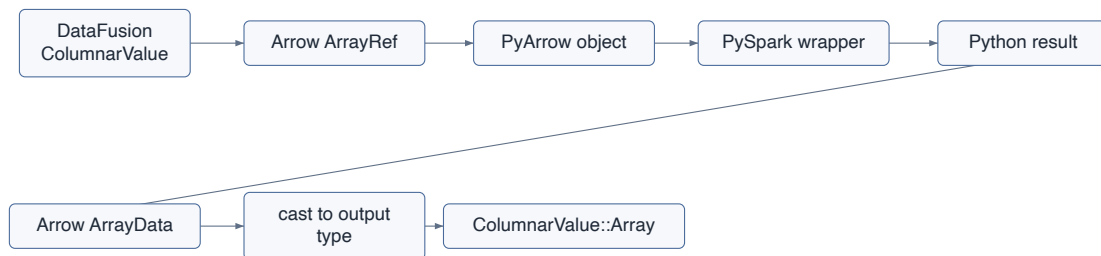


Figure 63: Flowchart 11.3

This is the practical meaning of “Arrow UDF” in Sail. Arrow is the boundary format between Rust execution and Python execution.

11.9 Grouped Aggregate Python UDFs

Grouped aggregate UDFs use:

```
crates/sail-python-udf/src/udf/pyspark_udaf.rs
```

The central object is PySparkGroupAggregateUDF, which implements AggregateUDFImpl.

It supports two modes:

```

pub enum PySparkGroupAggKind {
    Pandas,

```

```
    Arrow,  
}
```

The accumulator path is important. DataFusion aggregate functions do not call a scalar function once per input batch. They create accumulators. Sail uses `BatchAggregateAccumulator` with a `BatchAggregator` implementation that calls Python over collected Arrow arrays.

The resolver also enforces a Spark analysis rule: aggregate UDF arguments cannot contain nested aggregate functions.

There is another small but revealing workaround: DataFusion requires at least one input to an aggregate function. For a zero-argument Python aggregate UDF, Sail injects a dummy `Int64` literal and records the actual argument count separately so the Python function still receives the right argument list.

That is the kind of adapter code a compatibility engine accumulates. DataFusion and Spark are close enough to compose, but not identical.

11.10 UDTFs And Stream UDFs

Python table functions are not scalar expressions. They take input rows or batches and emit zero or more output rows. Sail models them as stream transformations.

The common trait is:

```
pub trait StreamUDF: DynObject + Debug + Send + Sync {  
    fn name(&self) -> &str;  
    fn output_schema(&self) -> SchemaRef;  
    fn invoke(&self, input: SendableRecordBatchStream) ->  
    Result<SendableRecordBatchStream>;  
}
```

That trait lives in:

`crates/sail-common-datafusion/src/udf.rs`

`PySparkUDTF` implements `StreamUDF` in:

`crates/sail-python-udf/src/udf/pyspark_udtf.rs`

The resolver creates a `MapPartitionsNode`, which the physical planner turns into `MapPartitionsExec`.

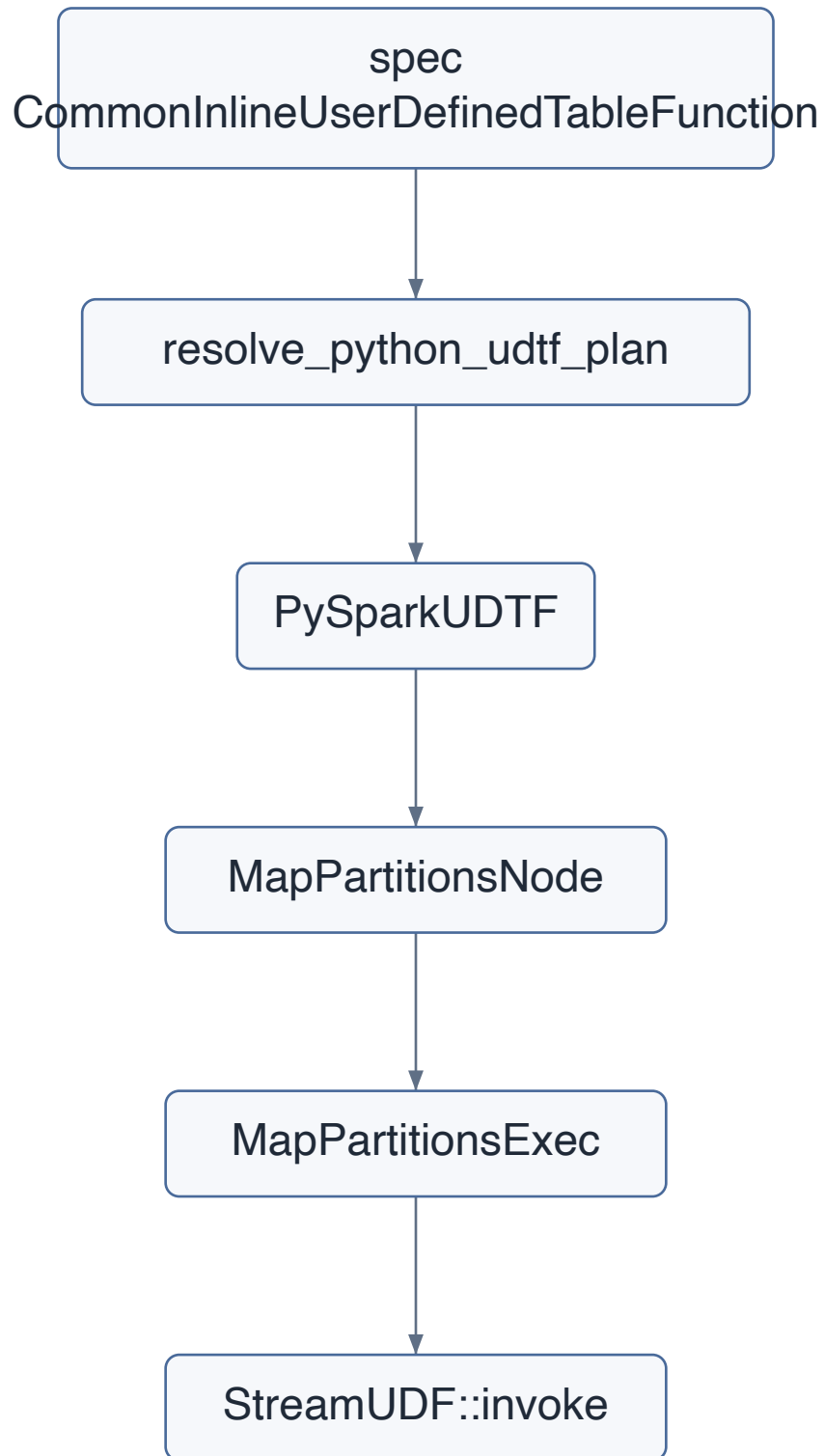


Figure 64: Flowchart 11.4

MapPartitionsExec is simple:

1. Execute the input physical plan for one partition.
2. Pass the resulting RecordBatch stream to the StreamUDF.
3. Wrap the output stream with the expected schema.

That is exactly the right abstraction for UDTFs. A UDTF is not “one input value -> one output value.” It is “one stream partition -> another stream partition.”

11.11 The Python Stream Bridge

The stream bridge lives in:

`crates/sail-python-udf/src/stream.rs`

PyMapStream converts a Rust RecordBatch stream into a Python iterator of PyArrow batches, then converts the Python output iterator back into a Rust RecordBatch stream.

The bridge uses a separate thread:

```
Rust input stream
-> PyInputStream.__next__
-> Python function iterator
-> output channel
-> Rust RecordBatchStream
```

The separate thread exists because the Python iterator performs blocking calls into a Tokio stream. The bridge uses a stop signal so the Rust side can tell the Python input iterator to stop.

The output path:

1. Calls the Python function with the input iterator.
2. Iterates over Python output batches.
3. Ignores empty batches in compatibility-sensitive cases.
4. Converts PyArrow batches to Arrow RecordBatch.
5. Casts batches positionally to the declared output schema.
6. Sends them through a channel to the Rust stream.

This is a useful pattern for any extension that needs to cross a runtime boundary: make the boundary stream-shaped, schema-aware, and cancellation-aware.

11.12 The Remote Execution Codec

So far, we have talked about resolving and executing functions in one process. But Sail has a distributed runtime. The driver builds a physical plan. Workers execute tasks. Workers must reconstruct every custom physical plan node and every custom function that appears inside physical expressions.

That is the job of RemoteExecutionCodec:

`crates/sail-execution/src/proto/codec.rs`

It implements DataFusion’s PhysicalExtensionCodec.

The codec handles:

- extended physical plan nodes,
- extended physical expressions,
- scalar UDFs,
- aggregate UDFs,
- window UDFs,
- stream UDFs,
- schemas,
- data types,
- scalar values,
- partitioning,
- statistics,
- file scan configs,
- Sail lakehouse execution nodes,
- Python UDF configuration and payloads.

The protobuf definitions live in:

`crates/sail-execution/proto/sail/plan/physical.proto`

The file starts with a telling comment: DataFusion data structures are often stored as opaque bytes because DataFusion’s protobuf definitions can change. Sail uses its own extended protobuf messages for Sail-specific nodes and wraps DataFusion’s protobuf encoding where needed.

11.13 Encoding Scalar UDFs

Scalar UDF encoding happens in `try_encode_udf()`.

The current design has two broad paths:

1. If the UDF is a known Sail/DataFusion built-in that the worker can reconstruct by name, encode it as `StandardUdf`.
2. If the UDF carries state or custom configuration, encode the required fields into a specific protobuf variant.

Examples of stateful scalar UDF variants:

```
PySparkUdf
PySparkCoGroupMapUdf
DropStructFieldUdf
ExplodeUdf
SparkUnixTimestampUdf
StructFunctionUdf
ArraysZipUdf
UpdateStructFieldUdf
TimestampNowUdf
SparkTimestampUdf
SparkDateUdf
SparkFromCsvUdf
```

```
SparkFromJsonUdf
...
```

For PySparkUDF, the codec stores:

```
kind
name
payload
deterministic
input_types
output_type
config
```

That is enough for a worker to reconstruct:

```
PySparkUDF::new(
  kind,
  name,
  payload,
  deterministic,
  input_types,
  output_type,
  Arc::new(config),
)
```

For a UDF like StructFunction, the codec only needs the field names. For a UDF like SparkTimestamp, it needs timezone and is_try. The encoding shape follows the state needed to reconstruct the function.

11.14 Decoding Scalar UDFs

Decoding happens in `try_decode_udf()`.

The codec receives a function name and an extension buffer. If the extension buffer is StandardUdf, it reconstructs the function by matching on the name:

```
"spark_array" -> SparkArray::new()
"spark_split" -> SparkSplit::new()
"spark_xxhash64" -> SparkXxhash64::new()
...
```

If the buffer contains a richer variant, the codec decodes the fields and constructs the object directly.

That distinction is a bit manual today. The code even has a TODO:

Implement custom registry to avoid codec for built-in functions

This is another bright signpost for discussion #2001. A third-party extension should not need to patch a giant match statement in RemoteExecutionCodec just to make a custom function work on workers.

11.15 Native Package Codecs on the Experimental Branch

The implemented branch adds explicit native codec paths before the builtin extended-plan dispatch. `RemoteExecutionCodec::try_decode` recognizes `WORKER_CODEC_PREFIX` and invokes `WorkerExtensionExec::decode` with the host child plans and task context. It separately recognizes the driver-native prefix. The encoder identifies those concrete plan nodes and delegates to their encoders. These are distinct placement contracts, not interchangeable process-local handles.

Native scalar functions likewise have a dedicated scalar codec prefix and `encode_scalar/decode_scalar` helpers in `crates/sail-common-datafusion/src/native_scalar.rs`. The worker must have the matching package registration and identity. Resolving a coincidentally identical function name is not enough to prove it is the same implementation.

The host codec owns validation and reconstruction at the boundary. The package owns its domain payload and algorithm. A worker-native descriptor transports schemas, routing and operation identity rather than a pointer into the driver's address space. Its decoder uses authoritative task scope when binding retained worker state.

This is the concrete correction to the earlier proposal's generic registry sketch. It supports the tested native scalar and relation contracts; it does not establish arbitrary optimizer-rule or aggregate-function plugin compatibility. Add a new contribution type only with its own planning, serialization, identity and lifecycle evidence.

11.16 Encoding Aggregate And Window UDFs

Aggregate UDFs follow the same idea.

Known standard aggregate UDFs encode as:

`StandardUdaf`

Then the worker decodes by name:

```
bitmap_and_agg
histogram_numeric
kurtosis
max_by
mode
percentile
product
try_avg
try_sum
...
```

Python grouped aggregate UDFs carry their payload, input names, input types, output type, deterministic flag, kind, config, and actual argument count.

Window UDF support is narrower. The codec has `ExtendedWindowUdf`, and currently the standard custom path includes `ntile` through `SparkNtile`.

The broader lesson is the same: every non-standard callable object needs a worker-side reconstruction story.

11.17 Encoding Stream UDFs

Stream UDFs are not DataFusion scalar expressions, so they have their own codec path:

```
fn try_encode_stream_udf(&self, udf: &dyn StreamUDF) -> Result<ExtendedStreamUdf>
fn try_decode_stream_udf(&self, udf: ExtendedStreamUdf) -> Result<Arc<dyn
StreamUDF>>
```

The current variants include:

```
PySparkMapIterUdf
PySparkUdtf
```

For PySparkUDTF, the codec stores:

```
kind
name
payload
input_names
input_types
passthrough_columns
function_return_type
function_output_names
deterministic
config
```

That mirrors the constructor for `PySparkUDTF::try_new()`.

This design is clean in one important way: stream UDFs are encoded with the physical operator that uses them. `MapPartitionsExec` does not need to know how a Python UDTF works. It only needs a `StreamUDF`.

11.18 Worker Sessions And Built-Ins

The server session factory deliberately does not add all DataFusion default features:

```
We do not add default features to the session state,
since we manage table formats and functions ourselves.
```

But the worker session factory does add default features:

```
We still add default features for the worker session
since we need built-in functions to be available for the codec
when decoding the execution plan.
```

That comment is small but important. It tells us that decoding a physical plan is not only a bytes-to-struct operation. It may depend on what functions and features are registered in the worker's `SessionState`.

This is one of the hard requirements for extensions:

The driver and every worker must agree on the callable universe.

If the driver can plan a function the worker cannot decode, the query fails at task startup. If the worker can decode but not execute the function, the query fails during batch execution.

11.19 Distributed Example: Built-In Function

Consider:

```
SELECT xxhash64(customer_id)
FROM orders
```

The path is:

1. Spark Connect or SQL creates `spec::Expr::UnresolvedFunction`.
2. The resolver normalizes the function name.
3. The built-in scalar function registry maps it to a Spark-compatible expression.
4. DataFusion physical planning creates a physical expression containing a UDF.
5. The job graph planner splits the plan into distributed stages if needed.
6. The task definition serializes the physical plan with `RemoteExecutionCodec`.
7. The worker decodes the UDF as a known standard UDF by name.
8. The worker executes the function against Arrow arrays.

The function itself may feel local, but in cluster mode it has to survive this trip:

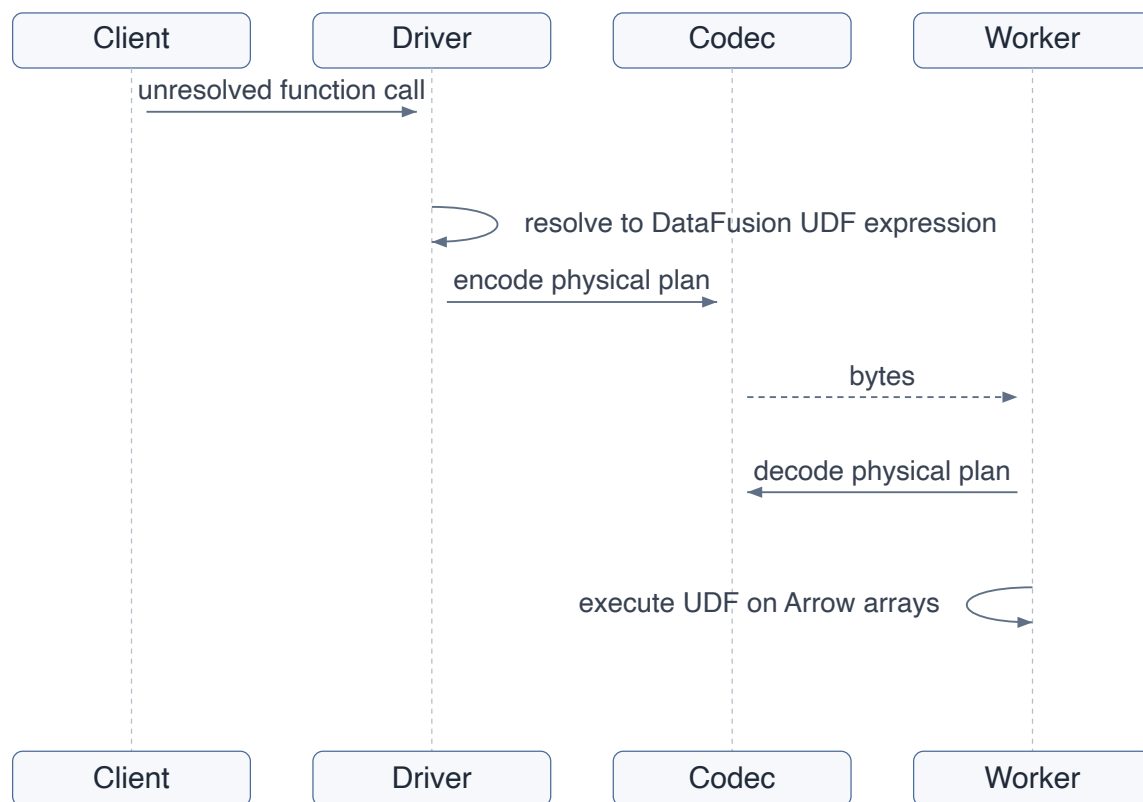


Figure 65: Sequence diagram 11.5

11.20 Distributed Example: Python Scalar UDF

Now consider:

```
@udf("long")
def plus_one(x):
    return x + 1

df.select(plus_one("amount"))
```

The path is richer:

1. PySpark serializes the Python function command.
2. Spark Connect sends the UDF information to Sail.
3. Sail stores or resolves the UDF as `CommonInlineUserDefinedFunction`.
4. The resolver computes input Arrow types from the call site.
5. `PySparkUdfPayload::build()` writes a PySpark-compatible payload:
 - eval type,
 - config,
 - optional input type JSON for certain PySpark versions,
 - profiling flag for PySpark 4,
 - argument offsets,
 - keyword argument names,
 - serialized command bytes.
6. Sail creates `PySparkUDF`.
7. The physical plan is encoded.
8. The codec encodes kind, payload, input types, output type, and config.
9. The worker decodes `PySparkUDF`.
10. At execution time, the UDF loads the Python callable lazily.
11. Arrow arrays cross into Python.
12. Python returns Arrow-compatible data.
13. Sail casts the result to the declared output type.

That is a lot of machinery, but each part has a job:

```
spec      -> preserve user intent and Python command
resolver  -> bind arguments and types
UDF object -> implement DataFusion execution
codec     -> move the object to workers
Python    -> run user code over Arrow-shaped data
```

11.21 Distributed Example: Python UDTF

A UDTF is stream-shaped:

```
class SplitWords:
    def eval(self, text):
        for word in text.split():
            yield (word,)
```

The Sail path is:

1. Resolve the UDTF definition and arguments.

2. Determine the return type. If no static return type exists, call the Python `analyze` method during query analysis.
3. Build a `PySparkUdtPayload`.
4. Create a `PySparkUDTF` stream UDF.
5. Build a `MapPartitionsNode`.
6. Physical planning creates `MapPartitionsExec`.
7. The codec encodes both the physical operator and its stream UDF.
8. The worker decodes `MapPartitionsExec` and `PySparkUDTF`.
9. At runtime, the input partition stream is passed into Python.
10. Python yields output batches, and Rust turns them back into `RecordBatch` streams.

Diagram:

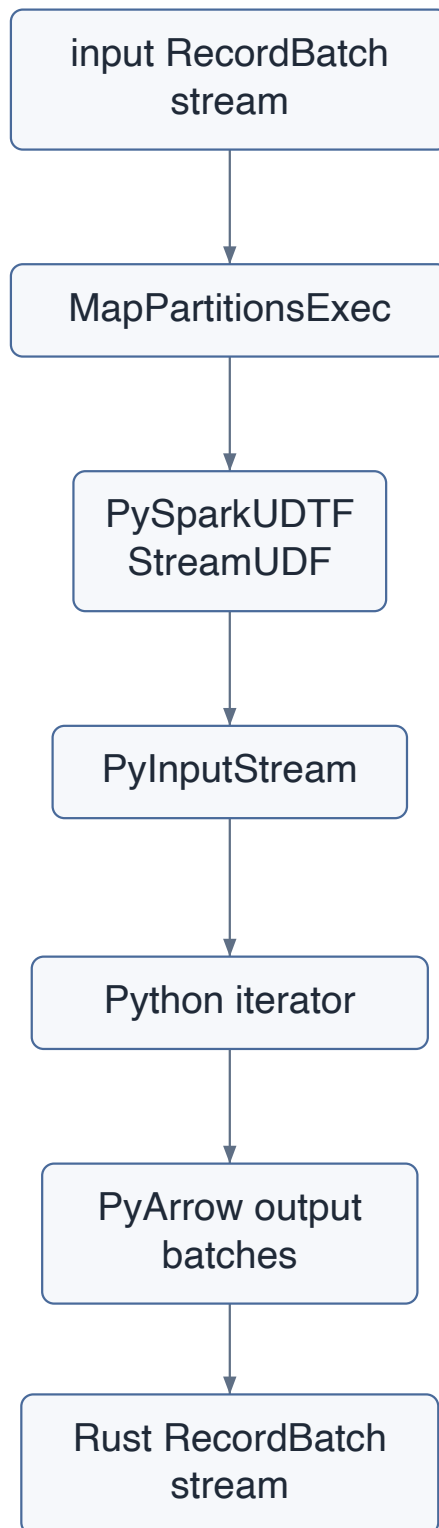


Figure 66: Flowchart 11.6

This is the same distributed principle again: the operator and the callable object must both be serializable.

11.22 Codec Design Lessons

`RemoteExecutionCodec` is not glamorous code, but it is the backbone of cluster execution.

It teaches several lessons:

First, function identity is not enough. Some functions need state:

- timezone,
- field names,
- ANSI mode,
- safe/try flags,
- input types,
- output types,
- Python payload bytes,
- PySpark config,
- passthrough column counts.

Second, the worker must reconstruct the same behavior, not merely a function with the same name.

Third, built-ins and extensions need different handling. Built-ins can sometimes be reconstructed by name. Extension functions need explicit registration and encoding.

Fourth, codecs are versioned contracts even when no version field is visible. If you change a function's fields, you have changed the bytes a worker expects.

Fifth, `DataFusion`'s extension codec hooks are exactly the right place to integrate custom behavior, but `Sail` needs a registry around them to avoid central matches.

11.23 Extension Implications

For discussion #2001, functions and codecs expose the sharpest edge of the design.

A third-party extension may want to add:

- a scalar UDF,
- an aggregate UDF,
- a window UDF,
- a table function,
- a stream UDF,
- a physical expression,
- a physical operator,
- a logical optimizer rewrite,
- a Spark Connect custom expression,
- a Python package that registers all of the above.

To work in local mode, registering a `DataFusion` UDF may be enough.

To work in cluster mode, that is not enough.

A distributed extension needs:

Need	Why
Logical name registration	So the resolver can bind function calls.
Type inference	So the logical plan can be checked and named.
Execution implementation	So DataFusion can evaluate the function.
Physical encoding	So the driver can serialize plans.
Physical decoding	So workers can reconstruct plans.
Worker installation	So the implementation exists in worker processes.
Version compatibility	So encoded bytes match decoder expectations.
Collision policy	So two extensions cannot silently claim the same name.
Ordering policy	So optimizer and planner hooks run deterministically.

This is the difference between a plugin that works in a notebook and an extension that works in a distributed query engine.

11.24 Extension Codecs on the Implementation Branch

Planning and execution have separate entry points. The branch described in chapter 13 accepts extension relations through `Relation.extension`; raw `Command.extension` and `Expression.extension` dispatch remain unsupported. Native scalar functions enter through registered function calls. Resolving a relation may produce existing relational operators or an extension execution node. A codec reconstructs executable objects; it does not resolve the original Connect request again.

The implemented dispatch is in `crates/sail-execution/src/proto/codec.rs`:

- **Worker extension plan:** `WORKER_CODEC_PREFIX` selects `WorkerExtensionExec::decode`. The implementation is in `crates/sail-common-datafusion/src/worker_extension.rs`.
- **Driver extension plan:** `DRIVER_CODEC_PREFIX` selects `DriverExtensionExec::decode`. The implementation is in `crates/sail-common-datafusion/src/driver_extension.rs`.
- **Native scalar UDF:** `SCALAR_CODEC_PREFIX` selects `decode_scalar` in `crates/sail-common-datafusion/src/native_scalar.rs`.
- **Existing Sail plan or UDF:** the codec retains its existing `protobuf` and `builtin` dispatch in `crates/sail-execution/src/proto/codec.rs`.

These are focused additions to the existing physical codec. They are not a public `FunctionCodec` or `SailFunctionExtension` trait covering every DataFusion extension category. In particular, the

native scalar path does not establish a corresponding packaged aggregate, window, or stream-UDF ABI.

A worker plan carries a descriptor and receives its decoded input plans from the physical-plan decoder. The descriptor identifies the extension work to bind in the worker's execution context. Native scalar encoding instead records the identity needed to retrieve an installed scalar implementation. Neither path ships a native library inside a query plan. The matching package must already be available in each process that needs it.

11.25 Registration and Review Boundaries

The Python bootstrap discovers opted-in packages through `pysail.extensions` entry points. Its manifest and bindings describe relation registrations and native objects. Chapter 13 explains the manifest, capsule types, collision checks and package compatibility requirements. That package contract is separate from Sail's existing builtin function maps and Python UDF payloads.

For a distributed extension, review three boundaries independently:

1. **Resolve:** does the enabled package bind the requested relation or function, with the expected input cardinality and types?
2. **Reconstruct:** can the receiving process decode the physical object using its installed package, and reject an incompatible identity?
3. **Execute and release:** does the object produce the declared Arrow schema, honor cancellation and admission, and retain ownership until its final consumer releases the buffers?

A codec round-trip test proves only reconstruction. A local function result proves neither worker installation nor distributed cleanup. The qualification examples in chapter 17 therefore exercise actual worker processes as well as codec and manifest failures. Pure relational expansion can use existing codecs, but any custom objects it leaves in the physical plan still need a supported reconstruction path.

11.26 Reading Exercise

Trace this query:

```
df.select(my_udf("x"))
```

Suggested path:

1. Start with `CommonInlineUserDefinedFunction` in `crates/sail-common/src/spec/expression.rs`.
2. Follow `resolve_expression_common_inline_udf()` in `crates/sail-plan/src/resolver/expression/udf.rs`.
3. Watch how input types are computed from resolved arguments.
4. Open `PySparkUdfPayload::build()` in `crates/sail-python-udf/src/cereal/pyspark_udf.rs`.
5. Follow the creation of `PySparkUDF`.
6. Open `PySparkUDF::invoke_with_args()`.
7. Then jump to `RemoteExecutionCodec::try_encode_udf()`.

8. Follow `UdfKind::PySpark`.
9. Read `RemoteExecutionCodec::try_decode_udf()`.
10. Confirm that the worker reconstructs the same `PySparkUDF`.

The key question:

What data must cross the driver-worker boundary for this function to behave the same on the worker as it did in the driver's plan?

That question is the whole chapter in miniature.

11.27 Takeaways

Functions in Sail are distributed execution contracts:

- Built-ins are resolved through Sail's Spark-compatible function maps.
- Registered PySpark UDFs are stored in the catalog and materialized at call sites.
- Inline PySpark UDFs carry command bytes directly in the spec expression.
- Scalar Python UDFs implement `DataFusion ScalarUDFImpl`.
- Grouped Python aggregate UDFs implement `DataFusion AggregateUDFImpl`.
- Python UDTFs use Sail's `StreamUDF` abstraction and `MapPartitionsExec`.
- Arrow arrays and record batches are the runtime boundary between Rust and Python.
- `RemoteExecutionCodec` makes custom plans and functions executable on workers.
- Packaged extensions require registration, physical reconstruction, worker compatibility and runtime ownership checks; adding a function name is only one part of that contract.

The next chapter moves from callable behavior to tables: catalogs, table formats, lakehouse scans and writes, and how file and table providers cross the Sail/DataFusion boundary.

12 Catalogs, Lakehouse Tables, and Data Sources

Storage has several identities: a catalog name, a table location, a source implementation, a logical query, and the physical files or transaction metadata that execution reads or changes. Treating these as one object makes it hard to explain where validation and side effects belong.

This chapter follows upstream [b2470ea4b](#), still declaring Sail 0.7.1 while the book prepares for 0.7.2. Its source interface is `DataSource`, with optional `LakeSource` capability. The earlier edition's `TableFormat` trait and physical writer sketch are obsolete; they are not the current embedding API.

12.1 Code Map

Responsibility	Current source
Data source contract and registry	<code>crates/sail-common-datafusion/src/datasource.rs</code>
Lake metadata, DDL and DML capability	<code>crates/sail-common-datafusion/src/lakesource.rs</code>
Session source registration	<code>crates/sail-session/src/formats.rs</code>
Listing source implementation	<code>crates/sail-data-source/src/listing/source.rs</code>
Listing logical write node	<code>crates/sail-data-source/src/listing/write.rs</code>
Listing physical planner	<code>crates/sail-data-source/src/listing/planner.rs</code>
Delta source	<code>crates/sail-delta-lake/src/lake_source.rs</code>
Iceberg source	<code>crates/sail-iceberg/src/lake_source.rs</code>
Python adapter	<code>crates/sail-data-source/src/formats/python/adapter.rs</code>
Generic row-level logical nodes	<code>crates/sail-logical-plan/src/row_level.rs</code>

12.2 Catalogs: Names Before Data

The `CatalogManager` in `crates/sail-catalog/src/manager/mod.rs` is a session extension. It owns:

- the configured catalog providers,
- the default catalog,
- the default database,
- the global temporary database,
- temporary views,
- registered functions,
- tracked logical plans and function objects.

Its most important job is name resolution. A query like this:

```
SELECT * FROM sales.orders
```

does not say whether `sales` is a catalog or a database. Sail follows Spark-style resolution:

```
[name]
  -> default catalog + default database + table

[prefix..., table]
```

- > if prefix starts with a known catalog, use that catalog
- > otherwise use default catalog and treat prefix as database

The result of resolution is not data. It is metadata: a `TableStatus`. The table can be a physical table, a view, a temporary view, or a global temporary view.

```
pub enum TableKind {
    Table { ... },
    View { ... },
    TemporaryView { ... },
    GlobalTemporaryView { ... },
}
```

That enum is small, but it carries a lot of Spark compatibility:

Kind	What Sail does with it
Table	Builds a table scan using the table's format, location, schema, properties, and partitioning.
View	Parses the stored SQL definition and resolves it again into a logical plan.
TemporaryView	Reuses a stored logical plan from the session.
GlobalTemporaryView	Reuses a stored logical plan from the configured global temporary database.

This is why catalogs come before file formats. Sail cannot decide whether to invoke Parquet, Delta, Iceberg, or Python data source code until it knows what the name refers to.

12.3 Catalog Providers

Session startup builds the catalog manager in `crates/sail-session/src/catalog.rs`. The configured catalog list can include:

- memory catalog,
- Iceberg REST catalog,
- Unity catalog,
- OneLake catalog,
- Glue catalog,
- Hive Metastore catalog,
- the built-in system catalog.

Some catalog providers are wrapped in `RuntimeAwareCatalogProvider`, which lets them perform blocking or IO-heavy setup on the runtime intended for IO. Some are wrapped in `CachingCatalogProvider`, depending on whether the configuration asks for global or session cache behavior.

The key Rust idea here is trait objects:

```
HashMap<Arc<str>, Arc<dyn CatalogProvider>>
```

Sail does not need every catalog backend to have the same concrete Rust type. It needs each backend to implement the `CatalogProvider` contract.

That same shape reappears for named data sources:

```
HashMap<String, Arc<dyn DataSource>>
```

This is a pattern worth remembering for extension design: Sail favors small traits plus session registries over large enums that must know every implementation.

12.4 DataSource Is a Logical Planning Interface

`DataSource` names a source and plans its reads and writes. The essential return types distinguish it from the old physical-writer model:

```
async fn create_source(
    &self,
    ctx: &dyn Session,
    info: SourceInfo,
) -> Result<Arc<dyn TableSource>>;
```

```
async fn create_writer(
    &self,
    ctx: &dyn Session,
    info: SinkInfo,
) -> Result<LogicalPlan>;
```

These are method excerpts from the trait in `datasource.rs`. A read returns a logical table source; a write returns a logical plan. Physical planning happens later. The default `infer_schema` obtains a source and reads its schema, while an implementation can specialize schema inference.

A source may expose `as_lake_source(self: Arc<Self>)`. Its default is `None`. The owned receiver keeps the source alive while a caller uses the capability across asynchronous operations. A plain file or streaming source need not pretend to implement lakehouse metadata or row-level transaction semantics.

12.5 Registration and Capabilities

`DataSourceRegistry` stores `Arc<dyn DataSource>` values under lowercase names behind an `RwLock`. `get_data_source` retrieves a source; `get_lake_source` also requires the optional lake capability. A registered Parquet source can therefore be a valid data source while being invalid for a lake operation.

The registry's `register_data_source` uses map insertion. It does not implement the native extension bootstrap's duplicate-name refusal policy. Reusing a registry pattern does not mean that two registries have identical collision semantics.

`crates/sail-session/src/formats.rs` registers Arrow, Avro, binary, CSV, JSON, Parquet, text, socket, rate, console and noop sources, plus `DeltaLakeSource` and `IcebergLakeSource`. It then discovers Python sources and registers them through

`PythonDataSourceAdapter::register_all`. The accompanying test checks that ordinary sources are not accepted as lake sources and that Delta and Iceberg are.

12.6 Options, Locations and Time Travel

`OptionLayer` preserves where a setting came from: table properties, operation options, table location, or a timestamp/integer/string time-travel selector. Later option layers override earlier ones when consumed. Keeping these layers separate lets the source interpret a path differently from an ordinary option.

Converting a layer to opaque key/value options does not carry every semantic field: the implementation turns property and option lists into maps, while location and time-travel variants return empty maps. A new source must consume those typed variants deliberately rather than assuming map conversion preserves the entire read request.

`SourceInfo` and `SinkInfo` are planning inputs. Their source-specific interpretation belongs with the registered source. A catalog table contributes metadata and location; it does not remove the source's responsibility to validate its own read or write semantics.

12.7 Listing Writes

The listing implementation creates `FileWriteNode` in `crates/sail-data-source/src/listing/write.rs`. Its current options are:

```
pub struct FileWriteOptions {
    pub format: Arc<dyn WriteFormat>,
    pub url: Url,
    pub overwrite: bool,
    pub partition_by: Vec<CatalogPartitionField>,
    pub sort_by: Vec<Sort>,
}
```

This excerpt omits derive attributes, not fields. The node has one logical input and an empty result schema. Its `with_exprs_and_inputs` requires zero replacement expressions and exactly one replacement input, preserving its write options.

`listing/planner.rs` recognizes this node and checks both logical and physical input cardinality. This path belongs to listing data sources. It is not a generic claim that every Delta or Iceberg write first becomes this listing node.

```
resolved write request
-> DataSource.create_writer -> source-specific LogicalPlan
-> physical planner -> writer and any required commit work
-> Sail job planning and execution
```

Generating a logical write plan is not a commit. Execution and format-specific transaction rules still determine when data and metadata become durable.

12.8 LakeSource: Metadata and Row-Level Semantics

`LakeSource: DataSource` adds metadata inference, table metadata creation, row-level planning and alteration operations. Its default row-level planner returns explicit not-implemented errors

for DELETE, UPDATE and MERGE. Supporting a scan is not evidence that a source supports those mutations.

Table metadata creation is separate from registering the catalog object. The trait permits a source to create required storage metadata before catalog registration; its default implementation does no work. This is one reason that catalog and storage behavior must remain distinct in the planner and tests.

Generic row-level planning uses `RowLevelWriteNode` in `crates/sail-logical-plan/src/row_level.rs`. Source-specific implementations then choose the appropriate physical writers and commit behavior. The shared `RowLevelWriteMode` distinguishes copy-on-write from merge-on-read. When rows must be materialized, the first rewrites affected data files; the second writes row-level delete artifacts alongside new data. Metadata-only optimizations may bypass materialization in either mode.

Internal columns are not user data. `__sail_file_path` identifies source files; `__sail_file_row_index` identifies file-local rows for deletion vectors; `__sail_operation_type` carries write intent. Source writers must remove the operation column before persisting user rows. Its integer tags are internal physical-plan conventions, not external storage protocol values.

12.9 Delta and Iceberg Have Their Own Write Paths

Read `DeltaLakeSource` and `IcebergLakeSource` before following their lower-level writers. Delta keeps scan construction under `datasource/`, logical operations under `logical/`, and write/delete planning under `physical_plan/planner/`. Its transaction and snapshot modules own different responsibilities from a `DataFusion` batch writer.

Iceberg likewise separates its lake-source entry point, data source planning, logical nodes, physical writers and commit modules. Position/equality delete writers, write locations and manifest-related logic are not interchangeable with a plain Parquet sink. A reader following only the file writer will miss the metadata operation that makes those files part of a table.

For distributed changes, test both where batches are written and where the transaction commits. Driver placement for a metadata operation does not imply that every input scan or data writer runs on the driver. Conversely, successful worker output does not prove that a table commit succeeded.

12.10 Python Data Sources

`PythonDataSourceAdapter` implements `DataSource`. Its module contains a `PythonWriteNode`; the package separates table-provider, execution, write and commit code. Discovery is part of session source registration. This is an existing extension mechanism, distinct from the experimental `pysail.extensions` native-package bootstrap in Chapter 13.

The distinction prevents two common mistakes: expecting the native relation manifest to register an arbitrary Python data source, and assuming a Python source's discovery hook supplies native worker codec compatibility. Each path has its own deployment and execution contract.

12.11 Storage in the Graph Extensions

Pecan owns input snapshots, intermediate materialization and final output paths. These are ordinary query storage operations. Native Banda staging additionally owns native graph state. Argentea's worker adjacency lives within a bounded job; it is not a durable lakehouse table or a recovery log.

Shared storage can make inputs and results accessible to both hosts without serializing them through the client. It does not establish stable native owner placement, replay safety, budget admission, or the lifetime of exported Arrow buffers. Those are separate contracts in the extension and execution layers. A storage-sharing feature should therefore be assessed as storage infrastructure, not taken as proof that a stateful graph algorithm is distributed correctly.

12.12 Contributor Checks

For a new source, verify ordinary reads, schema inference, option precedence, logical writes and explicit unsupported operations. If it exposes LakeSource, add metadata and row-level cases appropriate to that capability. For distributed writes, include codec reconstruction, worker output, driver-side metadata work, failed attempts and cleanup. Use an independent read or metadata inspection to confirm the final table state.

For graph staging, test owned-path cleanup after success, cancellation and failed writes. Retain the failure receipt and distinguish deferred session cleanup from immediate cleanup. A later empty directory is useful evidence, but cannot by itself prove that a native memory lease was released.

Follow a concrete operation through the current source: registry lookup, `create_source` or `create_writer`, the returned logical node, physical planning, job placement, execution and final metadata state. This trace makes the boundary between reusable Sail infrastructure and source-owned behavior reviewable.

13 Extension Architecture in the Implemented Branch

An extension crosses several boundaries: client intent, session registration, planning, serialization, worker execution, memory ownership, and teardown. A scalar function exercises fewer of these than a distributed graph algorithm. The host contract should expose the boundary each needs without importing its domain implementation into Sail.

This chapter describes the experimental `querygraph/sail` extension branch pinned in the reader guide. It is not a description of released Sail 0.7.2. Upstream already includes the session-factory injection from merged #2630; the protocol, package bootstrap, native placement, and resource contracts below are additional branch work. The earlier proposal's broad Rust trait sketches are not the implemented package interface.

13.1 Four Graph Paths and a Spatial Consumer

Consumer	Where computation lives	Main host responsibility
SedonaDB scalar functions	Native DataFusion UDFs on executing workers	Registration, package identity, expression codecs and geometry fields
Pecan	Ordinary Sail queries submitted by a Python algorithm client	Existing relational execution and owned materialization
Nutmeg Banda	Native graph kernels over explicitly staged graph state	Native admission, placement, cancellation and retained ownership
Nutmeg Grenada	Relational graph computation inside Sail/DataFusion	Existing plans, joins, aggregates and execution resources
Argentea	Native partitions retained on Sail workers during one bounded job	Worker-native decoding, stable placement, job lifetime and fail-fast cleanup

Banda and Grenada name execution paths within the Nutmeg experiment. Pecan is a portable graph client; it does not require a new graph execution engine in Sail. Argentea extends the native path to worker-owned partitions. It is not a second scheduler or transport service.

The distinction matters for memory. Relational plans use Sail’s DataFusion runtime and can expose their operators to its optimizer. A native kernel keeps its own state and must account it explicitly. Running that kernel in the same process does not automatically add its allocations to a query memory pool.

13.2 Code Map

These paths refer to the pinned extension branch unless marked upstream.

Boundary	Source to read
Upstream embedding entry point	<code>crates/sail-spark-connect/src/entrypoint.rs</code>
Connect envelope decoding	<code>crates/sail-spark-connect/src/proto/extension.rs</code>
Relation resolution	<code>crates/sail-plan/src/resolver/query/extension.rs</code>
Session protocol contract	<code>crates/sail-common-datafusion/src/connect_extension.rs</code>
Driver-native contract	<code>crates/sail-common-datafusion/src/driver_extension.rs</code>
Worker-native contract	<code>crates/sail-common-datafusion/src/worker_extension.rs</code>
Native accounting	<code>crates/sail-common-datafusion/src/native_resource.rs</code>
Resource lease ABI	<code>crates/sail-native-resource-ffi/src/lib.rs</code>
Worker session construction	<code>crates/sail-session/src/session_factory/worker.rs</code>
Job partitioning	<code>crates/sail-execution/src/job_graph/planner.rs</code>
Native graph package	<code>examples/extensions/nutmeg/</code>
Distributed algorithm core	<code>examples/extensions/argentea/src/</code>
Portable graph client	<code>examples/extensions/graph-algorithms/src/pyspark_pecan/</code>

The extension examples have independent Cargo workspaces. Sedona has no Sail engine dependency. Nutmeg shares the small resource-lease ABI definition, rather than depending on Sail's planner or scheduler crates.

13.3 The Protocol Actually Implemented

Spark Connect offers relation, expression and command extension messages. The branch does not implement all three raw dispatch paths.

Client operation	Implemented representation
Extension relation	Relation.extension carrying a registered type URL
Native scalar expression	Ordinary unresolved function call, resolved by name
Mutation with a receipt	A relation whose execution performs the operation

Raw Expression.extension and Command.extension dispatch are not implemented. A command-shaped relation must be executed, for example by collecting its receipt. Constructing a DataFrame or asking for its schema must not mutate graph state.

A relation with DataFrame inputs uses an outer google.protobuf.Any type URL type.googleapis.com/sail.extension.v1.SailExtensionRequest. Its payload is:

```
message SailExtensionRequest {
  string payload_type_url = 1;
  bytes payload = 2;
  repeated spark.connect.Plan inputs = 3;
  repeated spark.connect.Expression input_expressions = 4;
  uint32 envelope_version = 5;
}
```

Version 1 is required. Inputs are root plans; input expressions are rejected by this implementation. Sail resolves the input plans and restores their column names before dispatch. The inner payload belongs to the extension, such as Nutmeg's versioned JSON request. Bare payloads are accepted only where the manifest explicitly permits them.

The pinned client environment uses Python 3.12, PySpark 4.0.1, protobuf runtime 7.36.2, grpcio 1.84.0 and PyArrow 21.0.0. PySpark supplies generated Connect messages; client code does not need to regenerate them with protoc. The protobuf Python runtime version is separate from Sail's build-time protobuf compiler. The example clients use internal PySpark plan APIs, so this pin is part of reproduction.

13.4 Package Discovery and Compatibility

With SAIL_EXPERIMENTAL_EXTENSIONS=1, Sail loads installed pysail.extensions entry points in name order. A package bootstrap returns a factory with a manifest and binding methods. Without the opt-in, ordinary Sail remains available. A package declaration looks like:

```
[project.entry-points."pysail.extensions"]
nutmeg = "sail_nutmeg:extension"
```

The manifest includes API, DataFusion and Arrow versions, package identity, placement, and relation registrations. Each registration names a type URL and its accepted input cardinality. Colliding names or URLs are refused rather than silently overriding builtins. The pinned examples

use DataFusion 55.1.0 and Arrow 59.3.0. Matching these declarations is necessary, but does not constitute a universal stable-ABI guarantee.

The loader checks package content and configuration as well as declared identity when tasks cross to workers. Changing package bytes under an unchanged version must not silently change an executing job. Deployment must record the native wheel for each platform; an ARM wheel and an Intel wheel cannot have identical bytes. Consult the compatibility matrix for the particular artifact combination. A same-version manifest alone does not establish cross-platform compatibility.

Installed native packages are trusted code. Manifest validation does not sandbox Rust, Python bootstrap code, GEOS, filesystem access, or network access.

13.5 Planning Without Eager Execution

A bound relation handler receives the payload and named `datafusion_execution_plan` capsules for its inputs. A driver relation returns a `datafusion_table_provider` capsule containing an `FFI_TableProvider`. The provider describes work; execution consumes inputs and produces Arrow batches. Native scalar objects expose a `datafusion_scalar_udf` capsule containing an `FFI_ScalarUDF`.

This separation allows schema inspection and optimizer traversal without executing an algorithm or changing a staged graph. Native state capture, algorithm work, and drop operations happen during execution. Reconnects and repeated actions do not imply exactly-once mutation semantics. Clients must consume receipts and respect the documented graph/session lifetime.

The small buildable packages are the best implementation examples. A new package should start from their pinned capsule contracts rather than implementing an imagined public Rust `SailExtension` trait.

13.6 Sedona: Fields Are Part of Correctness

The example imports actual SedonaDB native/GEOS scalar functions. It exposes 128 functions plus aliases at the recorded revision, with selected semantic checks. The count is not a claim that every function has exhaustive coverage. Ordinary Sail joins can evaluate spatial predicates; an indexed spatial join, raster surface and the full Spark Sedona API are not implemented here.

Geometry is represented by WKB with GeoArrow field metadata. Correctness therefore requires more than transporting binary bytes. Compatible fields must survive selected expression composition, arrays and shuffle serialization. Arbitrarily copying metadata from an input would mislabel incompatible binary, geography or coordinate-reference data. Collision policy also matters: excluded names retain Sail's existing implementation or placeholder rather than silently taking on a plugin's semantics.

The pinned Sedona source required a DataFusion/Arrow compatibility port, and its wheels bundle GEOS. Dependency closure and platform tags are part of the artifact qualification. Loading a wheel successfully is weaker evidence than executing its geometry functions on the actual workers.

13.7 Pecan and Grenada: Use the Existing Relational Engine

A graph table is still a relation. Degree, triplet and bounded-walk operations can be expressed as joins and aggregates over vertex and edge DataFrames. The Nutmeg graph-table helper can run these ordinary plans without native extension loading or CSR construction.

Pecan implements iterative graph algorithms by submitting ordinary relational work and retaining owned materialized state between iterations. Its control flow is in the client; each query still executes through Sail. Grenada evaluates the relational graph path through Sail/DataFusion. These paths expose relational work to the existing engine instead of exporting it to an external graph service.

These choices incur different costs from a native adjacency representation: joins, repartitioning, materialization, and repeated actions can dominate. They also reuse ordinary execution and memory mechanisms. Neither advantage proves a performance result without the same graph, semantics and resource envelope.

13.8 Banda: Explicit Native State and Memory Ownership

Banda stages graph input and invokes native kernels. The driver-native path can consume distributed input without distributing the kernel itself. Gathering input from workers is not distributed PageRank or WCC.

The host reserves a native quota from an explicit resource domain and passes a lease through `bind_with_resources`. Native state and retained output buffers must keep that lease alive until their final owner releases it. Accounting must precede expansion, rather than estimating memory only after allocation succeeds. A shared CSR cache and Arrow output can outlive a temporary computation object; releasing the session's reference alone is therefore insufficient.

The quota is non-spillable and is not an operating-system RSS ceiling. A global allocator such as `mimalloc` changes allocation behavior, not which query pays for an allocation. The experimental contract supplies accounting and ownership, while the allocator supplies storage. Cancellation checks, job close and final buffer release are distinct events that need distinct tests.

13.9 Argentea: Worker-Local Native Partitions

Argentea retains source-owned native graph partitions in workers during one job. The client builds a bounded sequence of native regions linked by existing Sail exchanges. Each owner participates in phase completion, including owners with no vertices. Requests and messages carry operation, snapshot, generation, owner and phase information so that stale or misrouted work can be rejected.

The host additions provide registration and decoding on workers, admission from the worker's resource domain, stable task-group placement, and lifecycle closure. The extension owns adjacency, frontier or component state, algorithm messages, certificates and convergence. It reuses Sail's planning, task scheduling and Arrow streams. It does not introduce a graph transport or a general iteration service.

Phase count is bounded before execution. BFS uses $2K+4$ stages; its defaults remain 14 levels and 32 stages, with explicit support up to 62 levels and 128 stages. The larger limit passes ARM process-cluster tests for all three BFS methods, including a chain that consumes all 62 expansions and a graph that converges after four. Lazy session views keep individual Connect plans shallow under the existing wire guard. View registration is not an action per graph iteration: one terminal materialization executes the composed native job.

The algorithm families must remain explicit:

- PageRank has reference and residual/frontier implementations. Termination and normalization semantics belong in the request and correctness oracle.
- BFS has reference, frontier and direction-optimizing methods. Choosing the last does not prove that a particular run entered pull mode; receipts record modes.
- WCC has reference label propagation and seeded star contraction. Argentea's advanced form operates over original edges; it is not identical to the fused physical edge contraction used in other graph paths.
- SSSP has reference relaxation and an advanced all-edge delta-star method. The latter is not classical light/heavy delta stepping. It avoids scanning inactive edges, but current barrier work still includes local label scans and copies.

An iteration cap is a refusal to certify a complete result, not permission to return partial answers as success. Cap tests check the typed native cause, completed topology or barriers, failed job and cleanup. Algorithm names alone cannot establish equivalent work or equivalent stopping criteria across paths.

13.10 Failure Semantics and Observability

The prototype uses whole-query failure before attempting recovery of retained native state. A worker loss must not cause a silent task replay against missing or inconsistent partition state. Native identity, stage inventory, task attempts and final job status are checked together.

One focused host repair marks nonterminal tasks canceled before unassigning them when a job fails or is canceled. Previously a failed job could leave task records showing RUNNING. The repair preserves successful-job ordering and existing terminal states. This is a scheduler bookkeeping responsibility, not graph logic.

The first client error can be a scheduler-wrapped failure or an earlier transport error. A peer's cancellation can also arrive before the original native memory refusal. WCC and SSSP therefore preserve a causal `native_memory_budget` receipt with owner identity and whether state had initialized. The adapter matches its pinned dependency's exact local error; it does not infer a memory cause from a small quota or a generic cancellation.

Receipts report accounted live and peak reservations after the failed call unwinds. Those numbers are neither RSS nor the size of the refused allocation. A memory test requires initialized owners, the causal record, terminal tasks, closed state and successful subsequent work on the same live workers.

All four reference/advanced WCC and SSSP variants pass that reuse sequence on two ARM worker processes. A 32 MiB native quota sits inside a 48 MiB Sail pool: a leaked full reservation would prevent another such admission. The test retains readable Parquet results while issuing later jobs, then checks session, staging and process cleanup. It does not claim universal leak freedom or physical two-host coverage.

13.11 Build, Run and Review

Use the branch's source-distribution tutorial to build Sail and the separately packaged extensions. The build requires Rust 1.97.1, Python 3.12 with a shared library, uv, Git, protobuf tooling, a C/C++ toolchain and GEOS development files. Pin the commit before building; the original `sail-extensions-1` tag is an earlier snapshot and does not contain every later graph feature.

```
git clone --branch work/extensions-traversal-bench https://github.com/querygraph/sail.git
cd sail
git checkout dcd44f287b8422a62abd06d2410aaf96e225e136
bash examples/extensions/scripts/build.sh
```

The script builds a development-profile Sail executable and separate Sedona and Nutmeg wheels, installs Pecan, and reports their locations. The combined Nutmeg wheel also registers the Argentea worker factory. This is a functional review build; performance comparisons require the disclosed release-profile artifacts and benchmark settings. Read `examples/extensions/TUTORIAL.md` for local/process-cluster and multi-host startup. Use `examples/extensions/WRITING-AN-EXTENSION.md` for the minimal scalar, relation and command-shaped examples. Its original driver-native examples are not the Argentea worker protocol. Use `examples/extensions/argentea/PYTHON.md`, `BFS.md`, `WCC_ADAPTER.md`, `SSSP_ADAPTER.md`, `FAULTS.md` and `GRAPH_RESOURCES.md` for that protocol's independently executable qualification commands.

The evidence is intentionally scoped. PageRank and the original bounded BFS have physical two-host functional results. WCC, SSSP, graph fault/resource sequences and expanded BFS currently have ARM process-cluster evidence; remaining Linux and physical-host qualification is not supplied by a successful unit test. The large Pecan/Banda/Grenada traversal campaign is separate from these functional checks.

13.12 Review the Host Contract by Responsibility

A maintainable review isolates each host responsibility and its executable invariant: dispatch and naming; package identity; geometry field preservation; placement and codec reconstruction; native admission and retained ownership; job-scoped cleanup and fail-fast behavior. Domain algorithms stay in example packages. Pecan's relational path is a useful control: it demonstrates which graph operations need no new host hook at all.

The branch contains the combined experiment. Its design review describes logical review units; it does not claim that independently extracted upstream patches have already been validated. Keep that distinction when planning integration. The smallest defensible change is one whose missing capability has been observed, whose owner in Sail is clear, and whose invariant has an executable test.

14 Arrow Flight SQL

Spark Connect is Sail's primary front door, but it is not the only one. Sail also exposes Apache Arrow Flight SQL for clients that want SQL over Arrow-native transport rather than PySpark over Spark Connect.

This chapter is deliberately short and concrete. Its job is to place `sail-flight` in the definitive architecture: it is a second protocol surface that converges on the same SQL parser, Sail spec layer, DataFusion physical planning, JobService, and Arrow RecordBatch streams.

14.1 Code Map

Concern	File
Flight SQL service	<code>crates/sail-flight/src/service.rs</code>
Query handle state	<code>crates/sail-flight/src/state.rs</code>
Metrics wrapper	<code>crates/sail-flight/src/metrics.rs</code>
SQL parser entry point	<code>crates/sail-sql-analyzer/src/parser.rs</code>
SQL AST to spec	<code>crates/sail-sql-analyzer/ src/statement.rs</code>
Planning entry point	<code>crates/sail-plan/src/lib.rs</code>
Job runner extension	<code>crates/sail-common-datafusion/src/ session/job.rs</code>
Session manager	<code>crates/sail-session/ src/session_manager/</code>

14.2 Why Flight SQL Exists Beside Spark Connect

Spark Connect is Spark-shaped. It carries unresolved Spark relations, commands, expressions, config operations, reattachable execution, artifacts, and session semantics. That is exactly what PySpark needs.

Flight SQL is SQL-shaped. It is useful for ADBC, JDBC-style tools, BI clients, and systems that already understand Arrow Flight. The protocol does not try to model the Spark DataFrame API. It asks a simpler question:

Given this SQL statement, what schema will it produce, and where can I fetch the Arrow stream?

That makes `sail-flight` an excellent example of Sail's internal layering. The protocol is different, but the planning core is the same.

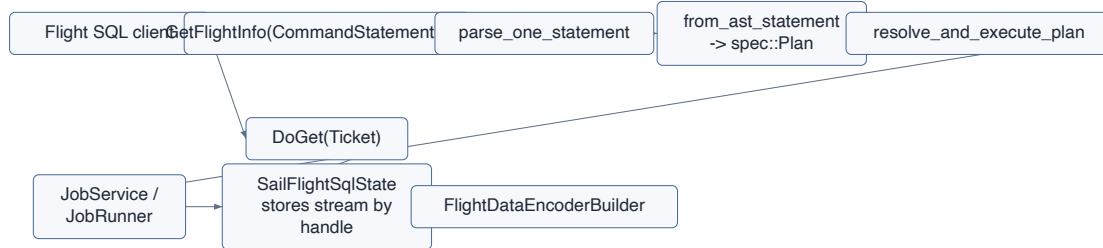


Figure 67: Flowchart 14.1

14.3 The Service Shape

`SailFlightSqlService` is small compared with `SparkConnectServer`:

```
pub struct SailFlightSqlService {
    session_manager: SessionManager,
    config: Arc<PlanConfig>,
    metrics: Option<Arc<MetricRegistry>>,
    state: Arc<Mutex<SailFlightSqlState>>,
}
```

The service implements `FlightSqlService` from the `arrow-flight` crate. The key methods for query execution are:

- `get_flight_info_statement`
- `do_get_statement`
- `do_handshake`

Other Flight SQL methods can remain unimplemented until Sail needs them. This is a different compatibility posture than Spark Connect, where PySpark expects a wider service surface.

14.4 Phase One: `GetFlightInfo`

Flight SQL separates planning from fetching. The first request is `GetFlightInfo(CommandStatementQuery)`. Sail parses the SQL text, converts it to `spec::Plan`, creates a physical plan, starts execution through the session's `JobService`, and stores the resulting stream under an opaque handle.

The key sequence is:

```
let statement = parse_one_statement(&query.query)?;
let plan = from_ast_statement(statement)?;
let ctx = self.get_session_context().await?;
let (plan, _) = resolve_and_execute_plan(&ctx, self.config.clone(), plan).await?;
let schema = plan.schema();
let service = ctx.extension::<JobService>()?;
let stream = service.runner().execute(&ctx, plan).await?;
```

That is the same core path used elsewhere:

SQL text

- > SQL AST
- > Sail spec
- > DataFusion physical plan
- > JobRunner stream

The service then stores the stream in `SailFlightSqlState`:

```
let handle = QueryHandle::new();
self.state.lock().await.add_stream(handle.clone(), stream);
```

The returned `FlightInfo` contains the schema and a `TicketStatementQuery` carrying that handle.

14.5 Phase Two: DoGet

The second request presents the ticket:

```
let handle = QueryHandle::try_from(ticket.statement_handle.as_ref())?;
let stream = self
    .state
    .lock()
    .await
    .remove_stream(&handle)
    .ok_or_else(|| Status::not_found(...))?;
```

The handle is consumed once. This keeps server state simple: Flight SQL clients can re-run a query if a fetch fails, while Spark Connect clients use reattachable execution and response buffering.

After retrieving the stream, Sail encodes batches as Flight data:

```
let output = FlightDataEncoderBuilder::new()
    .with_schema(schema)
    .build(output)
    .map(|result| result.map_err(|e| Status::internal(format!("encoding error: {e}"))));
```

This is where Flight SQL differs from Spark Connect at the wire level. Spark Connect wraps Arrow IPC bytes in Spark protobuf messages. Flight SQL sends Arrow Flight frames directly.

14.6 The Shared Session Model

The current Flight SQL service uses a shared default session:

```
const DEFAULT_SESSION_ID: &'static str = "flight-default";
const DEFAULT_USER_ID: &'static str = "flight-user";
```

That means Flight SQL queries share session configuration and catalog state. It is a reasonable initial model, but a definitive guide should call it out because it is not the same as Spark Connect's client-provided session IDs.

A future version could carry session identity in Flight headers. The important architectural point is that the same `SessionManager` and `SessionContext` machinery still applies.

14.7 Commands

SQL commands are different from queries because the effect matters more than the result stream. Sail classifies the parsed plan:

```
let statement_type = match &plan {  
  spec::Plan::Query(_) => StatementType::Query,  
  spec::Plan::Command(_) => StatementType::Command,  
};
```

For commands, the service drains the stream eagerly in `get_flight_info_statement`. That ensures the command has completed before the client receives `FlightInfo`.

This detail matters for DDL. A BI client that sends `CREATE TABLE` should not have to call `DoGet` just to make the side effect happen.

14.8 Metrics

When OpenTelemetry metrics are available, `sail-flight` wraps the result stream in `MetricsRecordingStream`. The wrapper records row counts, batch counts, elapsed time, and statement type.

This mirrors a general Sail design habit: protocol crates should add protocol-level metrics around streams without changing the core execution layer.

14.9 Spark Connect Versus Flight SQL

Concern	Spark Connect	Flight SQL
Main client	PySpark	ADBC/JDBC/BI tools
Request payload	Spark protobuf plan	SQL statement
Result transport	Spark <code>ExecutePlanResponse</code> with <code>ArrowBatch</code>	Arrow Flight <code>FlightData</code>
Session identity	Client-provided session ID	Shared default session today
Retry model	Reattachable operation stream	Re-execute query
Planning convergence	<code>spec::Plan</code>	<code>spec::Plan</code>
Execution convergence	<code>JobService / JobRunner</code>	<code>JobService / JobRunner</code>

14.10 Extension Boundaries Across Front Doors

Flight SQL and Spark Connect converge on common planning and execution services, but their protocol-specific dispatch remains different. The upstream Flight entry point calls `create_flight_session_manager`; Spark Connect's merged #2630 factory-selection entry point does not automatically replace Flight startup.

A scalar or data-source registration installed in the appropriate session can be used by the ordinary resolver. A Spark Connect `Relation.extension` envelope, however, is not a Flight SQL

request. The experimental Nutmeg and Argentea clients use that Connect path. Their successful Connect qualification is not evidence that a Flight client can send the same payload or invoke the same operations.

To expose an operation through another front door, define its client-visible representation and wire it to the shared semantic layer, then test that protocol’s session setup, plan creation, execution and cleanup. Keep worker package decoding and resource lifetime common where possible. This preserves the shared engine without pretending the protocols have identical extension surfaces.

14.11 Takeaways

Flight SQL is not a separate query engine inside Sail. It is another way to reach the same SQL parser, Sail spec IR, DataFusion planning path, and job runner. The protocol differences are real, especially around session identity and fetch handles, but the internal convergence is the point.

Navigation: Previous: Chapter 13, Extension Architecture | Next: Chapter 15, Custom Nodes And Optimizers | Reader Guide

15 Custom Nodes And Optimizers

Sail uses DataFusion as its query kernel, but Spark compatibility requires plan constructs DataFusion does not provide directly. The solution is a set of custom logical nodes, custom physical plans, logical optimizer rules, physical optimizer rules, and extension planners that connect the pieces.

This chapter collects that machinery in one place. Earlier chapters introduced it as part of DataFusion and plan resolution; here we treat it as a contributor’s map.

15.1 Code Map

Concern	File
Logical nodes	<code>crates/sail-logical-plan/src/</code>
Physical nodes	<code>crates/sail-physical-plan/src/</code>
Logical optimizer rules	<code>crates/sail-logical-optimizer/src/</code> <code>lib.rs</code>
Lakehouse physical planners	<code>crates/sail-session/src/planner.rs</code> , <code>crates/sail-delta-lake/</code> , <code>crates/sail-iceberg/</code>
Physical optimizer rules	<code>crates/sail-physical-optimizer/src/</code>
Physical optimizer pipeline	<code>crates/sail-physical-optimizer/src/</code> <code>lib.rs</code>
Extension physical planner	<code>crates/sail-session/src/planner.rs</code>
Session optimizer registration	<code>crates/sail-session/src/</code> <code>session_factory/server.rs</code>

15.2 The Pattern

A Spark-specific plan feature usually crosses five layers:

spec node or resolver condition

- > DataFusion LogicalPlan::Extension(UserDefinedLogicalNodeCore)
- > optional logical optimizer rewrite
- > ExtensionPhysicalPlanner downcast
- > DataFusion ExecutionPlan
- > optional physical optimizer rewrite

That is a lot of plumbing, but it buys a clear boundary: Sail can keep using DataFusion's optimizer and execution APIs while adding Spark-shaped behavior.

15.3 Logical Node Inventory

The current Sail logical extension nodes include:

Logical node	Main purpose
RangeNode	Spark range
ExplicitRepartitionNode	repartition, coalesce, repartitionByRange
ShowStringNode	df.show() table-string formatting
MapPartitionsNode	stream and Python/Pandas map-style UDF execution
FileWriteNode	writes through table/data-source formats
FileDeleteNode	DELETE planning
MergeIntoNode	MERGE planning before row-level expansion
MonotonicIdNode	monotonically_increasing_id()
SparkPartitionIdNode	spark_partition_id()
SortWithinPartitionsNode	Spark partition-preserving sort
SchemaPivotNode	schema-producing pivot behavior
CatalogCommandNode	DDL/catalog commands as physical work
BarrierNode	streaming barrier/checkpoint coordination
streaming source/filter/limit/collector nodes	structured streaming flow-event plans

The exact list can change, so readers should treat `crates/sail-session/src/planner.rs` as the authoritative dispatch table. If a logical node cannot be downcast there, it will not become a physical plan.

15.4 Example: Range

`spark.range(start, end, step, numPartitions)` is a good first node because it is a leaf plan. The logical node carries the range parameters and schema. The physical planner downcasts it and constructs `RangeExec`.

The lesson is simple:

Spark has a relation; Sail models it as a `DataFusion` extension node; physical planning turns it into an executable operator.

`Range` also contains partitioning logic, which makes it distributed-friendly. Each partition receives a slice of the range rather than every worker scanning the whole sequence.

15.5 Example: Explicit Repartition

Spark repartitioning semantics are more explicit than `DataFusion`'s default optimizer choices. Sail models user-requested repartitioning as `ExplicitRepartitionNode`.

That node survives logical planning so the physical optimizer can later decide the right concrete physical shape:

- hash repartition,
- round-robin repartition,
- coalesce,
- or passthrough.

This is a recurring Sail technique: preserve Spark intent long enough that a later stage can lower it correctly.

15.6 The Extension Physical Planner

`ExtensionPhysicalPlanner` in `sail-session` is the central bridge. It implements `DataFusion`'s `ExtensionPlanner` trait and performs a sequence of downcasts:

```
if let Some(node) = node.as_any().downcast_ref::<RangeNode>() {
    ...
} else if let Some(node) = node.as_any().downcast_ref::<ShowStringNode>() {
    ...
} else if let Some(node) = node.as_any().downcast_ref::<MapPartitionsNode>() {
    ...
}
```

This is not glamorous code, but it is one of the most important files in Sail. It tells you which extension nodes are executable and where each one enters the physical layer.

The planner chain is assembled in `ExtensionQueryPlanner`:

```
lakehouse extension planners
-> system table planner
-> listing table planner
-> Sail custom extension physical planner
```

Ordering matters. Delta and Iceberg table planners get a chance to handle table- format-specific nodes before the generic Sail planner handles ordinary logical extension nodes.

15.7 Logical Optimizers

Sail has a small logical optimizer layer in front of DataFusion’s defaults.

`DecorrelateLateralProjection` handles a Spark lateral-subquery case before DataFusion’s broader decorrelation rule runs. The important point is not just the rule itself, but its placement. It must run before DataFusion’s `DecorrelateLateralJoin` because it handles a simpler projection-only case.

Lakehouse writes add another logical rule: `ExpandRowLevelOp`. It rewrites lakehouse MERGE and DELETE nodes into row-level write plans that format-specific planners can execute.

That rule is the bridge between Spark SQL commands and Delta/Iceberg physical planning.

15.8 Physical Optimizers

Sail’s physical optimizer pipeline is more deliberate than “append some rules to DataFusion.” `sail-physical-optimizer` reconstructs the DataFusion rule order and adds Sail-specific rules at selected points.

The custom rules include:

Rule	Purpose
<code>JoinReorder</code>	Dynamic-programming join reorder with safeguards
<code>RewriteExplicitRepartition</code>	Lowers Sail’s explicit repartition placeholder
<code>RewriteCollectLeftHashJoin</code>	Ensures collect-left joins have valid partitioning
<code>EnforceBarrierPartitioning</code>	Enforces streaming barrier partition requirements

This means contributors must distinguish logical optimizer changes from physical optimizer changes. A logical rule rewrites `LogicalPlan`. A physical rule rewrites `ExecutionPlan`.

15.9 Contributor Checklist

When adding a new custom operator, ask:

1. Does the Sail spec need a new representation?
2. Does the resolver need to produce a logical extension node?
3. Does the logical node implement `UserDefinedLogicalNodeCore` correctly?
4. Does projection pushdown need `necessary_children_exprs`?
5. Does the physical node implement `ExecutionPlan` and declare `PlanProperties`?
6. Does `ExtensionPhysicalPlanner` downcast and plan it?
7. Does the physical plan need codec support for distributed execution?

8. Does it need logical or physical optimizer support?
9. Does it need tests in local and cluster execution?

The codec question is easy to miss. A node that works locally can still fail in cluster mode if workers cannot decode it.

15.10 Extension Implications

Third-party extensions need this same multi-layer path. A physical operator alone is not enough. A logical optimizer rule alone is not enough. A function registry entry alone is not enough if the physical plan later runs on a worker that cannot decode it.

That is why the extension architecture chapter treats extension registration as a bundle of contributions:

- functions,
- logical rules,
- physical rules,
- extension planners,
- session config,
- table formats,
- codecs.

15.11 Opaque Native Regions and Visible Relational Work

The experimental graph paths illustrate a useful planning boundary. Pecan and Nutmeg’s graphable helpers emit ordinary relational plans, so Sail and DataFusion can optimize their joins, projections and aggregates. Native Banda and Argentea regions expose input/output contracts while their kernel internals remain opaque. The host cannot infer an arbitrary native algorithm’s cost, ordering or retry safety merely from its output schema.

Preserve truthful `PlanProperties` and required distributions. Argentea’s descriptor validates integer-range routing, and its worker stages reuse explicit slot-sharing groups for stateful ownership. Those are execution requirements, not optional optimizer hints. A rewrite that changes partition ownership or duplicates a stateful region can invalidate the algorithm even if the resulting schema matches.

Test both plan shape and execution. The expanded BFS qualification caught a client-side bound that prevented the composed plan from reaching native execution; other classes of defect require observing actual worker placement and phase completion. A schema-only test cannot certify either property.

15.12 Takeaways

Custom nodes are the places where Spark semantics become DataFusion-compatible plans. Optimizer rules preserve or lower those semantics at the right stage. `sail-session/src/planner.rs` is the physical dispatch map, and `sail-physical-optimizer/src/lib.rs` is the physical rewrite map.

16 Local And Streaming Execution

The distributed chapters explain the cluster path in detail: job graphs, drivers, workers, task regions, streams, and shuffles. This chapter fills in two execution views that deserve their own concise treatment:

- local execution, where Sail directly asks DataFusion to execute a physical plan,
- structured streaming, where Sail rewrites a logical plan into a flow-event plan before physical execution.

Both use the same JobRunner abstraction, which is exactly why the abstraction is valuable.

16.1 Code Map

Concern	File
Job runner trait	<code>crates/sail-common-datafusion/src/session/job.rs</code>
Local and cluster runners	<code>crates/sail-execution/src/job_runner.rs</code>
Streaming rewriter	<code>crates/sail-plan/src/streaming/rewriter.rs</code>
Streaming source trait	<code>crates/sail-common-datafusion/src/streaming/source.rs</code>
Flow event schema	<code>crates/sail-common-datafusion/src/streaming/event/schema.rs</code>
Flow event streams	<code>crates/sail-common-datafusion/src/streaming/event/stream.rs</code>
Streaming logical nodes	<code>crates/sail-logical-plan/src/streaming/</code>
Streaming physical nodes	<code>crates/sail-physical-plan/src/streaming/</code>
Streaming query manager	<code>crates/sail-spark-connect/src/streaming.rs</code>
Rate source	<code>crates/sail-data-source/src/formats/rate/reader.rs</code>
Socket source	<code>crates/sail-data-source/src/formats/socket/reader.rs</code>

16.2 One Trait, Two Execution Modes

The JobRunner trait hides the execution backend:

```
#[tonic::async_trait]
pub trait JobRunner: StateObservable<JobRunnerObserver> + Send + Sync + 'static {
    async fn execute(
        &self,
        ctx: &SessionContext,
        plan: Arc<dyn ExecutionPlan>,
    ) -> Result<SendableRecordBatchStream>;

    async fn stop(&self, history: oneshot::Sender<JobRunnerHistory>);
}
```

The local and cluster runners implement the same method. Protocol code does not need to know which mode is active. It retrieves `JobService` from the session and calls:

```
service.runner().execute(ctx, plan).await
```

That line is one of Sail's key architecture compression points.

16.3 Local Execution

`LocalJobRunner` is intentionally small:

```
let plan = trace_execution_plan(plan, options)?;
Ok(execute_stream(plan, ctx.task_ctx())?)
```

The runner wraps the plan with telemetry tracing and then delegates to `DataFusion`'s `execute_stream`. `DataFusion` handles partition execution inside the process, and `Sail` receives a `SendableRecordBatchStream`.

Local mode is not a lesser engine. It is the same planning path with a simpler execution backend. That makes it useful for:

- development,
- compatibility tests,
- small deployments,
- debugging physical plans before cluster concerns enter.

16.4 Cluster Execution

`ClusterJobRunner` sends the physical plan to the driver actor:

```
self.driver.send(DriverMessage::ExecuteJob {
    plan,
    context: ctx.task_ctx(),
    result: tx,
}).await?;
```

The driver builds a job graph and eventually returns a stream. From the caller's perspective, local and cluster modes both produce `SendableRecordBatchStream`.

The difference is below the trait boundary.

16.5 Streaming Starts As A Logical Rewrite

Streaming is not a different protocol. A streaming query still enters through Spark Connect, resolves to a logical plan, and becomes a physical plan. The key difference is that Sail rewrites the logical plan before physical planning:

```
let plan = if is_streaming_plan(&plan)? {  
    rewrite_streaming_plan(plan)?  
} else {  
    plan  
};
```

The rewriter turns ordinary plan nodes into streaming-aware extension nodes where needed. For example:

- batch table scans over streaming sources become `StreamSourceWrapperNode`,
- `RangeNode` can be wrapped by `StreamSourceAdapterNode`,
- filters and limits become streaming filter/limit nodes,
- sinks and display paths use collectors and barriers.

16.6 Stream Sources

A streaming source implements `StreamSource`:

```
#[async_trait::async_trait]  
pub trait StreamSource: Send + Sync + fmt::Debug {  
    fn data_schema(&self) -> SchemaRef;  
  
    async fn scan(  
        &self,  
        state: &dyn Session,  
        projection: Option<&Vec<usize>>,  
        filters: &[Expr],  
        limit: Option<usize>,  
    ) -> Result<Arc<dyn ExecutionPlan>>;  
}
```

The source returns an `ExecutionPlan`, not a raw stream. That keeps it inside the DataFusion physical-planning model. Current concrete examples include Sail's rate and socket sources.

16.7 Flow Event Schema

Streaming records carry more than user columns. Sail prepends flow-event fields:

```
_marker  
_retracted  
<user columns...>
```

`_marker` is for control messages. `_retracted` distinguishes normal insert events from retraction/delete events.

This design lets streaming physical operators process one Arrow RecordBatch shape while preserving event semantics. It also gives future stateful operators a place to represent retract-mode updates.

16.8 Streaming Physical Nodes

The streaming logical nodes are planned by `ExtensionPhysicalPlanner`, just like other custom nodes. Their physical counterparts live under `crates/sail-physical-plan/src/streaming/`.

The important nodes are:

Node	Role
<code>StreamSourceWrapperNode</code>	Scans a real streaming source
<code>StreamSourceAdapterNode</code>	Adapts a bounded source into flow events
<code>StreamFilterNode</code>	Filters flow-event batches
<code>StreamLimitNode</code>	Applies streaming limit/offset behavior
<code>StreamCollectorNode</code>	Collects or strips flow-event fields near output

`BarrierNode` and `BarrierExec` provide checkpoint-like coordination points.

16.9 Streaming Query Lifecycle

Spark Connect exposes streaming operations such as start, stop, status, and await. Sail tracks running queries with `StreamingQuery` and `StreamingQueryManager`.

The lifecycle uses asynchronous coordination primitives:

- watch channels for stopped/error state,
- oneshot channels for stop signals,
- `JoinHandle` for the background task.

The manager lives in `SparkSessionState`, so streaming query state is scoped to the Spark session.

16.10 Current Boundaries

Streaming support is real, but not all Spark streaming semantics are complete. Readers should distinguish:

- the architecture, which is present and coherent,
- the feature surface, which is still evolving.

The flow-event schema, streaming rewriter, and query manager are the architectural foundation. Full stateful aggregations, event-time triggers, and continuous-mode coverage are areas to verify against the current code before making claims.

16.11 Execution Modes of the Graph Extensions

An ordinary distributed input does not make every downstream algorithm distributed. Pecan submits relational graph work through Sail's existing query execution. Grenada follows the relational `DataFusion` path. Banda's original native relation can gather distributed input and run

its kernel on the driver. That is driver-native execution with distributed input, not a partitioned native algorithm.

Argentea uses worker-native partitions and rejects local-only Sail execution. Its process-cluster qualifier uses two separate worker processes; physical multi-host qualification additionally verifies host identity and ownership across machines. These deployment claims should remain separate in documentation and benchmark labels.

The bounded graph job is also different from a streaming query. Argentea's native state lasts for its job, with static phase exchanges and a terminal result or certified failure. The current implementation does not provide unbounded stream state, checkpoint recovery or automatic replay of native mutations. Cancellation and worker loss use whole-query failure with cleanup.

Lazy session views keep each Connect request shallow while composing the bounded plan. Registering a view is catalog work; the terminal action executes the job. Pecan's client-controlled iterative actions are a different execution strategy. Both can reuse Sail without requiring a new graph scheduler, but their timing, staging and memory boundaries must be reported separately.

16.12 Takeaways

Local execution and cluster execution share JobRunner. Streaming and batch execution share the planning/execution stack below a logical rewrite. That is the pattern to preserve: new execution behavior should enter through clear boundaries, not by bypassing the common plan and stream model.

Navigation: Previous: Chapter 15, Custom Nodes And Optimizers | Next: Chapter 17, Testing Spark Compatibility | Reader Guide

17 Testing Spark Compatibility

Sail's promise is compatibility with Spark-facing behavior, not merely successful Rust compilation. Testing therefore has to compare Sail against Spark semantics: SQL output, DataFrame behavior, functions, errors, type coercion, schema names, streaming state, and protocol responses.

This chapter gives contributors a testing map.

17.1 Code Map

Concern	File or directory
Gold test crate	crates/sail-gold-test/
Spark gold data scripts	scripts/spark-gold-data/
Common gold report scripts	scripts/common-gold-data/
PySpark tests	python/pysail/tests/spark/
Flight tests	python/pysail/tests/flight/
Streaming tests	python/pysail/tests/spark/streaming/
SQL docs and feature docs	docs/guide/sql/
Spark test recipes	docs/development/spark-tests/
Function support utilities	python/pysail/spark/utils/

17.2 The Test Pyramid

Sail has several useful test layers:

Layer	What it catches
Rust unit tests	local invariants, parser behavior, optimizer rewrites, codecs
Gold tests	Spark SQL/function output compatibility
PySpark integration tests	DataFrame API, Connect behavior, Python UDFs
Flight tests	Flight SQL protocol and Arrow transport
Feature files	behavior-oriented execution scenarios
Manual plan inspection	logical/physical plan regressions

No single layer is enough. A function can pass unit tests and still fail Spark compatibility because Spark's null handling, string formatting, overflow behavior, or timestamp display rules differ from DataFusion defaults.

17.3 Gold Tests

Gold tests are the strongest compatibility signal. The workflow is:

1. Run real Spark examples or documentation-derived queries.
2. Store the expected output.
3. Replay the same query through Sail.
4. Compare schemas, values, ordering behavior, and display semantics.

The point is not only “does this expression run?” The point is “does this expression behave like Spark?”

This is especially important for:

- string functions,
- timestamp and interval functions,
- decimal precision/scale,
- arrays/maps/structs,
- aggregate edge cases,
- null handling,
- ANSI versus non-ANSI behavior.

17.4 Parser Round Trips

The SQL parser has a second testing dimension: syntax preservation. TreeText lets tests parse SQL and unparse it back to normalized text.

Round-trip tests catch grammar regressions that a semantic query test might miss. For example, a parser can still produce a plan for a common query while losing support for a rare Spark syntax form.

Use parser round trips for:

- DDL syntax,
- Hive compatibility clauses,
- interval literals,
- complex expressions,
- identifiers and quoting,
- function-call variants.

17.5 PySpark Integration Tests

The Python tests exercise the surface users actually touch. They are especially important for:

- Spark Connect DataFrame APIs,
- Python UDF registration and execution,
- Pandas and Arrow UDF paths,
- UDTFs,
- streaming commands,
- config behavior,
- error messages as seen by PySpark.

When a test failure shows up here, debug the path in layers:

PySpark call

- > Spark Connect protobuf
- > proto-to-spec conversion
- > PlanResolver
- > DataFusion plan

- > JobRunner
- > Arrow IPC response
- > PySpark decoding

Do not assume the bug is in the final function implementation. Many compatibility bugs are conversion or type-resolution bugs.

17.6 Flight Tests

Flight SQL tests should verify:

- GetFlightInfo schema,
- ticket creation,
- DoGet fetch behavior,
- handle consumption,
- command execution,
- Arrow batch encoding,
- basic session/catalog state expectations.

Flight SQL enters through SQL, so it shares parser/analyzer coverage with Spark SQL. Its unique risk is protocol handling and Arrow Flight framing.

17.7 Plan Inspection

Sail records plan strings for explain output:

- initial logical plan,
- final logical plan,
- final physical plan.

Plan inspection is useful when the output is wrong but no panic occurs. Ask:

1. Did the proto or SQL path produce the right spec?
2. Did the resolver choose the right DataFusion expression or Sail extension node?
3. Did the optimizer rewrite away something Spark required?
4. Did physical planning choose the expected custom operator?
5. Did cluster execution introduce a repartition or shuffle issue?

Plan bugs often look like data bugs until you inspect the tree.

17.8 Local Versus Cluster Testing

Local mode is necessary but not sufficient. Cluster mode adds:

- physical plan encoding and decoding,
- worker session setup,
- function re-resolution,
- stream locations,
- shuffle channels,
- task attempts,
- remote data movement.

Any feature that creates a custom physical operator, UDF, UDAF, table format, or shuffle-sensitive distribution should be tested in cluster mode before being treated as complete.

17.9 Testing New Functions

For a new Spark function:

1. Add focused Rust unit tests for implementation details.
2. Add gold examples based on Spark behavior.
3. Test nulls, empty inputs, nested types, and edge values.
4. Verify type coercion and return type.
5. Test both SQL and DataFrame entry paths if they differ.
6. If the function creates a UDF or aggregate state, verify distributed execution.

Spark compatibility failures tend to hide in boring edge cases. That is where the tests earn their keep.

17.10 Testing New Table Formats Or Catalogs

For storage work, test both metadata and execution:

- name resolution,
- create/list/drop behavior,
- schema conversion,
- path and option handling,
- table properties,
- scan projection and filters,
- write modes,
- row-level operations if supported,
- object-store URI behavior.

Catalog code can be correct while the resulting `TableProvider` is wrong. Table format code can scan correctly while catalog metadata is wrong. Test both sides of the boundary.

17.11 Testing the Implemented Extension Boundaries

The experimental branch provides executable examples under `examples/extensions/`. Its scalar and native graph paths need different tests. A registration count or successful import does not establish worker execution. A correct returned graph vector does not establish owner placement or cleanup.

Boundary	Evidence to retain
Scalar registration	SQL and DataFrame results, collision refusal, worker package identity
Geometry fields	Values and compatible field metadata through composition and shuffle
Relation planning	Schema/explain without native execution; validated inputs and payload
Physical reconstruction	Actual worker decoding, package identity and complete stage inventory
Graph answer	Independent oracle, unreachable/null semantics and deterministic witnesses
Stateful worker ownership	Stable owner/worker/process/adjacency identity through every phase
Cancellation or loss	Observed initialization before injection; failed job, no replay, surviving closes
Native memory refusal	Original causal receipt, terminal tasks and successful same-worker reuse
Owned storage	View removal and staging cleanup, including deferred session cleanup

Reference algorithms are useful correctness controls, but are not automatically performance references. Argentea's star WCC is not the same implementation as physical edge contraction in another path. Delta-star SSSP is not classical light/heavy delta stepping. Compare outputs against an independent oracle and state the implementation distinction before comparing timings.

17.12 Reproduce a Worker Qualification

Build the pinned Sail executable and native wheel first. Keep the executable and wheel build receipts; a command-line source SHA does not prove installed bytes. For a fresh, unmodified build of both artifacts from the checkout in chapter 13, with the build script's default paths, run from the repository root:

```
SAIL_BINARY="$PWD/target/extensions-poc/host/debug/sail"
RUNTIME_SOURCE_SHA="$(git rev-parse HEAD)"
NATIVE_SOURCE_SHA="$RUNTIME_SOURCE_SHA"
SPARK_CONNECT_MODE_ENABLED=1 .venv/bin/python examples/extensions/argentea/
python/qualify_bfs.py \
  --case chain-62 --method direction --worker-task-slots 512 \
  --sail-binary "$SAIL_BINARY" \
  --runtime-source-sha "$RUNTIME_SOURCE_SHA" \
  --native-source-sha "$NATIVE_SOURCE_SHA" \
  --output /tmp/argentea-bfs128-review
```

If reusing an older binary or wheel, substitute its recorded source revision; do not label it with the current checkout hash. Custom build paths also require the corresponding executable and Python environment.

The output directory must be new. This case exercises 62 actual expansions and 128 native stages. The ordinary small graph with the same bound checks early convergence and DONE propagation instead. Both are needed to distinguish active algorithm work from a merely large reserved plan.

Use the separate WCC, SSSP, fault and graph-resource qualifiers documented in Chapter 13. Their process-cluster evidence uses distinct worker processes on one host. A two-host claim additionally requires observed host identity and graph traffic crossing the ownership boundary. A process count cannot prove it.

17.13 Failure Evidence Must Establish the Cause

A small memory quota plus a failed RPC is insufficient to prove a memory refusal. The first error can be peer cancellation or transport loss. The graph-resource auditor requires a native `native_memory_budget` receipt after initialization, matching operation identity, terminal tasks, and owner closure. Its reuse sequence then runs three successful queries on the same workers and session.

Likewise, a worker-loss test must demonstrate which process was killed and that the native state existed before the kill. Argentea's functional controls stop the supervised workers in an observed post-init window before cancellation or loss. They do not infer the window from a fixed sleep. Every failed attempt is retained, including failures of the test harness itself.

The first expanded BFS run exposed a client bug before native execution: the larger request bound was not forwarded to the view composer. Those six failures remain in the evidence bundle alongside the corrected runs and a reproducing public-client regression test. Replacing failed receipts with later successes would hide the tested boundary and its repair.

17.14 Measurements and Qualification Are Separate

A functional development build can establish an answer or cleanup invariant; it cannot supply a release-performance claim. Preserve compiler/profile, host binary, wheel bytes, dataset identity, algorithm options, process placement, thread count, memory limits, repetitions and the original outcome of every cell.

Report elapsed-time boundaries explicitly: end-to-end input validation and staging are different from kernel time. Report memory by what was observed: native reservation peaks, process RSS and container peaks are different measurements. For two hosts, retain per-host observations and explain how an aggregate peak was formed; summing two independent peak values need not equal simultaneous usage.

Do not erase timeout, error, unsupported or unavailable cells. A fresh successful control can investigate a failure but does not replace its original observation. The retained large PageRank/WCC campaign includes one timeout among 180 trials; its later successful control remains separate.

The traversal campaign is another frozen workload and does not qualify Argentea performance by association.

17.15 Takeaways

Testing Sail means testing conversions. Every query crosses protocol, spec, planning, execution, Arrow, and client boundaries. Good tests identify which boundary failed.

Navigation: Previous: Chapter 16, Local And Streaming Execution | Next: Chapter 18, Feature Playbooks | Reader Guide

18 Feature Playbooks

This chapter is a practical contributor guide. It turns the architecture from the previous chapters into checklists for common changes.

The point is not to replace code review. The point is to help a contributor avoid the classic Sail mistake: implementing one layer of a feature and forgetting the other four.

18.1 Adding A Spark Function

Start by classifying the function.

Kind	Typical location
Scalar expression that maps to DataFusion	<code>crates/sail-plan/src/function/scalar/</code>
Scalar function requiring Spark-specific run-time behavior	<code>crates/sail-function/src/scalar/</code>
Aggregate function	<code>crates/sail-function/src/aggregate/</code> and function registry
Window function	<code>crates/sail-function/src/window/</code> and function registry
Table/generator function	<code>crates/sail-plan/src/function/table/</code> and resolver paths

Then follow the lifecycle:

1. Confirm Spark syntax and behavior.
2. Decide whether the function can be expressed as a DataFusion Expr.
3. If yes, register it through the `ScalarFunctionBuilder` closure model.
4. If no, implement the appropriate DataFusion UDF/UDAF/window function.
5. Add type coercion and return-type logic.
6. Add SQL/gold tests.
7. Add DataFrame/PySpark tests if the API path differs.
8. Verify distributed behavior if aggregate state or custom UDF encoding is involved.

Prefer expression-level registration when possible. It keeps the function visible to DataFusion optimization.

18.2 Adding A Python UDF Path

Python UDF work crosses a wide boundary:

Spark Connect function payload

- > Sail spec
- > resolver UDF object
- > Python serialization/deserialization
- > Arrow/Python conversion
- > local or worker execution
- > codec support if distributed

Before editing, identify which UDF family you are touching:

- scalar row-by-row,
- Arrow scalar,
- Pandas scalar,
- grouped aggregate,
- grouped map,
- co-grouped map,
- iterator UDF,
- UDTF.

Then check:

1. Does Spark Connect carry enough metadata?
2. Does the spec type preserve it?
3. Does resolver logic construct the right UDF object?
4. Does sail-python-udf know how to call it?
5. Does the codec preserve it for workers?
6. Are Python exceptions mapped back to Spark-friendly errors?

18.3 Adding A Catalog Backend

Catalogs answer questions about names, databases, tables, and views.

1. Add or update configuration in Sail's config model.
2. Implement CatalogProvider.
3. Register the provider in `crates/sail-session/src/catalog.rs`.
4. Decide whether it needs runtime-aware wrapping for I/O runtime use.
5. Decide whether catalog caching applies.
6. Map backend errors to `CatalogError`.
7. Return `TableStatus` with accurate format, schema, location, and properties.
8. Test create/list/get/drop behavior.
9. Test table scans through the DataFusion bridge.

Do not stop at metadata tests. A catalog backend is only useful if its `TableStatus` leads to the right table provider.

18.4 Adding a Data Source

Table formats translate table metadata into read and write behavior.

1. Implement or extend `DataSource`; writes return logical plans.
2. Register it in `crates/sail-session/src/formats.rs`.
3. Define source and sink option resolution.
4. Implement scan creation.
5. Implement write planning if supported.
6. Add object-store handling if needed.
7. Add projection/filter tests.
8. Add write-mode tests.
9. Add catalog integration tests.

For lakehouse formats, expose the optional `LakeSource` capability and test its metadata, DDL and row-level contracts separately. Also ask:

- Does this format support row-level operations?
- Does it need logical expansion rules?
- Does it need an extension physical planner?
- Does it need metadata columns for merge/delete?

18.5 Adding A Logical Plan Node

Use this path when Spark has a logical concept DataFusion does not represent natively.

1. Add the spec representation if needed.
2. Add resolver logic that creates a `LogicalPlan::Extension`.
3. Implement `UserDefinedLogicalNodeCore`.
4. Preserve schema and expressions correctly.
5. Implement projection-pushdown hints if relevant.
6. Add the physical `ExecutionPlan`.
7. Register downcast planning in `ExtensionPhysicalPlanner`.
8. Add optimizer rules only if planning alone is insufficient.
9. Add codec support for cluster mode if the physical node can reach workers.

The physical planner registration belongs in `sail-session`, not `sail-plan`.

18.6 Adding A Physical Optimizer Rule

Physical optimizer rules rewrite executable plans. They should be used when:

- DataFusion's physical plan is valid but not Spark-compatible,
- a Sail placeholder exec must be lowered,
- a partitioning/distribution contract must be enforced,
- cluster execution needs a safer physical shape.

Checklist:

1. Write down the exact physical invariant.

2. Add a focused rule implementation.
3. Insert it in `get_physical_optimizers` at the correct point.
4. Add tests for before/after plan shape.
5. Confirm the rule does not fight DataFusion's own rules.
6. Test local and cluster execution when distribution changes.

18.7 Adding A Streaming Source

Streaming sources implement `StreamSource`.

1. Define source options and schema.
2. Implement `data_schema`.
3. Implement `scan` to return an `ExecutionPlan`.
4. Ensure execution emits flow-event batches.
5. Register source discovery/read resolution.
6. Test projection behavior.
7. Test query lifecycle: start, status, stop, await.
8. Test interactions with filters, limits, and sinks.

The source should not return ordinary user-schema batches from physical execution. It must produce the flow-event schema expected by streaming physical operators.

18.8 Adding Distributed Codec Support

If a feature creates a physical object that workers need to execute, the remote execution codec has to know about it.

Ask:

1. Does the object appear inside an `ExecutionPlan` sent to workers?
2. Does DataFusion's protobuf codec already support it?
3. If not, does Sail's codec need a custom representation?
4. Does the worker session have all function/data-source registrations required to decode or re-resolve it?
5. Are version mismatches possible?

Codec work is often the difference between “works locally” and “works in Sail.”

18.9 Adding a Separately Packaged Native Extension

Start from the pinned Sedona scalar package or Nutmeg relation package under `examples/extensions/`. Keep the package's Cargo workspace and native wheel independent of Sail engine crates. Declare the entry point, manifest versions, relation input bounds and placement before adding domain code.

For a scalar contribution, test resolution by ordinary function name and actual worker execution after codec reconstruction. For a relation, return a provider that describes execution; schema inspection must not stage a graph or run an algorithm. For a mutation, return and consume a receipt relation. Do not introduce raw command/expression dispatch accidentally through client examples when the host does not implement it.

If the operation retains native state, first identify its owner and release path. Request an admitted resource lease before expansion and keep it with retained buffers. Test cancellation, state drop, retained output, session shutdown and subsequent admission. The graph-resource qualifier is a concrete example of proving reuse on live workers after a post-initialization refusal.

18.10 Adding a Worker-Native Algorithm

Use Argentea's request/core/adaptor/client separation. The core owns the algorithm and validates its phase messages. The native adaptor binds that core to worker scope and Arrow batches. The client composes bounded phases and owns temporary views and staging. Sail supplies decoding, scheduling, placement and lifecycle.

Before changing Sail, try to express the operation through ordinary relations. Pecan is the control example. If native state is required, document the missing host invariant and its existing owner: worker session registration, codec, job-group placement, memory domain, task scope or close lifecycle. Reuse the existing shuffle path for messages rather than adding graph-specific transport.

Validate an independent answer oracle first, then empty owners, skew, signed-ID extrema, duplicates, unreachable vertices, nonconvergence caps and malformed messages. Exercise the maximum supported plan and actual algorithm work at that bound; a large plan that immediately converges is a different test. Finally, validate cancellation, loss and memory refusal before measuring performance.

For performance work, freeze the exact artifacts and workload first. Distinguish reference and advanced implementations, including differences in convergence and normalization. Record every outcome and the time/memory observation boundary. A native local kernel and a distributed relational query are different execution classes even when both return the same graph result.

18.11 Debugging A Compatibility Bug

Use this order:

1. Reproduce with a minimal SQL query if possible.
2. Compare with Spark output.
3. Inspect the Spark Connect or SQL entry path.
4. Inspect the Sail spec.
5. Inspect initial and final logical plans.
6. Inspect the physical plan.
7. Run in local mode.
8. Run in cluster mode if the feature is distributed-sensitive.
9. Add the smallest regression test that would have caught it.

18.12 Takeaways

Sail features are pipelines. A complete contribution usually needs protocol/spec, resolver, Data-Fusion planning, execution, tests, and sometimes distributed codec support. The playbook is there to keep those layers visible.

Navigation: Previous: Chapter 17, Testing Spark Compatibility | Next: Chapter 19, Roadmap And Codebase Navigation | Reader Guide

19 Roadmap And Codebase Navigation

This chapter is a field guide. It tells you where to start, what is solid, what is evolving, and how to keep your mental model synchronized with the code.

Sail moves quickly, so treat exact capability claims as snapshots. The stable part is the architecture: front doors converge to spec, spec resolves to DataFusion, DataFusion plans run locally or through the distributed job runner, and Arrow batches move through every layer.

19.1 Start By Task

Goal	Start here
Understand a PySpark query	<code>crates/sail-spark-connect/src/service/plan_executor.rs</code>
Understand SQL parsing	<code>crates/sail-sql-parser/src/parser.rs</code>
Understand SQL to spec	<code>crates/sail-sql-analyzer/src/statement.rs</code>
Understand spec to logical plan	<code>crates/sail-plan/src/resolver/</code>
Understand session setup	<code>crates/sail-session/src/session_factory/server.rs</code>
Understand physical planning	<code>crates/sail-session/src/planner.rs</code>
Understand local execution	<code>crates/sail-execution/src/job_runner.rs</code>
Understand distributed execution	<code>crates/sail-execution/src/driver/</code>
Understand job graph splitting	<code>crates/sail-execution/src/job_graph/</code>
Understand shuffles	<code>crates/sail-execution/src/plan/shuffle_*.rs</code> and <code>crates/sail-execution/src/stream/</code>
Understand Python UDFs	<code>crates/sail-python-udf/src/</code>
Understand catalogs	<code>crates/sail-catalog/</code> and <code>crates/sail-session/src/catalog.rs</code>
Understand table formats	<code>crates/sail-common-datafusion/src/datasource.rs</code> and <code>crates/sail-session/src/formats.rs</code>
Understand Delta and Iceberg	<code>crates/sail-delta-lake/</code> , <code>crates/sail-iceberg/</code> , and <code>crates/sail-session/src/planner.rs</code>
Understand extensions	<code>crates/sail-session</code> , <code>crates/sail-plan</code> , <code>crates/sail-execution/src/proto/codec.rs</code>

19.2 Route Extension Work by Ownership

Read the upstream embedding entry point before adding session hooks: `crates/sail-spark-connect/src/entrypoint.rs` already exposes factory selection through merged #2630. Then use the experimental branch's protocol and native contracts as separate examples, rather than assuming they are upstream APIs.

Task on the experimental branch	Start here
Encode a relation request	examples/extensions/WRITING-AN-EXTENSION.md
Understand worker descriptors and scope	crates/sail-common-datafusion/src/worker_extension.rs
Understand stable native stage grouping	crates/sail-execution/src/job_graph/worker_groups.rs
Understand admitted native ownership	crates/sail-common-datafusion/src/native_resource.rs
Trace a distributed graph algorithm	examples/extensions/argentea/src/ and its Python qualifier
Reproduce quota release after failure	examples/extensions/argentea/GRAPH_RESOURCES.md

For an ABI question, inspect the package build receipt and compatibility matrix. For a distributed correctness question, inspect the complete stage/task and owner records. For a performance question, inspect the frozen matrix and all outcomes. These artifacts answer different questions; none is a substitute for the others.

19.3 Start By Symptom

Symptom	Likely area
PySpark API call fails before planning	Spark Connect service/proto conversion
SQL text parses incorrectly	SQL parser or analyzer
Column cannot be resolved	resolver state or attribute resolution
Function gives Spark-incompatible output	function registry or implementation
Works in SQL but not DataFrame API	protocol-to-spec conversion mismatch
Works locally but not in cluster	codec, worker session, shuffle, or task execution
Table name resolves incorrectly	catalog manager or namespace handling
File scan has wrong schema	table format or source option resolution
Merge/delete fails	lakehouse optimizer/planner path
Streaming query starts but never finishes/stops	streaming query manager or background task

19.4 What Is Mature

The following areas are central and well established architecturally:

- Spark Connect request handling,

- Sail spec as the internal semantic boundary,
- DataFusion logical and physical planning,
- Arrow RecordBatch execution,
- local JobRunner,
- catalog/table-format separation,
- custom logical/physical node pattern,
- Python UDF architecture,
- distributed job graph architecture.

Mature does not mean bug-free. It means the architecture is settled enough that new work should usually fit into the existing pattern.

19.5 What Is Evolving

Several areas are active design surfaces:

- third-party extension architecture,
- DataFusion FFI or other execution-time plugin boundaries,
- worker-side extension registration and codec negotiation,
- full streaming feature coverage,
- lakehouse write breadth,
- Iceberg writes,
- broader Flight SQL session semantics,
- Kubernetes worker lifecycle and fault tolerance,
- remote stream storage and shuffle spill behavior.

When working in these areas, prefer small changes that preserve future extension options.

19.6 Capability Snapshot

At the time this guide was prepared, the important capability shape is:

Area	Current shape
PySpark/DataFrame	Primary target through Spark Connect
SQL	Custom parser/analyzer, Spark syntax focus
Flight SQL	Secondary SQL front door
Arrow	Core memory and wire model
DataFusion	Query kernel
Local execution	Direct DataFusion stream execution
Cluster execution	Driver/worker/job graph/task stream architecture
Python UDFs	Multiple PySpark UDF/UDTF paths
Catalogs	Memory, Glue, HMS, Iceberg REST, Unity, OneLake, system
Formats	Listing file formats, Delta, Iceberg, Python data sources
Delta	Reads, append/overwrite writes, row-level paths for MERGE/DELETE, variant-related work
Iceberg	Read path, write support evolving
Streaming	Architecture present, feature coverage evolving

Always verify exact support in the current repository before documenting a public claim. Capability surfaces move faster than architecture chapters.

19.7 Reading The Crate Graph

The crate graph has a useful directional shape:

```
protocol crates
  -> spec/resolver/session crates
  -> DataFusion extension crates
  -> execution/storage/support crates
```

The most important rule is separation:

- `sail-spark-connect` should know about Spark protobufs.
- `sail-plan` should know about Sail spec and DataFusion logical plans.
- `sail-session` should assemble DataFusion session state and physical planners.
- `sail-execution` should execute physical plans without caring which protocol produced them.

When a change violates that separation, pause. Sometimes it is necessary, but it usually signals that a boundary type or extension point is missing.

19.8 The Files Worth Bookmarking

Bookmark these first:

- `crates/sail-spark-connect/src/service/plan_executor.rs`
- `crates/sail-common/src/spec/plan.rs`
- `crates/sail-plan/src/lib.rs`
- `crates/sail-plan/src/resolver/plan.rs`
- `crates/sail-plan/src/resolver/query/mod.rs`
- `crates/sail-session/src/session_factory/server.rs`
- `crates/sail-session/src/planner.rs`
- `crates/sail-execution/src/job_runner.rs`
- `crates/sail-execution/src/job_graph/planner.rs`
- `crates/sail-execution/src/proto/codec.rs`
- `crates/sail-common-datafusion/src/session/job.rs`
- `crates/sail-common-datafusion/src/datasource.rs`

These files are not the whole system. They are the quickest route back to the architecture when you feel lost.

19.9 The Definitive Mental Model

The entire guide can be compressed to one path:

```
client intent
  -> protocol-specific message or SQL
  -> Sail spec
  -> DataFusion logical plan with Sail extensions
  -> optimized logical plan
  -> physical plan with Sail extensions
  -> local stream or distributed job graph
  -> Arrow RecordBatch stream
  -> protocol-specific response
```

And one warning:

If a feature cannot survive every boundary it crosses, it is not complete.

19.10 Closing

Sail is interesting because it is not merely a Rust rewrite of Spark. It is a careful compatibility layer over a modern Rust query stack. The best way to learn it is to follow conversions: protobuf to spec, SQL to spec, spec to logical plan, logical plan to physical plan, physical plan to job graph, job graph to tasks, tasks to Arrow streams, Arrow streams back to the client.

That is the shape to preserve as Sail grows.

Navigation: Previous: Chapter 18, Feature Playbooks | Reader Guide

20 The 0.7.2 Preparation Snapshot

This edition separates two source trees. Upstream lakehq/sail is pinned at 85d06ce08825dfba4e066e9a5f54123e40829d43; its workspace declares 0.7.1 and the book prepares for the planned 0.7.2 release. The experimental extension tree is querygraph/sail at dcd44f287b8422a62abd06d2410aaf96e225e136. An experimental feature's presence in that tree does not make it part of an upstream release.

The architecture remains a sequence of translations:

```
client intent -> protocol or SQL -> Sail planning representation
-> DataFusion logical and physical plans -> local or distributed execution
-> Arrow results -> client protocol
```

The ownership of those translations has changed since the July edition. The current source uses DataSource and optional LakeSource capabilities, separate task-runner actors and stream managers, and codec code under execution's proto/ module. Following an obsolete filename can obscure an API change rather than merely delay navigation.

20.1 Changes That Matter for Extension Authors

Merged upstream #2630 lets an embedder select the Spark server's session factory. The default entry point still selects the ordinary Spark factory. This provides a way to compose session setup; it does not by itself deploy native packages to workers or serialize their operators.

Upstream #2675 adds mimalloc as the Rust global allocator for the CLI and Python extension. It does not turn a separately built wheel's allocations into admitted query memory. The experimental resource-domain and lease contract remains a separate accounting/ownership mechanism. Existing measured artifacts predate this allocator change and must keep their original source attribution.

Upstream #2676 adds SelectSemiJoinBuildSide after DataFusion's JoinSelection. For a partitioned, non-null-aware left semi-join with inconclusive size statistics, it can build from the right input when that input is an aggregate grouped on the join keys. It preserves partitioned execution; it does not infer that the aggregate is small enough to broadcast. The rule is in crates/sail-physical-optimizer/src/select_semi_join_build_side.rs. The newer main snapshot also updates npm dependencies and migration guidance. These changes do not retroactively apply to the experimental branch or its measured binaries.

The extension branch tests that boundary with Sedona scalar functions and native graph kernels. Pecan provides the relational control path: graph algorithms can run as ordinary distributed Sail queries without adding a native graph operator. Argentea then isolates what retained worker-native graph state actually needs: package decoding, task scope, stable ownership, admission and lifecycle cleanup.

20.2 The Current Crate Map

The current Sail repository has a larger set of crates than a first read of the architecture suggests. A practical contributor map is:

Area	Crates
Protocol front doors	sail-spark-connect, sail-flight, sail-cli
SQL and functions	sail-sql-parser, sail-sql-analyzer, sail-sql-macro, sail-function
Spec and planning	sail-common, sail-plan, sail-logical-plan, sail-logical-optimizer
Session and DataFusion integration	sail-session, sail-common-datafusion
Physical execution	sail-physical-plan, sail-physical-optimizer, sail-execution
Python and Arrow interop	sail-python, sail-python-udf
Catalogs	sail-catalog, sail-catalog-memory, sail-catalog-system, sail-catalog-hms, sail-catalog-glue, sail-catalog-iceberg, sail-catalog-unity, sail-catalog-onelake
Lakehouse formats	sail-delta-lake, sail-iceberg
Storage and cache	sail-data-source, sail-object-store, sail-cache, sail-system-store
Shuffle service integration	sail-celeborn
Support	sail-build-scripts, sail-gold-test, sail-telemetry, sail-common-hms, sail-mimalloc

This map is more useful than a dependency graph when you are trying to make a change. Start with the area that owns the user’s observable behavior, then walk inward until you find the semantic boundary:

- protocol conversion lives near Spark Connect, Flight SQL, or SQL parser code;
- Spark semantic decisions usually land in analyzer, spec, resolver, function, or catalog layers;
- DataFusion integration lands in session, logical/physical extension nodes, or physical planners;
- distributed behavior lands in job graphs, codecs, workers, task streams, and shuffle paths;
- lakehouse writes land in table-format code and driver-side commit rules.

20.3 Reading Source and Evidence Together

For an implementation claim, begin with the pinned tree and the owning module. For a qualification claim, begin with the receipt’s host binary, native wheel, source revision and command. These are related but different questions. A new source commit does not retroactively change which executable an older receipt tested.

The graph qualification records include exact vectors, owner identities, native phases, Sail stage/task inventories, cleanup and failed attempts. PageRank and the original BFS bound have physical two-host results. The later WCC/SSSP fault and memory-reuse checks and expanded BFS bound have ARM process-cluster evidence; remaining platform gates stay explicit in the extension integration document. Performance comparisons have their own datasets, semantics, resource envelopes, profiles and time/memory boundaries.

Source-path validation in this book is intentionally limited. The script `sail-rust-book/scripts/audit-source-paths.py` resolves explicit inline code paths against both pinned trees. A successful check proves path existence, not that a Rust excerpt compiles or that its explanation is correct. Those require source review and, for runnable examples, execution against the corresponding artifact.

20.4 Build the Ordinary Book Formats

The source repository owns the manuscript, diagrams, metadata and source-specific preparation hook. `FIRSTPAIR.md` defines the delivery identity and points to the shared FirstPair builder. From a checkout with the required FirstPair tooling:

```
repo_root="$(git rev-parse --show-toplevel)"
"$HOME/src/firstpair/publishing/scripts/build-library-book.sh" \
  --repo-root "$repo_root"
```

`book.build.json` selects the preparation hook and output formats. A local build produces the ordinary book artifacts; it does not publish the public library. Validate the PDF visually and check EPUB, HTML and links as well as successful process exit. A source revision and an existing generated PDF can represent different editions until the build and its validation have completed.

20.5 The Code-Navigation Vault

The Obsidian vault is a separate generated product linking chapters, code files, fragments and symbols. A vault generated from an older source pin is not a current code index merely because the Markdown manuscript has changed. Its source identity and fragment targets require their own regeneration and validation.

Vault mutation has an additional operational requirement: close it in Obsidian and confirm closure before generation, because the application can rewrite index and workspace files concurrently. The source contract and shared FirstPair workflow define candidate validation and publication. Ordinary manuscript and PDF/EPUB work can proceed independently of vault replacement.

20.6 Continue from a Concrete Boundary

To add a scalar function, follow resolution, field semantics and codec reconstruction. To add a source, follow logical reads/writes and any optional lake capability. To add native state, follow admission, owner lifetime, task placement and failure behavior. Keep domain computation outside the host when an existing plan can express it, and add a host hook only for an observed missing contract.

This makes the extension experiment useful beyond graphs: it provides executable examples of where Sail's existing abstractions suffice and where a focused lifecycle or serialization contract is required.